

O'REILLY®

AI at the Edge

Solving Real-World Problems with
Embedded Machine Learning



Compliments of



EDGE IMPULSE

**Daniel Situnayake
& Jenny Plunkett**
Foreword by Pete Warden

AI at the Edge

Edge AI is transforming the way computers interact with the real world, allowing IoT devices to make decisions using the 99% of sensor data that was previously discarded due to cost, bandwidth, or power limitations. With techniques like embedded machine learning, developers can capture human intuition and deploy it to any target—from ultra-low power microcontrollers to embedded Linux devices.

This practical guide gives engineering professionals, including product managers and technology leaders, an end-to-end framework for solving real-world industrial, commercial, and scientific problems with edge AI. You'll explore every stage of the process, from data collection to model optimization to tuning and testing, as you learn how to design and support edge AI and embedded ML products. Edge AI is destined to become a standard tool for systems engineers. This high-level road map helps you get started.

- Develop your expertise in AI and ML for edge devices
- Understand which projects are best solved with edge AI
- Explore key design patterns for edge AI apps
- Learn an iterative workflow for developing AI systems
- Build a team with the skills to solve real-world problems
- Follow a responsible AI process to create effective products

Daniel Situnayake is head of machine learning at Edge Impulse, where he leads embedded machine learning R&D.

Jenny Plunkett, senior developer relations engineer at Edge Impulse, is a technical speaker, developer evangelist, and technical content creator.

"AI at the Edge provides an excellent introduction on combining modern AI techniques and embedded systems."

—Elexia White

Author of *Making Embedded Systems*
and host of the *Embedded* podcast

"Anyone joining this exciting new field will benefit from the deep insights and clarity of thought this book provides."

—Aurélien Geron

Former lead of YouTube's automatic
video classification team
and best-selling author

"I can imagine it being used as a reference book, returning to it time and time again."

—Fran Baker

Director of Sustainability
and Social Impact, Arm

MACHINE LEARNING

ISBN: 978-1-098-16476-8

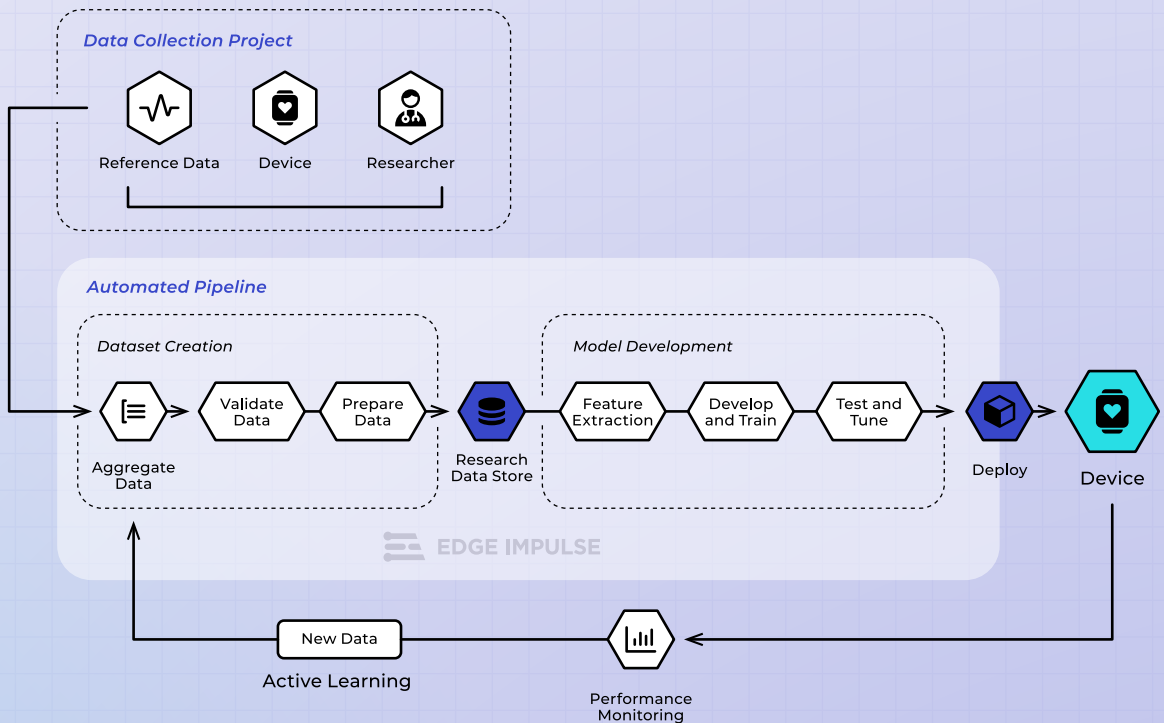


Twitter: @oreillymedia
[linkedin.com/company/oreilly-media](https://www.linkedin.com/company/oreilly-media)
[youtube.com/oreillymedia](https://www.youtube.com/oreillymedia)

Build. Train. Optimize. AI for the edge.

Make your product smarter with:

- An end-to-end engineering platform for Edge AI
- Tools for data quality, feature engineering, training, edge optimization, deployment, and MLOps



Your ML Workflow with Edge Impulse

Praise for *AI at the Edge*

AI at the Edge introduces the new and fast-growing field of edge AI in a practical, easy-to-follow way. It demystifies jargon and highlights real challenges that you are likely to encounter when building edge AI applications. The book offers an essential guide to going from concept to deployment—a must-read for getting started in the field.

—Wiebke Hutiri, Delft University of Technology

I really love the writing style which makes complex technical topics approachable and digestible. I can imagine it being used as a reference book, returning to it time and time again—which I will certainly be doing!

—Fran Baker, Director of Sustainability and Social Impact, Arm

What a wonderfully accessible and thorough introduction to the emerging field of edge AI! It covers an impressive breadth of topics, from the core concepts to the latest hardware and software tools, it's full of actionable advice, and includes several end-to-end examples. Anyone joining this exciting new field will benefit from the deep insights and clarity of thought this book provides.

—Aurélien Geron, former lead of YouTube's automatic video classification team and best-selling author

This is the guide to creating smarter devices: *AI at the Edge* provides an excellent introduction on combining modern AI techniques and embedded systems.

—Elecia White, author of *Making Embedded Systems* and host of the *Embedded* podcast

AI at the Edge

*Solving Real-World Problems with
Embedded Machine Learning*

Daniel Situnayake and Jenny Plunkett

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

AI at the Edge

by Daniel Situnayake and Jenny Plunkett

Copyright © 2023 Daniel Situnayake and Jenny Plunkett. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Nicole Butterfield

Development Editor: Angela Rufino

Production Editor: Elizabeth Faerm

Copyeditor: nSight, Inc.

Proofreader: Charles Roumeliotis

Indexer: WordCo Indexing Services, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

January 2023: First Edition

Revision History for the First Edition

2023-01-10: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098120207> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *AI at the Edge*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and Edge Impulse. See our [statement of editorial independence](#).

978-1-098-12020-7

[LSI]

Jenny would like to dedicate this book to every woman who is currently pursuing, or is interested in pursuing, an engineering degree—you can do anything you put your mind to.

Dan would like to dedicate this book to the Situnayake family. It's been a tough few years, but we always make it through as a team.

Table of Contents

Foreword from Edge Impulse.....	xv
Foreword.....	xvii
Preface.....	xix
1. A Brief Introduction to Edge AI.....	1
Defining Key Terms	1
Embedded	1
The Edge (and the Internet of Things)	3
Artificial Intelligence	6
Machine Learning	8
Edge AI	10
Embedded Machine Learning and Tiny Machine Learning	12
Digital Signal Processing	12
Why Do We Need Edge AI?	13
To Understand the Benefits of Edge AI, Just BLERP	14
Edge AI for Good	18
Key Differences Between Edge AI and Regular AI	19
Summary	23
2. Edge AI in the Real World.....	25
Common Use Cases for Edge AI	25
Greenfield and Brownfield Projects	26
Real-World Products	27
Types of Applications	32
Keeping Track of Objects	32
Understanding and Controlling Systems	34

Understanding People and Living Things	36
Transforming Signals	39
Building Applications Responsibly	41
Responsible Design and AI Ethics	43
Black Boxes and Bias	46
Technology That Harms, Not Helps	49
Summary	53
3. The Hardware of Edge AI.....	55
Sensors, Signals, and Sources of Data	55
Types of Sensors and Signals	58
Acoustic and Vibration	59
Visual and Scene	60
Motion and Position	62
Force and Tactile	63
Optical, Electromagnetic, and Radiation	64
Environmental, Biological, and Chemical	65
Other Signals	66
Processors for Edge AI	68
Edge AI Hardware Architecture	68
Microcontrollers and Digital Signal Processors	70
System-on-Chip	75
Deep Learning Accelerators	77
FPGAs and ASICs	78
Edge Servers	81
Multi-Device Architectures	82
Devices and Workloads	84
Summary	84
4. Algorithms for Edge AI.....	85
Feature Engineering	85
Working with Data Streams	86
Digital Signal Processing Algorithms	88
Combining Features and Sensors	93
Artificial Intelligence Algorithms	95
Algorithm Types by Functionality	96
Algorithm Types by Implementation	101
Optimization for Edge Devices	116
On-Device Training	119
Summary	121

5. Tools and Expertise.....	123
Building a Team for AI at the Edge	123
Domain Expertise	124
Diversity	126
Stakeholders	128
Roles and Responsibilities	129
Hiring for Edge AI	132
Learning Edge AI Skills	134
Tools of the Trade	136
Software Engineering	137
Working with Data	141
Algorithm Development	143
Running Algorithms On-Device	153
Embedded Software Engineering and Electronics	157
End-to-End Platforms for Edge AI	162
Summary	167
6. Understanding and Framing Problems.....	169
The Edge AI Workflow	169
Responsible AI in the Edge AI Workflow	171
Do I Need Edge AI?	172
Describing a Problem	172
Do I Need to Deploy to the Edge?	173
Do I Need Machine Learning?	178
Practical Exercise	185
Determining Feasibility	186
Moral Feasibility	187
Business Feasibility	189
Dataset Feasibility	191
Technological Feasibility	192
Making a Final Decision	196
Planning an Edge AI Project	197
Summary	199
7. How to Build a Dataset.....	201
What Does a Dataset Look Like?	201
The Ideal Dataset	203
Datasets and Domain Expertise	205
Data, Ethics, and Responsible AI	206
Minimizing Unknowns	208
Ensuring Domain Expertise	209

Data-Centric Machine Learning	210
Estimating Data Requirements	211
A Practical Workflow for Estimating Data Requirements	213
Getting Your Hands on Data	215
The Unique Challenges of Capturing Data at the Edge	217
Storing and Retrieving Data	220
Getting Data into Stores	222
Collecting Metadata	223
Ensuring Data Quality	225
Ensuring Representative Datasets	225
Reviewing Data by Sampling	227
Label Noise	229
Common Data Errors	231
Drift and Shift	233
The Uneven Distribution of Errors	234
Preparing Data	235
Labeling	235
Formatting	246
Data Cleaning	248
Feature Engineering	255
Splitting Your Data	256
Data Augmentation	261
Data Pipelines	263
Building a Dataset over Time	265
Summary	266
8. Designing Edge AI Applications.....	267
Product and Experience Design	268
Design Principles	270
Scoping a Solution	271
Setting Design Goals	274
Architectural Design	278
Hardware, Software, and Services	278
Basic Application Architectures	279
Complex Application Architectures and Design Patterns	286
Working with Design Patterns	291
Accounting for Choices in Design	292
Design Deliverables	295
Summary	296

9. Developing Edge AI Applications.....	297
An Iterative Workflow for Edge AI Development	297
Exploration	298
Goal Setting	300
Bootstrapping	302
Test and Iterate	306
Deployment	313
Support	315
Summary	315
10. Evaluating, Deploying, and Supporting Edge AI Applications.....	317
Evaluating Edge AI Systems	317
Ways to Evaluate a System	319
Useful Metrics	322
Techniques for Evaluation	334
Evaluation and Responsible AI	337
Deploying Edge AI Applications	338
Predeployment Tasks	339
Mid-Deployment Tasks	341
Postdeployment Tasks	342
Supporting Edge AI Applications	343
Postdeployment Monitoring	343
Improving a Live Application	350
Ethics and Long-Term Support	353
What Comes Next	356
11. Use Case: Wildlife Monitoring.....	357
Problem Exploration	358
Solution Exploration	359
Goal Setting	359
Solution Design	360
What Solutions Already Exist?	360
Solution Design Approaches	361
Design Considerations	363
Environmental Impact	364
Bootstrapping	366
Define Your Machine Learning Classes	367
Dataset Gathering	367
Edge Impulse	368
Choose Your Hardware and Sensors	369
Data Collection	370

iNaturalist	372
Dataset Limitations	375
Dataset Licensing and Legal Obligations	376
Cleaning Your Dataset	376
Uploading Data to Edge Impulse	377
DSP and Machine Learning Workflow	379
Digital Signal Processing Block	380
Machine Learning Block	381
Testing the Model	392
Live Classification	392
Model Testing	393
Test Your Model Locally	395
Deployment	395
Create Library	396
Mobile Phone and Computer	397
Prebuilt Binary Flashing	400
Impulse Runner	401
GitHub Source Code	401
Iterate and Feedback Loops	401
AI for Good	403
Related Works	404
Datasets	404
Research	404
12. Use Case: Food Quality Assurance.....	407
Problem Exploration	407
Solution Exploration	408
Goal Setting	409
Solution Design	409
What Solutions Already Exist?	410
Solution Design Approaches	410
Design Considerations	412
Environmental and Social Impact	413
Bootstrapping	414
Define Your Machine Learning Classes	414
Dataset Gathering	415
Edge Impulse	415
Choose Your Hardware and Sensors	416
Data Collection	417
Data Ingestion Firmware	417
Uploading Data to Edge Impulse	418

Cleaning Your Dataset	420
Dataset Licensing and Legal Obligations	422
DSP and Machine Learning Workflow	423
Digital Signal Processing Block	424
Machine Learning Block	426
Testing the Model	429
Live Classification	430
Model Testing	431
Deployment	432
Prebuilt Binary Flashing	433
GitHub Source Code	437
Iterate and Feedback Loops	437
Related Works	439
Research	439
News and Other Articles	440
13. Use Case: Consumer Products.....	441
Problem Exploration	441
Goal Setting	442
Solution Design	442
What Solutions Already Exist?	443
Solution Design Approaches	443
Design Considerations	445
Environmental and Social Impact	446
Bootstrapping	447
Define Your Machine Learning Classes	447
Dataset Gathering	448
Edge Impulse	448
Choose Your Hardware and Sensors	449
Data Collection	450
Data Ingestion Firmware	450
Cleaning Your Dataset	452
Dataset Licensing and Legal Obligations	453
DSP and Machine Learning Workflow	454
Digital Signal Processing Block	455
Machine Learning Blocks	459
Testing the Model	463
Live Classification	463
Model Testing	465
Deployment	466
Prebuilt Binary Flashing	466

GitHub Source Code	466
Iterate and Feedback Loops	467
Related Works	467
Research	468
News and Other Articles	468
Index.....	471

Foreword from Edge Impulse

In 2023 it has felt like *everyone* is talking about artificial intelligence. A convergence of new and existing technologies has created novel capabilities, and business and technical leaders are applying them to build mind-blowing products and experiences. The public is taking note.

While much of the attention is on generative AI, with its gigantic models that output natural-sounding text and images, there's an even larger revolution happening at a far smaller scale. Advances in software and hardware mean that embedded systems—the tiny, hidden computers that run our world—have become capable of hosting powerful AI algorithms that can read and interpret sensor data. With 28 billion microcontrollers produced *every year*, the opportunity is tremendous.

What started in the labs of companies like Google, where I first worked on edge AI, is now bringing new capabilities to every corner of our world. From the phone in your pocket to the factory floor, AI and machine learning are making it possible for devices to recognize voice commands, identify dangerous situations, monitor health conditions, and protect endangered species. The most innovative companies are using edge AI to reduce costs, improve customer experiences, and build products that were never possible before. The rest are still catching up.

Despite this potential, the unfortunate fact is that most AI projects fail. Edge AI is the intersection of two challenging disciplines: AI and machine learning, and embedded engineering. Very few orgs have the skills to be productive with both. Much of the success of AI projects depends on building the right team, and giving them access to the right tools and skills for the job.

At Edge Impulse, where I lead the machine learning team, we build tools that help organizations be successful with edge AI. We enable a team's ML experts to make sense of sensor data and to adapt their models to run on embedded systems, and we empower the experienced embedded engineers to develop their own algorithms, no prior knowledge required.

I started at Edge Impulse as the first full-time employee, nearly four years ago. We've since grown to a large, multidisciplinary team, including experts from across many industries. This book, like the tools we've built, represents the knowledge we've collectively uncovered about how to build successful projects in the edge AI space. It draws insight from the thousands of real-world projects that companies have built with our product.

Our goal as a company is to enable your existing developers to succeed with edge AI—whether they're ML practitioners or embedded software engineers—without having to hire or train a whole new team. We use our deep organizational knowledge to help you navigate the pitfalls and challenges of developing an AI product, lower your time to market, and help your engineers feel immediately productive. As these chapters show, our tools can both lower the costs of development and reduce the risks associated with delivering AI projects.

As a truly transformative technology, edge AI represents vast potential for impact. Some of those who read this book will go on to create products that transform their respective industries, create huge economic value, and deliver improvements in quality of life to millions of people. We hope that with our help, it's your team that succeeds.

— *Daniel Situnayake*
Head of ML at Edge Impulse,
coauthor of AI at the Edge and TinyML

Foreword

In 2022, GitHub CEO Thomas Dohmke said, “I think the shift to the cloud will happen at such a rapid rate, that in just a few years I predict there will be no more code on your local computer.”¹ This book does a great job of explaining why I and a lot of other people in the emerging field of edge ML think he’s dead wrong.

We’re starting to see the emergence of many practical applications like high-quality voice recognition, forest fire prevention, and smart home controls that are only possible because local devices are now capable of running advanced machine learning algorithms. Jenny and Dan have put together a wonderful book that not only explains why adding intelligence to edge applications is so crucial to solving important problems, but also walks the reader through the steps required to design, implement, and test these kinds of applications.

It can feel pretty intimidating when you first start looking at a machine learning project on the edge. The field involves a lot of jargon, is changing rapidly, and requires knowledge from domains like embedded systems and artificial intelligence that have traditionally not been well integrated. What the authors have achieved is a gentle but thorough introduction to everything you need to know to work effectively on an application. They’ve also managed to make it accessible to a wide range of readers thanks to their emphasis on examples from the real world and use of plain English instead of math or code to explain even complex topics. This makes the book easy to recommend to product managers, executives, and designers, as well as engineers.

They’ve managed to take a lot of hard-won knowledge gained from experience and distill it down to lessons that will give any team working on these kinds of applications a big head start.

¹ From [GitHub’s Twitter account](#).

They also manage to explore beyond the practical concerns of how to build an edge ML application, and will help you understand how to avoid causing harm with your work. The ethical concerns around AI can seem overwhelming, but the authors manage to break them down into questions you can apply in a straightforward way as part of the project planning and testing process. This will help all the stakeholders on your project collaborate, and hopefully avoid a lot of the potential dangers involved in giving computers more decision-making power over our lives.

I've been working on edge ML applications for over ten years now, first at a startup, then as a tech lead at Google—and now as the founder of another startup, and I will be asking everyone who joins our team to read this book. If you have any interest at all in this area, whether as a coder, designer, manager, or just someone who cares about this new technology that's emerging in our world, I can't recommend this book highly enough. I guarantee that reading it will introduce you to a lot of fascinating ideas, as well as help you build the next generation of smart devices.

— *Pete Warden, CEO at Useful Sensors Inc.,
creator of TensorFlow Lite for Microcontrollers*

Preface

Over the past few years, a growing community of engineers and researchers have quietly rewritten the rules for how computers interact with the physical world. The result, a technology known as “edge artificial intelligence,” promises to upend a century of computer history and touch the lives of every human being.

With a tiny software update, edge AI technology can grant cheap, energy-efficient processors—already inside everything from dishwashers to thermostats—the ability to perceive and understand the world. We can empower everyday objects with their own intelligence, no longer dependent on data-hungry centralized servers. And next-generation tools put this magic in reach of everyone, from high school students to conservation researchers.

There are already many edge AI products out there in the world. Here are some that we’ll meet in the pages of this book:

- Smart devices that help prevent forest fires caused by electricity transmission, by mounting to electricity pylons and predicting when a fault may occur
- Wearable bands that keep firefighters safe by warning when they’re at risk from heat strain and overexertion
- Voice user interfaces that provide hands-free control of technology, no internet connection required
- Smart collars that monitor the movements of wild elephants, helping researchers understand their behavior and protect them from conflict
- Wildlife cameras that identify specific animal species and help scientists understand their behavior

The technology of edge AI is still fresh and new, and these existing applications are just a glimpse of what is possible. As more people learn how to work with edge AI, they’ll create applications that solve problems across every avenue of human activity.

The goal of this book is to empower you to be one of them. We want to help you create successful edge AI products based on your own unique perspectives.

About This Book

This book is designed for the engineers, scientists, product managers, and decision makers who will drive this revolution. It's a high-level guide to the entire space, providing a workflow and a framework for solving real-world problems using edge AI.

Among other things, we hope to teach you:

- The opportunities, limitations, and risks inherent to various edge AI technologies
- A framework for analyzing problems and designing solutions using AI and embedded machine learning
- An end-to-end practical workflow for successfully developing edge AI applications

In the first part of the book, the initial chapters will introduce and discuss the key concepts, helping you understand the lay of the land. The next few will take you through the practical processes that will help you design and implement your own applications.

In the second part of the book, starting in **Chapter 11**, we'll use three end-to-end walkthroughs to demonstrate how to apply your knowledge to solve real problems in scientific, industrial, and consumer projects.

By the end of the book, you'll feel confident in viewing the world through the lens of edge AI, and you'll have a solid set of tools you can use to help build effective solutions.



This book covers a lot of topics! For an overview of everything we've included, take a quick look at the table of contents.

What to Expect

This isn't a programming book or a tutorial for a particular set of tools, so don't expect a ton of line-by-line code explanations or step-by-step guides to using specific software. Instead, you'll learn how to apply general frameworks to solve problems using whichever tools are best suited to the job.

That said, this is a topic that benefits greatly from tangible, interactive examples that can be explored, customized, and built upon. In the course of the book, we'll provide all sorts of artifacts you can explore—from Git repositories to free online datasets and example training pipelines.

Many of these will be hosted in **Edge Impulse**, which is an engineering tool for building edge AI applications.¹ It's built on open source technologies and standard best practices, so you'll be able to understand the principles even if you do your own work on a different platform. The book's authors are both big fans of Edge Impulse—but they may be biased, since they are part of the team that built it!



To guarantee portability, all the artifacts of the machine learning pipeline can be exported from Edge Impulse in open formats, including the datasets, machine learning models, and C++ implementations of any signal processing code.

What You Need to Know Already

This book is about building software that runs on edge devices, so some familiarity with the high-level concepts of embedded development will be helpful. This could be on either resource-constrained devices such as microcontrollers or digital signal processors (DSPs), or on general-purpose devices such as embedded Linux computers.

That said, if you're just getting started with embedded software, you should have no trouble keeping up! We'll keep things simple and introduce new topics as they come up.

Beyond that, no particular knowledge is assumed. Since the goal of this book is to provide a practical road map for an entire field of engineering, we'll cover a lot of topics at a high level. If you're interested in digging deeper into anything we mention—from the fundamentals of machine learning to the essentials of ML application design—we'll provide lots of resources that we've found useful in our own learning.

Responsible, Ethical, and Effective AI

The most important part of building any kind of application is ensuring that it works correctly in the real world. Unfortunately, AI applications are especially vulnerable to a class of issues that make them *appear* to work well when in reality they are failing—often in very harmful ways.

¹ Edge Impulse is described in the academic paper “**Edge Impulse: An MLOps Platform for Tiny Machine Learning**” (S. Hymel et. al, 2022).

Avoiding this class of problems will be a core theme—if not *the* core theme—of this book. Because modern AI development is an iterative process, it isn’t enough to test your system at the end of the workflow to see whether it works. Instead, you need to be thinking about the potential pitfalls at every step along the way. You’ll have to understand where the risks lie, critically review your intermediate results, and make informed decisions that take the needs of your stakeholders into account.

Over the course of the book, we’ll introduce a strong framework that will help you understand, reason, measure performance, and make decisions based on an awareness of the things that can go wrong when building AI applications. It will be the foundation for our entire development process and will shape the way we design our applications.

This process begins at the very inception of a project. To build effective applications, it’s critical to understand that there are certain use cases for which our current approach to artificial intelligence is simply not an appropriate tool. In many situations, the risk of causing harm—physical, financial, or societal—outweighs the potential benefit of deploying AI. This book will teach you how to identify these risks and take them into account when exploring the feasibility of a project.

As domain experts, we have the responsibility to make sure the technology we create is used appropriately. Nobody else is better positioned to do this work, so it falls on us to do it well. This book will help you make the right decisions and create applications that perform well, avoid harm, and benefit the wider world.

Further Resources

A book that covered all of embedded AI, from low-level implementation to high-level design patterns, would be the size of an entire bookshelf! Instead of trying to squeeze everything into one volume, the book you’re reading will provide a detailed but high-level road map of the whole space.

To zoom in on the minutiae that are relevant for your particular project, “[Learning Edge AI Skills](#)” on page 134 recommends plenty of further resources.

Reaching Out

The authors would love to hear from you; get in touch at hello@edgeaibook.com.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/ai-at-the-edge>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*AI at the Edge* by Daniel Situnayake and Jenny Plunkett (O'Reilly). Copyright 2023 Daniel Situnayake and Jenny Plunkett, 978-1-098-12020-7."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/ai-at-the-edge>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>.

Follow us on Twitter: <https://twitter.com/oreillymedia>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

This book wouldn't have been possible without the hard work and support of a large number of people to whom we are very grateful.

We've been honored with a foreword by the one and only **Pete Warden**, who—beyond being a visionary technologist who deserves much of the credit for launching this field—is a wonderful human being and a great friend. Thank you so much for your support, Pete!

We extend our deep gratitude to **Wiebke (Toussaint) Hutiri**, who went truly above and beyond in helping shape and inform the responsible AI content in this book, including contributing a fantastic introduction to “**Responsible Design and AI Ethics**” on page 43. You are a star in your field.

We are indebted to our incredible panel of technical reviewers and advisors whose wisdom and insight has had such a huge impact on this book. Their names are Alex Elium, Aurélien Geron, Carlos Roberto Lacerda, David J. Groom, Elecia White, Fran Baker, Jen Fox, Leonardo Cavagnis, Mat Kelcey, Pete Warden, Vijay Janapa Reddi, and Wiebke (Toussaint) Hutiri. An additional big thanks to Benjamin Cabé for allowing us to feature his artificial nose project. Any inaccuracies are entirely the responsibility of the authors.

We'd also like to thank the amazing team at O'Reilly, especially Angela Rufino, who has shepherded us through the writing process with the utmost understanding and care. Major gratitude to Elizabeth Faerm, Kristen Brown, Mike Loukides, Nicole Taché, and Rebecca Novack.

This book would not exist without the support of our team at Edge Impulse, an all-star cast of absolute heroes. Special thanks to the founders, Zach Shelby and Jan Jongboom, for believing in our vision for this book, supporting us in making it happen, and creating a space where ideas can bloom. Much love to the entire team, which at the time of writing includes: Adam Benzion, Alessandro Grande, Alex Elium, Amir Sherman, Arjan Kamphuis, Artie Beavis, Arun Rajasekaran,

Ashvin Roharia, Aurelien Lequertier, Carl Ward, Clinton Oduor, David Schwarz, David Tischler, Dimi Tomov, Dmitry Maslov, Emile Bosch, Eoin Jordan, Evan Rust, Fernando Jiménez Moreno, Francesco Varani, Jed Huang, Jim Edson, Jim van der Voort, Jodie Lane, John Pura, Jorge Silva, Joshua Buck, Juliette Okel, Keelin Murphy, Kirtana Moorthy, Louis Moreau, Louise Paul, Maggi Yang, Mat Kelcey, Mateusz Majchrzycki, Mathijs Baaijens, Mihajlo Raljac, Mike Senese, Mikey Beavis, MJ Lee, Nabil Koroghli, Nick Famighetti, Omar Shrit, Othman Mekhannene, Paige Holvik, Raul James, Raul Vergara, RJ Vissers, Ross Lowe, Sally Atkinson, Saniea Akhtar, Sara Olsson, Sergi Mansilla, Shams Mansoor, Shawn Hanscom, Shawn Hymel, Sheena Patel, Tyler Hoyle, Vojislav Milivojevic, William DeLey, Yan Li, Yana Vibe, and Zin Kyaw. You make magic happen.

Jenny would like to thank her Texas family and friends for being super supportive over the years, and her cats Blue Gene and Beatrice for being the best coworkers. She especially would like to thank her dad, Michael Plunkett, who encouraged her to pursue electrical engineering at The University of Texas at Austin, and who inspired her lifelong curiosity in new technologies.

Dan would like to thank his family and friends for being supportive of every big adventure. He's deeply grateful to Lauren Ward for her love and partnership throughout all of our journeys. And he thanks Minicat for her calming feline presence—and permission to use her photographs in this book.

A Brief Introduction to Edge AI

Welcome on board! In this chapter, we'll be taking a comprehensive tour of the edge AI world. We'll define the key terms, learn what makes "edge AI" different from other AI, and explore some of the most important use cases. Our goal for this chapter is to answer these two important questions:

- What is edge AI, anyway?
- Why would I ever need it?

Defining Key Terms

Each area of technology has its own taxonomy of buzzwords, and edge AI is no different. In fact, the term *edge AI* is a union of two buzzwords, fused together into one mighty term. It's often heard alongside its siblings, *embedded machine learning* and *TinyML*.

Before we move on, we better spend some time defining these terms and understanding what they mean. Since we're dealing with compound buzzwords, let's deal with the most fundamental parts first.

Embedded

What is "embedded"? Depending on your background, this may be the most familiar of all the terms we're trying to describe. *Embedded systems* are the computers that control the electronics of all sorts of physical devices, from Bluetooth headphones to the engine control unit of a modern car. *Embedded software* is software that runs on them. [Figure 1-1](#) shows a few places where embedded systems can be found.

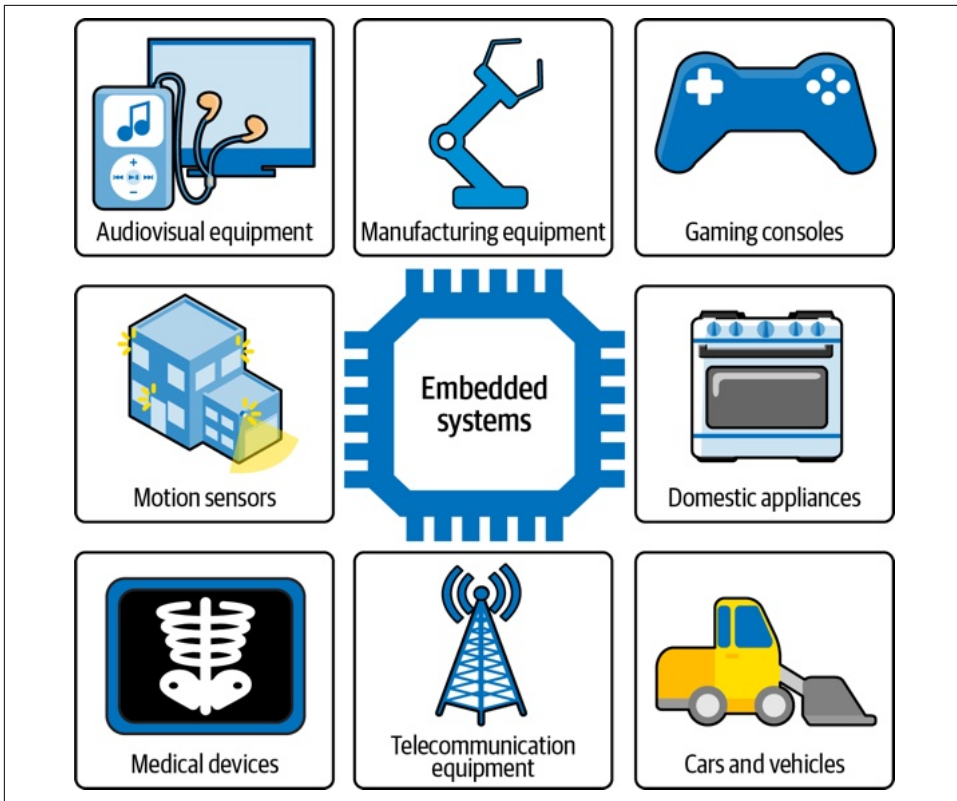


Figure 1-1. Embedded systems are present in every part of our world, including the home and the workplace

Embedded systems can be tiny and simple, like the microcontroller that controls a digital watch, or large and sophisticated, like the embedded Linux computer inside a smart TV. In contrast to general-purpose computers, like a laptop or smartphone, embedded systems are usually meant to perform one specific, dedicated task.

Since they power much of our modern technology, embedded systems are extraordinarily widespread. In fact, there were over 28 billion microcontrollers shipped in the year 2020¹—just one type of embedded processor. They’re in our homes, our vehicles, our factories, and our city streets. It’s likely you are never more than a few feet from an embedded system.

¹ As reported by [Business Wire](#).

It's common for embedded systems to reflect the constraints of the environments into which they are deployed. For example, many embedded systems are required to run on battery power, so they're designed with energy efficiency in mind—perhaps with limited memory or an extremely slow clock rate.

Programming embedded systems is the art of navigating these constraints, writing software that performs the task required while making the most out of limited resources. This can be incredibly difficult. Embedded systems engineers are the unsung heroes of the modern world. If you happen to be one, thank you for your hard work!

The Edge (and the Internet of Things)

The history of computer networks has been a gigantic tug of war. In the first systems—individual computers the size of a room—computation was inherently centralized. There was one machine, and that one machine did all the work.

Eventually, however, computers were connected to terminals (as shown in [Figure 1-2](#)) that took over some of their responsibilities. Most of the computation was happening in the central mainframe, but some simple tasks—like figuring out how to render letters onto a cathode-ray tube screen—were done by the terminal's electronics.

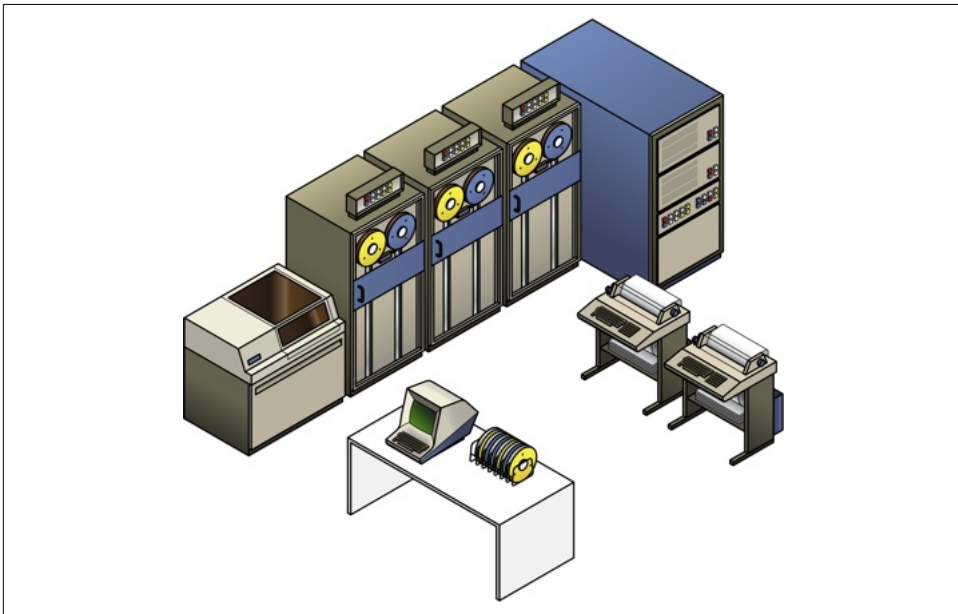


Figure 1-2. Mainframe computers performed the bulk of the computation, while simple terminals processed input, printed output, and rendered basic graphics

Over time, terminals became more and more sophisticated, taking over more and more functions that were previously the job of the central computer. The tug-of-war had begun! Once the personal computer was invented, small computers could do useful work without even being connected to another machine. The rope had been pulled to the opposite extreme—from the center of the network to the *edge*.

The growth of the internet, along with web applications and services, made it possible to do some really cool stuff—from streaming video to social networking. All of this depends on computers being connected to servers, which have gradually taken over more and more of the work. Over the past decade, most of our computing has become centralized again—this time in the “cloud.” When the internet goes down, our modern computers aren’t much use!

But the computers we use for work and play are not our only connected devices. In fact, it is estimated that in 2021 there were 12.2 billion assorted items connected to the internet,² creating and consuming data. This vast network of objects is called the Internet of Things (IoT), and it includes everything you can think of: industrial sensors, smart refrigerators, internet-connected security cameras, personal automobiles, shipping containers, fitness trackers, and coffee machines.



The first ever IoT device was created in 1982. Students at Carnegie Mellon University **connected a Coke vending machine to the ARPANET**—an early precursor to the internet—so they could check whether it was empty without leaving their lab.

All of these devices are embedded systems containing microprocessors that run software written by embedded software engineers. Since they’re at the edge of the network, we can also call them *edge devices*. Performing computation on edge devices is known as *edge computing*.

The edge isn’t a single place; it’s more like a broad region. Devices at the edge of the network can communicate with each other, and they can communicate with remote servers, too. There are even servers that live at the edge of the network. **Figure 1-3** shows how this looks.

² Expected to grow to 27 billion by 2025, according to **IoT Analytics**.

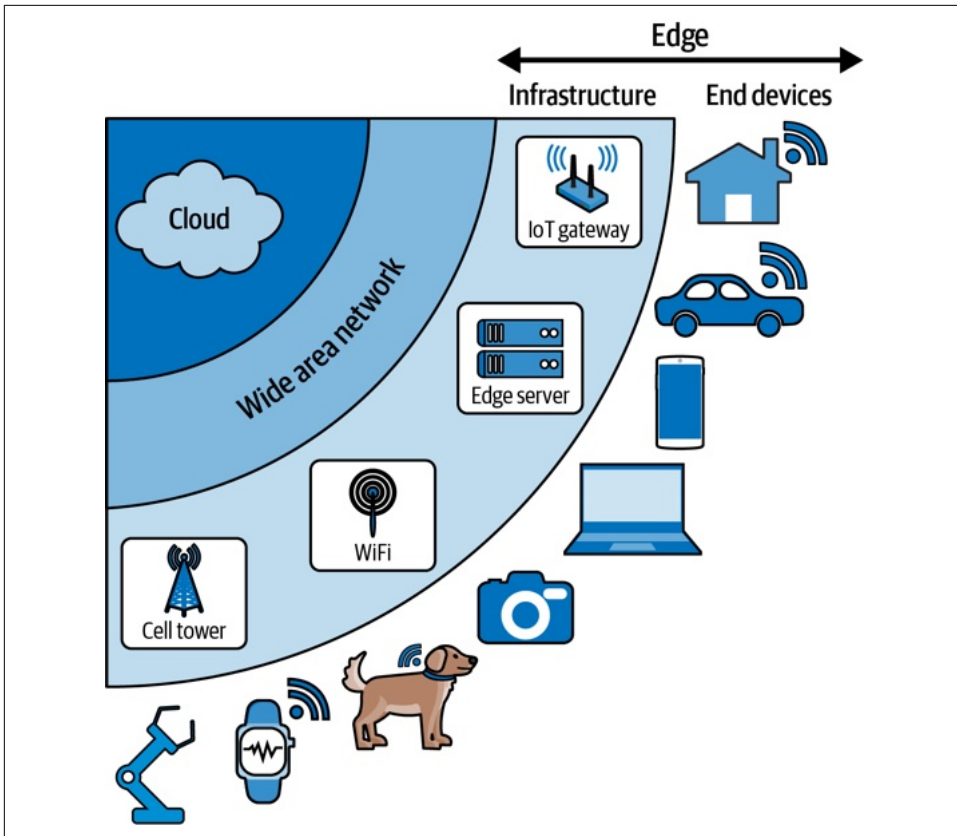


Figure 1-3. Devices at the edge of the network can communicate with the cloud, with edge infrastructure, and with each other; edge applications generally span multiple locations within this map (for example, data might be sent from a sensor-equipped IoT device to a local edge server for processing)

There are some major benefits to being at the edge of the network. For one, it's where all the data comes from! Edge devices are our link between the internet and the physical world. They can use sensors to collect data based on what is going on around them, be that the heart rate of a runner or the temperature of a cold drink. They can make decisions on that data locally and send it to other locations. Edge devices have access to data that nobody else does.



Are Mobile Phones and Tablets Edge Devices?

As portable computers that live at the edge of the network, mobile phones, tablets, and even personal computers are all edge devices. Mobile phones were one of the first platforms to feature edge AI: modern mobile phones use it for many purposes, from voice activation to smart photography.³

We'll come back to edge devices later (since they're the focus of this book). Until then, let's continue to define some terms.

Artificial Intelligence

Phew! This is a big one. Artificial intelligence (AI) is a very big idea, and it's terribly hard to define. Since the dawn of time, humans have dreamed of creating intelligent entities that can help us in our struggle to survive. In the modern world we dream of robot sidekicks who assist with our adventures: hyperintelligent, synthetic minds that will solve all of our problems, and miraculous enterprise products that will optimize our business processes and guarantee us rapid promotion.

But to define AI, we have to define intelligence—which turns out to be particularly tough. What does it mean to be intelligent? Does it mean that we can talk, or think? Clearly not—just ask the slime mold (see [Figure 1-4](#)), a simple organism with no central nervous system that is capable of solving a maze.

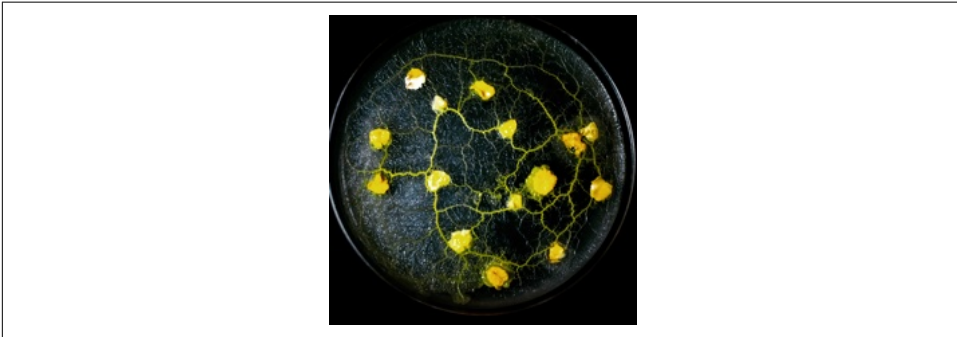


Figure 1-4. Slime molds are single-celled organisms that have been documented as being able to solve mazes in order to locate food, via a process of biological computation—as shown in “[Slime Mould Solves Maze in One Pass Assisted by Gradient of Chemo-Attractants](#)” (Andrew Adamatzky, arXiv, 2011)

³ Embedded engineering and mobile development are typically separate disciplines. Even within a mobile device, the embedded firmware and operating system are distinct from mobile applications. This book focuses on embedded engineering, so we won't talk much about building mobile apps—but we will cover techniques that are relevant in both cases.

Since this isn't a philosophy book, we don't have the time to fully explore the topic of intelligence. Instead, we want to suggest a quick-and-dirty definition:

Intelligence means knowing the right thing to do at the right time.

This probably doesn't stand up to academic debate, but that's fine with us. It gives us a tool to explore the subject. Here are some tasks that require intelligence, according to our definition:

- Taking a photo when an animal is in the frame
- Applying the brakes when a driver is about to crash
- Informing an operator when a machine sounds broken
- Answering a question with relevant information
- Creating an accompaniment to a musical performance
- Turning on a faucet when someone wants to wash their hands

Each of these problems involves both an action (turning on a faucet) and a precondition (when someone wants to wash their hands). Within their own context, most of these problems sound relatively simple—but, as anyone who has used an airport restroom knows, they are not always straightforward to solve.

It's pretty easy for most humans to perform most of these tasks. We're highly capable creatures with *general* intelligence. But it's possible for smaller systems with more *narrow* intelligence to perform the tasks, too. Take our slime mold—it may not understand why it is solving a maze, but it's certainly able to do it.

That said, the slime mold is unlikely to also know the right moment to turn on a faucet. Generally speaking, it's a lot easier to perform a single, tightly scoped task (like turning on a faucet) than to be able to perform a diverse set of entirely different tasks.

Creating an artificial *general* intelligence, equivalent to a human being, would be super difficult—as decades of unsuccessful attempts have shown. But creating something that operates at slime mold level can be much easier. For example, preventing a driver from crashing is, in theory, quite a simple task. If you have access to both their current speed and their distance from a wall, you can do it with simple conditional logic:

```
current_speed = 10 # In meters per second
distance_from_wall = 50 # In meters
seconds_to_stop = 3 # The minimum time in seconds required to stop the car
safety_buffer = 1 # The safety margin in seconds before hitting the brakes

# Calculate how long we've got before we hit the wall
seconds_until_crash = distance_from_wall / current_speed

# Make sure we apply the brakes if we're likely to crash soon
```

```
if seconds_until_crash < seconds_to_stop + safety_buffer:
    applyBrakes()
```

Clearly, this simplified example doesn't account for a lot of factors. But with a little more complexity, a modern car with a driver assistance system based on this conditional logic could arguably be marketed as AI.⁴

There are two points we are trying to make here: the first is that intelligence is quite hard to define, and many rather simple problems require a degree of intelligence to solve. The second is that the programs that implement this intelligence do not necessarily need to be particularly complex. Sometimes, a slime mold will do.

So, what is AI? In simple terms, it's an artificial system that makes intelligent decisions based on some kind of input. And one way to create AI is with machine learning.

Machine Learning

At its heart, machine learning (ML) is a pretty simple concept. It's a way to discover patterns in how the world works—but automatically, by running data through algorithms.

We often hear AI and machine learning used interchangeably, as if they are the same thing—but this isn't the case. AI doesn't always involve machine learning, and machine learning doesn't always involve AI. That said, they pair together very nicely!

The best way to introduce machine learning is through an example. Imagine you're building a fitness tracker—it's a little wristband that an athlete can wear. It contains an accelerometer, which tells you how much acceleration is happening on each axis (x, y, and z) at a given moment in time—as shown in [Figure 1-5](#).

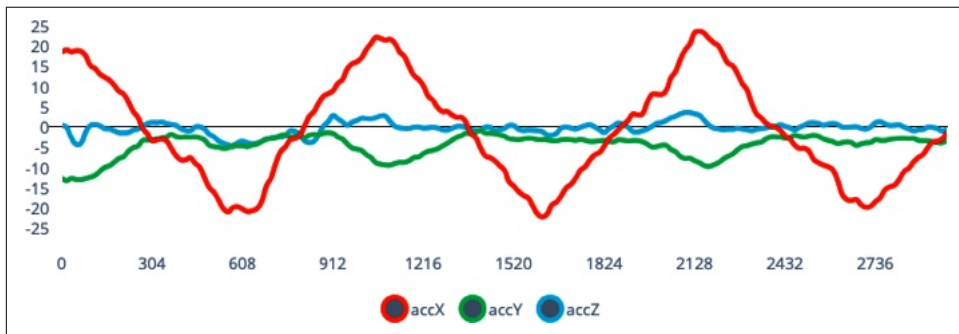


Figure 1-5. The output of a three-axis accelerometer sampled at 6.25 Hz

⁴ For many years it was hoped that artificial general intelligence could be achieved by complex conditional logic, hand-tuned by engineers. It has turned out to be a lot more complicated than that!

To help your athletes, you want to keep an automatic log of the activities they are doing. For example, an athlete might spend an hour running on Monday and then an hour swimming on Tuesday.

Since our movements while swimming are quite different from our movements while running, you theorize that you might be able to tell these activities apart based on the output of the accelerometer in your wristband. To collect some data, you give prototype wristbands to a dozen athletes and have them perform specific activities—either swimming, running, or doing nothing—while the wristbands log data (see [Figure 1-6](#)).

Now that you have a dataset, you want to try to determine some rules that will help you understand whether a particular athlete is swimming, running, or just chilling out. One way to do this is by hand: analyzing and inspecting the data to see if anything stands out to you. Perhaps you notice that running involves more rapid acceleration on a particular axis than swimming. You can use this information to write some conditional logic that determines the activity based on the reading from that axis.

Analyzing data by hand can be tricky, and it generally requires expert knowledge about the domain (such as human movements during sport). An alternative to manual analysis might be to use machine learning.

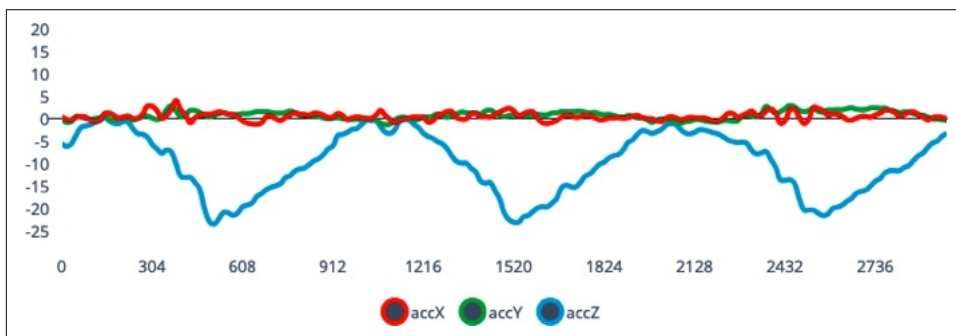


Figure 1-6. The output of a three-axis accelerometer showing a different activity than in [Figure 1-5](#); each activity can be characterized by a pattern of changes in acceleration on each axis over time

With an ML approach, you feed all of your athletes' data into a training algorithm. When provided with both the accelerometer data and information about which activity the athlete is currently performing, the algorithm does its best to learn a mapping between the two. This mapping is called a *model*.

Hopefully, if the training was successful, your new machine learning model can take a brand new, never-seen-before input—a sample of accelerometer data from a

particular window in time—and tell you which activity an athlete is performing. This process is known as *inference*.

This ability to understand *new* inputs is called *generalization*. During training, the model has learned the characteristics that distinguish running from swimming. You can then use the model in your fitness tracker to understand fresh data, in the same way that you might use the conditional logic we mentioned earlier.

There are lots of different machine learning algorithms, each with their own strengths and drawbacks—and ML isn't always the best tool for the job. Later in this chapter we'll discuss the scenarios where machine learning is the most helpful. But a nice rule of thumb is that machine learning really shines when our data is really complex.

Edge AI

Congratulations, we've made it to our first compound buzzword! Edge AI is, unsurprisingly, the combination of edge devices and artificial intelligence.

As we discussed earlier, edge devices are the embedded systems that provide the link between our digital and physical worlds. They typically feature sensors that feed them information about the environment they are close to. This gives them access to a metaphorical fire hose of high-frequency data.

We're often told that data is the lifeblood of our modern economy, flowing throughout our infrastructure and enabling organizations to function. That's definitely true—but all data is not created equally. The data obtained from sensors tends to have a very high volume but a relatively low informational content.

Imagine the accelerometer-based wristband sensor we described in the previous section. The accelerometer is capable of taking a reading many hundreds of times per second. Each individual reading tells us very little about the activity currently taking place—it's only in aggregate, over thousands of readings, that we can begin to understand what is going on.

Typically, IoT devices have been viewed as simple nodes that collect data from sensors and then transmit it to a central location for processing. The problem with this approach is that sending such large volumes of low-value information is extraordinarily costly. Not only is connectivity expensive, but transmitting data uses a ton of energy—which is a big problem for battery-powered IoT devices.

Because of this problem, the vast majority of data collected by IoT sensors has usually been discarded. We're collecting a ton of sensor data, but we're unable to do anything with it.

Edge AI is the solution to this problem. Instead of having to send data off to some distant location for processing, what if we do it directly on-device, where the data is

being generated? Now, instead of relying on a central server, we can make decisions locally—no connectivity required.

And if we still want to report information back to upstream servers, or the cloud, we can transmit just the important information instead of having to send every single sensor reading. That should save a lot of cost and energy.

There are many different ways to deploy intelligence to the edge. **Figure 1-7** shows the continuum from cloud AI to fully on-device intelligence. As we'll see later in this book, edge AI can be spread across entire distributed computing architectures—including some nodes at the very edge, and others in local gateways or the cloud.

As we've seen, artificial intelligence can mean many different things. It can be super simple: a touch of human insight encoded in a little simple conditional logic. It can also be super sophisticated, based on the latest developments in deep learning.

Edge AI is exactly the same. At its most basic, edge AI is about making some decisions on the edge of the network, close to where the data is made. But it can also take advantage of some really cool stuff. And that brings us nicely to the next section!

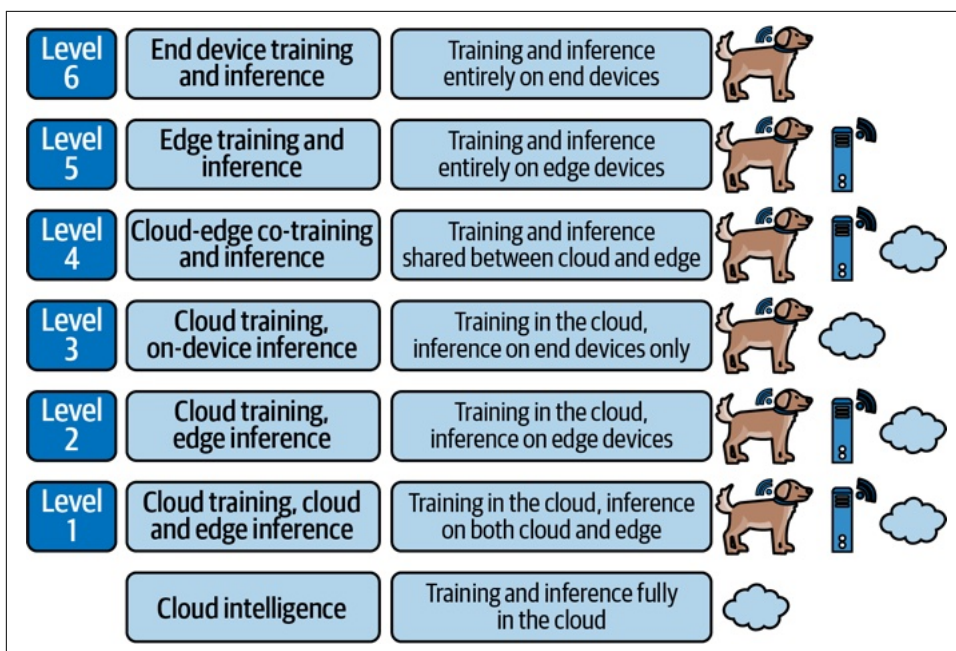


Figure 1-7. The continuum between cloud intelligence and fully on-device intelligence; these six levels were described by “*Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing*” (Zhou et. al., *Proceedings of the IEEE*, 2019)

Embedded Machine Learning and Tiny Machine Learning

Embedded ML is the art and science of running machine learning models on embedded systems. Tiny machine learning, or TinyML,⁵ is the concept of doing this on the most constrained embedded hardware available—think microcontrollers, digital signal processors, and small field programmable gate arrays (FPGAs).

When we talk about embedded ML, we’re usually referring to machine learning inference—the process of taking an input and coming up with a prediction (like guessing a physical activity based on accelerometer data). The training part usually still takes place on a conventional computer.

Embedded systems often have limited memory. This raises a challenge for running many types of machine learning models, which often have high requirements for both read-only memory (ROM) (to store the model) and RAM (to handle the intermediate results generated during inference).

They are often also limited in terms of computation power. Since many types of machine learning models are quite computationally intensive, this can also raise problems.

Luckily, over the past few years there have been many advances in optimization that have made it possible to run quite large and sophisticated machine learning models on some very small, low-power embedded systems. We’ll learn about some of those techniques over the next few chapters!

Embedded machine learning is often deployed alongside its trusty companion, *digital signal processing*. Before we move on, let’s define that term, too.

Digital Signal Processing

In the embedded world we often work with the digital representations of signals. For example, an accelerometer gives us a stream of digital values that correspond to acceleration on three axes, and a digital microphone gives us a stream of values that correspond to sound levels at a particular moment in time.

Digital signal processing (DSP) is the practice of using algorithms to manipulate these streams of data. When paired with embedded machine learning, we often use DSP to modify signals before feeding them into machine learning models. There are a few reasons why we might want to do this:

- Cleaning up a noisy signal
- Removing spikes or outlying values that might be caused by hardware issues

⁵ The term *TinyML* is a registered trademark of the TinyML Foundation.

- Extracting the most important information from a signal
- Transforming the data from the time domain to the frequency domain⁶

DSP is so common for embedded systems that often embedded chips have super fast hardware implementations of common DSP algorithms, just in case you need them.

We now share a solid understanding of the most important terms in this book. **Figure 1-8** shows how they fit together in context.

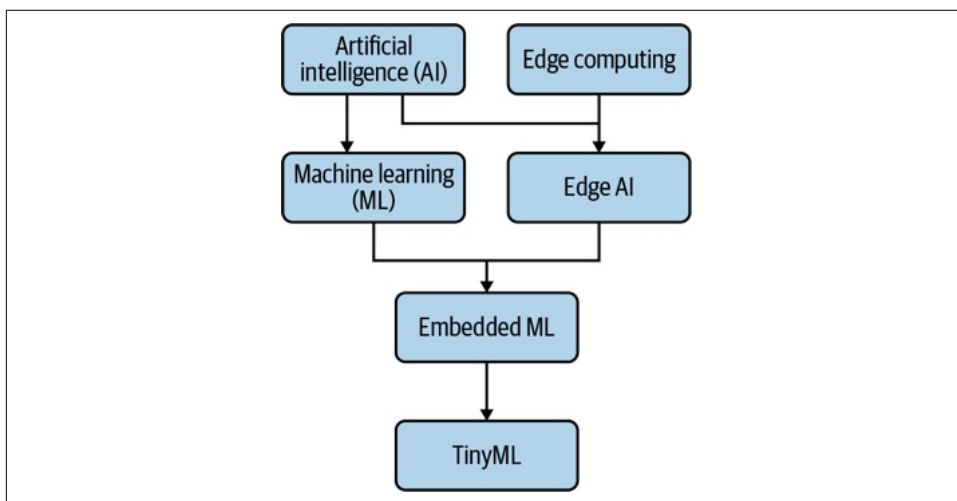


Figure 1-8. This diagram shows some of the most important concepts in edge AI in context with each other—from most general at the top to most specific at the bottom

In the next section, we’ll dive deep into the topic of edge AI and start to break down what makes it such an important technology.

Why Do We Need Edge AI?

Imagine that this morning you went on a trail run through Joshua Tree National Park, a vast expanse of wilderness in the Southern California desert. You listened to music the whole time, streamed to your phone via an uninterrupted cellular connection. At a particularly beautiful spot, deep in the mountains, you snapped a photograph and sent it to your partner. A few minutes later you received their reply.

In a world where even the most remote places have some form of data connection, why do we need edge AI? What is the point of tiny devices that can make their own

⁶ This will be explained in “Spectral analysis” on page 91.

decisions if the internet's beefy servers are only a radio burst away? With all of the added complication, aren't we just making life more difficult for ourselves?

As you may have guessed, the answer is no! Edge AI solves some very real problems that otherwise stand in the way of making our technology work better for human beings. Our favorite framework for explaining the benefits of edge AI is a rude-sounding mnemonic: BLERP.

To Understand the Benefits of Edge AI, Just BLERP

BLERP? Jeff Bier, founder of the Edge AI and Vision Alliance, [created this excellent tool](#) for expressing the benefits of edge AI. It consists of five words:

- Bandwidth
- Latency
- Economics
- Reliability
- Privacy

Armed with BLERP, anyone can easily remember and explain the benefits of edge AI. It's also useful as a filter to help decide whether edge AI is well suited for a particular application.

Let's go through it, word by word.

Bandwidth

IoT devices often capture more data than they have bandwidth to transmit. This means the vast majority of sensor data they capture is not even used—it's just thrown away! Imagine a smart sensor that monitors the vibration of an industrial machine to determine if it is operating correctly. It might use a simple thresholding algorithm to understand when the machine is vibrating too much, or not enough, and then communicate this information via a low bandwidth radio connection.

This already sounds useful. But what if you could identify patterns in the data that give you a clue that the machine might be about to fail? If we had a lot of bandwidth, we could send the sensor data up to the cloud and do some kind of analysis to understand whether a failure is imminent.

In many cases, though, there isn't enough bandwidth or energy budget available to send a constant stream of data to the cloud. That means that we'll be forced to discard most of our sensor data, even though it contains useful signals.

Bandwidth limitations are very common. It's not just about available connectivity—it's also about power. Networked communication is often the most energy-intensive task an embedded system can perform, meaning that battery life is often the limiting function. Some machine learning models can be quite compute intensive, but they tend to still use less energy than transmitting a signal.

This is where edge AI comes in. What if we could run the data analysis on the IoT device itself, without having to upload the data? In that case, if the analysis showed that the machine was about to fail, we could send a notification using our limited bandwidth. This is much more feasible than trying to stream all of the data.

Of course, it's also quite common for devices to have no network connection at all! In this case, edge AI enables a whole galaxy of use cases that were previously impossible. We'll hear more about this later.

Latency

Transmitting data takes time. Even if you have a lot of available bandwidth it can take tens or hundreds of milliseconds for a round-trip from a device to an internet server. In some cases, latency can be measured in minutes, hours, or days—think satellite communications, or store-and-forward messaging.

Some applications demand a faster response. For example, it might be impractical for a moving vehicle to be controlled by a remote server. Controlling a vehicle as it navigates an environment requires constant feedback between steering adjustments and the vehicle's position. Under significant latency, steering becomes a major challenge!

Edge AI solves this problem by removing the round-trip time altogether. A great example of this is a self-driving car. The car's AI systems run on onboard computers. This allows it to react nearly instantly to changing conditions, like the driver in front slamming on their brakes.

One of the most compelling examples of edge AI as a weapon against latency is in robotic space exploration. Mars is so distant from Earth that it takes *minutes* for a radio transmission to reach it at the speed of light. Even worse, direct communication is often impossible due to the arrangement of the planets. This makes controlling a Mars rover very hard. NASA solved this problem by using edge AI—their rovers use **sophisticated artificial intelligence systems** to plan their tasks, navigate their environments, and search for life on the surface of another world. If you have some spare time, you can even **help future Mars rovers navigate** by labeling data to improve their algorithms!

Economics

Connectivity costs a lot of money. Connected products are more expensive to use, and the infrastructure they rely on costs their manufacturers money. The more

bandwidth required, the steeper the cost. Things get especially bad for devices deployed on remote locations that require long-range connectivity via satellite.

By processing data on-device, edge AI systems reduce or avoid the costs of transmitting data over a network and processing it in the cloud. This can unlock a lot of use cases that would have previously been out of reach.

In some cases, the only “connectivity” that works is sending out a human being to perform some manual task. For example, it’s common for conservation researchers to use camera traps to monitor wildlife in remote locations. These devices take photos when they detect motion and store them to an SD card. It’s too expensive to upload every photo via satellite internet, so researchers have to travel out to their camera traps to collect the images and clear the storage.

Because traditional camera traps are motion activated, they take a lot of unnecessary photos—they might be triggered by branches moving in the wind, hikers walking past, and creatures the researchers aren’t interested in. But some teams are now using edge AI to identify only the animals they care about, so they can discard the other images. This means they don’t have to fly out to the middle of nowhere to change an SD card *quite* so often.

In other cases, the cost of connectivity might not be a concern. However, for products that depend on server-side AI, the cost of maintaining server-side infrastructure can complicate your business model. If you have to support a fleet of devices that need to “phone home” to make decisions, you may be forced into a subscription model. You’ll also have to commit to maintaining servers for a long period of time—at the risk of your customers finding themselves with “bricked” devices if you decide to pull the plug.⁷

Don’t underestimate the impact of economics. By reducing the cost of long-term support, edge AI enables a vast number of use cases that would otherwise be infeasible.

Reliability

Systems controlled by on-device AI are potentially more reliable than those that depend on a connection to the cloud. When you add wireless connectivity to a device, you’re adding a vast, overwhelmingly complex web of dependencies, from link-layer communications technologies to the internet servers that may run your application.

Many pieces of this puzzle are outside of your control, so even if you make all the right decisions you will still be exposed to the reliability risk associated with technologies that make up your distributed computing stack.

⁷ Not all edge AI applications are immune to this since it’s often necessary to monitor devices and push updates to algorithms. That said, there are certainly many cases where edge AI can reduce the burden of maintenance.

For some applications, this might be tolerable. If you're building a smart speaker that responds to voice commands, your users might understand if it stops recognizing their commands when their home internet connection goes down. That said, it can still be a frustrating experience!

But in other cases, safety is paramount. Imagine an AI-based system that monitors an industrial machine to make sure that it is being operated within safe parameters. If it stops working when the internet goes down, it could endanger human lives. It would be much safer if the AI is based entirely on-device, so it still operates in the event of a connectivity problem.

Reliability is often a compromise, and the required level of reliability varies depending on use case. Edge AI can be a powerful tool in improving the reliability of your products. While AI is inherently complex, it represents a different type of complexity than global connectivity, and its risk is easier to manage in many situations.

Privacy

Over the past few years, many people have begrudgingly resigned themselves to a trade-off between convenience and privacy. The theory is that if we want our technology products to be smarter and more helpful, we have to give up our data. Because smart products traditionally make decisions on remote servers, they very often end up sending streams of sensor data to the cloud.

This may be fine for some applications—for example, we might not worry that an IoT thermostat is reporting temperature data to a remote server.⁸ But for other applications, privacy is a huge concern. For example, many people would hesitate to install an internet-connected security camera inside their home. It might provide some reassuring security, but the trade-off—that a live video and audio feed of their most private spaces is being broadcast to the internet—does not seem worth it. Even if the camera's manufacturer is entirely trustworthy, there's always a chance of the data being exposed through security vulnerabilities.⁹

Edge AI provides an alternative. Rather than streaming live video and audio to a remote server, a security camera could use some onboard intelligence to identify that an intruder is present when the owners are out at work. It could then alert the owners in an appropriate way. When data is processed on an embedded system and is never transmitted to the cloud, user privacy is protected and there is less chance of abuse.

⁸ Even in this innocuous example, a malicious person accessing your thermostat data could use it to recognize when you're on vacation so they can break into your house.

⁹ This exact scenario unfolded in 2022 with the Ring home security system, which was found to be vulnerable to an attack ("[Amazon's Ring Quietly Fixed Security Flaw That Put Users' Camera Recordings at Risk of Exposure](#)", TechCrunch, 2022).

The ability of edge AI to enable privacy unlocks a huge number of exciting use cases. It's an especially important factor for applications in security, industry, childcare, education, and healthcare. In fact, since some of these fields involve tight regulations (or customer expectations) around data security, the product with the best privacy is one that *avoids* collecting data altogether.

Using BLERP

As we'll start to see in [Chapter 2](#), BLERP can be a handy tool for understanding whether a particular problem is well suited for edge AI. There doesn't have to be a strong argument for every word of the acronym: even meeting just one or two criteria, if compelling enough, can give merit to a use case.

Edge AI for Good

The unique benefits of edge AI provide a new set of tools that can be applied to some of our world's biggest problems. Technologists in areas like conservation, healthcare, and education are already using edge AI to make a big impact. Here are just a few examples we're personally excited about:

- [Smart Parks](#) is using [collars running machine learning models](#) to better understand elephant behavior in wildlife parks around the world.
- Izoelektro's [RAM-1](#) helps prevent forest fires caused by power transmission hardware by using embedded machine learning to detect upcoming faults.
- Researchers like Dr. Mohammed Zubair Shamim from King Khalid University in Saudi Arabia are training models that can [screen patients for life-threatening medical conditions such as oral cancer](#) using low-cost devices.
- Students across the world are developing solutions for their local industries. João Vitor Yukio Bordin Yamashita, from UNIFEI in Brazil, created a system for [identifying diseases that affect coffee plants](#) using embedded hardware.

The properties of edge AI make it especially well-suited for application to global problems. Since reliable connectivity is expensive and not universally available, many current generation smart technologies only benefit people living in industrialized, wealthy, and well-connected regions. By removing the need for a reliable internet connection, edge AI increases access to technologies that can benefit people and the planet.

When machine learning is part of the mix, edge AI generally involves small models—which are often quick and cheap to train. Since there's also no need to maintain expensive backend server infrastructure, edge AI makes it possible for developers with limited resources to build cutting-edge solutions for the local markets that they know better than anyone. To learn more about these opportunities, we recommend

watching “TinyML and the Developing World”, an excellent talk given by Pete Warden at the TinyML Kenya meetup.

As we saw in “Privacy” on page 17, edge AI also creates an opportunity to improve privacy for users. In our networked world, many companies treat user data as a valuable resource to be extracted and mined. Consumers and business owners are often required to barter away their privacy in order to use AI products, putting their data in the hands of unknown third parties.

With edge AI, data does not need to leave the device. This enables a more trusting relationship between user and product, giving users ownership of their own data. This is especially important for products designed to serve vulnerable people, who may feel skeptical of services that seem to be harvesting their data.

TinyML for Developing Countries

If you’re interested in the global benefits of edge AI, the **TinyML for Developing Countries (TinyML4D)** initiative is building a network of researchers and practitioners who are focused on solving developing world challenges using edge AI.

As we’ll see in later sections, there are many potential pitfalls that must be navigated in order to build ethical AI systems. That said, the technology provides a tremendous opportunity to make the world a better place.



If you’re thinking about using edge AI to solve problems for your local community, the authors would love to hear from you. We’ve provided support for a number of impactful projects and would love to identify more. Send an email to the authors at hello@edgeaibook.com.

Key Differences Between Edge AI and Regular AI

Edge AI is a subset of regular AI, so a lot of the same principles apply. That said, there are some special things to consider when thinking about artificial intelligence on edge devices. Here are our top points.

Training on the edge is rare

A lot of AI applications are powered by machine learning. Most of the time, machine learning involves *training* a model to make predictions based on a set of labeled data. Once the model has been trained, it can be used for *inference*: making new predictions on data it has not seen before.

When we talk about edge AI and machine learning, we are usually talking about *inference*. Training models requires a lot more computation and memory than inference does, and it often requires a labeled dataset. All of these things are hard to come by on the edge, where devices are resource-constrained and data is raw and unfiltered.

For this reason, the models used in edge AI are often trained before they are deployed to devices, using relatively powerful compute and datasets that have been cleaned and labeled—often by hand. It's technically possible to train machine learning models on the edge devices themselves, but it's quite rare—mostly due to the lack of labeled data, which is required for training and evaluation.

There are two subtypes of on-device training that are more widespread. One of these is used commonly in tasks such as facial or fingerprint verification on mobile phones, to map a set of biometrics to a particular user. The second is used in predictive maintenance, where an on-device algorithm learns a machine's "normal" state so that it can act if the state becomes abnormal. There'll be more detail on the topic of on-device learning in ["On-Device Training" on page 119](#).

The focus of edge AI is on sensor data

The exciting thing about edge devices is that they live close to where the data is made. Often, edge devices are equipped with sensors that give them an immediate connection to their environments. The goal of an edge AI deployment is to make sense of this data, identifying patterns and using them to make decisions.

By its nature, sensor data tends to be big, noisy, and difficult to manage. It arrives at a high frequency—potentially many thousands of times per second. An embedded device running an edge AI application has a limited time frame in which to collect this data, process it, feed it into some kind of AI algorithm, and act on the results. This is a major challenge, especially given that most embedded devices are resource-constrained and don't have the RAM to store large amounts of data.

The need to tame raw sensor data makes digital signal processing a critical part of most edge AI deployments. In any efficient and effective implementation, the signal processing and AI components must be designed together as a single system, balancing trade-offs between performance and accuracy.

A lot of traditional machine learning and data science tools are focused on tabular data—things like company financials or consumer product reviews. In contrast, edge AI tools are built to handle constant streams of sensor data. This means that a whole different set of skills and techniques is required for building edge AI applications.

ML models can get very small

Edge devices are often designed to limit cost and power consumption. This means that, generally, they have much slower processors and smaller amounts of memory than personal computers or web servers.

The constraints of the target devices mean that, when machine learning is used to implement edge AI, the machine learning models must be quite small. On a midrange microcontroller, there may only be a hundred kilobytes or so of ROM available to store a model, and some devices have far smaller amounts. Since larger models take more time to execute, the slow processors of devices can also push developers toward deploying smaller models.

Making models smaller involves some trade-offs. To begin with, larger models have more capacity to learn. When you make a model smaller, it starts to lose some of its ability to represent its training dataset and may not be as accurate. Because of this, developers creating embedded machine learning applications have to balance the size of their model against the accuracy they require.

Various technologies exist for compressing models, reducing their size so that they fit on smaller hardware and take less time to compute. These compression technologies can be very useful, but they also impact models' accuracy—sometimes in subtle but risky ways. [“Compression and optimization” on page 117](#) will talk about these techniques in detail.

That said, not all applications require big, complex models. The ones that do tend to be around things like image processing, since interpreting visual information involves a lot of nuance. Often, for simpler data, a few kilobytes (or less) of model is all you need.

Learning from feedback is limited

As we'll see later, AI applications are built through a series of iterative feedback loops. We do some work, measure how it performs, and then figure out what's needed to improve it.

For example, imagine we build a fitness monitor that can estimate your 10K running time based on data collected from onboard sensors. To test whether it's working well, we can wait until you run an actual 10K and see whether the prediction was correct. If it's not, we can add your data to our training dataset and try to train a better model.

If we have a reliable internet connection, this shouldn't be too hard—we can just upload the data to our servers. But part of the magic of edge AI is that we can deploy intelligence to devices that have limited connectivity. In this case, we might not have the bandwidth to upload new training data. In many cases, we might not be able to upload anything at all.

This presents a big challenge for our application development workflow. How do we make sure our system is performing well in the real world when we have limited access to it? And how can we improve our system when it's so difficult to collect more data? This is a core topic of edge AI development and something we'll be covering heavily throughout this book.

Compute is diverse and heterogeneous

The majority of server-side AI applications run on plain old x86 processors, with some graphics processing units (GPUs) thrown in to help with any deep learning inference. There's a small amount of diversity thanks to Arm's recent server CPUs, and exotic deep learning accelerators such as Google's TPUs (tensor processing units), but most workloads run on fairly ordinary hardware.

In contrast, the embedded world includes a dizzying array of device types:

- Microcontrollers, including tiny 8-bit chips and fancy 32-bit processors
- System-on-chip (SoC) devices running embedded Linux
- General-purpose accelerators based on GPU technology
- Field programmable gate arrays (FPGAs)
- Fixed architecture accelerators that run a single model architecture blazing fast

Each category includes countless devices from many different manufacturers, each with a unique set of build tools, programming environments, and interface options. It can be quite overwhelming.

The diversity of hardware means there are likely to be multiple suitable systems for any given use case. The hard part is choosing one! We'll cover this challenge over the course of the book.

"Good enough" is often the goal

With traditional AI, the goal is often to get the best possible performance—no matter the cost. Production deep learning models used in server-side applications can potentially be *gigabytes* in size, and they lean on powerful GPU compute to be able to run in a timely manner. When compute is not an obstacle, the most accurate model is often the best choice.

The benefits of edge AI come with some serious constraints. Edge devices have less capable compute, and there are often tricky choices involved with trading off between on-device performance and accuracy.

This is certainly a challenge—but it's not a barrier. There are huge benefits to running AI at the edge, and for a vast number of use cases they easily outweigh the penalty

of a little reduced accuracy. Even a small amount of on-device intelligence can be infinitely better than none at all.

The goal is to build applications that make the most of this “good enough” performance—an approach described elegantly by Alasdair Allan as **Capable Computing**. The key to doing this successfully is using tools that help us understand the performance of our applications in the real world, once any performance penalties have been factored in. We’ll be covering this topic at length.

Tools and best practices are still evolving

As a brand-new technology that has only begun to reach mass adoption, edge AI still depends on tools and approaches that were developed for large-scale, server-side AI. In fact, the majority of AI research is still focused on building large models on giant datasets. This has a couple of implications.

First, as we’ll see in **Chapter 5**, we’ll often find ourselves using existing development tools from the fields of data science and machine learning. On the positive side, this means we can draw from a rich ecosystem of libraries and frameworks that is proven to work well. However, few of the existing tools prioritize things that are important on the edge—like small model sizes, computational efficiency, and the ability to train on small amounts of data. We often have to do some extra work to make these the focus.

Second, since edge AI research is fairly new we’re likely to see extremely rapid evolution. As the field grows, and more researchers and engineers turn to focus on it, new approaches for improving efficiency are emerging—along with best practices and techniques for building effective applications. This promise of rapid change makes edge AI a very exciting field to work in.

Summary

In this chapter, we’ve explored the terminology that defines edge AI, learned a handy tool for reasoning about its benefits, explored how moving compute to the edge can increase access to technology, and outlined the factors that make edge AI different from traditional AI.

From the next chapter onward, we’ll be dealing with specifics. Get ready to learn about the use cases, devices, and algorithms that power edge AI today.

Edge AI in the Real World

We now have a basic understanding of what edge AI means and what makes it—in theory—a useful set of technologies. In this coming chapter, we’ll see what that theory looks like when it makes contact with the real world. We’ll start by examining some actual products that are out in the field today. After that, we’ll explore the top application areas for edge AI products. Finally, we’ll learn more about the ethical considerations required to make any product a success.

Common Use Cases for Edge AI

As we learned in the previous chapter, edge AI is especially valuable for devices with an abundance of sensor data but a lack of compute or connectivity. Luckily for us, these conditions can be found nearly everywhere.

In modern cities, it can feel like we’re never very far from a power socket or a wireless access point. But even when high bandwidth network connections and reliable power are available, there are huge advantages to limiting the communications and power consumption of devices. As we saw in *“To Understand the Benefits of Edge AI, Just BLERP” on page 14*, the pursuit of desirable features such as portability, reliability, privacy, and cost can drive product development toward devices that are designed to minimize the use amount of connectivity and energy usage.

Despite our seemingly global internet, there are plenty of places on the planet that are limited in connectivity or power. At the time of writing, *50% of the Earth’s land is relatively untouched by human development*. Only a small percentage of the planet’s surface has cellular or wireless coverage, and *billions of people do not have reliable access to power*.

But beyond the obviously remote regions, there are plenty of hidden corners in our most built-up regions that fall into this category. In our modern industrial supply chains, there are places where it's impractical to provide hardwired DC power for embedded devices—making efficient, battery-powered devices the perfect fit (see [Figure 2-1](#)).



Figure 2-1. There are many places on our planet where battery power is required

At the same time, sensors are becoming cheaper, more sophisticated, and less power hungry. Often, even simple embedded devices ship with highly capable sensors that remain underutilized due to the challenges in getting the data off the system for remote processing. For example, imagine a basic fitness wearable that uses an accelerometer to count steps. Even this simple device might be equipped with a sensitive multiaxis accelerometer that has a very high sample rate, capable of recording the most subtle movements. Unless the device's software is equipped to interpret this data, most of it will be thrown away: it would be too energy intensive to send the raw data to another device for processing.

Greenfield and Brownfield Projects

Conditions such as those discussed above produce nearly endless opportunities for deploying edge AI. In practical terms, it can be helpful to group these opportunities into two categories: *greenfield* and *brownfield*. These terms are borrowed from urban planning. A greenfield project is one that takes place on a site that has yet to be developed and is still a grassy, green field. A brownfield project takes place on a site that has already been developed and may have some existing legacy infrastructure.

In the edge AI world, greenfield projects are ones where the hardware and software are designed together from the ground up. Since there's no existing hardware, greenfield projects can make use of the latest and greatest innovations in compute and sensing—which we'll learn more about later in this chapter. The developers have

more freedom to design the ideal solution for the use case they are trying to target. For instance, modern cellphones are designed to include dedicated low-power digital signal processing hardware so that they can continually listen out for a wake word (such as “OK, Google” or “Hey, Siri”) without draining the battery. The hardware is chosen with the specific wake word–detection algorithm in mind.

In contrast, brownfield edge AI projects begin with existing hardware that was originally designed for a different purpose. Developers must work within the constraints of the existing hardware to bring AI capabilities to a product. This reduces developers’ freedom, but it avoids the major costs and risks associated with designing new hardware. For example, a developer could add wake word detection to a Bluetooth audio headset that is already on the market by making use of spare cycles in the device’s existing embedded processor. This new functionality could even be added to existing devices with a firmware update.

Greenfield projects are exciting because they allow us to push the limits of what is possible by pairing the latest edge AI hardware and algorithms. On the other hand, brownfield projects allow us to bring new capabilities to existing hardware, delighting customers and making the most of existing designs.

Real-World Products

The best way to understand a technology is to see how it’s applied in the real world. We’re still in the early days of edge AI, but it’s already being used across a huge range of applications and industries. Here are three brief overviews of real-world systems that have been developed using edge AI. Perhaps your own work will be featured in a future edition of this book!

Preventing forest fires using power line fault detection

Power lines transmit electricity across vast swathes of wilderness, including Europe’s ancient forests. Equipment failure can potentially ignite vegetation and cause wild-fires. With thousands of miles of towers and power lines, often in very remote areas, electrical equipment can be difficult to monitor.

Izoelektro’s **RAM-1** device uses edge AI to help solve this problem (**Figure 2-2**). A package of sensors monitors conditions at each electrical pylon, including temperature, inclination, and voltage, and uses a deep learning classification model (**Chapter 4**) to identify when a fault may be developing. Technicians can visit the pylon and make a repair before there is any danger of fire. The device has a rugged construction designed to withstand extreme weather conditions over many years of service.



Figure 2-2. Izoelektro's RAM-1 device (Credit: *Izoelektro*)

There are two main factors that make this a perfect application for edge AI. The first is the lack of connectivity in wilderness locations. It would be prohibitively expensive to transmit raw sensor data from thousands of remotely located pylons in real time. Instead, the elegant solution is to interpret the sensor data at the source and transmit *only when a fault is predicted*—a maximum of around 250 kilobytes per month. The device is able to understand which data is crucial enough to require immediate attention, sending less important information in periodic batch transmissions.

This selective communication helps with the second, slightly unintuitive factor. Although the RAM-1 is mounted on an electricity pylon, it actually makes use of battery power. This ensures it keeps working even if there's a fault in the power lines, and it reduces the cost and complexity of installation. Since radio transmission uses a lot of energy, the RAM-1's ability to avoid unnecessary transmission helps it preserve battery life. In fact, with the help of edge AI, its battery can last for twenty years.

Here's how the RAM-1 fits the BLERP model:

Bandwidth

Connectivity is limited in remote locations where RAM-1 is deployed.

Latency

It's critical to identify failures as soon as they happen, as opposed to waiting for a periodic data transmission.

Economics

Avoiding unnecessary communication saves money and means the device can run on battery power, which reduces the cost of installation.

Reliability

The ability to run on battery power improves reliability.

Privacy

Not a major consideration for this use case.

Protecting first responders with intelligent wearables

The nature of their work means that firefighters are often exposed to high temperatures, and the extreme heat conditions can have a major impact on their long-term health. In fact, according to FEMA **the leading cause of firefighter line-of-duty deaths is sudden cardiac events**.

SlateSafety's **BioTrac Band** is a wearable device designed for workers, like firefighters, who are exposed to extreme conditions (**Figure 2-3**). It provides an early warning system that can help alert individuals and teams to conditions that may result in heat strain and overexertion. The BioTrac Band uses an embedded machine learning model alongside heuristic algorithms¹ to analyze data from multiple sensors—including signals from the wearer's body—and predict when an injury is about to happen. This intelligence made the device one of *Time* magazine's **100 best inventions of 2021**.



*Figure 2-3. SlateSafety's BioTrac Band (Credit: **SlateSafety**)*

The extreme environments that the BioTrac Band is deployed in make it a fantastic use case for edge AI. By analyzing data on-device, the band can continue to function and warn its wearer even when connectivity becomes limited or unavailable during the course of an emergency. In addition, the ability to interpret data on-device means that unnecessary transmission of data can be avoided—which saves energy and improves battery life, while allowing the size and weight of the device to be kept to a minimum. It also saves costs, which means the device can be more widely adopted.

¹ We'll learn about these soon, in "**Conditionals and heuristics**" on page 101.

Here's how the BioTrac Band fits the BLERP model:

Bandwidth

Connectivity is limited in extreme environments where firefighters work.

Latency

Health issues are time-critical and must be identified immediately.

Economics

Streaming raw data from sensors would require expensive high bandwidth connections.

Reliability

The device can continue to warn firefighters of potential risks even if connectivity drops, and it can function for a long time on a small battery.

Privacy

Raw biosignal data can be kept on-device, with only critical information being transmitted.

Understanding elephant behavior with smart collars

With increased pressure on their natural habitat, wild elephants are increasingly coming into contact with human beings. These interactions often end badly for the animals, with poaching or conflict with farmers and other people frequently leading to injury and death. To reduce the likelihood of these events, conservation workers and scientists are trying to learn more about elephant behavior and the types of conditions that lead to dangerous encounters.

ElephantEdge is an open source project to create a **tracking collar** designed to help researchers understand elephant behavior (**Figure 2-4**). The collar, fit around the neck of an elephant, can provide insight into the animal's location, health, and activities using a combination of embedded sensors and machine learning models. This data can be used for scientific study—and it can also be used to alert humans to the presence of animals so that conflict can be avoided.

Since the device is attached to a wild elephant, replacing the battery is a difficult task! Edge AI technology helps by minimizing the amount of energy consumed by the device. Instead of transmitting large amounts of raw sensor data, the machine learning–equipped collar is able to transmit high-level information about the animal's activities—for example, whether it is walking, eating, drinking, or engaging in other behaviors. The models that allow it to do this were prototyped by a community of citizen scientists working with public datasets.



Figure 2-4. The OpenCollar Edge tracking collar being fitted to a sedated elephant (Credit: IRNAS)

These low bandwidth requirements mean the collar can take advantage of an extremely low-power wireless communication technology named LoRa.² The collar is able to communicate with LoRa-equipped satellites that pass over once per day, sending a summary of the animal's activities since the last transmission. This means that the system can work reliably even in places with no traditional connectivity, but the battery can last for an estimated five years.

Here's how the OpenCollar Edge fits the BLERP model:

Bandwidth

Connectivity is limited in elephant habitats; on-device analysis enables use of low-energy wireless technologies.

Latency

Even though the device only transmits once per day, this is very frequent compared to traditional tracking collars that require manual downloading.

Economics

The device saves money by replacing traditional methods for monitoring elephants, which are labor intensive.

Reliability

Infrequent transmission means the battery can last for years, and makes satellite technology economically viable, increasing range.

² LoRa is a trademark derived from the phrase “long range,” since it is designed for long-range, low-power communications.

Privacy

Tracking of elephants directly is less intrusive to local people than setting up cameras to monitor animal activity, which is another proposed solution.

These three use cases represent a tiny sample of what is possible. In the next section, we'll talk through some general high-level categories of applications.

Types of Applications

There are opportunities to deploy edge AI across every part of our modern world, from heavy industry to healthcare, agriculture to art. The possibilities are nearly endless! To make things easier to discuss, the roles that edge AI technologies play within these applications can be grouped into a few high-level categories:

- Keeping track of objects
- Understanding and controlling systems
- Understanding people and living things
- Generating and transforming signals

Let's walk through each of these categories and understand where edge AI fits.

Keeping Track of Objects

From vast container ships to individual grains of rice, our civilization depends on the movement of objects from one place to another. This might occur in the controlled conditions of a warehouse, where items are moved carefully from storage to shipment. It may also occur under the most extreme conditions, like the motion of weather systems across the face of the planet.

Tracking and interpreting the state of objects, both man-made and natural, is a key application area for edge AI. Intelligent sensors can help encode the state of the physical world in a form computers can understand, allowing us to do a better job of coordinating our activity.

Table 2-1 discusses edge AI use cases that involve keeping track of objects.

Table 2-1. Edge AI use cases for keeping track of objects

Use case	Key sensors
Monitoring shipments using smart packaging to detect damage during transit	Accelerometer, vibration, GPS, temperature, humidity
Counting products on store shelves using embedded cameras, so items can be restocked before they run out	Vision
Analyzing the movement of plastic waste in the ocean so it can be cleaned up	Vision

Use case	Key sensors
Identifying and tracking obstacles at sea to help ships avoid collisions	Radar
Locating buried natural resources using geophysical sensors	Electromagnetic, acoustic

Deep Dive: Monitoring Shipments Using Smart Packaging

It's common for manufactured products to travel thousands of miles on their way to a customer—and they don't always make it in one piece. Damage during shipping costs businesses money, but when a shipment arrives damaged after a long voyage, it isn't always easy to figure out what happened.

With edge AI, a logistics company could attach a device to high-value shipments that can recognize when an expensive item is at risk of damage. For example, if equipped with an accelerometer, the device could use a machine learning model to distinguish between normal bumps and jolts and specific types of rough handling that might lead to damage. Any rough handling event could be logged, along with a timestamp and a GPS location.

The logs could be uploaded periodically, whenever the device is able to get a wireless connection. Upon arrival, if there is any damage, the company could analyze the logs to discover the time and place where the damage happened—allowing them to find and fix the cause of the issue.

What makes this a good use case for edge AI? Let's think about it in terms of BLERP:

Bandwidth

To detect sudden bumps, the accelerometer data would have to be quite high frequency. This makes it difficult to transmit from low-power wireless radios, which are generally low bandwidth. By processing data on-device, we can massively lower the bandwidth requirements.

Latency

Not a major consideration for this use case.

Economics

It's expensive to transmit data wirelessly, especially since the device could be anywhere in the world. Using edge AI helps conserve data and lower costs.

Reliability

Shipments in transit are unlikely to have reliable connectivity, so it's important that the device can keep logging even when out of range. If we don't have to store raw data, we can log all the interesting events in a small amount of memory.

Privacy

Not a major consideration for this use case.

Key benefits for object tracking

Object tracking tends to make use of the connectivity and cost-related benefits of edge AI. The world has many objects, and they’re not always in convenient places. Cheap edge AI sensors making use of low-cost, opportunistic connectivity can provide high-resolution visibility into gaps in the supply chain that would otherwise be too expensive to monitor.

Of course, the exact benefits of deploying edge AI vary from project to project. For example, a system using cameras to monitor stock on store shelves might use edge AI for privacy. If internet-connected cameras were used to monitor store shelves, employees might feel like they are under constant scrutiny by HQ. But a stock tracking system that works offline, solely for the benefit of the store’s team, could be a welcome aid.

Understanding and Controlling Systems

Our modern world is built on millions of complex, interconnected systems—everything from production lines to transportation networks, climate control to smart home appliances. The well-being of our economies is tied intimately to these systems. A breakdown in production can cost vast amounts of time and money and improvements in efficiency can lead to huge savings in costs, labor, and emissions.

The monitoring, control, and maintenance of complex systems is a vast opportunity for edge AI. The ability to make rapid, reliable decisions at the edge can improve the responsiveness and resilience of our systems, and fine-grained insights into system state can help us better plan for the future.

Some edge AI use cases that involve understanding and controlling systems can be found in [Table 2-2](#).

Table 2-2. Edge AI use cases for understanding and controlling systems

Use case	Key sensors
Monitoring an oil rig for signs that it needs maintenance, avoiding downtime and reducing leaks and spillage	Accelerometer, vibration, load, temperature, audio, vision, and more
Autonomously driving a combine harvester, helping a farmer quickly harvest their crops	Vision, GPS
Understanding and shaping traffic flow on a busy highway, using variable speed limits to keep cars moving	Vision, magnetometer
Directing a mechanical tool using sensor feedback	Accelerometer, vibration, load
Identifying faulty items on a production line using computer vision, improving quality control and quickly recognizing issues	Vision
Cleaning a carpet using a robot vacuum, saving time for the owner of the home	Vision, proximity, touch, current ^a
Fetching items in a warehouse using robots, reducing labor costs and workplace health risks	Vision, proximity, touch, light

Use case	Key sensors
Detecting intrusions in computer networks using traffic analysis, automatically responding to security threats	Network logs ^b
Estimating vehicle tire wear based on vibration during motion	Accelerometer

^a Analysis of motor current can be used to identify when a robot's wheels or cleaning tools are stuck.

^b Edge AI doesn't always require sensor data: any locally available data streams can be used as input to algorithms.

This is a truly enormous category of applications, including many of the things we associate with our vision of “the future”: self-driving vehicles, industrial robots, and smart factories. What they have in common is the use of edge AI to monitor the state of a complex system and to provide feedback and control when change is required.

Key benefits for understanding and controlling systems

A broad category, the automated monitoring and control of systems makes use of most of the benefits of edge AI. Economics and reliability are particularly important to many business use cases, and the benefits of low-bandwidth, low-latency solutions provide further justification where otherwise a server-side system might be used.

Deep Dive: Predictive Maintenance at an Oil Rig

If a piece of industrial equipment suddenly fails, the resulting downtime and disruption to processes can cost tremendous amounts of money. In some cases, it may also pose a threat to the health of human beings and the environment. Predictive maintenance is the art of identifying when a system is starting to fail—so that steps can be taken before it does.

An oil well is an incredibly complicated piece of machinery that operates under extreme conditions. Due to its precarious position in the middle of the ocean, faults can result in more than just costly downtime—the lives of the rig's crew are at stake, and oil spills can contaminate the ocean environment.

Using edge AI, sensor-equipped devices can be deployed to monitor key components of an oil rig, measuring factors such as vibration, temperature, and noise. They can learn the “normal” state of each part of the system, building a model of what a nominal operation looks like. If conditions start to deviate, they can alert a maintenance team to investigate further. Particularly sophisticated predictive maintenance systems might even have some control over the equipment, automatically halting operation if a dangerous situation is detected.

To understand why this is a good fit for edge AI, we can use the BLERP model:

Bandwidth

Most oil rigs rely on satellites for connectivity, making it challenging to stream large amounts of sensor data from thousands of rig components into the cloud. Further, there are places within a drilling operation that have *very* limited

connectivity—for example, a drilling bit might be miles beneath the ocean floor! On-device predictive maintenance can turn a vast stream of noisy data into a lightweight sequence of events that are easy to transmit.

Latency

It's expensive to pay expert human beings to travel to an oil rig and inspect equipment. This means that inspection happens periodically, limiting how quickly a problem can be identified. With constant monitoring from an edge AI system, issues can be identified and addressed as soon as they present.

Economics

Predictive maintenance can save vast amounts of money that might otherwise be lost to downtime. In addition, monitoring via AI-equipped smart sensors is a lot cheaper than paying humans to perform the dangerous work of inspecting heavy machinery.

Reliability

In the extreme off-shore environment, you can't always depend on reliable transportation or communications. Using edge AI, insight into equipment health can continue even when usual operations are disrupted.

Privacy

Not a major consideration for this use case.

Understanding People and Living Things

The biological world is complex, messy, and can change quickly. There's huge value in being able to understand and react to it in real time. This category includes human-oriented technologies, like fitness tracking watches and educational toys, as well as systems for monitoring nature, agriculture, and the microscopic world.

These applications help bridge the gap between biology and technology, allowing our rigid computer systems to interface with the dynamic and flexible world of life on Earth. As our understanding of biology improves, this field will continue to grow.

Table 2-3 shows examples of edge AI use cases that help people and computers understand one another.

Table 2-3. Edge AI use cases involving people

Use case	Key sensors
Alerting workers in a dangerous environment when they are missing protective equipment	Vision
Understanding human gestures to control a video game	Vision, accelerometer, radar
Identifying when an intensive care patient's health is deteriorating and notifying a medical attendant	Biosignals, medical equipment

Use case	Key sensors
Recognizing when a thief has broken into a home and alerting the authorities	Vision, audio, accelerometer, magnetic sensors
Categorizing physical activities using sensors in a smart watch	Accelerometer, GPS, heart rate
Recognizing a user's voice commands and controlling an appliance	Audio
Counting the number of people who are waiting at a bus stop	Vision
Warning a driver when they are falling asleep at the wheel of a car	Vision

Our world is filled with plants, animals, and other living things. [Table 2-4](#) shows examples of edge AI use cases that help us make sense of them.

Table 2-4. Edge AI use cases involving living things

Use case	Key sensors
Informing researchers when wildlife of interest is spotted by a remote trail camera	Vision, audio
Diagnosing crop diseases in a remote rural location with no cellphone coverage	Vision, volatile organic compound
Recognizing sounds made by marine mammals to track their movements and understand their behavior	Acoustic
Warning villagers of an approaching elephant so they can avoid human-animal conflict	Thermal imaging, vision
Categorizing farm animal behavior using a smart collar to understand health	Accelerometer
Cooking food to perfection by monitoring and controlling a sensor-equipped kitchen appliance	Vision, temperature, volatile organic compound ^a

^a Volatile organic compound (VOC) sensors can detect various types of gases.

Key benefits for understanding people and living things

Another large area, applications involving people and living things, makes use of every aspect of the BLERP model. That said, this is a category where privacy can be especially important. There are many applications that are technically feasible using server-side AI, but only become socially acceptable when done on-device.

The most widespread example of this is digital personal assistants, such as Apple's Siri or Google's Google Assistant. As discussed earlier, personal assistants work by using on-device models to constantly listen for wake words. Only after the wake word is detected is any audio streamed to the cloud. Without the on-device component, the assistant would have to constantly stream audio to the service provider. This would be incompatible with most people's expectations around privacy.

By moving functionality onto devices, and avoiding the transmission of data, we unlock massive possibilities—especially in vision, which until recently required large models that could only be run in the cloud.

Deep Dive: Spotting Rare Wildlife with Trail Cameras

A trail camera, or camera trap, is a special type of camera designed for monitoring wildlife. It has a tough, weatherproof housing, a high-capacity battery, and a motion sensor. Installed with a view of a trail, it automatically snaps photos whenever it detects motion.

Researchers who are monitoring specific species install camera traps in remote locations and leave them for months at a time. When they return, they download the photos from the camera and use them to better understand their target species; for example, they may try to estimate how many individuals exist.

There are some significant problems with camera traps that cost a lot of time and money:

- Most of the photos captured do not feature the target species. Instead, the capture was triggered by nontarget species or by random motion in the field of view.
- Due to the high number of false positives, it would not be helpful to send notifications of captures via a network connection. Instead, researchers must travel out to the remote location to collect the saved photos. This is extremely expensive and can result in missing data if the memory card gets full, or unnecessary trips if nothing interesting has been photographed.
- Researchers must trawl through thousands of useless photographs to find the few that matter.

Using edge AI, camera traps can be equipped with deep-learning vision models trained to identify target species and reject any images that do not contain them. This means that researchers no longer have to worry about filling up memory cards with useless images. Even better, it means that cameras can potentially be equipped with low-power or cellular radio transmitters that allow them to report back on animal sightings without anyone having to visit the field. This can massively reduce the cost of a study and increase the amount of scientific work that can be done.

The BLERP model can tell us exactly why this is a great application for edge AI:

Bandwidth

Camera traps are often deployed in remote areas with low connectivity—perhaps with expensive, low-bandwidth satellites as the only option. With edge AI, the number of photos taken can be reduced enough to make it possible to transmit them all.

Latency

Without edge AI, the latency involved with sending a researcher to collect photos from camera traps could be measured in months! With edge AI and a low-power

radio connection, it's possible to analyze photos immediately and obtain useful information without having to wait.

Economics

Avoiding trips out into the field saves large amounts of money; so does avoiding unnecessary use of expensive satellite radios.

Reliability

If useless photos can be discarded, the memory card will take longer to fill up.

Privacy

An edge AI camera can discard photos of humans on the trail, preserving the privacy of other trail users (such as local people or hikers).

Transforming Signals

To a computer, our world is made up of signals: time series of sensor readings that each describe a small fragment of a situation or environment. Our previous categories of applications are mostly focused on interpreting these signals and responding to them accordingly. Data from one or more sensors is assimilated, and a simple output is constructed that either facilitates human interpretation or can be used as a control signal for an automated system.

This final category is a little different. Sometimes, rather than converting a raw signal into an instantaneous decision, we simply want to transform one signal into another (Table 2-5). As discussed in “[Digital Signal Processing](#)” on page 12, digital signal processing is an important part of embedded applications. In these use cases, which go much further than the traditional DSP pipeline, it is the end goal rather than a side effect.

Table 2-5. Edge AI use cases for transforming signals

Use case	Signal type
Filtering background noise to improve call quality on a cell phone	Audio
Removing noise from photographs captured with a smartphone camera	Vision
Generating music to accompany a musician during practice	Audio
Blurring the background of a video stream during a remote work meeting	Vision
Generating realistic human speech from text	Audio
Translating one written language into another using a smartphone camera	Vision, text
Upsampling low-resolution audio so that it sounds better to the human ear	Audio
Compressing video using deep learning so that it can be transmitted via a low bandwidth connection	Video
Creating a spoken representation of a visual scene for visually impaired people	Audio
Transcribing a spoken conversation into text for convenience of note-taking	Audio
Using data from cheap sensors to simulate the output of an expensive one	Time series

Key benefits for transforming signals

Since digital signals are expressed over time, applications in this area tend to benefit from the latency benefits of edge AI. Bandwidth is also particularly important, since access to the original signal is required; transmitting the transformed signal often requires the same amount of bandwidth, if not more.

Deep Dive: Blurring the Background During a Remote Work Meeting

With the growth of remote work and videoconferencing, employees have had to get used to their previously private home spaces being broadcast to their coworkers. To help maintain some privacy, many videoconferencing tools now support blurring the background of a video stream while leaving the subject of the video intact.

These tools depend on a technique named *segmentation*, which uses deep learning models to identify the pixels in a stream of video that belong to one category or another. In this case, the model is trained to distinguish between a person and their background scenery. The input is the raw stream of video from a camera. The output is a stream of video with the same resolution but with the background pixels blurred together, making it hard to see what is there.

To preserve privacy, it's important that this technique uses edge AI—otherwise, the unblurred video would be transmitted outside of the user's home. Instead, the segmentation and blurring is performed on-device before the data is transmitted.

Here's how this use case maps onto our BLERP model:

Bandwidth

The transformation works best if it happens on the high-resolution original video stream rather than a compressed, low-resolution version that may contain visual artifacts. It's often not feasible to transmit high-resolution video, so the transformation must be done on-device.

Latency

Performing the transformation on a remote server may add additional latency versus directly sending the video stream to a peer. Performing it on-device removes this potential extra step.

Economics

It's cheaper to perform the required computation on the device sending video as opposed to in the cloud, where the service provider would have to pay for it.

Reliability

With a cloud server as a middleman, the video streaming pipeline is more complex and has a higher probability of outages. By processing on-device, the pipeline is simpler and may be less likely to fail.

Privacy

When the data is transformed on-device, the user can be guaranteed that nobody will ever see the original video.

Another interesting application for transforming data is the concept of a *virtual sensor*. In some situations, engineering or cost constraints may prevent you from outfitting a device with all of the sensors you would like to. For example, perhaps your design would benefit from a particularly accurate sensor—but that sensor is too expensive for production use.

To get around this problem, it may be possible to create a virtual sensor—an artificial stream of data that provides signals that are almost as good as the real thing. To do this, an edge AI algorithm might process other signals (for example, it could combine readings from several cheaper sensors) and attempt to reconstruct the signal of the desired sensor based on the information they contain.

For example, in **monocular depth estimation** a model is trained to estimate the distance of objects from a simple image sensor. This would usually require a more expensive solution, such as a stereoscopic camera or a laser-based distance sensor.

We’ve now explored the four high-level categories that most edge AI applications can be grouped into. As edge AI technologies continue to evolve, we’ll see many more potential use cases open up. But technological feasibility does not automatically make something a good idea. In the next section, we’ll talk about the importance of responsible design—and learn some of the pitfalls that can result in edge AI applications that cause more harm than good.

Building Applications Responsibly

The first part of this chapter has covered some of the most interesting potential applications for edge AI, and the next chapter will provide a framework for breaking down problems and deciding whether they are a good fit for edge AI to solve.

But as we heard in **“Responsible, Ethical, and Effective AI”** on page xxi, it’s vital that any project is analyzed at every step along the way to make sure that its design and use are responsible. This isn’t some warm-and-fuzzy process where we pat ourselves on the back for ticking some boxes and then continue with our work. Poorly designed technology products can be life-destroying, career-ending disasters—for the end users of the products, the businesses selling them, and the developers creating them.

An example of this is **Uber’s self-driving car division**. The rideshare company launched an aggressive drive toward developing a self-driving car, hiring industry luminaries, and investing billions of dollars. In its rush to test a system on real streets, the company’s flawed safety procedures and ineffective software led to the tragic

death of a pedestrian. This disaster resulted in the shutdown of Uber’s self-driving program, layoffs of hundreds of employees, and the fire sale of the self-driving car division to another business.³

Building a self-driving car, if done well, could result in safer roads and reduced emissions. It seems like a noble mission. But the complex environment of edge AI can lead to potential pitfalls that are challenging to navigate. When these risks are factored in, a well-intentioned technology project can become a deadly minefield.

In Uber’s case, their self-driving car was subject to an incredibly common failure mode of machine learning systems: it was incapable of understanding situations that had not appeared in its training dataset. According to the National Transportation Safety Board, Uber’s self-driving car lacked “the capability to classify an object as a pedestrian unless that object was near a crosswalk”.

There are many factors that contribute to a catastrophic failure like this. On the part of the developers, it shows incompetence and negligence to drive a self-driving car on public roads when it was not tested for its ability to handle even the most common operating conditions. In Uber’s case, this led directly to the death of a human being and the failure of a company division. We may assume that the team behind Uber’s self-driving software were intelligent, capable people—they were recruited as the best in the business. So how can it be that capable people miss the obvious when building and deploying technology?

The unfortunate truth is that building technology well is hard, and it’s difficult to solve complex problems with technologies that—by nature—can only reflect partial considerations. Beyond the fundamental technical challenges, it’s your responsibility as a professional to know the limits of your technology, to scrutinize your processes, ruthlessly evaluate your work, and be willing to shut down a project if it doesn’t seem to be going the right way. A product that unintentionally harms people is a bad product, no matter how brilliant the team that designed it.

In a business setting, you may be fighting against organizational inertia that is more concerned with shipping something than making sure it is safe. But you should always remember that, at the end of the day, your livelihood, reputation, and freedom are on the line if you neglect your professional responsibilities. Even worse, you could build a product that ruins the lives of others and regret it for the rest of your days.

³ The head of the division, Anthony Levandowski, was later sentenced to eighteen months in prison for theft of intellectual property—suggesting that ethical issues were a systemic problem.

Responsible Design and AI Ethics

*Responsible design is critical to building effective products. To make sure it receives the introduction it deserves, the authors invited **Wiebke (Toussaint) Hutiri**, PhD researcher in the Cyber Physical Intelligence Lab at the Technical University of Delft, to write the following section. Wiebke does interdisciplinary research at the intersection of applied machine learning and edge computing, with a focus on designing trustworthy machine learning systems for the Internet of Things.*

Harmful AI failures, as described earlier in this chapter, have made AI ethics an important consideration for most companies integrating AI into their products. For developers, ethics are important, but it is often very difficult to know what ethics are and how to put them into practice. Scrutinizing the values that underpin the product development process (see the following sidebar) is one way of connecting ethics to edge AI development in a practical way. With values as a foundation, the next step is to practice responsible design.

Values in Design

Products cannot be divorced from the context in which they are used. This means that a product is only good if it is useful for doing the task for which it was designed. Of course, products can be repurposed for unexpected alternative uses that expand beyond the initially intended design. However, for a developer, it is a very risky undertaking to hope for unpredictable repurposing, rather than doing good engineering.

In reality, it is seldom enough for products to be only useful. Utility only presents the lowest bar of what users expect from a product. Depending on the context, products also need to be safe to use, have longevity, have low manufacturing and operating costs, and avoid creating harmful waste throughout production, use, and at the end of their life. These are just a few *nonfunctional* requirements (i.e., requirements that do not contribute to technical performance) that are as important for product success as technical performance.

The work of developers is difficult, because oftentimes multiple requirements cannot be met simultaneously and pose trade-offs. Navigating and prioritizing these trade-offs is a key aspect of what engineering design is all about. Individual people typically navigate trade-offs based on their values. Values, like sustainability, democracy, safety, privacy, or equality, are principles that you apply, often unconsciously, to guide decisions in your life (e.g., if privacy is one of your core values, you may be really motivated to learn about edge AI because you don't want your personal data to be shared with third parties).

It is only natural that developers bring their values into the edge AI design process. But there is a big caveat to doing this. Values are personal and vary across people and cultures. You thus cannot rely on your own decision-making heuristics, or on

those of a team of developers who all have the same perspective of life, to result in a successful product that addresses the needs of diverse users. Having a process for reaching consensus on how and which values should guide your design decisions is important and should be part of your development process.

This breakout is only a teaser, and you can learn more about designing for values in [Design for Values—An Introduction](#), from Delft University of Technology.

To design responsibly, a developer needs to know the limits of their “construction” materials and master their tools. Moreover, they need to measure and evaluate whether their product meets the set of functional and nonfunctional requirements they set out to achieve. That’s where machine learning is nothing like any of the hardware or software applications that you have developed before. In ML, data are your “bricks” and your “thermometer” at the same time. This has unique consequences for edge AI.

Data is your “bricks”: For machine learning models in edge AI applications, your training data is your building blocks. The quality of your data affects the quality of your product. Put simply, if it’s not in the training data, you cannot predict it. If it is underrepresented in the training data, you cannot predict it reliably. If it is overrepresented in the training data, you’ll mostly predict it, rather than other things. When you look at representation, it is important to consider the distribution of subgroups across target labels, not only subgroup representation. It is also important to look at the quality of target labels across subgroups, as mislabeled training samples will affect the quality of your models.

Data is your “thermometer”: Where training data is your bricks, evaluation data is the “thermometer” that measures your models. Using evaluation datasets that do not represent your application scenarios is like using an uncalibrated thermometer to measure the temperature. Having high-quality evaluation data is important. In statistics, small sample sizes require special treatment. The common metrics that are used to evaluate ML models assume sufficient sample representation. For evaluation data, it is important that all categories are represented sufficiently across labels, so that your evaluation is statistically meaningful. If a category is not evaluated across all target labels, the limits of the models are not known.

There are many implications of using data to build and evaluate models. For example, data is historic and represents the past, not the future. Data only represents a snapshot in time, while the world is changing. Data is specific to the location where it is collected and to the entity that has been measured. All of this implies that data is only a sample and never complete. Incomplete data is a major cause of bias and discrimination in AI, which has harmed people and caused scandals.

We'll take a deep dive into datasets, explaining all of these concepts from scratch, in [Chapter 7](#).

Important Concepts in Responsible Design

Here are some of the key terms used in machine learning fairness that are very relevant to responsible design:

Bias

In its most general sense, bias means a slanted or skewed perspective. In algorithmic systems, like AI, bias also describes a system that systematically produces outputs that favor some and are prejudiced against other groups or categories. Bias does not have to refer to people. Unless your application targets a particular group or category, bias is usually not a good thing.

Discrimination

Discrimination refers to outcomes of decision-making processes that treat individuals or groups of people differently, based on protected or sensitive attributes. The law provides formal definitions for which attributes are protected in which applications. Sensitive attributes can also be defined informally, based on what a community or society deems as worthy of protecting to ensure equality.

Fairness

Fairness is the most difficult concept to define. This is because no universal definition of fairness exists. Instead, fairness is highly context dependent and influenced by the type of application and its context, the risks and harms that can result from failure, and the stakeholders' values. People often value fairness alongside other values like equity, equality, and inclusiveness. Bias can result in unfairness, and many people consider discrimination to be unfair.

To mitigate bias, avoid discrimination, and evaluate potential unfairness, you need to understand the context in which your design will be used:

- Who will use your design, and how will they use it?
- What gap does your design fill?
- What are the environmental conditions your design will operate in?
- Are there adversaries who may seek to disrupt your product's operation?
- What can go wrong if your design doesn't work as intended?

Metaphorically speaking, are you building edge AI from bricks of unfired clay in a town where it rains for 200 days a year? Or have you carefully considered the context, and collected training data that allows you to build a structure suitable for

the environment? Is your evaluation data a broken piece of equipment, or a carefully calibrated thermometer?

To wrap up this brief introduction to responsible design and AI ethics, you can think of a responsible developer as someone who is *skilled at using their tools to craft their materials into a form that fills a gap that stakeholders want to have filled in accordance with the values agreed on by all that are affected by the project.*

It is easy to get started with responsible design. Know the limits of your data. Know the limits of your models. Talk to the people who will use your product. If there's only one takeaway for you from this section, it should be this: *KUDOs (Know Ur Data, Obviously) to those that develop responsible edge AI.*

—Wiebke (Toussaint) Hutiri, Technical University of Delft

Black Boxes and Bias

There are two aspects of edge AI that make it especially prone to causing unexpected harm in practice: *black boxes* and *bias*.

The term *black box* is a metaphor for a system that is opaque to analysis and understanding. Data goes in, decisions come out, but the processes within it that lead to those decisions are inscrutable. This is a common criticism of modern AI, especially deep learning models, which are famously difficult to dissect. Some algorithms, like random forests, are quite easy to interpret—if you have access to the model, you can read its internals to understand why it makes certain decisions. But things are very different on-device.

Edge devices are often, by design, invisible. They are intended to merge into the background of our built environments; they're embedded in our buildings, products, vehicles, and toys. They are literal black boxes; their contents are invisible, often protected by layers of security to avoid any detailed inspection.

Once an AI algorithm—no matter how simple—is deployed to an edge device, it becomes a black box to anyone who is using it. And if the device is deployed in different real-world conditions to those anticipated by its original developers, even they may have little insight into why it is behaving the way it does.

This is dangerous in several ways, depending on who you are. The device's users—the people who bought and installed it—are now reliant on a system they do not fully understand. They may trust the device to make the right decisions, but they don't have any guarantees that this trust is warranted.

In the example of Uber's self-driving car, the test driver was supposed to be able to intervene in a dangerous situation. Ultimately, however, it's not feasible for a human operator to be able to reliably compensate for a faulty automated system, no matter

their training. The experiment’s reliance on human intervention to avoid tragedy was an irresponsible design decision.

The pedestrian, an innocent bystander who just happened to be crossing the road, was also a victim of the black box nature of the model. If they had been alerted that the car approaching was an unreliable self-driving prototype, they may not have chosen to cross. But the edge AI system, hidden inside an ordinary-looking car, provided no inherent warning. There was no reason for a pedestrian to assume that the car would not behave like human-driven cars.

Finally, the black box nature of edge AI creates a risk for its developers. For example, imagine an edge AI camera trap being used to monitor the population of an invasive species. It might turn out that the camera suffers from false negatives—it fails to recognize one in every three invasive animals that come past. If the camera trap is deployed in a remote location, it may not be possible to validate the camera’s output against reality. The researchers will get an underestimate of the animals’ population—but they will have no way of knowing, since the raw data may no longer exist.

Unlike server-side AI, which can be deployed and monitored alongside the raw data it processes, edge AI is often deployed specifically in situations where raw data cannot be captured. In practice, this means that there’s sometimes no direct way for developers to measure whether an edge AI application is performing correctly once it is in the field.

In practice, conservation researchers solve this dilemma by storing all of the photos captured and performing manual reviews when the memory card is collected, until confidence is built in the system—but this costs time and money. If there’s no equivalent mechanism available to monitor an application, or if it isn’t feasible to implement one within the available budget, it may not be possible to deploy the application responsibly.

Black Boxes and Explainability

The opposite of a black box system is an *explainable* one. Some AI algorithms are more explainable than others, as we’ll learn in “[Classical machine learning](#)” on page 103. Even systems using relatively opaque algorithms can be designed in a way that makes them more explainable—although this can come at the cost of reduced performance or increased complexity. The right trade-off is unique to each application.

Explainability is increasingly a legal requirement for certain projects.⁴ It’s worth exploring if this is the case in your regulatory environment.

⁴ As described in “[Impact of Legal Requirements on Explainability in Machine Learning](#)” by Adrien Bibal et al. (arXiv, 2020).

The hazards of *black boxes* are compounded by the dangers of *bias*. Bias, in edge AI systems, results in the system being equipped with a model of its application area that does not represent the real world. Bias is very likely to occur in systems if developers do not pay attention to well-known sources of bias. The most common sources of bias are:

Human bias

All humans are biased toward a certain worldview based on their experiences (i.e., not appreciating that pedestrians may disregard traffic rules⁵).

Data bias

Datasets reflect the data collection process, not reality (i.e., a dataset may only contain examples of people crossing the street at crosswalks).

Algorithmic bias

All AI algorithms have innate limitations,⁶ and their selection and tuning can result in bias (i.e., the selected algorithm may not perform well on small, faint objects like distant pedestrians at night).

Testing bias

Because real-world testing is difficult and expensive, it often only covers common cases, leading to biased test coverage (i.e., doing exhaustive testing on an artificial test course is expensive, and developers wished to reduce costs, so a critical scenario was not tested).

Bias can be hard to avoid in AI systems. While we tend to associate the term with deliberate, intentional forms of discrimination (such as deliberate sexism in hiring processes), in technology projects, it most commonly occurs due to a lack of understanding of the application context, which is then amplified by the limitations of our resources.

To mitigate bias, a development team needs access to relevant domain experts, a carefully collected dataset (even if it cannot reflect the exact conditions of the real world), an algorithm that is appropriate for the task, and a sufficient budget for real-world testing. In reality, many developer teams only pay attention to these after they experience costly failures due to bias.

When coupled with black box edge AI products, bias creates a risky situation. As described earlier, users are likely to assume that a system *works*. They will trust in

5 Crossing a street outside of a crosswalk is illegal in many parts of the US, where the Uber experiment was performed.

6 Machine learning models have a property known as **inductive bias**, which reflects the internal assumptions they make about how the world works. This inductive bias is required in order for models to work—so selecting the appropriate model is very important.

the correct, safe, and reasonable operation of a product. With no way to inspect its mechanism of operation, they are unable to test this assumption themselves. The responsibility is entirely on developers to meet and manage user expectations.

A successful AI project must be aware of its own limitations and provide the necessary structure to protect users and the public from its potential failure. It's critical for the team behind a product to define the parameters within which the product will function—and to make sure that its users are aware of these parameters.

Over the course of this book, we'll learn a framework for ensuring this awareness and for putting the brakes on projects that are not safe to deploy. It's an ongoing process, and one that must run from conceptualization until end of life. Many projects will run into an ethical quagmire as their true effectiveness is revealed—but some projects are just wrong from the start.

Technology That Harms, Not Helps

Surveillance systems have become pervasive in our modern world, and the public has been forced to adapt to their presence without being asked for consent. The application of AI to surveillance is a complicated topic. While edge AI can potentially be used to preserve privacy, it can also be used to infringe on human rights.

In November 2019, it was discovered that a major supplier of video surveillance cameras, Hikvision, was **marketing a surveillance camera designed to classify the race of individuals**, including that of Uyghurs, a Chinese minority who have been subject to vicious repression by their government. *The New York Times* reported that Chinese government authorities are attempting to use edge AI technology to identify Uyghur people by their appearance and **“keep records of their comings and goings for search and review”**.

While Uber's self-driving experiment resulted in a tragedy caused by bad engineering, Hikvision's racial profiling technology is—to the authors of this book, who believe in a democratic society that promotes personal freedom and equality for all—fundamentally wrong.

When functioning perfectly, the system is designed to enforce a societal bias against a subgroup of people. There is no way to limit the system's bias; in fact, the bias is present as part of the design. While it may be argued that morality is subjective and that different societies have different values, the fact is that the millions of Uyghur people being tracked by this system have had no choice in the matter—and would likely reject it if asked.

Such clear violations of moral expectations may seem obvious, but human psychology—naivety, arrogance, or greed—makes it easy for a group of intelligent people to cross moral boundaries without considering the harm they might cause. One example of this is the service HireVue. Designed to reduce the cost of interviewing

job candidates, companies use HireVue's product to analyze recorded videos where candidates answer specific questions. The company claims to use AI algorithms to rate the likelihood of a candidate being successful in a given role.

Naively, the developers of HireVue did not consider the impact of human, data, algorithmic, and testing biases on their work. Their product, built to use audio-visual information in hiring decisions, inevitably incorporated the voice, accent, and appearance of candidates when making hiring decisions. The clear risk of discrimination this created led to a lawsuit and a backlash from the public, resulting in HireVue having to scrap features of their product and conduct a **third-party audit of their algorithms**.

A further aspect to consider is that an edge AI technology may be used by customers for purposes other than those for which they were designed—and these purposes may be unethical. For example, consider an edge AI camera trap designed for spotting an endangered species. While intended for scientific research, the camera trap might easily be repurposed by poachers as a tool for locating animals that they wish to capture and sell on the black market. It's important to consider these potential “off-label” uses when designing an application, since the risk may be so high that it outweighs the potential benefits of the product.

The costs of negligence

Technologies that use artificial intelligence are often designed to integrate deeply into our world, shaping the day-to-day interactions we have with our homes, places of work, businesses, governments, and each other. This means that the failure of these systems can have a profound impact on people.

There isn't space in this book for a full discussion of the myriad ways this can happen, but here are a few examples:

Violations due to negligence

- Medical hardware could misdiagnose patients, affecting their treatment.

- Surveillance equipment could direct enforcement against some groups of people more than others, leading to unequal justice.

- Educational toys could perform better for some children than others, reducing access to learning opportunity.

- Safety devices could fail due to lack of testing with different user groups, leading to bodily harm.

- Unsecured devices could be compromised by criminals, facilitating crime.

Deliberate ethical violations

Pervasive AI-powered surveillance could impact personal privacy.

Smart sensors could be used by poachers to target endangered wildlife.

Weapons augmented with edge AI could increase conflict deaths and disrupt the global balance of power.

Mitigating societal harms

The framework provided throughout this book will encourage you to take time during the development process to understand the societal implications of what you are building and to make go/no-go decisions based on your findings. Responsible design is very context specific, and mitigating societal harm should be done systematically and continuously to ensure you are on the right path.

A best practice for building responsible applications of AI is to assemble a product team with diverse perspectives in both technical expertise and lived experience. Human biases amplify technical biases, and a diverse team is less likely to have blind spots in their collective worldview. If you have a small team, it's important to budget time and money for diversity, and to reach out to the wider community to find people who are willing to help evaluate your ideas and provide feedback, adding their perspectives to the mix.

Psychological Safety and Ethical AI

Your team's insight is crucial in identifying potential harms, so it's vital that they feel like they have the ability to speak up and make their voices heard throughout the development process. Even in the best working environments it can feel risky for employees to speak up when they think their feedback may disadvantage them.

For example, imagine an employee who notices a potential risk but feels unable to mention it because they feel hesitant to derail an important project. In reality, the employee may save the company time, money, and reputation by pointing out a significant issue. However, if the employee feels afraid of a potential negative impact on their career, reputation, or the team's morale, they may choose not to say anything until it is too late.

Psychological safety is the feeling of being able to speak up and discuss issues without fearing negative consequences. This, along with a culture that reinforces the importance of ethics in AI, are necessary conditions for building successful AI projects. Some valuable resources on this topic are covered in [“Diversity” on page 126](#).

There is no way to benchmark the “ethicality” of a system.⁷ Instead, we need to understand the values underlying the creation of a system—including what those values are, who they belong to, and in which context they are intended to apply. This awareness allows us to shape our work into useful products that bring benefit, not harm.

Various companies and services exist to help guide teams through the process of developing AI responsibly, or to audit existing applications for potential harms. If you are concerned about harmful “off-label” use of your work, there are also some legal tools at your disposal. **Responsible AI Licenses** (RAIL) are technology licenses designed to help developers restrict the legal use of artificial intelligence products for harmful applications.

By attaching a RAIL to their product, developers create legal grounds to prevent its misuse in a specific list of applications, which can be extended to include any categories the developer would like to include. Some of the default prohibited options include surveillance, crime prediction, and generating fake photography. Of course, this only helps prevent unethical usage by entities that consider themselves bound by legal agreements.

Finally, there are many free, high-quality online resources that you can use to learn more about ethical and responsible AI and evaluate the work you are doing. To get you started, here’s a short list:

- **Berkeley Haas’ guide, Mitigating Bias in Artificial Intelligence**
- **Google’s recommended practices for responsible AI**
- **Microsoft’s responsible AI resources**
- **PwC’s responsible AI toolkit**
- **Google Brain’s “People + AI Research (PAIR)”**

For a detailed high-level summary of current approaches to principles in AI, we also recommend reading “**Principled Artificial Intelligence: Mapping Consensus in Ethical and Rights-Based Approaches to Principles for AI**” (J. Fjeld et al., Berkman Klein Center Research Publication, 2020).

⁷ Travis LaCroix and Alexandra Sasha Luccioni, “Metaethical Perspectives on ‘Benchmarking’ AI Ethics”, arXiv, 2022.

Summary

In this chapter, we've developed a solid understanding of how edge AI fits into our world. We know the top use cases, the key benefits, and the critical ethical considerations that need to be applied.

We're now ready to dive into some of the technical details. In the next chapter, we'll learn about the technology that makes edge AI work.

The Hardware of Edge AI

It's now time to meet the devices, algorithms, and optimization techniques that power edge AI applications. This chapter is designed to provide a broad overview of the most important technical elements of the field. By the end of it, you'll have the building blocks necessary to start the high-level planning of an edge AI product.

Sensors, Signals, and Sources of Data

Sensors are electronic components that give devices the power to measure their environments and detect human input. They range from extremely simple (trusty old switches and variable resistors) to mind-blowingly sophisticated (light detection and ranging [LIDAR] and thermal imaging cameras). Sensors provide our edge AI devices with the streams of data that they use to make decisions.

Beyond sensors, there are other sources of data that our devices can tap into. These include things like digital device logs, network packets, and radio transmissions. Although they have a different origin, these secondary data streams can be just as exciting as sources of information for AI algorithms.

Different sensors provide data in different formats. A few data formats are commonly encountered in edge AI applications. They can be summarized as follows:

Time series

Time series data represents the change in one or more values over time. A time series may contain multiple values from the same physical sensor—for example, a single sensor component may provide readings of both temperature and humidity. Time series data is often collected by polling a sensor at a specific rate, such as a certain number of times per second, to produce a signal. The rate of polling is known as the sampling rate, or frequency. It is common that the individual

readings (known as samples) are collected within a constant period, so the time interval between two samples is always the same.

Other time series may be aperiodic, meaning the samples are not collected at a constant rate. This might happen in the case of a sensor that detects specific events—for example, a proximity sensor that toggles a pin when something comes within a certain distance. In this case, it is common to capture the exact time when an event happened alongside the sensor value itself.

Time series may represent summary information. For example, a time series could consist of the number of times something happened during the interval since the last value.

Time series data is the most common form of sensor data for edge AI. It is particularly interesting because, in addition to the sensor values, the signal includes information about the timing of the values. This provides useful information when attempting to understand how a situation is changing. In addition to timing information being useful, time series data is valuable because it contains multiple readings from the same sensor, reducing the impact of momentary anomalous readings.

There is no typical frequency for a time series—it can range from a single sample a day to millions of samples per second.

Audio

A special case of time series data, audio signals represent the oscillation of sound waves as they travel through the air. They are generally captured at a very high frequency—thousands of times per second. Since hearing is a human sense, huge amounts of research and development have gone into innovations that make it easier to work with audio data on edge devices.

These technologies include special signal processing algorithms that make it easier to process audio data, which in its raw form is typically captured at an extremely high frequency. As we will see later, audio signal processing is so common that a lot of embedded hardware comes with built-in functionality for performing it efficiently.

One of the most widespread uses of edge AI audio processing is in speech detection and classification. That said, audio doesn't even have to be in the spectrum of human hearing. Sensors used by edge AI devices can potentially capture ultrasound (higher than audible by human hearing) and infrasound (lower than audible by humans) data.

Image

Images are data that represent the measurements taken by a sensor that captures an entire scene, as opposed to a single point. Some sensors, like cameras, use an array of tiny elements to capture data from the entire scene in one go. Other sensors, like LIDAR, build up an image by mechanically sweeping a single sensor element across the scene over a period of time.

Images have two or more dimensions. In their typical form, they can be thought of as a grid of “pixels,” where the value of each pixel represents some property of the scene at the corresponding point in space. A basic example of this is shown on the left side of [Figure 3-1](#). The size of the grid (for example, 96x96 pixels) is known as the *resolution* of the image.

A pixel may have multiple values, or channels. For example, while a grayscale image only has one value per pixel, representing how light or dark the pixel is, a color image may have three values per pixel (in the RGB model), representing three colors (red, blue, and green) that can be mixed to represent any other color in the visible spectrum. This structure is shown on the right side of [Figure 3-1](#).

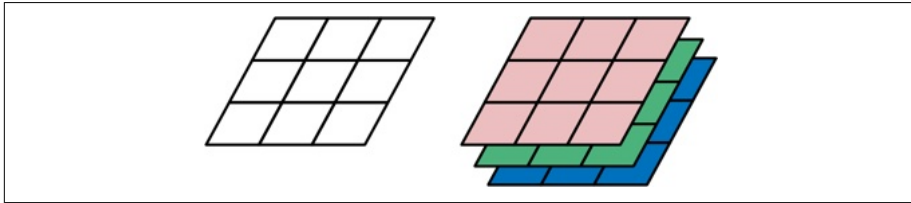


Figure 3-1. The diagram on the left represents the pixels of a single channel image; the diagram on the right represents the structure of a three-channel image, such as an RGB photograph

The typical representation of images, as an n -dimensional grid, means that they contain spatial information about the relative proximity of different aspects of a scene to one another. This information is extremely valuable in understanding what a scene contains. There are entire classes of *image processing* and *computer vision* algorithms that make use of this information.

Images don’t have to represent visible light, or even light at all. They can represent infrared light (often used to measure the temperature of parts of a scene), time of flight (in the case of LIDAR, which measures how long it takes light to bounce back from each part of a scene), or even radio waves (think of data collected by a radio telescope, or on a radar screen).

Video

Technically another special case of time series data, video deserves its own category due to its distinct utility. A video is a sequence of images, each representing a snapshot of a scene at a point in time. As a time series, video has a sampling rate—although in the case of video it is typically referred to as the frame rate, since each individual image in the sequence is known as a frame.

Video is a very rich format—it contains both spatial information (within each frame) and temporal information (between each frame). This richness means that it tends to occupy a lot of memory, so it tends to require more capable computing devices.

How Are Values Represented?

All of the preceding categories represent individual sensor readings using single numeric values. For example, a time series is a sequence of individual readings, and an image is a grid composed of individual readings.

Each reading is a number and can be represented on a computer in a variety of different ways. For example, here are some typical numeric types used to represent sensor data in C++:

- Boolean (1 bit): a number with two possible values
- 8-bit integer: a nondecimal number with 256 possible values
- 16-bit integer: a nondecimal number with 65,536 possible values
- 32-bit floating point:¹ can represent a wide range of numbers with up to seven decimal places, with a maximum of 3.4028235×10^{38}

By varying the numeric type used to represent a value, developers can trade numerical precision for reduced memory usage and computational complexity.

Types of Sensors and Signals

There are thousands of different types of sensors on the market. A nice way of grouping them is by their *modality*. According to Carnegie Mellon University, **modality refers to the way in which something happens or is experienced**. From a human perspective, our senses of sight, hearing, or touch all have different modalities.

¹ As described by the **IEEE 754** standard.

There's no strictly defined list of sensor modalities, and the best way to describe them may vary between industries and applications. In the following section, we'll explore some groupings that make sense from a broad edge AI perspective:

- Acoustic and vibration
- Visual and scene
- Motion and position
- Force and tactile
- Optical, electromagnetic, and radiation
- Environmental, biological, and chemical

There are also many nonsensor data sources available to edge devices—we'll go through those, too.

Acoustic and Vibration

The ability to “hear” vibrations allows edge AI devices to detect the effects of movement, vibration, and human and animal communication at a distance. This is done with acoustic sensors, which measure the effect of vibrations that are traveling through a medium that might range from air (in the case of microphones, like the one in [Figure 3-2](#)) to water (hydrophones) or even the ground (geophones and seismometers). Some vibration sensors are designed specifically for use with heavy industrial machinery.

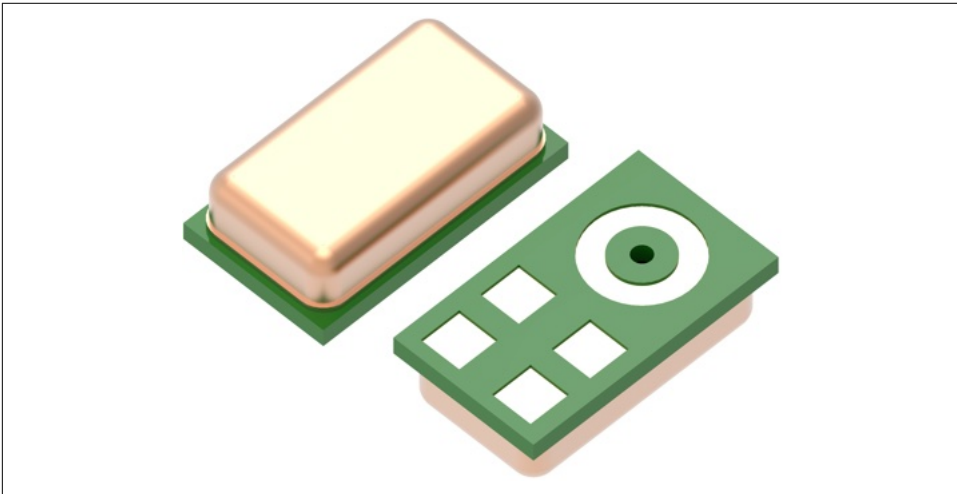


Figure 3-2. A 3D rendering of a surface-mount microelectromechanical systems (MEMS) microphone, found in many modern products

An acoustic sensor typically provides a time series that describes the variation of pressure in its medium. Acoustic signals contain information across various frequencies—for example, the high and low notes of a singing voice. Acoustic sensors generally operate in a certain frequency range, and they may not have a linear response to frequencies even within that range.

In addition to their nonlinear frequency response, the ability of acoustic sensors to capture high frequencies depends on their sample rate. To accurately capture a high-frequency signal, an acoustic sensor must have a sufficiently high sample rate. When building an edge AI application for acoustics, make sure you understand the properties of the signal you are trying to measure, and choose sensor hardware that is a good fit.

Visual and Scene

It is common for edge AI applications to need to understand the scenery around them in a passive manner, without reaching out to touch it. The most common sensors used for this task are image sensors, ranging from tiny, low-power cameras (as seen in [Figure 3-3](#)) to super high-quality, multimegapixel sensors. As described previously, the images obtained from image sensors are represented as arrays of pixel values.

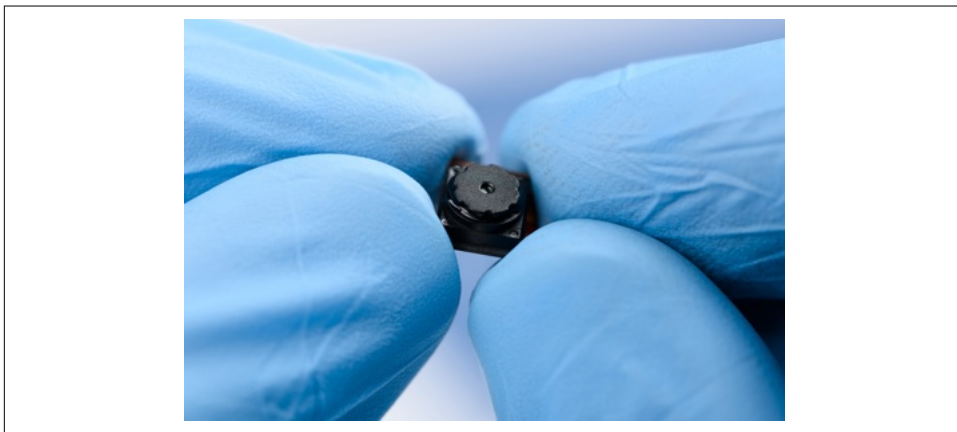


Figure 3-3. A tiny image sensor, the type of form factor that might be used in an embedded device

Image sensors capture light using a grid of sensor elements. In a camera, light from a scene is focused onto the sensor by a lens. The area that can be imaged by a camera is known as its field of view, and it depends on the size of the lens and the image sensor.

Some common variations in image sensors:

Color channels

For visual light, sensors can commonly capture data in grayscale or color (red, green, and blue, or RGB).

Spectral response

The wavelengths of light the image sensor is sensitive to, which may exceed the range of human vision. This can even include infrared radiation, allowing sensors known as thermal cameras to “see” heat.

Pixel size

Larger sensors can capture more light per pixel, increasing their sensitivity.

Sensor resolution

The more elements on a sensor, the finer detail it can capture.

Frame rate

How frequently a sensor can capture an image, typically in frames per second.

Since illumination of a scene is sometimes required, it is common to pair image sensors with light emitters—in both visible and invisible ranges of the spectrum. For example, an infrared LED can be used with an infrared-sensitive camera to illuminate dark scenes without disturbing humans or animals with visible light.

Larger, higher resolution sensors typically require more energy. High-resolution sensors produce large amounts of data, which can be difficult to process on smaller edge AI devices.

A relatively new group of image sensors, known as event cameras, work slightly differently. Instead of capturing the entire visual field at a specific frame rate, each pixel in the camera responds individually to changes in brightness but remains silent if nothing is happening. The result is a time series of individual pixel changes that can be easier for edge AI devices to process than a large sequence of full frames.

Another interesting type of image sensor is known as range imaging sensors. These allow devices to image their surroundings in three dimensions—often by emitting light and measuring how long it takes to bounce back, a technique known as “time of flight.” A common time-of-flight sensor technology is known as LIDAR. LIDAR sensors work by scanning their surroundings with a laser beam, measuring how much of the light is reflected back to the sensor. This allows them to visualize an area in three dimensions, as shown in [Figure 3-4](#).

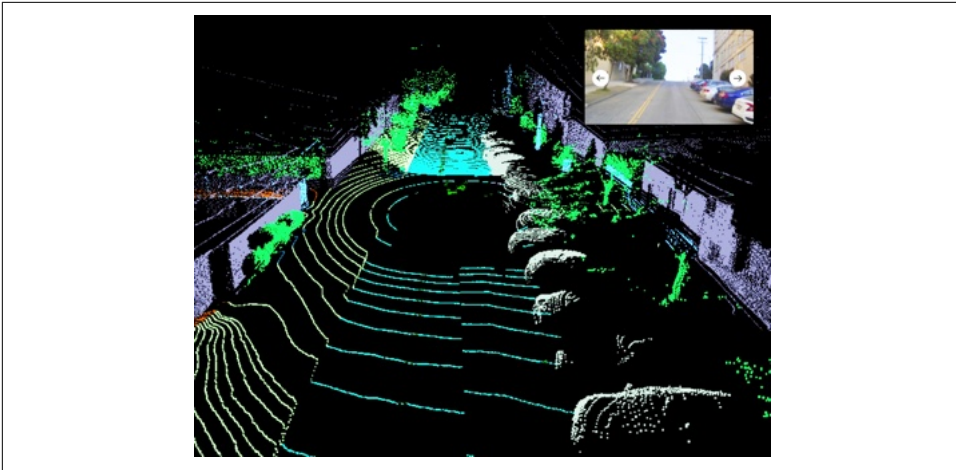


Figure 3-4. This image from the *PandaSet* open source LIDAR dataset shows a typical LIDAR “point cloud,” with each point in the 3D visualization representing a distance that was measured by the laser; the inset photograph at the top right shows the same scene from the perspective of an image sensor

LIDAR and other time-of-flight sensors are typically much larger, more complex, expensive, and energy intensive than standard image sensors. The large amounts of data they generate can be difficult to process and store on edge devices, which also limits their utility. LIDAR is typically used for mapping environments—including to help self-driving vehicles navigate the world.

Radar, or radio detection and ranging, is also occasionally used by edge devices to understand the position of surrounding objects in three dimensions, potentially at long range. Like LIDAR, it is complex and has high energy requirements—but is definitely an option if your use case requires it.

Motion and Position

It can be useful for edge AI devices to understand both where they are and where they might be headed. Fortunately, there are many different types of sensors that can help. This is a broad category, ranging from the simplest (mechanical tilt switches) to the most complicated (the satellite-enabled GPS [Global Positioning System]). As a whole, they allow devices to understand their position and motion within the world.

Here’s a list of typical motion and position sensors for edge AI applications:

Tilt sensor

A mechanical switch that is on or off depending on its orientation. Super cheap and easy to use.

Accelerometer

Measures the acceleration (the change in velocity over time) of an object across one or more axes, often at a high frequency. Accelerometers are the Swiss Army knives of motion sensing, used for everything from recognizing the characteristic motions of sporting activities (in smart watches) to sensing the vibrations of industrial equipment (in predictive maintenance). They also always know which way is down, thanks to the pull of gravity.

Gyroscope

Measures the rate of rotation of an object. Often paired with an accelerometer to give a picture of the motion of an object in 3D space.

Rotary or linear encoder

Measures the exact position of either a shaft or axle (rotary) or a linear mechanism (like the position of an inkjet printer head). Often used in robotics to capture the positions of robots' wheels, limbs, and other appendages.

Time of flight

A sensor that uses an electromagnetic emission (light or radio) to measure the distance from a sensor to whatever object is directly in its line of sight.

Real-time locating systems (RTLS)

Systems that use multiple transceivers in fixed locations around a building or site to track the position of individual objects, such as pallets in a warehouse.

Inertial measurement unit (IMU)

A system that uses multiple sensors to approximate the current position of a device based on its motion as measured from an internal frame of reference (as opposed to using external signals such as GPS).

Global Positioning System (GPS)

A passive system that uses radio signals from satellites to determine the location of a device, down to a few meters. Requires line of sight from the device to several satellites.

Motion and position are typically represented as a time series of sensor readings. Given the number of sensor types in this category, there are options for every cost and energy budget. Typically, the more confidence in absolute position required, the more cost and complexity involved.

Force and Tactile

From switches to load cells, force and tactile sensors help edge AI devices measure the physicality of their environment. They can be helpful in facilitating user interaction, understanding the flow of liquids and gases, or measuring the mechanical strain on an object.

Here are some typical force and tactile sensors:

Buttons and switches

Traditional switches used as simple buttons for human interaction, but also serve as sensors that provide a binary signal that indicates when a device is colliding with something.

Capacitive touch sensors

Measure the amount that a surface is being touched by a conductive object, like a human finger. This is how modern touchscreens work.

Strain gauges and flex sensors

Measure how much an object is being deformed, which can be interesting for detecting damage to objects and for building tactile human interface devices.

Load cells

Measure the precise amount of physical load that is applied to them. They come in a wide range of sizes, from tiny (useful for measuring the weight of small objects) to gigantic (measuring strain in bridges and skyscrapers).

Flow sensors

Designed to measure the rate of flow in liquids and gases, such as water in a pipe.

Pressure sensors

Used to measure pressure of a gas or liquid, either environmental (such as atmospheric pressure) or inside a system (such as inside a car tire).

Force and tactile sensors are typically simple, low energy, and easy to work with. Their measurements are easy to represent as time series. They are especially useful when building tactile user interfaces or detecting when a robot (or other device that can move around) has hit something.

Optical, Electromagnetic, and Radiation

This category includes sensors that are designed to measure electromagnetic radiation, magnetic fields, and high energy particles, in addition to basic electrical properties such as current and voltage. This may sound exotic, but it includes familiar things like measuring the color of light.

Here are some typical optical, electromagnetic, and radiation sensors:

Photosensor

A category of sensor that detects light at various wavelengths, both visible and invisible to the human eye. This can be useful for many things, from measuring ambient light levels to detecting when a beam of light has been broken.

Color sensor

Uses photosensors to measure the precise color of a surface, which can be helpful for recognizing different types of objects.

Spectroscopy sensor

Uses photosensors to measure the way that various wavelengths of light are absorbed and reflected by materials, giving an edge AI system insight into their composition.

Magnetometer

Measures the strength and direction of magnetic fields. A subtype of magnetometer is a digital compass, which can indicate the direction of north.

Inductive proximity sensor

Uses an electromagnetic field to detect nearby metal. It is commonly used to detect vehicles for traffic monitoring.

Electromagnetic field (EMF) meter

Measures the strength of electromagnetic fields. This includes those emitted incidentally, for example by industrial equipment, or those intentionally emitted by radio transmitters.

Current sensor

Measures the flow of current through a conductor. This can be useful in monitoring industrial equipment since fluctuations in current can provide information about the functioning of the equipment.

Voltage sensor

Measures the amount of voltage across an object.

Semiconductor detector

Measures ionizing radiation, which is composed of extremely fast-moving particles, typically created by the decay of radioactive substances.

As with many other sensors, this category generally provides a time series of measurements. While useful for measuring ambient conditions, the sensors described here can also be useful in arrangements where they detect emissions that are produced deliberately by a device. For example, a photosensor can be paired with a light emitter on the other side of a hallway to detect when someone is moving past.

Environmental, Biological, and Chemical

A loose category that includes many different types of sensors; environmental, biological, and chemical sensing allows edge AI devices to sniff the composition of the world around them. Some common types of sensors include:

Temperature sensor

Measures temperature, either of the device itself or of a distant source of infrared emissions.

Gas sensor

Many different sensors exist to measure concentrations of different gases. Common gas sensors include humidity sensors (which measure water vapor), volatile organic compound (VOC) sensors, which measure a selection of common organic compounds, and carbon dioxide sensors.

Particulate matter sensor

Measures the concentration of tiny particles in a sample of air and are commonly used to monitor pollution levels.

Biosignals sensor

Covers a vast range of signals that are present in the bodies of living things—for example, measurement of electrical activity in the human heart (electrocardiography) and brain (electroencephalography).

Chemical sensor

Many different sensors available that are designed to measure the presence or concentration of specific chemicals.

This category of sensors generally provides a time series of readings. Due to their need to interact chemically and physically with the environment, they can sometimes be difficult to work with—for example, calibration against known quantities of chemicals is often required, and sometimes sensors require a warm-up period before they can take a reliable reading. It is common for environmental sensors to degrade over time and require replacement.

Other Signals

In addition to gathering signals from the physical world, many edge AI devices have access to a rich feed of virtual data. This can be split roughly into two groups: introspective data, about the state of the device itself, and extrospective data, about the systems and networks that the device is connected to.

Depending on the device, various types of internal state may be available. These could include:

Device logs

These track the lifecycle of the device since it was powered up. This could provide information about many different things: configuration changes, duty cycle, interrupts, errors, or anything else you choose to log.

Internal resource utilization

This might include available memory, power consumption, clock speed, operating system resources, and usage of peripherals.

Communications

A device can keep track of its physical connections, radio communications, networking configuration and activity, and the resulting energy usage.

Internal sensors

Some devices have internal sensors; for example, many system-on-chip devices include a temperature sensor to monitor their CPU.

One interesting usage of introspective data is in preserving battery life. Lithium rechargeable batteries can lose capacity if they are continually held at 100% charge while plugged in. Apple's iPhone uses an edge AI feature known as **Optimized Battery Charging** in order to avoid this problem. It uses an on-device machine learning model to learn the user's charging routine, then uses this model to minimize the amount of time the battery spends full—while ensuring the battery is still charged when the user needs it.

Extrospective data streams, which come from outside the device, can be extremely rich in information. Here are some possible sources:

Data from connected systems

It's common for edge AI devices to be deployed in a network, and data forwarded by adjacent devices can be used as input to AI algorithms. For example, an IoT gateway could use edge AI to process and make decisions based on the data collected by its nodes.

Remote commands

An edge AI device might receive control instructions from another system or user. For instance, the user of a drone could request that it move to a certain coordinate in 3D space.

Data from APIs

An edge AI device can request data from remote servers to feed into its algorithms. For example, a home heating system equipped with edge AI might request weather forecast data from an online API and use the information to help decide when to turn the heating on.

Network data

This might include network structure, routing information, network activity, and even the contents of data packets.

Some of the most interesting edge AI systems make use of all of these data streams together. Imagine an agricultural technology system that helps a farmer take care of crops. It might include remote sensors out in the fields, connections to important

online data sources (like the weather forecast, or the price of fertilizer), and a control interface used by the farmer. As an edge AI system, it could potentially operate without an internet connection—but if it had one, it could make use of valuable information.

In more complex system architectures, edge AI also pairs nicely with server-side AI; we'll learn more about that later in this chapter.

Processors for Edge AI

One of the most exciting parts of edge AI is the vast—and growing—array of hardware that applications can make use of. In this section, we'll explore the high-level categories of hardware and learn what makes each one suited to a particular niche.

We're in the midst of a Cambrian explosion of edge AI hardware, so in the time since this book was published it is likely that there are even more options than what is printed here. With a spectrum that runs from cheap, low-power microcontrollers (so-called “thin edge” devices) to lightning-fast GPU-based accelerators and edge servers (known as “thick edge”), developers can find hardware that is the perfect fit for almost any application.

Edge AI Hardware Architecture

The *architecture* of a hardware system is the way its components connect to each other. **Figure 3-5** shows the typical hardware architecture of an edge device.

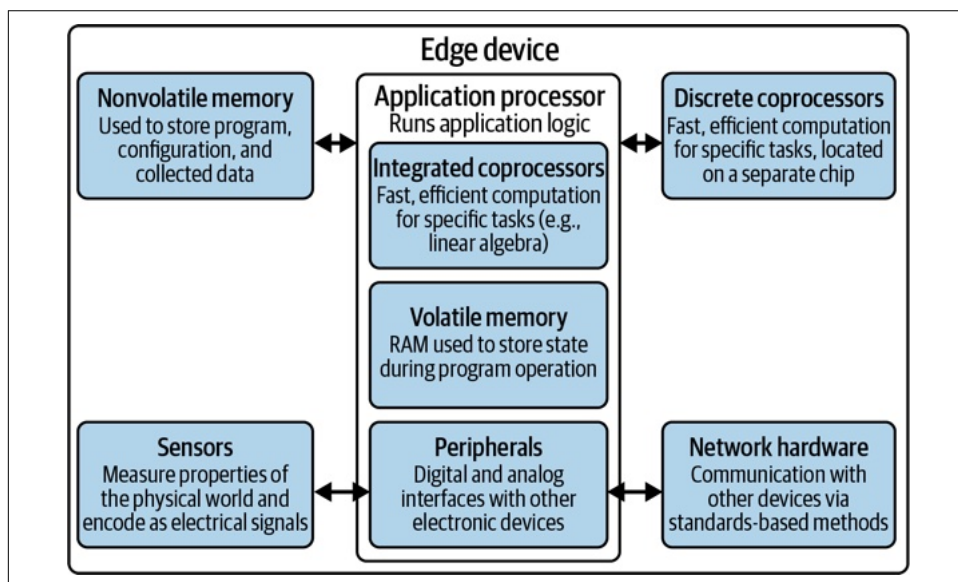


Figure 3-5. Architecture of an edge device

The beating heart of the device is the *application processor*. This is the general-purpose processor that coordinates the application and, by default, runs all of the algorithms and logic that make up its program.

In many cases, the application processor will have *integrated coprocessors*: built-in additional hardware that is highly efficient at performing certain computations. For example, many processors have a built-in floating-point unit (FPU) designed to quickly perform floating-point calculations. Medium- and high-end MCUs increasingly have integrated hardware that can accelerate functions relevant to edge AI, such as digital signal processing and linear algebra.

The application processor also integrates the *volatile memory* (such as RAM) that is used as working memory during program execution. That said, it's also common to have additional RAM that is external to the processor itself and lives on a separate chip.

On Die Versus Off Die

A computer system typically consists of integrated circuits (ICs) made from silicon chips attached to a printed circuit board (PCB). For example, the application processor is an IC. A silicon chip consists of a literal piece of silicon, upon which is etched a series of complex patterns that make up a processor. This piece of silicon is known as a *die*.

When referring to a processor and the other components it integrates with, it's common to hear the terms *on die* and *off die*. On-die components are located on the same piece of silicon as the processor itself, whereas off-die components are located in separate ICs attached to the same PCB.

Because on-die components are physically closer to the main processor, they're typically faster and more energy efficient: it takes less time and energy to send data back and forth between them. However, the more items located on the same die, the bigger the die has to be—and large dies tend to be expensive and power hungry.

Every embedded hardware design has to strike a balance between what is included on die and off die. For example, if efficiency is the highest priority it may make sense to choose a processor with on-die features. If cost is more important, off die may be the way to go.

Many designs make use of both on-die and off-die components. For example, a system may have a small amount of on-die RAM for program execution combined with a larger amount of off-die RAM used to buffer raw sensor data that is waiting to be processed.

RAM is very fast memory, but it uses a lot of energy, and the contents of RAM are lost when the device shuts down. It's quite expensive and takes up a lot of physical space, so it's often a very limited resource.

The application processor is connected to *nonvolatile memory*, commonly known as ROM (read-only memory) or flash,² which can similarly be located either on die or off die (in [Figure 3-5](#) it is shown off die). Nonvolatile memory is used to store things that don't often change and need to be preserved when the system is shut down. This might include software programs, user configuration, and machine learning models. It's slow to read and extremely slow to write.

Many designs have *discrete coprocessors*. Similar to integrated coprocessors, these exist to perform fast and efficient mathematics for specific purposes. Unlike integrated coprocessors, they are located off die. They may be far more powerful (and power hungry) than the application processor: for example, a low-power SoC may be combined with a powerful GPU.

The *peripherals* of the processor provide its interface with the rest of the world, via various standards.³ The most common things peripherals are used to connect with are *sensors* and *network hardware*.

Microcontrollers and Digital Signal Processors

It could be argued convincingly that microcontrollers are the foundation of our modern world. They're the tiny, cheap computers that animate everything from car engines to smart appliances. Microcontrollers are manufactured in astonishing volume; it's projected that **26.89 billion of them will be shipped in 2022**—that's three and a half for every human being on the planet.



MCUs

Microcontrollers are often referred to as MCUs, an acronym for microcontroller units.

Microcontrollers are typically used for single-purpose applications, like controlling a piece of machinery. This means that they can get away with being a lot simpler than other types of computers that need to run multiple programs—for example, they generally do not use an operating system.

² While flash memory can be reprogrammed, it's still referred to as ROM in an embedded context.

³ Peripheral interfaces, such as GPIO, I2C, SPI, and UART, are important when designing hardware but are beyond the scope of this book. Most modern embedded processors have decent peripheral support.

Instead, their software (known as *firmware*) is run directly on the hardware and incorporates the low-level instructions necessary to drive any peripherals.⁴ This can make software engineering for microcontrollers quite challenging, but it gives developers a lot more control over exactly what is going on when their programs run.

One of the distinguishing characteristics of microcontrollers is that the majority of their components are implemented on a single piece of silicon; this is key to their relatively low cost. In addition to a processor, a microcontroller is generally equipped with flash memory (for storing programs and other useful data), RAM (for storing state during program execution), and various technologies for communicating with other devices (such as sensors) using either digital or analog signals.

The microcontroller world is incredibly diverse—part of the reason they are so valuable is that they are available in variants to suit every imaginable situation. For the purposes of this book, we'll divide them into three main categories: low-end, high-end, and digital signal processors.

Low-end MCUs

Many MCUs are designed specifically for low cost, small size, and energy efficiency. The trade-off is that they have limited computational resources and capabilities. Here are some typical specifications:

- 4-bit to 16-bit architecture
- <100 MHz clock speed
- 2 KB to 64 KB of flash memory
- 64 bytes to 2 KB of RAM
- Digital input and output
- Current draw: single digits to tens of milliamps at ~1.5–5 volts when running, and microamps when sleeping while waiting for input
- Cost: one or two dollars per unit when bought in large volumes

⁴ Another characteristic of firmware is that, unlike an operating system, firmware is not typically intended to be modified by the end user.

A Note on Power

The amount of energy that a microcontroller consumes depends on many factors, most of which are in the developers' hands to control. Among other things, you can decrease power consumption by running the processor at a reduced speed, switching off features when not in use, and putting the entire microcontroller into idle mode when it is not currently processing data.

This flexibility, coupled with the general diversity of the microcontroller market, makes it tricky to quote exact numbers for power consumption. If you're designing against tight power constraints, you'll want to evaluate the hardware and measure energy use for yourself.

Many low-end MCUs used today are based on designs that have been in use since the 1980s.⁵ While technology has continued to improve, there's always a need for simple, low-cost and low-power hardware, so these chips are here to stay. They are extremely common across many industries.

Low-end MCUs have some significant disadvantages when it comes to edge AI. Since they lack memory and compute, they aren't well suited to dealing with large amounts of data or complex signal processing. They generally do not have any hardware implementation of floating-point arithmetic, meaning calculations involving rational numbers can be incredibly slow. These attributes limit the types of edge AI algorithms they can run.

The typical applications for low-end MCUs play to their benefits: high-reliability automotive and medical devices and low-cost appliances, gadgets, and infrastructure. One popular low-end MCU is the **Atmel 8-bit AVR platform**. While they are an important part of the MCU world, their computational limitations mean that low-end MCUs probably shouldn't be your first choice of target for an edge AI application.

That said, as we mentioned in the first section of the book, edge AI programs don't always have to be computationally challenging. Low-end MCUs are perfectly capable of running complex conditional logic, which may be enough for what you need to do. They can also form part of a network of connected devices that makes use of edge AI—for example, a low-end MCU could capture sensor data and forward it to a more sophisticated device for decision making.

⁵ The **Intel 8051** was first developed in 1980 and is still in use today.

High-end MCUs

At the other end of the MCU spectrum, today's most powerful microcontrollers have enough compute to give a '90s vintage personal computer a run for its money. In many cases, they still manage to be highly energy efficient. Here are some typical specs:

- 32-bit architecture
- <1000 MHz clock speed
- 16 KB to 2 MB of flash
- 2 KB to 1 MB of RAM
- Optional hardware support for faster math
- Floating-point unit (FPU)
- Single instruction, multiple data (SIMD) instructions
- Optional multiple processor cores
- Digital and analog input and output
- Current draw: low single digits to high tens of milliamps at ~1.5–5 volts; microamps while sleeping
- Cost: from low single digits to low tens of dollars per unit

High-end MCUs provide a big jump in performance, thanks to faster clock speeds and a 32-bit architecture.⁶ In addition, many models of MCUs have hardware support for some neat tricks that increase computation speed. One of these, SIMD, allows the processor to run several computations in parallel—which can be extremely helpful when running signal processing and machine learning applications, which involve a lot of computation.

Increasingly, high-end MCUs are designed with edge AI applications in mind. It's common for vendors to offer software and libraries that help optimize edge AI code to run efficiently on-device. Another big benefit is a trend toward providing larger amounts of flash and RAM—very helpful for manipulating data and storing large machine learning models.

High-end MCUs are used in a huge range of use cases, from sensing and IoT to digital gadgets, smart appliances, and wearables. At the time of writing, they represent the sweet spot for cost, energy usage, and computational ability for embedded machine learning. They have just enough power to run capable deep learning

⁶ A 32-bit processor can operate on twice the data in one go as a 16-bit processor can. This means data can be processed more quickly. It also supports a larger amount of RAM.

models—including deep learning models that can process visual information—but they remain simple enough to embed very cheaply into a wide range of applications.

Microcontrollers based on **Arm’s Cortex-M cores** are extremely popular, such as the **Nordic nRF52840** and the **STMicroelectronics STM32H743VI**. There are also popular options based on the **RISC-V** architecture, such as the **Expressif ESP32**.

As edge AI becomes more important, it’s increasingly common to pair general-purpose, high-end microcontrollers with purpose-built coprocessors designed to accelerate deep learning workloads. We’ll cover that more in “**Deep Learning Accelerators**” on page 77.



Performance Characteristics

An average high-end microcontroller can process audio using deep learning in near real time, and low-resolution video at a second or so per frame.

Digital signal processors (DSPs)

An interesting subcategory, DSPs are special microcontrollers that are designed to be highly efficient at transforming digital signals. Instead of general-purpose computation, their architecture is designed to run specific algorithms and mathematical operations as quickly as possible—including things like multiply–accumulates and Fourier transforms, which we’ll encounter in **Chapter 4**.

As luck would have it, many of those mathematical operations are very helpful in edge AI, both for processing data and for running machine learning models. This can make DSPs a valuable tool. The downside of DSPs is that they are not designed for general-purpose compute, meaning that they may not be suitable for running the non–edge AI parts of your application.

Today’s high-end MCUs often have some of the features of DSPs, such as SIMD instructions that can help increase throughput for signal processing tasks—in fact, some are described as “digital signal controllers” in order to highlight these abilities. However, dedicated DSPs can still be useful. For example, many smartphones that include voice assistants (such as the Google Assistant) include a DSP chip in order to run an always-on keyword-spotting model without hurting battery life.

Heterogeneous Compute

Hardware designers aren't limited to choosing a single microcontroller for a given application. It's actually quite common to combine multiple microprocessors in a single product. For instance, an edge AI device may include a small, low-power MCU to run its basic operations—alongside a large, powerful MCU that is used for occasional signal processing and machine learning workloads.

This type of setup is known as heterogeneous compute, and it's increasingly important in edge AI because it enables true concurrency: the ability to perform more than one task at a time. One of the big challenges with heterogeneous compute is deciding how to split a computational workload between two processors in order to maximize efficiency. If you can do it right, there are major rewards.

Some architectures for edge AI application, such as those that use cascading models (see [“Cascading flow” on page 283](#)), lend themselves particularly well to heterogeneous compute. The rise of deep learning accelerators (see section [“Deep Learning Accelerators” on page 77](#)) is making it an increasingly important concept.

System-on-Chip

After microcontrollers, the next most common type of edge compute comes in the form of system-on-chip (SoC) devices. While a microcontroller is a stripped-down, optimized version of a computer with all the fat trimmed away, SoC devices attempt to squeeze all of the functionality of an entire traditional computer system into a single chip.

Unlike microcontrollers, whose software interacts directly with the hardware, SoC devices run traditional operating systems that abstract away a lot of the hardware so that developers can focus entirely on their application code. Developers can use the same tools and environments they use to write server and desktop applications, including high-level languages like Python (modern microcontrollers are typically programmed in C or C++).

There are two costs associated with this ease of use: efficiency and complexity. SoCs are generally a lot less energy efficient than microcontrollers, which limits their fields of application. They are still an order of magnitude more efficient than traditional computer systems with separate peripherals, but they're nowhere near as good as microcontrollers for keeping energy usage to a minimum. This additional energy usage may also introduce heat management issues.

The additional complexity represented by an operating system is another burden on SoC devices. With huge amounts of OS code being run alongside a developer's application, it's more difficult to guarantee reliability in the field.

SoCs tend to be a lot more powerful than microcontrollers, and they have a lot more features. Here are some typical stats:

- 64-bit architecture
- >1 GHz clock speed
- Multiple processor cores
- External RAM and flash (generally multiple gigabytes)
- 2D or 3D graphics processing unit
- Wireless networking
- High-performance digital input and output
- Current draw: hundreds of milliamps at ~5 volts
- Cost: tens of dollars per unit



Performance Characteristics

An average SoC can process audio and high-resolution video using deep learning in near real time.

Despite being far less efficient than microcontrollers, SoCs have been revolutionary. They allow the power of a formidable general-purpose computer to be deployed in an extremely small form factor. In the modern world, SoCs are pervasive—they power our mobile phones, televisions, car entertainment systems, industrial hardware, security systems, IoT gateways, and pretty much anything else that requires flexible computational power in a small package.

Their power, flexibility, and ease of use makes them especially valuable for edge AI. Developers can use familiar tools to develop applications that run on SoCs, and they have enough memory and processing power to run complex algorithms, such as relatively large deep learning models. There are very few types of edge AI algorithm that will not run on an SoC. Ease of use makes SoCs a great choice for prototyping edge AI applications even if the end goal is to move to cheaper or more efficient hardware.

Well-known SoC products include the [Qualcomm Snapdragon](#) and the [Broadcom BCM58712](#), which is used in the Raspberry Pi development board (mentioned in the sidebar “[Boards and Devices](#)” on [page 79](#)). Many popular SoCs are based on [Arm Cortex-A](#) processor cores.



Embedded Linux

Linux has become a very common choice of operating system for SoC devices. It's open source, which means it's free to use, and has a lot of community support. Being able to use familiar Unix development tools makes it easy for those with Unix experience to work with embedded Linux systems.

Deep Learning Accelerators

Both microcontrollers and SoCs are typically general-purpose computers—they are designed to be as flexible as possible. However, if you're willing to sacrifice some flexibility, it's possible to design integrated circuits that run certain operations *extremely* fast.

With the advent of deep learning (see “[Deep learning](#)” on page 106) on embedded devices, semiconductor companies have started to produce accelerators that can be paired with microcontrollers and SoCs to allow deep learning models to be run faster and more efficiently. The mathematics of deep learning is based around linear algebra, so deep learning accelerators—also known as *neural processing units* (NPU)—are designed to perform linear algebra efficiently.

There are various types of deep learning accelerators with their own trade-offs between energy use and flexibility. At one end of the spectrum, devices like [Syntiant's NDP10x series](#) have hardware implementations of specific deep learning model architectures (we'll learn more about these later) that can run quickly with incredibly low energy. Since the algorithm itself is baked into the silicon, these devices are not very flexible—but they can be extremely efficient.

At the other end of the spectrum, devices based on graphics processing unit (GPU) technology, like [Nvidia's Jetson](#) and [Google's Coral](#), offer a huge amount of flexibility and can run basically any type of deep learning model. The trade-off for this flexibility is that they are nowhere near as energy efficient.

Between the two ends of the spectrum are many different types of devices with varying degrees of flexibility and efficiency—like [Syntiant's NDP120](#) or [Arm's Ethos-U55](#) design.

Some types of accelerator use alternatives to traditional deep learning mathematics. For example, [BrainChip's Akida](#), described as a neuromorphic processor, uses spiking neural networks (see “[Compression and optimization](#)” on page 117) to deliver a unique set of trade-offs, including greater energy efficiency.



Performance Characteristics

Deep learning accelerators tend to be extremely fast—you can expect enough computational power to process audio and video in real time. Some devices can even process multiple streams in parallel.

Generally, deep learning accelerators are paired with either microcontrollers or SoCs. The conventional processor runs the application logic, and the accelerator runs the deep learning workload. Many designs combine the microprocessor and accelerator in a single package and provide special tools to help developers split the processing between them.

Early deep learning accelerators provided very little freedom of choice with regards to the types of deep learning models that are supported, but as the field matures devices are becoming more flexible. We're still in the very early days, so you can expect big advances and efficiency gains over time. In the long term, expect absurdly capable devices with miniscule power budgets—real-time video processing or language transcription that can run for years on a small battery.

FPGAs and ASICs

For the ultimate performance and efficiency benefits, designing your own processor circuit is an option. It's difficult, time consuming, and expensive, so it isn't something to be taken lightly, but for certain applications it might make sense.

Field programmable gate arrays, or FPGAs, are silicon integrated circuits that can be reprogrammed on demand to implement custom hardware designs. They allow engineers to create a custom processor design that implements a specific algorithm as efficiently as possible, then load it onto a device for deployment. The designs are created using special programming languages called hardware description languages, or HDLs.

Application-specific integrated circuits, or ASICs, are integrated circuits that are customized for particular applications. Unlike FPGAs, they can't be reprogrammed—their logic is permanently written into silicon. You can buy predesigned ASICs designed for specific purposes, such as **Himax's WE-I Plus**, or design your own.

Development with FPGAs is substantially cheaper than with ASICs, but the per-device cost is higher, as is the power consumption. It's common for companies to use FPGAs for prototypes or small production runs and ASICs for high volume. The engineering cost of creating an ASIC puts them out of reach for the majority of companies.

FPGA developer tools are becoming easier to use and more accessible, but they're still a relatively niche option in edge AI. Researchers are working on tooling that can automatically convert deep learning models into efficient FPGA implementations, so it's likely that FPGAs will play an increasingly large role in edge AI over time. Here are some interesting projects in the space at the time of publication:

- Google's [CFU Playground](#), which helps developers create deep learning accelerators using FPGAs
- [Tensil.ai](#), a machine learning model compiler and hardware generator for FPGAs

Boards and Devices

A processor isn't much use by itself—it needs to be mounted onto a board along with the other components that make up a full device—power supply, sensors and peripherals, and connectors. Most mass-produced edge AI products use custom printed circuit boards designed for their specific application.

Since these custom boards take time to design and produce, a lot of early engineering work is done with *development boards*, also known as *dev boards*. These are ready-to-use devices, sold by hardware manufacturers, that feature a given processor along with everything needed to connect to it and develop software (see [Figure 3-6](#)).

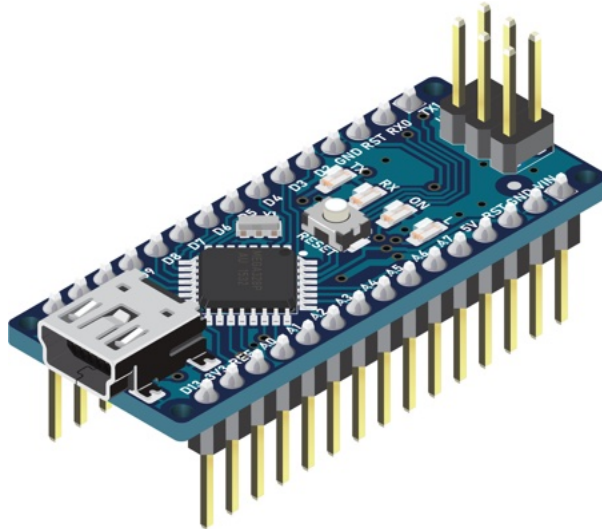


Figure 3-6. A typical dev board features a processor, power supply, input and output pins, connectors, and often some sensors that are connected to the processor and ready to use

They allow embedded engineers to evaluate various processors for a particular use case, and to rapidly build prototypes.

The computational requirements of edge AI algorithms means there's an interplay between hardware and algorithm choice, which makes development boards extremely valuable. With access to a few development boards, developers can quickly test their algorithms on real processors and find the ideal balance between performance, energy usage, and cost.

While traditionally only used during early prototyping, some manufacturers have realized the potential benefits of development boards for production use. If you're making a small batch of hardware, it may not be worth the extra cost and time to design a custom printed circuit board—which would only be economical if produced at large volume. Instead, you might choose to use a predesigned platform such as the **Arduino Portenta**, which features an MCU and a flexible set of inputs and outputs that allow you to easily integrate it into other systems.

These types of devices are available for SoCs and accelerators, too—for example, **Raspberry Pi** produce a range of fully integrated single board computers (SBCs) (see **Figure 3-7**) based on powerful SoCs, and **Nvidia's Jetson accelerators** allow developers to quickly run code on accelerator hardware. Many of these platforms provide a range of compatible devices, so you can prototype using a single board computer and then deploy to a system on module (SOM) designed to integrate into your own hardware without any code changes.

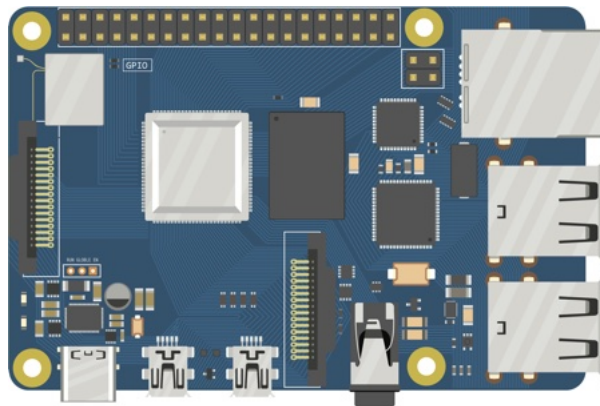


Figure 3-7. A single board computer includes a processor, memory, power supply, input and output connectors, network interfaces, and everything else required for plug and play use

Dev boards are typically just bare circuit boards without any enclosure, so they can't be used in the field without at least a bit of design work. If you'd prefer a fully finished device, industrial IoT gateways place SoCs in rugged enclosures with standard I/O ports, networking hardware, and power supply. They can be fairly expensive, but

they may save time and money versus designing and manufacturing a full piece of hardware.

The most common type of prebuilt edge AI device—by far—is the smartphone. There are entire books on integrating AI into smartphone apps, such as Laurence Moroney's *AI and Machine Learning for On-Device Development* (O'Reilly, 2021), so we won't be covering that topic here. More within our scope is the use of MCUs and DSPs to power specific smartphone features, such as digital assistants that wake up when specific keywords are spotted.

Beyond their integrated edge AI, smartphones can be a handy tool for edge AI developers who are prototyping applications. Since they are battery powered and have great connectivity, they can be useful for collecting initial data or testing out early versions of machine learning models during the initial stages of development, when proving feasibility is the most important thing.

Edge Servers

At the other end of the spectrum from custom silicon, it's possible to run conventional server hardware—the same that might be deployed in a data center—at the edge of the network. These powerful computers run full-scale server operating systems (typically Linux or Windows) and can be treated in the same way as any other cloud server. If they have access to AI-specific acceleration, it's likely in the form of GPUs. Some edge servers are sold in ruggedized form factors that are better suited to industrial settings (like a factory floor) than their data center dwelling equivalents.

The power of edge servers means that they can provide many of the benefits of cloud compute while maintaining the security, privacy, and convenience that comes with keeping data on-site. For some applications, they can provide the best of both worlds—high-capability hardware, low latency, reduced risk of data leakage, and economic use of bandwidth.

Another benefit of edge servers is that they can be treated as essentially just another piece of standard IT infrastructure. This means they can fit neatly into the procedures and skillsets of an existing IT department. In fact, it wasn't long ago that *all* commercial compute was done with on-premises servers. Edge compute used to be the norm for every business.

Edge servers have two major downsides: they use huge amounts of energy and they are very large. If you need vast amounts of compute located conveniently on-site, these trade-offs can be worth it. However, they are typically limited to fixed locations such as buildings and factories where there is spare room and a reliable power supply.

If full-sized edge servers feel like overkill for your application (and they probably are), Linux SoCs offer a great compromise. As standard Linux boxes, an IT

department can treat them like any other server—but they are available in tiny, power-efficient forms.

Multi-Device Architectures

Edge AI applications aren't always implemented directly on the devices that host the actual sensors. Sometimes, it makes sense to use a multi-device architecture. For example, sensors on a fleet of shipping pallets might use low-power radio to report data back to a gateway device mounted in a truck. The gateway, with less constraints on energy use and insight into the data from multiple pallets, could run the sophisticated edge AI logic that makes decisions with the data. **Figure 3-8** shows how this might look.

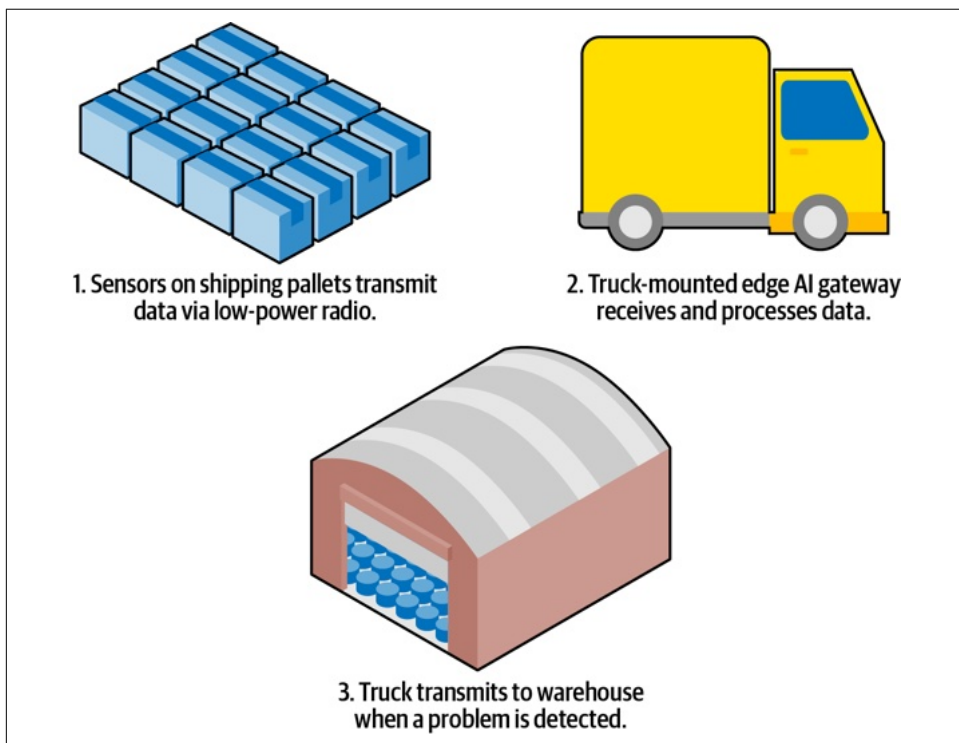


Figure 3-8. It's common to see architectures involve multiple devices

Things can get even funkier with heterogeneous compute (see the sidebar “**Heterogeneous Compute**” on page 75) in the mix. A single device might contain multiple types of processors: for example, one for running application code and another for running ML algorithms. A complete system might be composed of many devices, some with multiple processors, that collect and process data at many different points

depending on which BLERP benefits are needed. This type of solution can even involve cloud computation.

A great example of this type of architecture is a smart speaker with a voice assistant. Typically, they have at least two processors. The first is a low-power, always-on chip that runs DSP and a machine learning model to listen for wake words without using too much energy.

The second is an application processor, which is woken up by the always-on chip when the wake word is detected. The application processor might run a more sophisticated model to try to catch any false positives that got past the always-on chip. Together, these two processors can identify wake words without violating user privacy by streaming private conversations to the cloud.

Once the wake word has been confirmed, the application processor streams the audio to a cloud server, which performs speech recognition and natural language processing in order to come up with an appropriate response. The general flow is shown in [Figure 3-9](#).

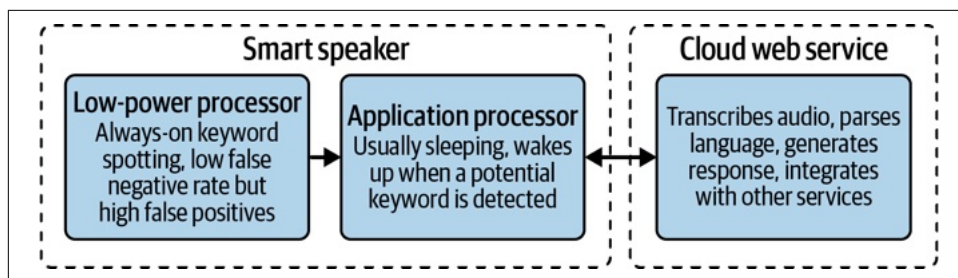


Figure 3-9. The low-power processor aims to catch as many potential keywords as possible; the application processor wakes up to evaluate any possible matches and invokes the cloud web service if a match is confirmed

When designing a system, don't be afraid to consider using multiple devices to tackle some of the trade-offs involved with different device types. Some common situations where it can be helpful are:

- Monitoring large numbers of individual entities: this can get expensive if high-end AI-capable hardware is used on every entity.
- Reducing energy use: sensors are battery powered and need to last a long time.
- Protecting privacy: sending data directly to a large device or cloud server might violate privacy norms.
- Integrating with legacy equipment: existing sensors or gateways might be supplemented with edge AI devices rather than being replaced.

Devices and Workloads

It’s important to understand what each type of device is capable of. [Table 3-1](#) provides a quick reference you can use to break down which types of devices are capable of processing which types of data. It shows the level of support for each data type on a given device: *Full*, *Limited*, or *None*.

However, bear in mind that each category is broad, and every individual device is unique. Not all high-end MCUs are the same. It’s also worth noting that the state of the art moves *fast* and that this reference may quickly become outdated!

Table 3-1. Data types and devices

Device type	Low-frequency time series	High-frequency time series	Audio	Low-resolution image	High-resolution image	Video
Low-end MCU	Limited	Limited	None	None	None	None
High-end MCU	Full	Full	Full	Full	Limited	Limited
High-end MCU with accelerator	Full	Full	Full	Full	Full	Limited
DSP	Full	Full	Full	Full	Limited	Limited
SoC	Full	Full	Full	Full	Full	Full
SoC with accelerator	Full	Full	Full	Full	Full	Full
FPGA/ASIC	Full	Full	Full	Full	Full	Full
Edge server	Full	Full	Full	Full	Full	Full
Cloud	Full	Full	Full	Full	Full	Full

Summary

This chapter has introduced the key hardware that hosts artificial intelligence on the edge and the sensors that keep it fed with data. In the next chapter, we’ll learn about the algorithms that make it work.

Algorithms for Edge AI

There are two main categories of algorithms that are important in edge AI: feature engineering and artificial intelligence. Both types have numerous subcategories; in this chapter we're going to explore a cross-section of them.

The goal is to provide an overview for each algorithm type from an engineering perspective, highlighting their typical usage, strengths, weaknesses, and suitability for deployment on edge hardware. This should give you a place to start when planning real-world projects, which we'll walk through in the coming chapters.

Feature Engineering

In data science, feature engineering is the process of turning raw data into inputs usable by the statistical tools we use to describe and model situations and processes. Feature engineering involves using your domain expertise to understand which parts of the raw data contain the relevant information, then extracting that signal from the surrounding noise.

From an edge AI perspective, feature engineering is all about transforming raw sensor data into usable information. The better your feature engineering, the easier life is for the AI algorithms that are attempting to interpret it. When working with sensor data, feature engineering naturally makes use of digital signal processing algorithms. It can also involve chopping the data into manageable chunks.

Working with Data Streams

As we've seen, the majority of sensors produce time series data. The goal of an edge AI application is to take these streams of time series data and make sense of them.

The most common way to manage streams is to chop a time series into chunks, often called windows, then analyze the chunks one at a time.¹ This produces a time series of results that you can interpret in order to understand what is going on. [Figure 4-1](#) shows how a window is taken from a stream of data.

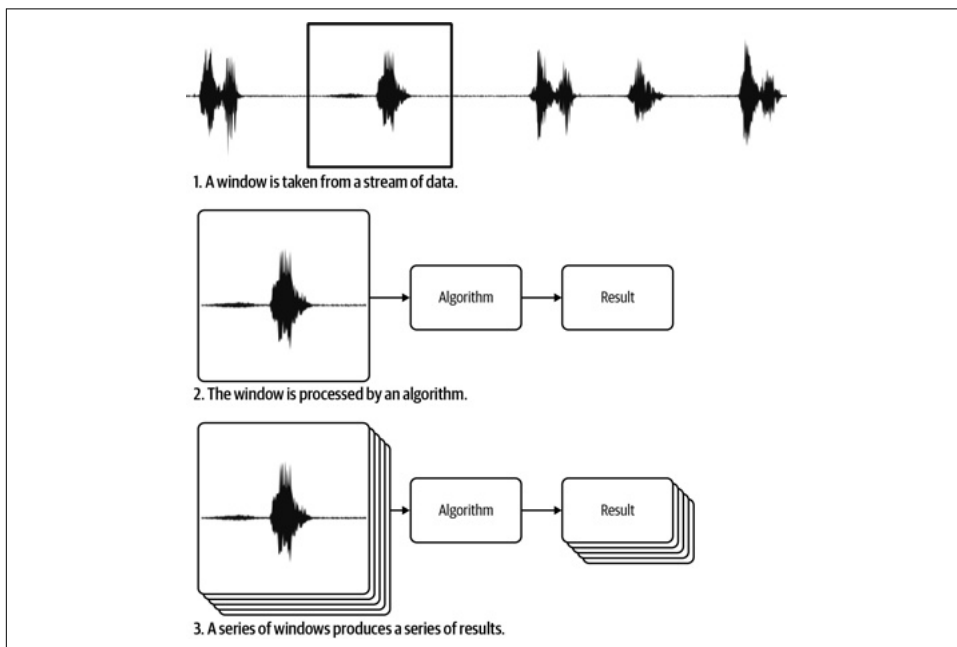


Figure 4-1. A time series is often broken into chunks, called windows, which are analyzed one at a time

It takes a certain amount of time to process a single chunk of data—we can call this the *latency* of our system. This limits how often we can take and process a window of data. The rate at which we can capture and process data is known as the *frame rate* of a system, often expressed in the number of windows that can be processed per second. Frames may be sequential or they may overlap, as shown in [Figure 4-2](#).

¹ The chunks can be discrete or overlapping, or even have gaps between them.

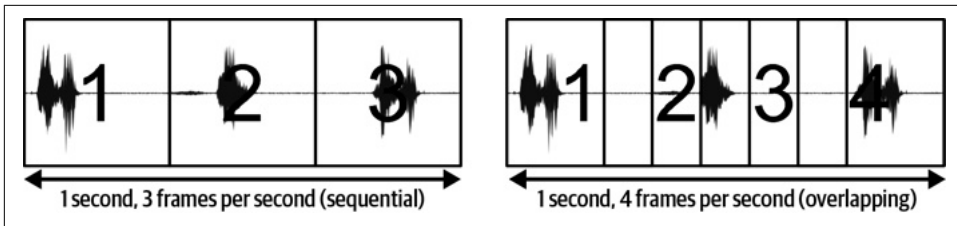


Figure 4-2. Depending on the frame rate, windows can potentially overlap; overlapping is desirable for data that contains events because it increases the chance that an entire event will fall within a window, rather than being cut short

The lower the latency, the more windows of data can be analyzed in a given period of time. The more analysis you can do, the more reliable the results. For example, imagine we are using a machine learning model to recognize a command. If the windows are too far apart, we might miss critical parts of a spoken command and not be able to recognize it (see [Figure 4-3](#)).

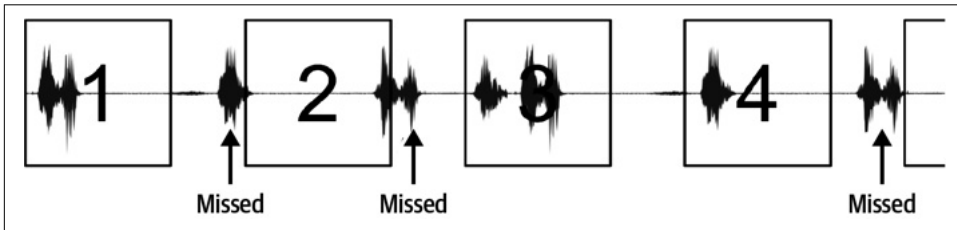


Figure 4-3. If the frame rate is too low, some parts of the signal will not be processed; if you are trying to detect short-lived events, this might mean that some events are missed

The choice of window size is very important. The larger the window, the longer it takes to process the data within it. However, larger windows contain more information about the signal—meaning they may make life easier for the signal processing and AI algorithms being used. The trade-off between window size and frame rate is an important thing to explore when you are developing a system.

As we'll see later, there are many different AI algorithms—and some of them are more sensitive to window size than others. Some algorithms (typically those that maintain an internal memory of what is occurring in a signal) are able to work well with very small window sizes, while others require large window sizes in order to properly parse a signal. Algorithm choice also impacts latency, which also constrains window size. It's a complex system of trade-offs between window size, latency, and algorithm choice.

Windowing also applies to video streams: in this case, each “window” of the video is a certain number of still images—typically a single one, but some AI algorithms can potentially analyze several images at the same time.

More sophisticated techniques for dealing with streams of data fall into the category of digital signal processing. These techniques can be combined with windowing in order to create data that feeds AI algorithms.

Digital Signal Processing Algorithms

There are hundreds of different signal processing algorithms that can help digest the signals produced by sensors. In this section, we'll cover some of the DSP algorithms that are most important for edge AI.

Resampling

All time series signals have a sample rate (also known as a frequency), often described in terms of the number of data samples per second (Hz). It's often necessary to change the sample rate of a signal. For example, you might want to reduce the rate of a signal (known as *downsampling*) if it is producing data faster than you can process it. On the other hand, you may want to increase the rate of a signal (*upsampling*) so that it can be conveniently analyzed alongside another signal that has a higher frequency.

Downsampling works by “throwing away” some of the samples in order to achieve the target frequency. For example, if you threw away every other frame of a 10 Hz (10 samples per second) signal it would become a 5 Hz signal. However, due to a phenomenon called aliasing, reducing the frequency in this way can lead to distortion in the output. To help combat this, signals must have some high-frequency information removed before they are downsampled. This is achieved using a low-pass filter, described in the next section.

Upsampling works in the opposite way—new samples are created and inserted to increase the frequency of a signal. For example, if an extra sample was inserted after every sample in a 10 Hz signal, it would become a 20 Hz signal. The difficult part is knowing what to insert! There's no way to know what would actually have been happening during the time between two samples, but a technique known as *interpolation* can be used to fill in the blanks with an approximation.

In addition to time series, images can also be upsampled and downsampled. In this case, it's the spatial resolution (pixels per image) that is being increased or decreased. Like time series resampling, the resizing of images also requires anti-aliasing or interpolation techniques.

Both upsampling and downsampling are important, but downsampling is more commonly encountered in edge AI. It's typical for sensors to produce an output at a set frequency, leaving it to the developer to downsample and obtain the frequency that best suits the rest of their signal processing pipeline.

For edge AI applications, upsampling is mostly useful if you wish to combine two signals with different frequencies into a single time series. However, this can also be achieved by downsampling the higher frequency signal, which might be computationally cheaper.

Resizing and Cropping Images

Different models of image sensors output images with varying sizes and shapes, and edge AI algorithms (such as deep learning vision models) often require images of very specific sizes. Cropping and resizing is commonly used to make images compatible with models and can involve both downsampling and upsampling, as well as throwing chunks of an image away.

Figure 4-4 shows some common ways that images are resized and cropped to fit a required input shape.

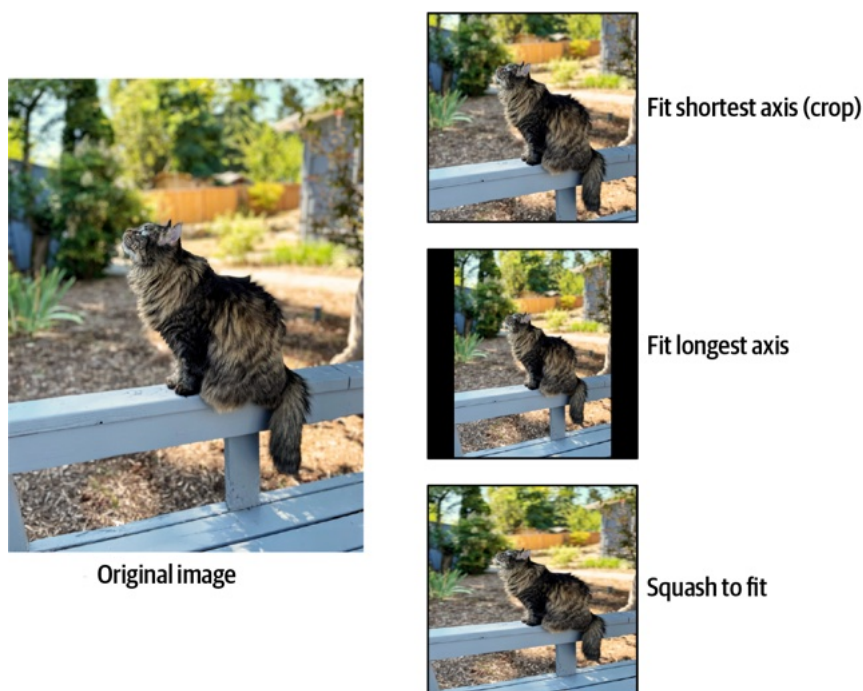


Figure 4-4. Three different ways that a rectangular image can be made to fit a square input shape

Filtering

A digital filter is a function that, applied to a time series signal, transforms it in certain ways. Many different types of filters exist, and they can be very useful in preparing data for edge AI algorithms.

Low-pass filters are designed to allow low-frequency elements of a signal to pass through, while removing high-frequency elements. The *cutoff frequency* of the filter describes the frequency beyond which high-frequency signals will be affected, and the *frequency response* describes how much those signals will be affected.

High-pass filters are the same thing in reverse, allowing frequencies *above* a cutoff frequency to pass, and attenuating (reducing) those below. A band-pass filter combines the two, allowing frequencies within a certain *band* but attenuating those outside of it.

The purpose of filtering in edge AI is to isolate the useful parts of a signal, removing parts that do not contribute to solving the problem. For example, a speech recognition application could use a band-pass filter to allow frequencies in the normal range of human speech (125 Hz to 8 kHz) while rejecting information in other frequencies. This could make it easier for a machine learning model to interpret the speech without being distracted by other information in the signal.

Filtering Noise

All signals from sensors contain some level of noise: random fluctuations in the data that happen due to slight inaccuracies in measurement. The background hum in audio recordings, or the speckles in a digital camera photograph taken at night, are typical examples of noise.

If the noise is present at specific frequencies, which is quite common, filters can be very useful in removing it. This can make it easier for some AI algorithms to interpret signals. However, some types of algorithms—such as deep learning models—are naturally able to cope with noise, so it isn't always necessary to filter it.

Filters can be applied to any type of data. For example, if a low-pass filter is applied to an image, it has a blurring or smoothing effect. If a high-pass filter is applied to the same image, it will “sharpen” details.

One type of low-pass filter is a *moving average filter*. Given a time series, it calculates a moving average of values within a certain window. In addition to smoothing the data, it has the effect of making a single value represent information from a wide range of time.

If several moving averages are calculated and stacked together, each with differing window lengths, a momentary snapshot of the signal (containing several different moving averages) contains information about changes in the signal across a window of time and a number of different frequencies. This can be a helpful technique in feature engineering, since it means an AI algorithm can observe a broad window of time using relatively few data points.

Filtering is an extremely common signal processing operation. Many embedded processors provide hardware support for some types of filtering, which reduces latency and energy usage.

Spectral analysis

A time series signal can be said to be in the *time domain*, meaning it represents how a set of variables change over time. Using some common mathematical tools, it's possible to transform a time series signal into the *frequency domain*. The values obtained through transformation describe how much of the signal lies in various frequency bands over a range of frequencies—a spectrum.

By slicing a signal into multiple, thin windows and then transforming each window into the frequency domain, as shown in [Figure 4-5](#), it's possible to create a map of how the signal's frequencies change over time. This map, known as a spectrogram, serves as a very effective input to machine learning models.

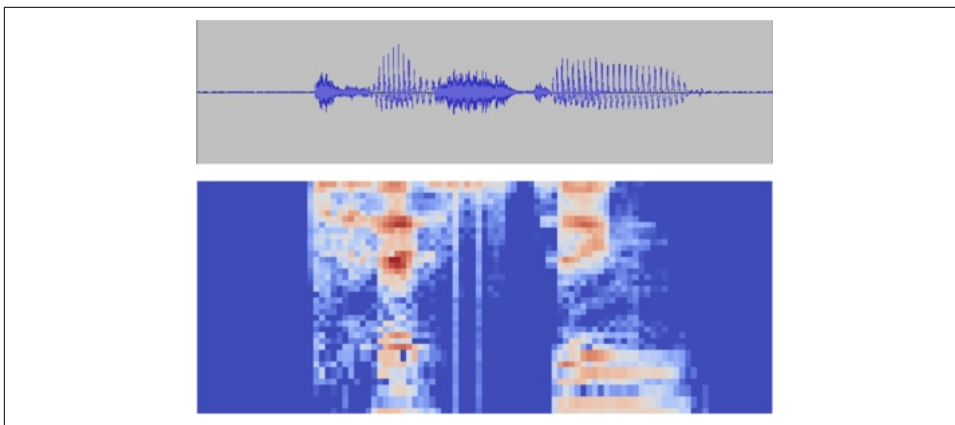


Figure 4-5. The same clip of audio represented as a waveform in the time domain (top) and a spectrogram in the frequency domain (bottom)

Spectrograms are commonly used in real-world applications, especially around audio. Separating the data into windowed frequency bands allows relatively small and simple models to interpret it.² It's also possible for humans to visually distinguish one word from another while looking at spectrograms—some people have even learned to read them.

There are many algorithms that can transform a signal from the time to the frequency domain, but the most common is the Fourier transform. It's a very common operation, and there's often hardware support (or at least optimized implementations) available for performing Fourier transforms on embedded devices.

There are a huge number of algorithms and techniques for digital signal processing and time series analysis; they're major fields of engineering and study. Some great resources on the subjects are:

- *The Scientist and Engineer's Guide to Digital Signal Processing*, by Steven W. Smith (California Technical, 1997)
- *Practical Time Series Analysis*, by Aileen Nielsen (O'Reilly, 2019)

Image feature detection

A whole subset of signal processing algorithms are concerned with the extraction of useful features³ from images. These have traditionally been referred to as *computer vision* algorithms. Some common examples include:

Edge detection

Used to identify boundaries in an image (see [Figure 4-6](#))

Corner detection

Used to find points in an image that have an interesting two-dimensional structure

Blob detection

Used to identify regions of an image that have something in common

Ridge detection

Used to identify curves within an image

² One reason for this is that the raw audio shown in [Figure 4-5](#) consists of 44,100 samples, while the equivalent spectrogram only has 3,960 elements. The smaller input means a smaller model.

³ In image processing, a feature is a particular piece of information about an image, such as the positions of certain visual structures. The Wikipedia page titled "[Feature \(computer vision\)](#)" lists many common image features.



Figure 4-6. Edge detection algorithms find boundaries between areas with different colors or intensities

Image feature detection reduces a big, messy image into a more compact representation of the visual structures that are present within it. This can potentially make life easier for any AI algorithms that are operating downstream.

Feature detection is not always necessary when working with images. Typically, deep learning models are able to learn their own ways of extracting features, reducing the utility of preprocessing. However, it's still common to perform feature detection when interpreting image data using other types of edge AI algorithm.

The **OpenCV project** provides a set of libraries for feature detection (and other image-processing tasks) that will run on most SoC devices. For microcontrollers, **OpenMV** provides an open source library of feature detection algorithm implementations along with hardware designed to run them.

Combining Features and Sensors

There's nothing stopping you from combining several different features and signals as the input to your AI algorithms. For example, you could calculate several moving averages of a time series over several different windows and pass them all into a machine learning model together. There are no hard-and-fast rules, so feel free to experiment and be creative with the way you slice and dice your data. The following chapters will provide a framework for experimentation.

Going beyond combining features from the same signal, *sensor fusion* is the concept of integrating data from multiple sensors together. For example, an edge AI fitness tracker could combine information from an accelerometer, gyroscope, and heart rate sensor to try to detect which sport a wearer is playing.

In a more complex edge AI scenario, the sensors don't even have to be integrated with the same device. Imagine a smart climate control system that makes use of temperature and occupancy sensors distributed throughout a building to optimize air conditioning usage.

There are three categories of sensor fusion:

Complementary

Where multiple sensors combine to deliver a more complete understanding of a situation than would be possible with a single sensor—for example, the various sensors on our hypothetical fitness tracker.

Competitive

Where multiple sensors measure the same exact thing in order to reduce the likelihood of bad measurements—for example, multiple redundant sensors monitoring the temperature of a critical piece of equipment.

Cooperative

Where information from multiple sensors combines to create a signal that was not otherwise available—for example, two cameras producing a stereo image that provides depth information.

The challenge inherent in sensor fusion is how to combine multiple signals that may even occur at different rates. You should consider the following:

1. Aligning the signals in time. For many algorithms, it's important that all of the signals we intend to fuse are sampled at the same frequency, and that the values reflect simultaneous measurements. This can be achieved through resampling—for example, upsampling a low-frequency signal so that it has the same rate as the high-frequency signal it is being fused with.
2. Scaling the signals. It's critical that the signals' values are on the same scale, so that a signal with typically large values does not overwhelm a signal with typically smaller ones.
3. Numerically combining the signals. This can be done using simple mathematical operations (addition, multiplication, or averaging) or with more sophisticated algorithms such as the Kalman filter (covered later)—or simply by concatenating the data together and passing it into the algorithm as a single matrix.

You can perform sensor fusion before or after other stages of feature engineering. For an arbitrary example: if you intended to fuse two time series, you might choose to run a low pass over one of them first, then scale them to the same scale, combine the two through averaging, and transform the combined values into the frequency domain. Don't be afraid to experiment!

Feature Scaling

A stream of data from a sensor can have a wide range of values. For example, if the sensor returns measurements as 16-bit unsigned integers, their value could be anywhere from 0 to 65,535.

Big ranges like this can make things tricky for some AI algorithms. For example, deep learning models can have a hard time training when their input values have a large magnitude.

Additionally, it can be hard to get good results from machine learning models when passing in features that have wildly different scales. The larger values outweigh the smaller ones, which reduces the benefit of having multiple input features. This is also a problem for sensor fusion.

To get around the issue, it's a very good idea to scale your inputs before combining them or sending them into AI algorithms. A common way to do this is called *normalization*. There are a few different varieties of normalization. In the simplest, known as *rescaling*, you determine the maximum and minimum values for a specific feature in a representative sample of your input data (typically using your training data, if you're working with machine learning models). You can then calculate the normalized values using the following formula:

$$\text{normalized_value} = (\text{raw_value} - \text{minimum}) / (\text{maximum} - \text{minimum})$$

This will provide a value between 0 and 1, which can be conveniently compared and combined with other normalized values on the same scale.

Some other common scaling methods include *mean normalization* and *standardization*. The Wikipedia article “[Feature Scaling](#)” provides a general overview.

One thing to note is that the values you encounter in the real world may turn out to have a different range from those in your training data. To avoid issues, you should clip any that are out of the expected range.

We now have some serious tools for processing data. In the next section, we'll explore the AI algorithms that will help us understand it.

Artificial Intelligence Algorithms

There are two ways to think about AI algorithms. One is based on functionality: what are they designed to do? The other is based on implementation: how do they work? Both aspects are important. Functionality is critical to the application you are trying to build, and implementation is important when thinking about your constraints—which generally means your dataset and the device you will be deploying to.

Algorithm Types by Functionality

First up, let's look at the most important types of algorithm from a functional perspective. Mapping the problem you are trying to solve to these algorithm types is known as *framing*, and we'll be diving deep into framing in [Chapter 6](#).

Classification

Classification algorithms try to solve the problem of distinguishing between various *types*, or *classes*, of things. This could mean:

- A fitness monitor with an accelerometer classifying walking versus running
- A security system with an image sensor classifying an empty room versus a room with a person present
- A wildlife camera classifying four different species of animal

[Figure 4-7](#) shows a classifier being used to determine whether a forklift truck is idle or moving, based on data collected by an accelerometer.



Figure 4-7. A classifier typically outputs a probability distribution including each possible class

Classification can be categorized in a few different ways, depending on the task:

Binary classification

The input belongs to one of two classes.

Multiclass classification

The input belongs to one of more than two classes.

Multilabel classification

The input belongs to zero or more of any number of classes.

The most common forms of classification are binary and multiclass. With these forms of classification, you always need at least two classes. Even if there's only one thing you care about (for example, a person in the room), you also need a class that represents everything that you *don't* care about (for instance, rooms that don't have people in them). Multilabel classification is comparatively rare.

Regression

Regression algorithms try to come up with numbers. This could mean:

- A smart thermostat that predicts the temperature in an hour's time
- A virtual scale that estimates the weight of a food product using a camera
- A virtual sensor that estimates a motor's speed of rotation based on its sound

Virtual sensors, like the latter two examples, are a particularly interesting case of regression. They can use available sensor data to predict measurements from different types of sensor—without actually requiring those sensors to be present.

Object detection and segmentation

Object detection algorithms take an image or video and identify the locations of specific objects within them, often by drawing *bounding boxes* around them. They combine classification and regression, identifying specific types of objects and predicting their numeric coordinates—as seen in [Figure 4-8](#).

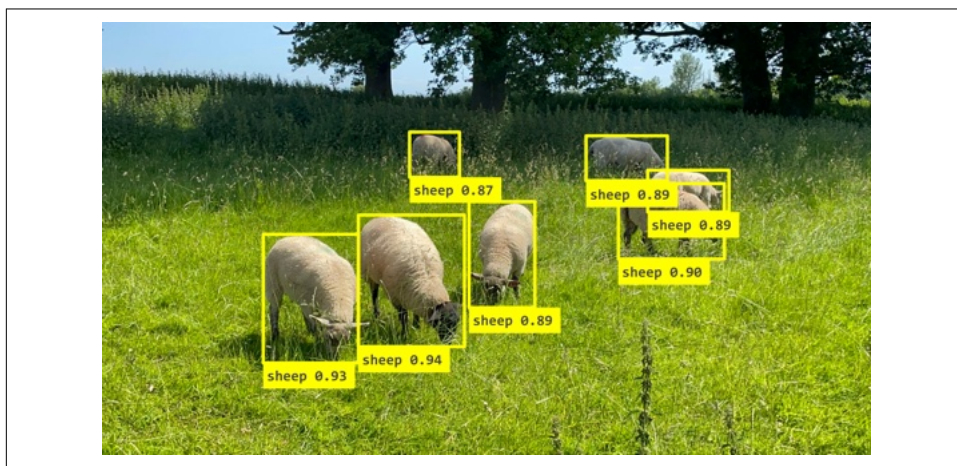


Figure 4-8. A common output for object detection models consists of bounding boxes drawn around detected objects, each with an individual confidence score

Specialized object detection algorithms exist for particular types of objects. For example, pose estimation models are designed to recognize human body parts and identify their locations within an image—as shown in [Figure 4-9](#).



Figure 4-9. Pose estimation identifies key points on a human body, the position of which can be used as input for other processes

Segmentation algorithms are similar to object detection algorithms, but they classify images at a pixel level. This results in a *segmentation map*, as seen in [Figure 4-10](#), which attempts to label areas of the input with their content.

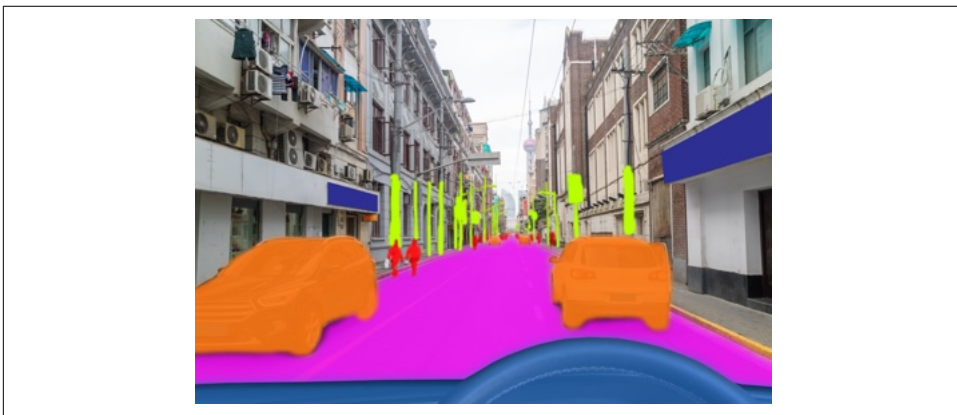


Figure 4-10. This street scene has been labeled with a segmentation map. Different areas, such as people and the road surface, are shown in different shades. A segmentation algorithm aims to predict which pixels belong to which type of object.

Here are some example use cases for object detection and segmentation:

- A farm monitor that uses cameras to count the number of animals in a field
- A home fitness system that gives people feedback on their form during workouts
- An industrial camera that measures how much of a container is filled with product

Anomaly detection

Anomaly detection algorithms recognize when a signal has deviated from its normal behavior. They are useful in many applications:

- An industrial predictive maintenance system that can recognize when a motor has started to break down by its current draw
- A robot vacuum that can identify when it is driving on an unusual surface using an accelerometer
- A trail camera that knows when an unknown animal has walked past

Anomaly detection algorithms are very useful for predictive maintenance. They're also very helpful when paired with machine learning models. Many machine learning models will produce spurious, random results if they are presented with an input that isn't in their training set.

To avoid this, an ML model can be paired with an anomaly detection algorithm that tells it when something is *out of distribution* so that its spurious results can be discarded. Some types of models can also be *calibrated* so that their output represents a true probability distribution that can be interpreted to recognize when the model is uncertain.

Clustering

Clustering algorithms try to group inputs by similarity and can recognize when an input is not similar to what it has seen before. They are often used when an edge AI device needs to learn from its environment, including for anomaly detection applications. For example, consider:

- A voice assistant that learns which voice belongs to each of its users
- A predictive maintenance application that learns a “normal” state of operation and can detect deviations from it
- A vending machine that can recommend drinks based on a user's previous choices

A clustering algorithm can either learn its clusters on the fly (after deployment) or have them configured ahead of time.

Dimensionality reduction

Dimensionality reduction algorithms take a signal and produce a representation of it that contains equivalent information but takes up a lot less space. The representations of two signals can then be compared to one another easily. Here are some example applications:

- Compression of audio, to make it cheaper to transmit sounds from a remote device
- Fingerprint recognition, ensuring a fingerprint matches the owner of a device
- Facial recognition, recognizing individual faces in a video feed

Dimensionality reduction tends to be used alongside other AI algorithms, as opposed to being used on its own. For example, it can be used in conjunction with a clustering algorithm to identify similar signals in complex data types, like audio and video.

Transformation

Transformation algorithms take one signal and output another. Here are some examples:

- Noise-canceling headphones that identify and remove specific noises in a signal
- A car reversing camera that enhances the image in dark or rainy conditions
- A speech recognition device that takes an audio signal and outputs a transcription

The input and output of transformation algorithms can be extremely different. In the case of transcription, the input is a stream of audio data and the output is a sequence of words.



Combining Algorithms

There's no reason you can't mix different types of algorithms in the same application. Later in this section we'll explore techniques for combining algorithms (see [“Combining algorithms” on page 113](#)).

Algorithm Types by Implementation

Exploring algorithms by functionality helps us understand what they are used for, but from an engineering perspective it's important to get a sense for the different ways these functionalities can be implemented. There are hundreds of different ways to build a classification algorithm, for example, resulting from decades of computer science research. Each method has its own unique strengths and weaknesses that are amplified by the constraints posed by edge AI hardware.

In the following section, we'll explore the most important ways that edge AI algorithms are implemented. Bear in mind that this isn't an exhaustive list—we're focused on edge AI, so we're focused on technologies that work well on-device.

Conditionals and heuristics

The simplest type of AI algorithms are based on conditional logic: simple `if` statements that result in decisions. Let's look back at the code snippet we explored in “Artificial Intelligence” on page 6:

```
current_speed = 10 # In meters per second
distance_from_wall = 50 # In meters
seconds_to_stop = 3 # The minimum time in seconds required to stop the car
safety_buffer = 1 # The safety margin in seconds before hitting the brakes

# Calculate how long we've got before we hit the wall
seconds_until_crash = distance_from_wall / current_speed

# Make sure we apply the brakes if we're likely to crash soon
if seconds_until_crash < seconds_to_stop + safety_buffer:
    applyBrakes()
```

This simple algorithm does a basic calculation using some human-defined values (`seconds_to_stop`, etc.) and decides whether to apply a car's brakes. Does this count as AI? It's a question that might stimulate debate—but the answer is emphatically yes.⁴

The common understanding of artificial intelligence is that it's a quest to create machines that can think like human beings. The engineering definition is much more realistic: AI allows computers to do tasks that typically require human intelligence. In this case, controlling a car's brakes to avoid a collision is definitely something that has typically required human intelligence. It would have been considered extremely impressive twenty years ago, but automatic braking is a common feature in modern vehicles.

⁴ There's a well-documented phenomenon known as the “AI effect”, where the moment AI researchers figure out how to make a computer do a task, critics no longer consider that task representative of intelligence.



Before you laugh at the idea that `if` statements can be artificial intelligence, consider that *decision trees*—one of the most popular and effective categories of machine learning algorithms—are just `if` statements under the hood. These days, even deep learning models can be implemented as binary neural networks, which are essentially conditional logic. Intelligence comes from the application, not the implementation!

The conditional logic in our car braking algorithm is actually an implementation of classification. Given an input (the speed of the car and the distance from a wall), the algorithm classifies the situation into one of two types: safe driving or impending crash. Conditional logic is naturally used for classification since its output is categorical; an `if` statement gives us either one output or another.

Conditional logic is connected to the idea of *heuristics*. A heuristic is a handcrafted rule that can be applied to a situation in order to help understand or react to it. For example, our car braking algorithm uses the heuristic that if we have less than four seconds before hitting a wall, we should apply the brakes.

Heuristics are designed by human beings using domain knowledge. This domain knowledge can be built on data that has been collected about a real-world situation. In that respect, our seemingly simple car braking algorithm might actually represent some deep, well-researched understanding of the real world. Perhaps the value of `seconds_to_stop` was arrived at after millions of dollars' worth of crash tests and represents the ideal value for the constant. With this in mind, it's easy to see how even an `if` statement can represent a significant amount of human intelligence and knowledge, captured and distilled into a simple and elegant piece of code.

Our car braking example is very simple—but when paired with signal processing, conditional logic can make some quite sophisticated decisions. For example, imagine you are building a predictive maintenance system that aims to alert workers of the health of an industrial machine based on the sounds it makes. Perhaps the machine makes a characteristic high-pitched whine when it is about to break down. If you capture audio and translate it into the frequency domain using a Fourier transform, you can use a simple `if` statement to determine when the whine is happening and let the workers know.

Beyond `if` statements, you can use more complex logic to interpret situations based on known rules. For example, an industrial machine may use a handcoded algorithm to avoid damage by varying its speed based on measurements of internal temperature and pressure. The algorithm might take the temperature and pressure and directly calculate an RPM, using human insight that is captured in the code.

If it works for your situation, conditional logic and other handcoded algorithms can be amazing. It is easy to understand, easy to debug, and easy to test. There's no risk

of unspecified behavior: the code either branches one way or another, and all paths can be exercised with automated tests. It runs incredibly fast and will work on any imaginable device.

There are two major downsides of heuristics. First, developing them may require significant domain knowledge and programming expertise. Domain knowledge is not always available—for example, a small company might not have the resources to conduct the expensive research necessary to understand the fundamental mathematical rules of a system. In addition, even given domain knowledge, not everyone has the expertise required to design and implement a heuristic algorithm in efficient code.

The second big downside is the idea of *combinatorial explosion*. The more variables that are present in a situation, the more difficult it is to model with traditional computer algorithms. A good example of this is the game of chess: there are so many pieces, and so many possible moves, that deciding what to do next requires a vast amount of computation. Even the most advanced chess computers built using conditional logic can easily be beaten by expert human players.

Some edge AI problems are *far* more complex than games of chess. For example, imagine trying to handwrite conditional logic that can determine whether a camera image shows an orange or a banana. With some tricks (“yellow means banana, orange means orange”) you might succeed for some categories of images—but it would be impossible to make it generalize beyond the simplest of scenes.

A good rule of thumb for handcoded logic is that the more data values you have to deal with, the more difficult it is going to be to get a satisfactory solution. Fortunately, there are plenty of algorithms that can step in when a handcoded approach fails.

Classical machine learning

Machine learning is a special approach to creating algorithms. Where heuristic algorithms are created by handcoding logic based on known rules, machine learning algorithms discover their own rules—by exploring large amounts of data.

The following description, taken from the book *TinyML*, introduces the basic ideas behind machine learning:

To create a machine learning program, a programmer feeds data into a special kind of algorithm and lets the algorithm discover the rules. This means that as programmers, we can create programs that make predictions based on complex data without having to understand all of the complexity ourselves. The machine learning algorithm builds a *model* of the system based on the data we provide, through a process we call *training*. The model is a type of computer program. We run data through this model to make predictions, in a process called *inference*.

—*TinyML* (O'Reilly, 2019)

Machine learning algorithms can perform all of the functional tasks described earlier in this chapter, from classification to transformation. The key requirement for using machine learning is that you have a *dataset*. This is a large store of data, generally collected under real-world conditions, that is used to train the model.

Typically, the data needed to train a machine learning model is gathered during the development process, aggregated from as many sources as possible. As we'll see in later chapters, a large and varied dataset is critical for working with edge AI—but especially machine learning.

Since machine learning depends on large datasets, and because *training* a machine learning model is computationally expensive, the training part generally happens before deployment, with *inference* happening on the edge. It's certainly possible to train machine learning models on-device, but the lack of data combined with the small amount of compute make it a challenge.

In edge AI, there are two main ways to work with machine learning datasets:

Supervised learning

Where the dataset has been *labeled* by an expert to assist the machine learning algorithm in understanding it

Unsupervised learning

Where the algorithm identifies structures in the data without human help

Machine learning has a major dataset-related drawback. ML algorithms depend entirely on their training data to know how to respond to inputs. As long as they are receiving inputs that are similar to their training data, they should work well. However, if they receive an input that is significantly dissimilar from their training dataset—known as an *out-of-distribution* input—they will produce an output that is completely useless.

The tricky part is that there is no obvious way of telling, from the output, that an input was out of distribution. This means that there's always a risk that a model is providing useless predictions. Avoiding this problem is a core concern when working with machine learning.

There are many different types of machine learning algorithms. *Classical* machine learning encompasses the vast majority of them used in practice, with the major exception of *deep learning* (which we'll explore in the next section).

Interpretability and Explainability

When a machine learning model makes a prediction, it's great if we can also understand *why* it made that particular prediction—as opposed to a different one. The property of making human-comprehensible decisions is known as *interpretability* or *explainability*.

Some machine learning algorithms are more interpretable than others. Whether this is important or not depends on your use case. For example, if a machine learning model is being used to assist with medical diagnosis, doctors may not trust it if it can't explain its predictions.

Interpretable algorithms are easier to work with, since debugging them is straightforward—if they produce an incorrect output, you can directly understand why and attempt to address it.

Here are some of the most useful types of classical ML algorithms for edge AI. The title indicates whether they are supervised or unsupervised algorithms:

Regression analysis (supervised)

Learns the mathematical relationships between input and output to predict a continuous value. Easy to train, fast to run, low data requirements, and highly interpretable, but can only learn simple systems.

Logistic regression (supervised)

A classification-oriented type of regression analysis, logistic regression learns the relationship between input values and *categories* of output—for relatively simple systems.

Support vector machine (supervised)

Uses fancy mathematics to learn much more complex relationships than basic regression analysis. Low data requirements, fast to run, can learn complex systems, but difficult to train and low interpretability.

Decision trees and random forests (supervised)

Uses an iterative process to construct a series of *if* statements that predict an output category or value. Easy to train, fast to run, highly interpretable, can learn complex systems but may require a lot of training data.

Kalman filter (supervised)

Predicts the next datapoint given a history of measurements. Can factor in multiple variables to improve precision. Often trained on-device, low data requirements, fast to run, easy to interpret, but can only model relatively simple systems.

Nearest neighbors (unsupervised)

Classifies data by how similar it is to known data points. Often trained on-device, low data requirements, easy to interpret, but can only model relatively simple systems and can be slow with lots of data points.

Clustering (unsupervised)

Learns to group inputs by similarity but does not require labels. Often trained on-device, low data requirements, fast to run, easy to interpret, but can only model relatively simple systems.

Classical ML algorithms are an incredible set of tools for interpreting the output of your feature engineering pipeline and making decisions with data. They cover the spectrum from highly efficient to highly flexible, and they can perform many functional tasks. Another major benefit is that they tend to be very explainable—it's easy to understand how they are making their decisions. And depending on the algorithm, the data requirements can be quite low (deep learning typically requires very large datasets).

The diverse pool of classical ML algorithms (there are literally hundreds) are both a blessing and a curse for edge AI. On the one hand, there are algorithms well suited to many different situations, which makes it possible to find one that is—theoretically—ideal for a particular use case. On the other hand, the large constellation of algorithms can be challenging to explore.

While libraries like **scikit-learn** make it easy to try out many different algorithms, there's an art and a science to tuning each one to perform optimally, and to interpreting their results. In addition, if you're hoping to deploy to a microcontroller, you may have to write your own efficient implementation of an algorithm—there are not many open source versions available yet.

A major downside of classical ML algorithms is that they run into a relatively low ceiling in terms of complexity of the systems they can model. This means that to get the best results, they often have to be paired with heavy feature engineering—which can be complex to design and computationally costly. Even with feature engineering, there are some tasks—such as the classification of image data—where classical ML algorithms just don't perform well.

That said, classical ML algorithms are a fantastic set of tools for making on-device decisions. But if you hit their limitations, deep learning might help.

Deep learning

Deep learning is a type of machine learning that focuses on neural networks. These have proven such an effective tool that deep learning has grown into a gigantic field, with deep neural networks being applied to many types of application.



This book focuses on the important properties of deep learning algorithms from an engineering perspective. The underlying mechanics of deep learning are interesting, but they're not required knowledge for building an edge AI product. Using modern tools, any engineer can deploy deep learning models without a formal background in machine learning. We'll share some of the tools for doing that in the tutorial chapters later on.

Deep learning shares the same principles as classical ML. A dataset is used to train a model, which can be implemented on a device to perform inference. There isn't anything magical about a model—it's just a combination of an algorithm and a collection of numbers that are fed into it, along with the model's input, in order to produce the desired output.

The numbers in the model are called *weights*, or *parameters*, and they're generated during the training process. The term *neural network* refers to the way that the model combines its input with its parameters, which was inspired by the way neurons in an animal brain connect to one another.

Many of the most mind-blowing feats of AI engineering that we've seen over the past decade have made use of deep learning models. Here are some popular highlights:

- **AlphaGo**, a computer program that used deep learning to beat the best players at **Go**, an ancient game once thought impossible for computers to master
- **GPT-3**, a model that can generate written language that is indistinguishable from human writing
- **Fusion reactor control**, using deep learning to control the shape of plasma within a fusion reactor
- **DALL•E**, a model that can generate realistic images and abstract art based on text prompts
- **GitHub Copilot**, software that assists software engineers by automatically writing code

Beyond the fancy stuff, deep learning excels at all of the tasks in our subsections of algorithm types (see “**Algorithm Types by Functionality**” on page 96). It has proven to be flexible, adaptable, and an incredibly useful tool in allowing computers to understand and influence the world.

Deep learning models are effective because they work as *universal function approximators*. It's been mathematically proven that, as long as you can describe something as a continuous function, **a deep learning network can model it**. This basically means that for any dataset that shows various inputs and desired outputs, there's a deep learning model out there that can convert one into the other.

A really exciting result of this ability is that during training, deep learning models can figure out how to do their own feature engineering. If a special transformation is needed to help interpret the data, a deep learning model can potentially learn how to do it. This doesn't make feature engineering obsolete, but it definitely reduces the burden on the developer to get things exactly right.

The reason deep learning models are so good at approximating functions is that they can have very large numbers of parameters. With each parameter, the model gets a little bit more flexibility, allowing it to describe a slightly more complex function.

This property leads to the two major drawbacks of deep learning models. First, finding the ideal values for all of these parameters is a difficult process. It involves training a model with lots of data. Data is often a rare and precious resource, difficult and expensive to obtain, so this can be a major obstacle. Fortunately, there are many techniques that can help make the most of limited data—we'll cover them later in the book.

The second major drawback is the risk of *overfitting*. Overfitting is when a machine learning model learns a dataset *too* well. Instead of modeling the general rules that lead from outputs to inputs in its dataset, it memorizes the dataset completely. This means that it won't perform well on data that it hasn't seen before.

Overfitting is a risk with all machine learning models, but it's especially a challenge for deep learning models because they can have so many parameters. Each additional parameter provides the model with slightly more ability to memorize its dataset.

There are a lot of different types of deep learning models. Here are some of the most important for edge AI:

Fully connected models

The simplest type of deep learning model, fully connected models consist of stacked *layers* of *neurons*. The input of a fully connected model is fed directly in as a long series of numbers. Fully connected models are capable of learning any function, but they are mostly blind to spatial relationships in their inputs (for example, which values in an input are next to one another).

In an embedded context, this means they work well for discrete values (for example, if the input features are a set of statistics about a time series) but they aren't as great with raw time series or image data.

Fully connected models are very well supported on embedded devices, with hardware and software optimizations commonly available.

Convolutional models

Convolutional models are designed to make use of the spatial information in their inputs. For example, they can learn to recognize shapes in images, or the structures of signals within time series sensor data. This makes them extremely useful in embedded applications since spatial information is important in so many of the signals we deal with.

Like fully connected models, convolutional models are very well supported on embedded devices.

Sequence models

Sequence models were designed originally for use on sequences of data, like time series signals or even written language. To help them recognize long-term patterns in time series, they often include some internal “memory.”

It turns out that sequence models are very flexible, and there’s increasing evidence that they can be very effective on any signal where spatial information is important. Many people believe they will eventually take over from convolutional models.

Sequence models are currently less well supported than convolutional and fully connected models on embedded devices; there are few open source libraries that provide optimized implementations for them. This is more due to inertia than technical limitations, so the situation is likely to change over the next couple of years.

Embedding models

An embedding model is a pretrained deep learning model that is designed for dimensionality reduction—it takes a big, messy input and represents it as a smaller set of numbers that describes it within a certain context. They are used in the same way a signal processing algorithm would be: they produce features that can be interpreted by another ML model.

Embedding models are available for many tasks, from image processing (turning a big messy image into a numeric description of its contents) to speech recognition (turning raw audio into a numeric description of the vocal sounds within it).

The most common use for embedding models is *transfer learning*, which is a way of reducing the amount of data required to train a model. We’ll learn more about that later.

Embedding models can be fully connected, convolutional, or sequence models, so their support on embedded devices varies—but convolutional embedding models are the most common.

Model Architectures

Deep learning models are flexible and modular—they are composed of *layers* and *operations* (also known as *ops*) that can be stacked and combined in an infinite number of ways.

Different arrangements are known as *architectures*, and many architectures have been designed that are optimized for different tasks. You'll often see references to deep learning model architectures in online articles and scientific literature.

Some noteworthy architectures for edge AI include:

- MobileNet and EfficientNet, families of architectures designed to run efficiently on mobile devices
- YOLO, a family of architectures designed to perform object detection
- Transformers, a family of architectures designed to translate between sequences of data

It's only in recent years that deep learning models have been brought to edge AI hardware. Since they are often large and involve significant computation to run, it's been the advent of high-end MCUs and SoCs with relatively powerful processors and large amounts of ROM and RAM that have enabled them to make the leap.

It's possible to run a small deep learning model using just a few kilobytes of memory, but for models that do more complex things—from audio classification to object detection—it is common for models to require dozens or hundreds of kilobytes as a minimum.

This is already impressive since traditional server-side machine learning models can be anywhere from tens of megabytes to several terabytes in size. Using clever optimization, and by limiting scope, embedded models can be made much smaller—we'll introduce some of these techniques shortly.

There are various ways to run a deep learning model on an embedded device. Here's a quick summary:

Interpreters

Deep learning interpreters, like **TensorFlow Lite for Microcontrollers**, use an interpreter to execute a model that is stored as a file. They are flexible and easy to work with, but they come with some computational and memory overhead, and they don't support every type of model.

Code generation

Code generation tools, like **EON**, take a trained deep learning model and translate it into optimized embedded source code. This is more efficient than an

interpreter-based approach, and the code is human-readable so it can still be debugged, but it still doesn't support every possible model type.

Compilers

Deep learning compilers, like **microTVM**, take a trained model and generate optimized bytecode that can be included into embedded applications. The implementation they generate can be highly efficient, but it's not as easy to debug and maintain as actual source code. They can support model types not explicitly supported by interpreters and code generation. It's common for embedded hardware vendors to provide custom interpreters or compilers to assist with running deep learning models on their hardware.

Handcoding

It's possible to implement a deep learning network by writing code by hand, incorporating the parameter values from a trained model. This is a difficult and time-consuming process, but it allows full control over optimization and allows you to support any model type.

The environment for deploying deep learning models is very different between SoCs and microcontrollers. Since SoCs run full, modern operating systems, they also support most of the tools that are used to run deep learning models on servers. This means that pretty much any type of model will run on a Linux SoC. That said, the latency of the model will vary depending on the architecture of the model and the SoC's processor.

There are also interpreters designed specifically for SoC devices. For example, **TensorFlow Lite** provides tools that allow deep learning models to be run more efficiently on SoCs—typically those that are used in smartphones. They include optimized implementations of deep learning operations that make use of features available in some SoCs, such as GPUs.

The SoCs that have integrated deep learning accelerators are a special case. Typically, the hardware vendor will provide a special compiler or interpreter that allows the model to make use of hardware acceleration. Accelerators typically only accelerate certain operations, so the amount of speedup depends on the architecture of the model.

Since microcontrollers don't run full operating systems, the standard tools for running deep learning models aren't available. Instead, frameworks like TensorFlow Lite for Microcontrollers provide a baseline of model support. They tend to lag behind the standard tools a little in terms of operator support, meaning they will not run some model architectures.



Operators and Kernels

In edge machine learning, an *operator*, or *kernel*, is an implementation of a particular mathematical operation used to run a deep learning model. These are overloaded terms with different meanings in other fields, including elsewhere in deep learning.

Typical high-end microcontrollers have hardware features such as SIMD instructions that will drastically improve the performance of deep learning models. TensorFlow Lite for Microcontrollers includes optimized implementations of operators, making use of these instructions, for several vendors. Like with SoCs, the vendors of microcontroller-based hardware accelerators often provide custom compilers or interpreters that allow models to run on their hardware.

The core advantages of deep learning are its flexibility, reduced requirements for feature engineering, and ability to make use of large amounts of data due to the high parameter counts of models. Deep learning is noteworthy for its ability to approximate complex systems, going beyond simple prediction to perform tasks such as generating art and accurately recognizing objects in images. Deep learning provides a lot of freedom, and researchers have only just begun to explore its potential.

The core disadvantages are its high data requirements, its propensity toward overfitting, the relatively large size and computational complexity of deep learning models, and the complexity of the training process. Additionally, deep learning models can be hard to interpret—it can be challenging to explain why they make one prediction over another. That said, there are tools and techniques that help mitigate most of these drawbacks.

Why Not Always Use Deep Learning?

Since deep learning is so capable, you may wonder why we would use any other machine learning algorithms. Deep learning is a powerful general-purpose tool that can model pretty much any relationship between input and output variables. However, just because it *can* doesn't mean it's always the *best* at it. Depending on the situation, classical ML algorithms can outperform deep learning in terms of:

Explainability

Nothing beats the interpretability of a decision tree, if your use case allows it.

Efficiency

Classical ML algorithms are typically much easier to compute than deep learning models.

Portability

Since they are simpler, classical ML algorithms can be deployed to the most basic devices (such as low-end MCUs).

Effectiveness

Some classical algorithms work better than deep learning in certain situations, especially when there is not much data available.

On-device training

Deep learning training is difficult to perform on-device, while some classical algorithms are easy to train in the field.

It all comes down to your individual use case. That said, if you were only going to take a deep dive into one technique for edge AI algorithm development, then it would probably make sense to choose deep learning.

Combining algorithms

A single edge AI application can make use of multiple different types of algorithms. Here are some typical ways this is done:

Ensembles

An *ensemble* is a collection of machine learning models that are fed the same input. Their outputs are combined mathematically in order to make a decision. Since every ML model has its own strengths and weaknesses, an ensemble of models is often more accurate together than its constituent parts. The downside of ensembles is the additional complexity, memory, and compute required to store and run multiple models.

Cascades

A *cascade* is a set of ML models that are run in sequence. For example, in a cellphone with a built-in digital assistant, a small, lightweight model is run constantly to detect any signs of human speech. Once speech is detected, a larger, more computationally expensive model is woken up in order to determine what was said.

Cascades are a great way of saving energy since they allow you to avoid unnecessary computation. In a heterogeneous compute environment, where multiple types of processor are available, the individual components of a cascade can even be run on different processors.

Feature extractors

As we learned earlier, embedding models take a high-dimensional input, like an image, and distill it down to a set of numbers that describe its content. The output of an embedding model can be fed into another model, designed to make predictions based on what the embedding model describes about the original input. In this case, the embedding model is being used as a *feature extractor*.

If a pretrained embedding model is used, this technique—known as *transfer learning*—can massively reduce the amount of data required to train a model.

Instead of learning how to interpret the original high-dimensional input, the model only needs to learn how to interpret the simple output returned by the feature extractor.

For example, imagine you wish to train a model to identify different species of birds from photographs. Rather than train an entire model from scratch, you could use the output of a pretrained feature extractor as the input to your model. This could reduce the amount of data and training time required in order to get good results.

Many pretrained deep learning feature extractors are available under open source licenses. They are commonly used for image related tasks, since large public image datasets are available for pretraining.

Multimodal models

A *multimodal model* is a single model that takes inputs of multiple types of data simultaneously. For example, a multimodal model might accept both audio and accelerometer data together. This technique can be used as a mechanism for sensor fusion, using a single model to combine disparate data types.

Postprocessing algorithms

On edge AI devices, we typically work with streams of data—for example, a continuous time series of audio data. When we run an edge AI algorithm on that stream of data, it will produce a second time series that represents the outputs of the algorithm over time.

This poses a problem. How do we interpret this second time series in order to decide? For example, imagine we are analyzing audio to detect when somebody says a keyword so that we can trigger some functionality on a product. What we *really* want to know is when did we hear the keyword?

Unfortunately, the time series of inference results is not ideal for this purpose. First, it contains many events that do not represent a keyword being detected. To clean these up, we can ignore any whose confidence that a keyword was spotted is below a certain threshold.

Second, the model may occasionally (and briefly) detect a keyword when a keyword was not actually spoken. We need to filter out these blips to clean up our output. This is equivalent to running a low-pass filter on the time series.

Finally, instead of telling us *each* time the keyword was spoken, the raw time series tells us at a set rate whether the keyword is *currently* being spoken. This means we need to do some output gating to get the information we really want.

After cleaning up the raw output, we now have a signal that tells us when a keyword was actually spotted. This is something we can use in our application logic to control our device.

This sort of postprocessing is extremely common in edge AI applications. The exact postprocessing algorithm used, and its particular parameters—for example, the threshold for considering something a match—can be determined on a case-by-case basis. Tools like Edge Impulse’s Performance Calibration (covered in “[Performance Calibration](#)” on page 337) allow developers to automate discovery of the ideal post-processing algorithm for their application.

Fail-safe design

There are many things that can go wrong with an edge AI application, so it’s critical that there are always safeguards in place to protect against unexpected issues.

For example, imagine a wildlife camera that uses a deep learning model to identify when an animal of interest has been photographed and uploads the animal’s image via a satellite connection. Under normal operation, it may send a few photographs a day—not costing very much in data fees.

But out in the field, a physical problem with the camera hardware—such as dirt or reflections on the lens—might result in images being taken that are very different from those in the original training dataset. These out-of-distribution images could lead to unspecified behavior from the deep learning model—which could mean that the model begins to constantly report that the animal of interest is present.

These false positives, caused by out-of-distribution inputs, might result in hundreds of images being uploaded via satellite connection. Not only would the camera be rendered useless, but it could potentially cost large amounts in data transfer fees.

In real-world applications, there’s no way to avoid things like damage to sensors or unexpected behavior from algorithms. Instead, it’s important that you design your application to be fail-safe. This means that if part of the system were to fail, the application would minimize harm.

The best way to do this varies between situations. In the case of a wildlife camera, it could be wise to build in a rate limit that kicks in if an unreasonable number of photographs are being uploaded. In another application, you might shut a system down entirely rather than risk harm being caused.

Building fail-safe applications is an important part of responsible AI—and good engineering in general. It is something to think about from the very beginning of any project.

Optimization for Edge Devices

With machine learning models, and particularly deep learning models, there's often a trade-off between how well a model performs its task and how much memory and compute the model requires.

This trade-off is extremely important for edge AI. Edge devices are typically computationally constrained. They are designed to minimize cost and energy usage, not to maximize compute. At the same time, they are expected to deal with real-time sensor data, often at high frequencies, and potentially react in real time to events in the data stream.

Larger machine learning models tend to be better at complex tasks, since they have more capacity—which is helpful for learning complicated relationships between inputs and outputs. This extra capacity means they may require more ROM and RAM, and it also means they take longer to compute. The additional compute time results in higher power consumption, as we'll learn in [“Duty cycle” on page 333](#).

Finding the correct balance between *task performance* and *computational performance* is essential in any application. It's a matter of juggling constraints. On the one hand, there's a minimum standard for performance at a given task. On the other hand, hardware choices create hard limits on available memory, latency, and energy.

Managing this trade-off is one of the difficult—but fascinating—parts of edge AI development. It's part of what makes the field uniquely interesting, and why tools for things like AutoML (which we'll learn about in [“Automated machine learning \(AutoML\)” on page 150](#)) need to be redesigned for edge AI.

Here are some of the factors that can help us minimize compute requirements while maximizing task performance.

Choice of algorithm

Every edge AI algorithm has a slightly different profile of memory usage and computational complexity. The constraints of your target hardware should inform your choice of algorithm. Typically, classical ML algorithms are smaller and more efficient than deep learning algorithms.

However, it's commonly the case that feature engineering algorithms use vastly more compute than either, making the choice between classical ML and deep learning less significant. The exception to this rule is the analysis of image data, which typically requires little feature engineering but relatively large deep learning models.

Here are some common ways to reduce the latency and memory required by your choice of algorithms:

- Reduce the complexity of feature engineering. More math means higher latency.
- Reduce the amount of data that reaches the AI algorithm.
- Use classical ML instead of deep learning.
- Trade complexity between feature engineering and machine learning model, depending on which runs more efficiently on your device.
- Reduce the size (the number of weights and layers) of deep learning models.
- Choose model types that have accelerator support on your device of choice.

Compression and optimization

There are many optimization techniques designed to reduce the amount of data and computation required by a given algorithm. Here are some of the most important types:

Quantization

One way to reduce the amount of memory and computation required by an algorithm or model is to decrease the precision of its numeric representations. As mentioned in [“How Are Values Represented?” on page 58](#), there are many different ways to represent numbers in computation—some that have more precision than others.

Quantization is the process of taking a set of values and reducing their precision while preserving the important information they contain. It can be done for both signal processing algorithms and ML models. It's especially useful for deep learning models, which by default tend to have 32-bit floating-point weights. By reducing the weights to 8-bit integers you can reduce a model to 1/4th its size—typically without much reduction in accuracy.

Another advantage of quantization is that the code to perform integer math is faster and more portable than the code for floating-point math. This means that quantization results in a significant speedup on many devices, and that quantized algorithms will run on devices that lack floating-point units.

Quantization is a lossy optimization, meaning that it typically reduces the task performance of the algorithm. In ML models, this can be mitigated by training at a lower precision so that the model learns to compensate.

Operator fusion

In operator fusion, a computation-aware algorithm is used to inspect the operators that are used when a deep learning model is run. When certain groups of operators are used together, it's possible to replace them with a single *fused* implementation that has been written to maximize computational efficiency.

Operator fusion is a lossless technique: it improves computational performance without causing any reduction in task performance. The downside is that fused implementations are only available for certain combinations of operators, so its impact depends greatly on the architecture of a model.

Pruning

Pruning is a lossy technique applied during the training of a deep learning model. It forces many of the model's weights to have a value of zero, creating what is known as a *sparse* model. In theory, this should allow for faster computation since any multiplication involving a zero weight will invariably result in a zero.

However, at this point in time there is very little edge AI hardware and software designed to take advantage of sparse weights. This will change over the next few years, but for now the main benefit of pruning is that sparse models are easier to compress, due to their large blocks of identical values. This is helpful when models need to be sent over the air.

Knowledge distillation

Knowledge distillation is another lossy deep learning training technique that enables a large “teacher” model to help train a smaller “student” model to reproduce its functionality. It takes advantage of the fact that there is typically a lot of redundancy in the weights of a deep learning model, meaning that it's possible to find an equivalent model that is smaller but performs almost as well.

Knowledge distillation is a bit fiddly, so it's not yet a common technique—but it's likely to become a best practice over the next few years.

Binary neural networks (BNNs)

BNNs are deep learning models where every weight is a single binary number. Since binary arithmetic is extremely fast on computers, binary neural networks can be very efficient to run. However, they are a relatively new technology and the tooling for training and running inference with them is not yet in broad use. Binarization is similar to quantization and is therefore a lossy technique.

Spiking neural networks (SNNs)

A spiking neural network is an artificial neural network where the signals transmitted through the network have a time component. As “neuromorphic” systems, they are designed to more closely resemble the way biological neurons

work. They have different trade-offs compared to traditional deep learning models, offering improved performance and efficiency for some tasks. However, they require specialized hardware (in the form of an accelerator) in order to offer a benefit.

SNNs can either be trained directly or be created from a traditional deep learning model in a conversion process. This process may be lossy.

Model compression has two major downsides. The first is that running compressed models often requires specific software, hardware, or a combination of the two. This can limit the devices that a compressed model can be deployed to.

The second downside is more dangerous. The lossy nature of compression often results in a subtle degradation of a model's predictive performance that can be difficult to spot. The reduction in precision can bias a model to perform well in common cases, but to lose performance in the “long tail” of less frequently encountered inputs.

This problem can amplify the biases inherent in datasets and algorithms. For example, if a dataset collected for training an ML-powered health wearable contains fewer examples from people in minority groups, model compression may lead to degraded performance for people in these groups. Since they are a minority, the impact on the model's overall accuracy might be hard to spot. This makes it extremely important to evaluate your system's performance on every subgroup within your dataset (see [“Collecting Metadata” on page 223](#)).

There are two excellent scientific papers on this topic from researcher Sara Hooker et al. One is [“What Do Compressed Deep Neural Networks Forget?”](#), and the other is [“Characterising Bias in Compressed Models”](#).

On-Device Training

In the vast majority of cases, machine learning models used in edge AI are trained before being deployed to a device. Training requires large amounts of data, typically annotated with labels, and involves significant computation—the equivalent of hundreds or thousands of inferences per data point. This limits the utility of on-device training, since by nature edge AI applications are subject to severe constraints in memory, compute, energy, and connectivity.

That said, there are a few scenarios where on-device training makes sense. Here's an overview:

Predictive maintenance

A common example of on-device training happens in predictive maintenance when a machine is being monitored to determine whether it is functioning normally. A small on-device model can be trained with data that represents a

“normal” state. If the machine’s signals start to deviate from that baseline, the application can notice and take action.

This use case is only possible when it can be assumed that abnormal signals are rare, and that at any given moment the machine is likely to be operating normally. This allows the device to treat the data being collected as having an implicit “normal” label. If abnormal states were common, it would be impossible to make assumptions about the state at any given moment.

Personalization

Another example where on-device training makes sense is when a user is asked to deliberately provide labels. For example, some smartphones use facial recognition as a security method. When the user sets up the device, they are asked to enroll images of their face. A numeric representation of these facial images is stored.

These types of applications tend to use carefully designed embedding models that convert raw data into compact numeric representations of their content. The embeddings are designed in such a way that the Euclidean distance between two embeddings⁵ corresponds to the similarity between them. In our face recognition example, this makes it easy to determine whether a new face matches the representations stored during setup: the distance between the new face and the enrolled face is calculated, and if it is sufficiently close then the faces are considered the same.

This form of personalization works well because, typically, the algorithm used to determine embedding similarity can be very simple; either a distance calculation or a nearest neighbors algorithm. The embedding model has done all the hard work.

Implicit association

A further example of on-device training is when labels are available by association. For example, battery management features such as Apple’s **Optimized Battery Charging** train models on-device to predict what time a user is likely to be using their device. One way to do this would be to train a forecasting model to output a probability of usage at a specific time, given a log of the previous few hours’ usage.

In this case, it’s easy to collect and label training data on a single device. Usage logs are collected in the background, and labels are applied according to some metric (such as whether the screen was activated). The implicit association

⁵ An embedding can be thought of as a coordinate in a multidimensional space. The Euclidean distance between two embeddings is the distance between the two coordinates.

between time and log content allow the data to be labeled. A simple model can then be trained.

Federated learning

One of the obstacles to training on-device is a lack of training data. In addition, on-device data is often private and users are not comfortable with it being transmitted. Federated learning is a way of training models in a distributed manner, across many devices, while preserving privacy. Instead of raw data being transmitted, partially trained models are passed around between devices (or between each device and a central server). The partially trained models can be combined and distributed back to devices once they are ready.

Federated learning often seems attractive since it appears to provide a way for models to learn and improve while in the field. However, it has some serious limitations. It is computationally expensive and requires large amounts of data transfer, which runs counter to the core benefits of edge AI. The training process is very complex and requires both on-device and server-side components, which increases project risk.

Since data is not stored globally, there is no way to validate that the trained model is performing well across the entire deployment. The fact that models are uploaded from local devices presents a vector for security attacks. Finally, and most importantly, it does not solve the problem of labels. If labeled data is not available, federated learning is useless.

Over-the-air updates

Although not actually an on-device training technique, the most common way to update models in the field is via over-the-air updates. A new model can be trained in the lab, using data collected from the field, and distributed to devices via firmware updates.

This depends on network communication, and it doesn't solve the problem of obtaining labeled data, but it's the most common way to keep models up to date over time.

Summary

We've now learned about the key AI algorithms that make edge AI possible, along with the hardware that runs them. The next chapter will walk through the tools—and the skills—that are needed to bring everything together.

Tools and Expertise

The edge AI development workflow includes many highly technical tasks, and most projects will require skills and expertise pooled by a team of experts.

The first section of this chapter is a guide to building the team that will turn your ideas into reality. Even if you're still at an early stage, it's helpful to understand the types of skills that will be important and the challenges you can expect to encounter. AI is all about automating human insights, so it's vital that you have the right insights on your team.

The second part of the chapter, starting with [“Tools of the Trade” on page 136](#), is designed to help get you to grips with the key technical tools for working with edge AI. If you're still early in your product development journey, you may want to skim over some of the details—and then use this chapter as a reference once you've come up with some concrete ideas and are ready to start.

Building a Team for AI at the Edge

Edge AI is a truly complete technology. As a topic, it makes use of knowledge from everything from the physical properties of semiconductor electronics all the way up to the engineering of high-level architectures that span devices and the cloud. It demands expertise in the most cutting-edge approaches to artificial intelligence and machine learning along with the most venerable skills of bare-metal embedded software engineering. It makes use of the entire history of computer science and electrical engineering, laid out end to end.

Nobody in the world holds deep expertise in every subfield of edge AI. Instead, the people working at the heart of the field rely on assembling networks of experts who they can look to for insight into other pieces of the puzzle. If you're building an edge AI product, you may have to do the same for yourself.

The best team for edge AI is one that has broad, cross-disciplinary knowledge, direct experience working on the problem domain, and comfort working in an iterative development process. The best executed products so far have come from teams with direct experience of the issue they are trying to solve: they've taken their existing knowledge and used it to inform their edge AI product.

It isn't necessary for a single team to have experts in every subfield of edge AI. The absolute bare minimum is probably two roles:

- A domain expert, who has deep insight into the problem to be solved
- An embedded engineer with experience developing for devices similar to the target

There's no reason why these two roles can't be filled by the same person. However, without experience working with machine learning or other AI algorithms, they'll have to rely very heavily on end-to-end platforms designed to guide non-ML experts through the process of algorithm creation.



If you're a solo developer without embedded development experience, you can level up your skills by building some non-AI projects on your target hardware. To make your life easier, you might consider sticking to SoC-level hardware, since embedded Linux development is much easier than bare metal. If you're using an end-to-end edge AI platform it should be relatively simple to deploy your model.

Determination and some scrappy improvisational skills can go a long way: we've seen plenty of scientific researchers build their own AI-powered hardware with relatively simple embedded skills.

While many problems can be solved by a minimal team, the most complex problems will require more heavy lifting. The remainder of this chapter lays out the roles and responsibilities that can potentially be important, which will hopefully give you a sense of what you need for your own team. It also talks through the challenges of hiring for edge AI.

Domain Expertise

Domain expertise, as we'll learn about in detail in [“Datasets and Domain Expertise” on page 205](#), is by far the most essential component of your team. If you have nothing but domain expertise and a budget, you can still hire developers and get a product built. But if nobody on your team has a deep understanding of the problem you're trying to solve, it's very unlikely you'll be able to solve it. In fact, there is a fair

chance you may end up trying to solve the wrong problem or creating a solution that nobody needs.

It would be difficult to build any kind of quality product without domain expertise, but building an AI product without domain expertise is almost impossible. The goal of edge AI is to distill expert knowledge into a piece of software and use it to automate a process. As we learned earlier in the book, intelligence means knowing the right thing to do at the right time. But how can we build a system that does that if we do not know it ourselves?

If you aren't a domain expert yourself, your first job is to find someone who is. Your second job is to have them validate the solution that you are planning to build. Here are some of the questions to ask them:

- Does the problem you wish to tackle really exist?
- If it exists, is it a useful problem to solve?
- Are there solutions to the problem that already exist?
- Would your proposed solution actually help solve the problem?
- Does your proposed solution sound feasible to build?
- If you build your solution, would anybody in the field want to buy it?

You should hopefully be able to ask someone these questions without having to pay too much money: they're the basic questions that any genuine domain expert would be thinking about if you were to offer them a job. You should make sure you pay attention to their answers, even if you disagree. If a genuine expert is telling you something is a bad idea, there's likely some truth to it.

Domain expertise should be at the heart of your organization and part of your core team. Your experts will be involved with so many aspects of the project that it isn't feasible for them to be peripheral members. That said, the ideal situation is that you have domain expertise at every level of your organization. For example, in addition to your core expertise you may have engineers, board members, and advisors who all have experience in the relevant area. Their combined insight will help your team anticipate and mitigate risk.

If you are unable to find anyone with the required expertise, you should abort your project before it gets started. There's simply no way to work ethically if you don't have the appropriate knowledge. Your project may be violating some golden rule of the field—and you wouldn't have any way to know about it. It's not acceptable to test unqualified functionality on your customers, as [Figure 5-1](#) makes clear. It's so difficult to establish a feedback loop with performance in the field that you likely won't know what's going wrong.



Figure 5-1. Using customers to validate your solution is a horrible idea (Twitter, 2022)

If you are truly convinced that you have a good idea, you may have to spend some time developing the required expertise yourself.

Diversity

In addition to domain expertise, the other essential property your team should aim for is diversity. As we discussed in “Mitigating societal harms” on page 51, one of the best defenses against societal issues is to build a team with diverse perspectives.

It can be helpful to think of workplace diversity in terms of four core areas:¹

Internal

Internal diversity reflects the things that a person is born with and didn’t choose for themselves. Some of these areas include age, nation of origin, race, ethnicity, sexual orientation, gender identity, physical ability, and personality types.

External

External diversity includes the things we pick up along the way, whether due to influence by external factors or due to conscious choices. Some examples are socioeconomic status, life experiences, education, personal interests, family status, location, and religious beliefs.

Organizational

Organizational diversity relates to a person’s role within an organization. This might include their place of work, job function, level within a hierarchy, pay level, seniority, or employment status.

¹ See “What Are the 4 Types of Diversity?” for more information on the four core areas.

Worldview

Diversity in worldview relates to how a person sees the world. It can include things like ethical frameworks, political beliefs, religious beliefs, personal philosophy, and general outlook on life.

As a result of differences in these four areas, every person has a different set of experiences that make their perspective unique. This unique viewpoint means that they will see the same situation in different ways. As a team building technology products, a diversity of perspectives is incredibly valuable because it allows the organization to view the problem and the proposed solutions from a multitude of different angles.

This provides a significant advantage over organizations that lack diversity. You'll be more capable of identifying all the nuance in a given situation, which has huge benefits when mapping out the space of possible solutions. Perhaps someone's personal experiences will translate into an amazing idea that nobody else would have thought of.

Even more importantly, diverse perspectives will help you identify issues with your own product. For example, you may find that different people naturally come up with different axes on which to evaluate your product's performance. A person with kids is more likely to consider the need for a product to cope well with family life, and someone with a physical disability may be more likely to think about accessibility.

This isn't to say that the members of your team should stand in for domain experts in these areas: just because a person has a disability does not mean they are automatically your official accessibility expert, a role they may neither want nor be qualified for. However, the fact that your team has diverse perspectives hopefully means that they are more likely to consider the *need* to bring on an accessibility expert.

It's not enough to just have a diverse team: individuals have to be comfortable sharing their input, and the rest of the organization has to actually listen to them. The work of building that type of environment is beyond the scope of this book, but there's plenty of literature on the subject. A good place to start is this [introduction to psychological safety from Google](#), who have found that teams where individuals can confidently speak up are **far more effective**.

Another key idea is that you should make use of perspectives from your entire organization. Beyond the people working directly on the product, you should draw feedback from everyone you can—from executives to entry-level workers. This will help you avoid blind spots in your insight. At many large tech companies, employees are encouraged to sign up to test new products that are still in development,² allowing development teams to access insight from across the entire company.

2 It's part of a strategy known as "dogfooding," covered in ["Real-world testing"](#) on page 321.

As with everything in iterative development, this process is all about building feedback loops that will help make your product over time. You should create systems to gather the perspectives of your diverse team from the very earliest stages when you're still sketching out ideas.

It's not always feasible for a single team to include all of the necessary diversity of perspectives. For example, you may need input on a product from young children, who are unlikely to be paid employees of your organization! One way to ensure these perspectives are included is to create a budget for having focus groups with these types of people throughout the course of your project.

Another way of broadening perspectives is to find a diverse group of advisors who can help inform your decisions. Assembling an advisory board that combines expertise in key areas with diverse representation is a powerful tool for helping you make the right decisions. They can act as a review board who can help you understand whether you are meeting your goals or veering off course.

Regardless of whether you have a large team, you should be relentless in seeking feedback from the people affected by your product—the most diverse group of all.

The Costs of Diversity

It's worth noting that diversity comes with some costs. Beyond the practical expense of compensating people for their time, diverse teams may find it harder to reach agreement on things like values and goals.

The leadership of a project needs to be prepared for this and may have to make some decisions that do not have full agreement. Documenting the reasoning behind any decisions, along with any dissenting views, is essential in ensuring a team can track its decision making and improve it over time.

That said, if there's fundamental disagreement on a particular issue, it can be a sign of significant risk.

Stakeholders

The stakeholders of your project are all of the people and communities who are potentially affected. This includes people within your organization, your customers, the end users of your system, and anybody who may be impacted—both directly and indirectly.

For a system to be effective, and for it to avoid causing harm, the needs and values of stakeholders must be considered. For example, if your system will come into contact with members of the public then it is important that they are considered as stakeholders, and that the project is designed with them in mind.

The best way to understand the needs and values of stakeholders is to ask them directly. They should be represented throughout your development workflow, from ideation to end of life.

Stakeholders can be identified using a well-established tool known as **stakeholder mapping**. You should make sure your team includes someone who is familiar with the process.

Roles and Responsibilities

Building a product takes a village full of people, and the next section of this chapter outlines some of the roles that are required. Your project may require roles that are not included here; these are just the most common ones that directly participate in the edge AI workflow.



You don't need to hire an individual person for each role. It's perfectly possible for the same person to play multiple roles in a project, and early on it may be the case that all of your prototyping is done by a single person.

For ease of digestion, we'll divide up the roles by type.

Knowledge and understanding

The roles in this category are critical to understanding the problem and solving it in the right way:

Domain expert

The starring role in the play, the domain expert brings a deep understanding of the project area. While a product manager's job is to understand how a project fits into the surrounding context (such as the market), a domain expert is the person who understands the science of the situation. For example, an industrial automation project might require a domain expert in the relevant industrial processes, and a healthcare project might need an expert in the related areas of medicine and biology.

Ethics and fairness expert

The ethics and fairness role is required in order to avoid making the types of mistakes that often result in harmful or ineffective products. They need a strong understanding of the technologies that will be used to solve a problem, the types of pitfalls that can emerge, and the processes that need to be followed. Domain expertise is important, too, since ethical issues can be specific to a domain.

Planning and execution

These high-level roles are important in guiding the project down the right path as it travels from ideation to launch and long-term support:

Product manager

A product manager is responsible for making decisions about the product: what it should be, what it should do, and who it should serve. Their job is to deeply understand the problem and the market, and work with those in technical roles to design and implement an effective solution. They lead through influence, pulling together different threads to weave a product that fits the right requirements.

Project manager

The project management role involves coordinating the execution of complex tasks across groups of people. For example, a project manager may organize the collection of a dataset that will be used to build and create a product.

Program manager

Program managers coordinate high-level strategies that are made up of multiple projects. For instance, a company planning to incorporate edge AI into multiple parts of its business to make cost savings may use a program manager to coordinate the process.

Algorithm development

These roles are involved in the exploration of datasets and the design of algorithms—along with mechanisms for evaluation of the system that is being built. This work can increasingly be done by nonexpert users who are using end-to-end platforms, but it's always good to have some solid experience to draw on to avoid making rookie mistakes:

Data scientist

The data science role is responsible for gathering, maintaining, and understanding the data that underlies an edge AI project. They have skills in data cleaning, analysis, and feature engineering. This role may often encompass machine learning work, but it could just as easily be distinct.

DSP engineer

A DSP engineer develops and implements DSP algorithms. They typically have strong skills in both algorithm development and low-level programming. DSP is extremely important in most edge AI projects—with the exception of those that combine deep learning with image data, since images are typically input without much processing.

ML practitioner

Machine learning practitioners spend their time trying to solve problems with ML. An ML practitioner will try to frame a problem in terms of different types of learning algorithms. They will then work with a dataset, attempting to develop algorithms that solve the problem. A key part of their role is determining how to evaluate algorithms and their performance, both in the lab and in the field.

In an edge AI project, DSP engineers and ML practitioners work very closely together, since DSP is a sophisticated form of feature engineering—which is a key part of the ML workflow.

Product engineering

This set of roles leads development of the product itself. They create the hardware and application code, and they implement the algorithm in a form that works efficiently on-device:

Hardware engineer

A hardware engineer designs the hardware that powers a product. This design includes both the sensors that capture raw data and the processors that attempt to make sense of it, along with the design and layout of printed circuit boards.

It's critical for hardware engineers to work closely with those in algorithm development roles so that the hardware and algorithms support each other. This is a two-way street: algorithm design must be informed by hardware constraints, and hardware design must be informed by algorithm design.

Embedded software engineer

Embedded software engineers write the low-level code that brings a piece of hardware to life. Their code has to interface with sensors, run algorithms, and interpret their output in order to make decisions. They implement the embedded application itself.

Embedded ML engineer

Some embedded software engineers focus specifically on machine learning. Their job is to make sure that ML algorithms run as efficiently as possible on a particular piece of hardware. They may have deep knowledge of the mathematics behind machine learning, along with experience with low-level software optimization. They aren't necessarily an expert in data science, although they can likely train simple ML models.

This is a very new role, but it's growing in step with the edge AI space.

Industrial designer

An industrial designer creates the physical design of the product. This is relevant to edge AI in that the physical design dictates many of the realities of sensor data collection: moving a sensor to another location on a product can completely change its typical output and make a dataset instantly obsolete. This means there needs to be significant communication between industrial design, electronics engineering, and algorithm development.

Software engineer

Many projects involve software engineering outside of the embedded space. For example, a lot of edge AI projects involve a server-side component. Writing this backend code requires different skills to developing embedded applications, so a different type of engineer is needed.

Technical services

These supporting roles help keep the technical side of the development process running smoothly and manage the tools that keep the team productive and safe:

MLOps engineer

An MLOps engineer is responsible for building and maintaining the MLOps solutions that are used by the rest of the team. It's essentially a **DevOps role**, but it requires strong understanding of the processes and demands of the edge AI workflow.

Security practitioner

This role attends to the security needs of the team, its data, and the products that are produced. It is both a consulting role—helping other roles understand how to be secure in what they do—and a proactive role, putting measures in place that help reduce security risk.

Quality assurance engineer

This role helps design and implement testing plans that put a product through its paces, allowing a team to understand whether the product is meeting its design goals. There's more about quality assurance in **“Real-world testing” on page 321**.

Hiring for Edge AI

A significant challenge of edge AI development is that as a very new field, there are not many people out there with experience working on it. At the time of writing, it is almost impossible to hire an engineer who has existing experience with edge AI: there are likely only a few hundred in the world, and most are still working on their first exciting edge AI projects and haven't had long enough to get itchy feet.

Fortunately, the fact that this is a new field means that even the most experienced engineers only have a couple of years' advantage. Recent advances in edge AI tooling, particularly in the form of end-to-end platforms, have massively reduced the barriers to entry. Hiring for edge AI has two main fronts where very specific knowledge is required: algorithm development and embedded engineering.

In the case of algorithm development, you'll likely be in the market for data scientists and ML practitioners. Some practitioners have backgrounds in applied engineering, solving practical problems in industry. Others may have a more academic background, investigating the principles underlying machine learning and coming up with new techniques.

Applied practitioners will have more experience with problem framing, which is very important in edge AI. This makes them a desirable choice, especially as an initial or solo hire. That said, academic researchers can still be a good fit for edge AI projects. They are less likely to have experience working within a typical software development environment and may take longer to ramp up. On the other hand, they are easier to hire than applied practitioners: there are simply more of them.



ML research is very different from applied ML, and some ML researchers may feel bored routinely applying existing techniques rather than attempting to come up with new ones. Make sure that it's clear to candidates what the expectations are around a role to avoid disappointment on both sides.

One difficulty is that not many people in data science and machine learning have much experience with sensor data. While vision is a common modality, audio is less so, and time series sensor data is likely to be a mystery to most practitioners: while time series analysis is common in data science, it's not typically the type of high-frequency time series that are produced by electronic sensors.

Fortunately, DSP engineers have a similar workflow and toolchain to ML practitioners, and they are already experts in feature engineering for sensor data. The skills and experience of DSP engineers makes them well suited to learning embedded ML, so one potential avenue is to recruit DSP engineers and have them learn the basics of machine learning. A team composed of both DSP engineers and ML practitioners will have a much easier time than either role alone.

In terms of embedded engineering, the challenges vary. While working with deep learning interpreters (or code generated by a deep learning compiler) is often simply a matter of library integration, embedded engineers may sometimes have to dig into the internals to figure out when something is going wrong. In these cases, some knowledge and understanding of deep learning is definitely helpful. Embedded engineers may also end up being responsible for the onerous task of converting a

model into the appropriate form to use on-device, which is definitely easier with some ML insight.

Another common task for embedded engineers is to implement classical ML models in software. There isn't yet a great embedded-specific C++ library for this but porting them is usually easy: there are reference implementations in higher-level languages that are simple to understand.

Unfortunately, finding an embedded engineer with existing ML knowledge is going to be a challenge for a while. That said, end-to-end platforms make things a lot easier, and eventually the number of experienced embedded ML engineers will grow. For now, it shouldn't be a blocker: a competent embedded engineer should be able to learn today's tools without too much trouble.

Learning Edge AI Skills

Over the last few years, some great resources have emerged for learning about AI at the edge. Like with most fields, there are two sides: theory and practice. Theory content will be most interesting to those who wish to contribute to advancing the field, while practical content is more helpful for those who wish to build products.

A word of caution: don't get lost in the weeds. Many people who wish to build AI products end up getting paralyzed by learning, exploring every rabbit hole they can rather than actually getting started on their projects. The reality is that this is a massive field, and you're never going to be able to learn it all. Be oriented toward action, learn enough to take your next step, and then reevaluate. Successful hardware products require teams, so figure out the minimum you need to know and then bring some experts on board.

Here are our top recommendations for both practical and theoretical content.

Practice

The final three chapters of this book, starting with [Chapter 11](#), will walk through the edge AI workflow end to end with three real-world use cases: wildlife monitoring, food quality assurance, and consumer products.

Once you're done with that, here's some further content:

Introduction to Embedded Machine Learning (Coursera course)

A highly rated online course intended as a practical introduction to the subject.

Computer Vision with Embedded Machine Learning (Coursera course)

A follow-up to the first course, focused specifically on vision.

Applied Machine Learning (TinyML) for Scale (HarvardX course)

This brilliant collection of courses focuses on the applied skills and big-picture expertise required for working with embedded ML.

TinyML Cookbook, a book by Gian M. Iodice (Packt, 2022)

A practical book based around useful “recipes” that demonstrate various concepts within embedded ML.

TinyML, a book by Pete Warden and Daniel Situnayake (O’Reilly, 2020)

A working introduction to embedded ML on microcontrollers, with examples focused on TensorFlow Lite for Microcontrollers.

Designing Machine Learning Systems, a book by Chip Huyen (O’Reilly, 2022)

A fantastic book about the machine learning development workflow, geared toward server-side applications but still very relevant.

Making Embedded Systems, a book by Elecia White (O’Reilly, 2011)

The best available practical introduction to developing embedded systems.

Theory

This content is for people who want to dig deeper into the theory of embedded machine learning. Remember, it’s not a prerequisite for successful product development—so don’t feel intimidated or get lost down the rabbit hole of studying.³

Tiny Machine Learning (TinyML) (HarvardX course)

This set of courses overlaps with Applied Tiny Machine Learning (TinyML) for Scale, referenced earlier, but starts with the absolute fundamentals—which may not be necessary if you want to get building as quickly as possible.

The Scientist and Engineer’s Guide to Digital Signal Processing, a book by Steven W. Smith (California Technical, 1997)

A truly comprehensive guide to digital signal processing, available for free and as a hardcover book. A good resource for any non-DSP engineer who will be working seriously with DSP algorithms.

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, a book by Aurélien Géron (O’Reilly, 2022)

A wonderful introduction to practical machine learning concepts and skills. A good resource for any non-ML engineer who will be working with ML algorithms.

³ Remember, the best way to learn is to build! Don’t fall into the common trap of thinking you need to memorize all of the theory first. This field develops so rapidly that you can never hope to learn it all.

Deep Learning with Python, a book by François Chollet (Manning, 2021)

Another fantastic introduction to ML, specifically focused on deep learning algorithms.

TinyML Foundation (YouTube channel)

The TinyML Foundation hosts regular presentations on embedded ML. Typically highly technical, this content reflects the cutting edge of research and engineering.

TinyML papers and projects (GitHub repository)

This repo is a goldmine of papers and resources related to the field.

Tools of the Trade

The story of edge AI is a story of tooling. In terms of raw technology, most of the basic ingredients required for putting artificial intelligence on edge devices have existed for a decade or more. However, these technologies—from capable embedded processors to deep learning models—tend to have a steep learning curve when they first become available.

Over time, however, our global technology ecosystem evolves tooling designed to manage the complexity and improve the usability of even the most challenging technologies. A rich combination of open source and commercial libraries, frameworks, and products have brought edge AI into the toolbox of the average embedded engineer.

A lot of this work has happened in the past two or three years, with libraries such as [TensorFlow Lite for Microcontrollers](#)⁴ and end-to-end development platforms like [Edge Impulse](#)⁵ bringing the technology over the threshold to enable mass adoption.

The following sections will walk through the tools that we think are most essential to AI at the edge. Successful teams will be at least passingly familiar with all of them.

End-to-End Platforms

End-to-end development platforms for edge AI incorporate many of the tools described in the following section, providing automated integration between them—along with a conscious, holistic design intended specifically for edge AI projects. They can massively reduce the complexity burden, making development much faster and less risky—and helping you avoid drowning in an ocean of unfamiliar tooling.

⁴ Founded by Pete Warden, then at Google, who kindly wrote the foreword for this book.

⁵ Which impressed this book's authors enough that we left jobs at Google and Arm to come and work on it.

End-to-end platforms are explained in their own section, “[End-to-End Platforms for Edge AI](#)” on page 162. While it’s helpful to have a basic understanding of low-level tools, it’s advisable to start with an end-to-end platform and only attempt to “roll your own” if the platform doesn’t fully meet your needs. In that case, the best end-to-end platforms will integrate with other industry standard tools so that you can extend their functionality without losing the benefits.

Software Engineering

A large portion of edge AI involves software development, so modern software engineering tools are incredibly important. Here are some of the key contributors.

Operating systems

It’s important to consider operating systems during both development and deployment. In development, your OS of choice will determine how easy it is to work with the extremely diverse set of software tools that make up the edge AI ecosystem. There’s a bit of minor conflict between two different engineering traditions.

In embedded engineering it’s historically common to use Windows as an operating system, and some embedded tools are written with this assumption. In contrast, the tools of data science and machine learning are typically best suited to a Unix-compatible environment such as Linux or macOS.

That said, this isn’t a huge problem in practice. It’s not strictly necessary for every member of a team to be able to run all of the tooling: for example, a machine learning engineer might train and optimize models with Linux and then hand them over to an embedded engineer who uses Windows. There are also plenty of tools for mixing environments, such as [Windows Subsystem for Linux](#). In addition, it’s common for more recent embedded toolchains to work fine in Unix environments—although embedded engineers may still prefer Windows as a familiar environment. The entire team at Edge Impulse, including both embedded and ML engineers, uses a combination of macOS and Linux virtual machines.

In deployment, operating systems are sometimes used on edge devices themselves. These are typically either embedded Linux (stripped-down distributions of Linux compiled to run on SoCs) or real-time operating systems (RTOS), which are special embedded-specific operating systems designed to run with minimal overhead. Both of these options, plus the option of no OS at all⁶ (which is the most common case for microcontrollers), are fully compatible with edge AI.

⁶ Known as “bare metal.”

Programming and scripting languages

The two most important programming languages for edge AI are Python and C++. Python is overwhelmingly the current language of choice for machine learning, thanks to a vast array of open source mathematical and scientific computing libraries and nearly 100% adoption by the machine learning research community. Since Python is also a first-class language for general software engineering, it beats domain-specific languages such as R.⁷ The two most important deep learning frameworks, TensorFlow and PyTorch, are both written in Python, as are the incredible tools we'll encounter in “[Mathematical and scientific computing libraries](#)” on page 143. Python has its quirks, but it's the right language to use for developing edge AI algorithms—from machine learning to DSP.

C++ (pronounced *C-plus-plus*) is a ubiquitous language in modern embedded software engineering. While some embedded platforms only support C (a simpler language than C++ that shares some characteristics), the high-end embedded devices that are typically used for edge AI are generally programmed with C++. The ecosystem around C++ features numerous tools and libraries that can make development easier, which is lucky—since it's the only game in town for most microcontroller-based systems.

C++ is a low-level language that provides a huge amount of control over the underlying hardware. It takes a skilled engineer to write good C++ code, but it can be much faster than the equivalent written in a higher-level language such as Python.



It's interesting to note that most of the mathematical heavy lifting done by Python libraries is actually implemented in C++ under the hood: the Python code is just used as a convenient wrapper. This gives developers the best of both worlds.

You're also likely to use scripting languages, such as Bash, during the development process. They are used to chain together and automate the complex tools and scripts that help build applications and deploy them to devices.

In terms of targets, you can expect to almost always use C++ when working with microcontrollers. SoCs, which run full operating systems, are often a lot more flexible—you may be able to run high-level languages such as Python. The trade-off is that they are far more expensive and consume a lot more energy than smaller devices.

Since most targets require C++, you'll need to port any algorithms developed in higher-level languages (like Python) in order to deploy your work. There are some tools explained later that make this easier, but it's not always a simple process.

⁷ A popular language for statistical computing that is not typically used for purposes outside data analysis.

Dependency management

Modern software typically has a lot of dependencies, and AI development takes this to the next level. Data science and machine learning tools often require absurd numbers of additional third-party libraries; installing a major deep learning framework such as TensorFlow brings everything from web servers to databases along for the ride.

Things can get complex on the embedded side, too, since signal processing and machine learning algorithms commonly require sophisticated, highly optimized mathematical computing libraries. In addition, the compilation and deployment of embedded C++ code often requires a rat's nest of dependencies to be present on a machine.

All of these dependencies can be an absolute nightmare and managing them is truly one of the most challenging parts of edge AI development. Various techniques exist to make it easier, from containerization (see the next section, “[Containerization](#)” on [page 139](#)) to language-specific environment management.

For Python, one of the most helpful tools is called [Poetry](#). It aims to simplify the process of specifying, installing, and isolating dependencies in multiple environments on a single machine.⁸ Other essential tools include OS-specific package management systems like [aptitude](#) (Debian GNU/Linux) and [Homebrew](#) (macOS).

One of the worst parts of dependency management comes when attempting to integrate different parts of a system together. For example, a model trained with one version of a deep learning framework may not be compatible with an inferencing framework released slightly later. This makes it extremely important to test systems end to end very early in the development process, to avoid nasty surprises later on.

Containerization

Containerization is the use of OS-level techniques to run software inside of sandboxed environments called *containers*. From inside, a container appears entirely distinct from the machine that is running it. It can have a different operating system and dependencies, and limited access to system resources.

Edge AI involves many different toolchains, used for everything from machine learning to embedded development. These toolchains often have mutually incompatible dependencies. For example, two toolchains might require entirely different versions of a language interpreter. Containerization is a powerful tool for enabling these incompatible toolchains to happily live side by side on a single machine.

⁸ The most common Python dependency management tools are [pip](#) and [Conda](#); Poetry is a relative newcomer but is highly recommended.

Containers are typically state-free and highly portable. This means that you can treat an entire painstakingly configured machine—described in a special syntax—as a command-line program that does a specific task. You can chain these together in order to perform useful work, and you can easily run them on different machines for a distributed computational environment.

It's also possible to run containers on embedded devices, typically within embedded Linux on an SoC. This can be an interesting way to package your software and its dependencies for distribution, although there is some overhead involved.

The most popular tools for containerization are **Docker** and **Kubernetes**. Docker is typically used locally on a development workstation, while Kubernetes is used to run clusters of containers within distributed computing infrastructure.

Distributed computing

Distributed computing is the idea of running different processes on different machines, potentially located anywhere in the world and connected via the internet. It's a more flexible way to approach computation than the use of single, high-powered mainframes and supercomputers, and it's the architectural style underlying the majority of modern computing.

Distributed compute is important to edge AI for many reasons. First, edge AI is an example of distributed computing! Computation is performed at the edge, where the data is created, and the results are either used locally or sent across the network.

Second, managing datasets, developing algorithms, and training machine learning models can be highly compute and storage intensive. This makes distributed computing a good fit for these parts of the process. For example, it's common to rent a highly capable remote server in order to train deep learning models—as opposed to having to buy and maintain a powerful machine for your office.

The task of organizing and controlling distributed computing infrastructure is called *orchestration*. There are many open source orchestration tools available, designed for different tasks. **Kubeflow** is an orchestration framework designed for running machine learning workloads across multiple machines.

Cloud providers

Businesses like **Amazon Web Services**, **Google Cloud**, and **Microsoft Azure** provide on-demand distributed computing resources that are available to anyone willing to pay for them. This type of distributed compute is known as “cloud compute,” since diagrams of computer networks typically use a cloud symbol to signify resources that are located outside of the local network.

Cloud providers host most of the world's websites. They take care of the physical hardware and the network configuration, allowing developers to focus on building

applications rather than managing equipment. They make heavy use of containerization to allow many different workloads to live side by side on the same infrastructure.

It's common for edge AI projects to use cloud compute for storing datasets, training machine learning models, and providing a backend from which edge devices can send and receive data. In some cases, such as “[Cascade to the cloud](#)” on page 288, AI algorithms running on cloud servers work in unison with those on edge devices in order to provide a service.

Working with Data

Data is a key ingredient of edge AI applications, and many tools exist for collecting, storing, and processing data.

Data capture

Obtaining data from the field can be difficult since there's often limited connectivity available at remote locations. Two useful tools are [data loggers](#) and [mobile broadband modems](#).

Data loggers are small devices designed to capture and log data collected by sensors in the field. They typically have a large amount of persistent storage for collecting sensor readings and can either be battery powered or connected to a permanent power source. The benefit of using a data logger is that you can begin collecting data immediately, before designing and building any of your own hardware. The downside is that data needs to be collected manually, by physically connecting to the logger.

Mobile broadband modems provide a wireless internet connection, typically via cellular networks—although satellite connections are also available. They can potentially transmit data from almost anywhere in the world, although connectivity depends on local availability and conditions. They offer the convenience of immediate data availability. However, data rates can be quite expensive, and wireless communication consumes a lot of energy, so they are not feasible for use in all situations.

IoT device management

Many platforms exist for communicating with IoT devices, managing their operation, and collecting data from them. Using them typically involves integrating either libraries or APIs into your embedded software. The software then connects with a cloud server that you can use to control the device.

These platforms can be convenient for collecting sensor data, especially in brownfield deployments where device management software may already be in use.

Data storage and management

As you collect your dataset, you'll need somewhere to store it. This can be as simple as comma-separated files on a hard disk—or as complex as a time series database designed specifically for storing and querying time series data. We will cover some of these options in [“Storing and Retrieving Data” on page 220](#).

Data storage solutions are designed for various purposes. Some are intended to be extremely fast at real-time querying of data, while others are designed to be as robust as possible against data loss. For edge AI applications, you're typically dealing with data in a “batch” mode, so performance isn't usually the most important factor. Instead, you should aim for a simple solution that fits the type of data you are collecting.

It's pretty common for AI datasets to be stored in the filesystem, without any type of database at all. Filesystems are designed for this type of data, and filesystem tools such as those available for the Unix command line can be helpful in manipulating it efficiently. Python's scientific computing ecosystem includes a lot of tools that are great at reading data from disk and helping you explore and visualize it.

While a fancy database isn't necessary, storing data in the right format is still important. As we will learn in [“Formatting” on page 246](#), sensor readings themselves should be stored in an efficient, compact binary representation such as [CBOR](#), [NPY](#), or perhaps [TFRecord](#)—which is specifically designed for high performance during machine learning training. Metadata about readings should be stored in separate files (known as *manifest files*) or in a simple database. Separating data from metadata in this way allows you to efficiently explore and manipulate datasets without reading massive files into memory.

Data pipelines

A data pipeline is a process that takes raw data and transforms it for use in a task, such as training a machine learning model. It's the way that data engineers automate things like data cleaning and wrangling. A typical data pipeline might take raw sensor data, filter it, combine it with other data, and write it into the correct format for training a machine learning model.

Many tools exist for defining data pipelines, some more complex than others. Edge AI data pipelines tend to involve very large amounts of relatively simple data, so avoid tools that are designed for working with structured data (such as data stored in relational databases). Instead of querying capabilities, look for high throughput and enough flexibility to run arbitrary signal processing algorithms.

Many cloud providers have features for running data pipelines in their distributed infrastructure. Some end-to-end platforms for edge AI make data pipelines a core feature and are designed specifically for the characteristics of sensor data.

Algorithm Development

Algorithm development is where most of the tooling complexity lives; there's a real galaxy of software available to help with the process. Some software is better suited to edge AI than others.

Mathematical and scientific computing libraries

The Python community has created some legitimate marvels of software engineering in the form of various open source libraries for performing mathematics and analysis of numbers. Some of the most important ones are:

NumPy

NumPy describes itself as “the fundamental package for scientific computing with Python,” and it's absolutely true. It provides the high-performance backbone for most Python-based numerical computing, and it has a wonderful API that lets you do complex things to large arrays of numbers with minimal effort. Its file format, NPY, is a convenient way to store sensor data.

pandas

What NumPy is to arrays, pandas is to tables of data. It provides an almost magically intuitive syntax for querying and transforming any information that can be organized into rows and columns. Pandas works with NumPy, so you can use it to help explore your sensor data; it's super-fast.

SciPy

SciPy provides a collection of fast implementations of algorithms that are essential to scientific computing. It's used heavily in developing DSP algorithms, and it's the magic that powers many other tools.

scikit-learn

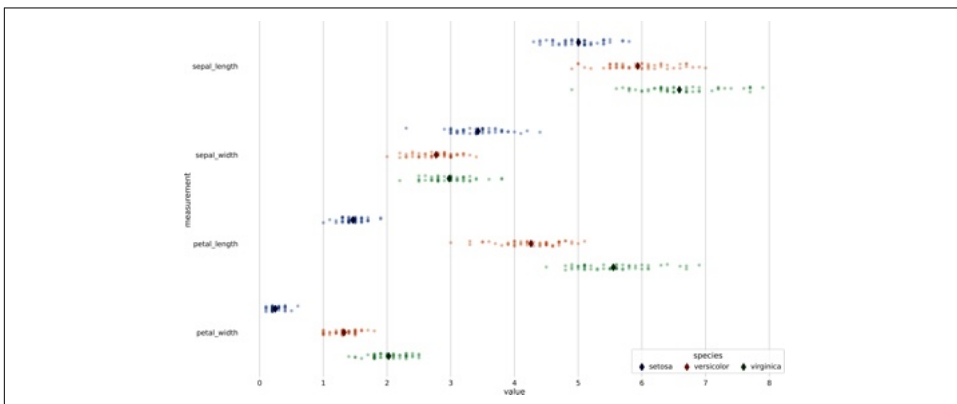
The library scikit-learn, built using NumPy and SciPy, provides a huge library of implementations of machine learning algorithms, along with the tools needed to feed them with processed data and evaluate their performance. Its API is designed so that you can plug its components together interchangeably, meaning you can easily compare and combine different algorithms. It's the gold standard for classical machine learning in Python, and its data processing and evaluation tools are often used even when training deep learning models with other frameworks.

Data visualization

When working with data, visualization is an essential tool—especially when the data concerned is digital signals. Graphs and charts allow us to represent and interpret numeric information that would otherwise be incomprehensible. The Python ecosystem has some fantastic libraries for visualizing data. They can be quite complex to get to grips with—especially if you want to customize visualizations beyond the provided defaults—but once you get the hang of them they can quickly turn rows of numbers into clear insight.

The two most common libraries are **Matplotlib** and **seaborn**. Matplotlib provides a million different ways to create data visualizations; it's commonly used to create the figures in scientific publications. Its syntax can be a little challenging, but it's so popular that a quick web search will usually help you figure out what you're trying to do.

Seaborn is built on top of Matplotlib and is designed to tame some of the complexity, making it easier to build attractive visualizations like the one in **Figure 5-2** without getting tangled in difficult APIs. It's made specifically to pair well with pandas.



*Figure 5-2. This plot shows the ranges and means for various columns in a dataset of plant measurements; it's one of the visualizations in **seaborn's example gallery***

Seaborn and Matplotlib output image files—but some visualization libraries, such as **Plotly**, produce interactive visualizations that can be explored dynamically.

Interactive computing environments

Edge AI development involves a lot of exploration that lives outside of the context of routine software engineering. Exploratory data analysis, digital signal processing, and machine learning all have a workflow that involves trying different ideas and quickly visualizing the results.

Various interactive environments exist for this purpose. Rather than just running a script and writing the results to a file or having to build an entire web application just to express information visually, interactive computing environments allow code and visualizations to exist side by side in the same editor.

The most important interactive environment for Python code is called **Jupyter Notebook**. Inside a notebook you can write and run Python code, and the output of the code is displayed alongside. This includes any visualizations you generate using libraries such as Matplotlib—as seen in [Figure 5-3](#).

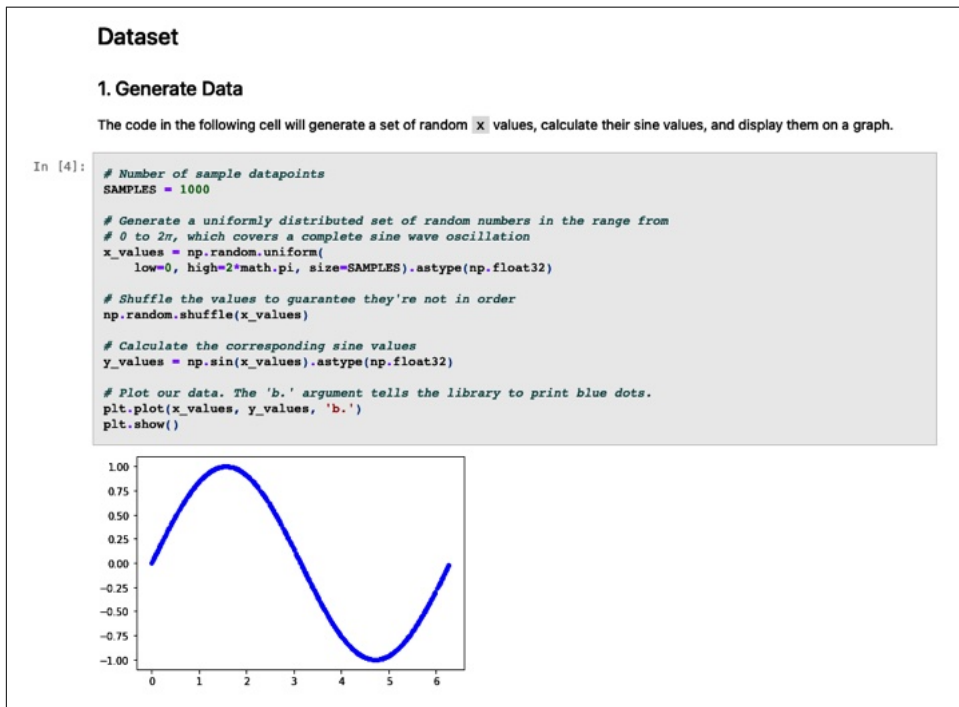


Figure 5-3. A screenshot from a Jupyter Notebook, showing a mixture of rich text, code, and the output of the code; the featured notebook is from the TensorFlow Lite for Microcontrollers [Hello World example](#)

This allows you to build interactive living documents that contain both the implementation of algorithms and the results of running them. They're valuable as both an interactive tool for experimentation and as documentary evidence of the work that you've done. A common workflow is to experiment with algorithms in a notebook until you find the best candidate, then port the code into regular Python scripts once you know it works well.

Jupyter can be run locally, but there are also Jupyter-based hosted environments. One of these is [Google Colab](#), and another is [Amazon SageMaker](#). Both can be used without cost but will provide additional compute for a fee.

Another common environment for interactive computing is [MATLAB](#), which combines a similar interactive environment with its own programming language. It's common in academia and engineering, but as a closed-source commercial product that costs money to license, it's less popular with software engineers. It's quite likely that those with a background adjacent to electrical engineering are familiar with MATLAB, including DSP engineers.

There's even an interactive environment designed specifically for edge AI. The [OpenMV IDE](#) is an open source product created by the OpenMV team to support development of machine vision applications. It makes it easy to test and implement algorithms that interpret visual information, which can subsequently be deployed to both OpenMV's hardware, and to other targets. The OpenMV IDE is unique in that it can be connected to a camera-equipped hardware device and display the results of algorithms running in real time.

Digital signal processing

DSP algorithm development is typically performed in Python or MATLAB. Either environment can be used, with individual DSP engineers typically preferring one over the other.

In Python, SciPy's [scipy.signal](#) module provides implementations of a lot of important DSP algorithms. In MATLAB, the [signal processing](#) and [image processing](#) toolboxes are very helpful.

MATLAB has some nice GUI-based tools that reduce the amount of programming required for algorithm development, but Python has the advantage of being directly compatible with the toolchains used for training machine learning models—as well as being free.

An increasingly popular third choice is [GNU Octave](#), designed to be a free, open source alternative MATLAB.

Deep learning frameworks

The ecosystem of deep learning tools is dominated by two wildly popular open source frameworks, written for Python: **TensorFlow**, created by Google, and **PyTorch**, created by Meta.⁹ Each framework originated as an in-house system for training deep learning models, and they both reflect the priorities of their respective sponsors.

Deep learning frameworks are different from typical software libraries (like NumPy or scikit-learn) in that they attempt to provide entire suites of tools under a single banner. TensorFlow and PyTorch both include systems for defining and training machine learning models, handling data, coordinating distributed systems, deploying to different types of compute, and much more.

Example 5-1. A simple deep learning model architecture being defined and trained using Keras, the high level API of TensorFlow

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Defining the model architecture
model = Sequential()
model.add(Dense(units=64, activation='relu'))
model.add(Dense(units=10, activation='softmax'))

# Setting up the training process
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])

# Training the model
model.fit(x_train, y_train, epochs=5, batch_size=32)

# Evaluating the model
loss_and_metrics = model.evaluate(x_test, y_test, batch_size=128)
```

The history of both tools has led to TensorFlow being the primary framework for industry, while PyTorch is the preferred tool of deep learning researchers.¹⁰ A big part of this is due to the TensorFlow ecosystem having more options available for model deployment, and this is especially relevant for edge AI.

At the time of writing, the majority of tools for optimizing models for efficiency and deploying them to edge devices have been written to integrate with the TensorFlow ecosystem. TensorFlow and PyTorch have different formats for storing models, and

⁹ Back when they were known as Facebook.

¹⁰ The history and comparison between the two frameworks is quite interesting, and [this fantastic blog post from AssemblyAI](#) does a great job of summarizing it.

while there are ways to convert between them, it isn't a straightforward process.¹¹ This means that the majority of ML engineers working in edge AI currently use TensorFlow.

Since PyTorch is the framework of choice for researchers, many of the newest model architectures are available first in a PyTorch format. This can be frustrating for industry developers who are using TensorFlow for its deployment capabilities. Fortunately, most of the researchers focusing on producing smaller, more efficient models that are suited for edge deployment are doing so within the TensorFlow ecosystem. The area where model incompatibility is most frustrating is in visual object detection since the training code for object detection models tends to be complex and difficult to port from one framework to another.

At the time of writing, TensorFlow is the best choice of framework for edge AI development. Developers using PyTorch will struggle with a complex and unreliable conversion process when they attempt to deploy their models. It will be interesting to see how this evolves over time as the PyTorch ecosystem matures.

Model compression and optimization

Edge devices typically require small, efficient models—especially in a deep learning context, where parameter count and computational requirements can scale up fast. In “[Compression and optimization](#)” on page 117, we learned about the various techniques available for improving the performance of models. Some of them are applied during training, while others happen afterwards.

Compression and optimization tools are generally available either as part of deep learning frameworks or by hardware vendors whose hardware supports particular optimizations. The [TensorFlow Lite converter](#) has become the de facto standard for operator fusion and basic quantization, with the TensorFlow Lite model file format becoming close to a standard in the industry.¹² Still in the TensorFlow ecosystem, the [TensorFlow Model Optimization Toolkit](#) provides a collection of open source tools that cover other types of optimization and compression.

It's worth remembering that most optimization approaches also require special tooling at inference time, covered later in the sidebar “[Inference and Model Optimization](#)” on page 156. At the time of writing the best supported optimization approach is quantization, with 8-bit quantized operator implementations widely available. Other techniques are less well supported, with sparsity being the biggest red herring: it sounds impressive, but there's currently very little hardware that supports it.

11 In fact, it can be an absolute nightmare even for the most experienced developers.

12 Other formats are in use, such as [ONNX](#), but the TensorFlow Lite format is by far the most popular.

Experiment tracking

Algorithm development is an iterative, exploratory process, and over the course of a project you're likely to make hundreds or thousands of different attempts at getting something that works acceptably well. It's important to keep things scientific, testing ideas systematically rather than just making random changes and hoping for the best. To achieve this, you'll need some kind of system for tracking experiments.

A typical experiment might involve taking a specific set of data samples, applying a particular DSP algorithm, using the features to train a machine learning model with a unique set of hyperparameters, and then testing the model on a standard test dataset. This situation has a lot of variables: the choice of samples, the DSP algorithm, the model, and its parameters.

Experiment tracking tools are designed to keep a log of which experiments are run, how their variables are set up, and what the results are. They try to organize what would otherwise be an unreliable, informal process of taking notes in a notebook and trying not to forget any details. Experiment trackers can also store the artifacts that result from experiments: training scripts, datasets, and trained models. This is helpful in understanding and reproducing your work at a later stage.

Experiment trackers are available as both open source packages and hosted commercial products. One of the simplest options is **TensorBoard**, an official part of the TensorFlow ecosystem.¹³ TensorBoard provides a simple web interface for visualizing and comparing the logs collected during training runs, along with some very powerful tools for optimizing and debugging training code. It's useful for keeping track of basic experiments, although it isn't designed as a persistent datastore that will last the lifetime of a project, and it doesn't work well if you are running very large numbers of trials.

A more sophisticated open source option is **MLflow**. It's a complex web application, backed by a database that can track experiments, store trained models, and package data science code so that experiments can be reproduced easily. It's better suited to long-term use than TensorBoard, and it can scale to track many thousands of experiments. It doesn't have the same optimization and debugging features as TensorBoard, which remains the tool of choice for improving the computational performance of training.

Many commercial products exist to help with experiment tracking. A notable option is **Weights & Biases**, which has a simple API and a well-designed web interface (along with many features that fit into the MLOps category, which we'll explore in “**Machine learning operations (MLOps)**” on page 151). A nice benefit of commercial tools is that you don't have to host them on your own infrastructure; you just pay a monthly

¹³ TensorBoard works with both TensorFlow and PyTorch.

fee and someone else performs the setup and maintenance and makes sure they are secure.

Automated machine learning (AutoML)

Once you begin tracking experiments using software, it's a simple step to start running them from software, too. AutoML tools are designed to automate the process of iteratively exploring a design space. Given a dataset and some constraints, they'll design experiments to test different combinations of variables in order to try and find the best model or algorithm.

This process is called *hyperparameter optimization*,¹⁴ and it's a highly effective way to find the best model for a particular dataset. There are many different algorithms that guide hyperparameter optimization, from a simple grid search (where every possible combination of variable is tried in turn) to named algorithms such as **Hyperband** that aim to intelligently control the process for maximum efficiency.

AutoML isn't a magic wand that will solve problems for you. It still takes domain expertise to frame a problem and set up the design space in the correct way. What AutoML *can* do is take the guesswork and tedium out of the ML workflow: it's a way to automate the trial and error while you focus on more productive things.

Some AutoML systems just take a design space as input and output a list of experiments to run, while others take it a step further toward the MLOps world (see the next section, “**Machine learning operations (MLOps)**” on page 151) by orchestrating the process of running the experiments using distributed computing techniques. A particularly complex flavor of AutoML is neural architecture search (NAS), which incorporates machine learning into the process of exploring the design space.

In terms of specifics, we recommend **Ray Tune** as a popular open source framework for hyperparameter tuning that is able to orchestrate the task of running hyperparameter optimization within your distributed infrastructure. **Sweeps by Weights & Biases** is a commercial, hosted product that helps orchestrate experiments on your own hardware.

AutoML is especially powerful for edge AI. This is because models designed for edge devices tend to be small and quick to train, which makes it easy to try a lot of different options. It's also especially important, since in edge AI we are optimizing for more than just model accuracy: we also need to find the smallest, fastest, and lowest power-consuming models that we can.

Typical AutoML tools don't account for these things, but some end-to-end edge AI platforms do.¹⁵

¹⁴ Or *hyperparameter tuning*.

Machine learning operations (MLOps)

The machine learning workflow has a lot of moving parts, and MLOps is the art and science of keeping track of them all. It encompasses many of the types of tools that we've covered in this chapter, from data storage systems to experiment tracking and AutoML capabilities.

As an engineer on an ML project, you're doing MLOps whether you're conscious of it or not. Even in the simplest projects, keeping track of your dataset, training scripts, and your current best model can be a challenge. In more complex projects, where every part of the workflow is constantly evolving as the result of feedback loops, keeping a handle on what is going on can be nearly impossible without effective tools.

ML Pipelines

One of the key pieces of functionality that makes up an MLOps solution is the ability to define and run ML pipelines. An ML pipeline is a scripted process that takes data, applies transformations (including signal processing or any other feature engineering), uses it to train a machine learning model, and evaluates the results. It's an extension of a data pipeline that includes the ML parts, too.

While initial experimentation often takes place in a notebook or in local scripts, it's common to start defining a formal pipeline once you want to begin automating the process of training a model. For example, pipelines make it easier to run repeated experiments to try different hyperparameters, and they are convenient if you are continually adding fresh data and want to automatically train and compare new models.

The simplest ML pipelines are implemented using scripting languages, either Python or Bash, and run on a single machine. More complex pipelines may be designed to run in distributed infrastructure, potentially with steps running in parallel to improve performance. It's common for sophisticated ML pipelines to make use of containerization (see [“Containerization” on page 139](#)): each step of the pipeline may be defined in a separate container that contains all of its required dependencies, and the containers are invoked one after the other.

An MLOps system can be built from individual components: you may choose one tool for dataset management, another tool for experiment tracking, and a different tool to store your best models. It is equally common to use comprehensive frameworks that take care of every stage in the process. It's also possible to use a mixture of comprehensive frameworks and whichever individual tools fit your specific needs.

15 Introduced in [“End-to-End Platforms for Edge AI” on page 162](#).

MLOps is a big area that encompasses many categories of tools, including some that we've seen earlier in this chapter. The website ml-ops.org, a great resource for understanding MLOps, says that MLOps includes the following tasks:¹⁶

- Data engineering
- Version control of data, ML models, and code
- Continuous integration and continuous delivery pipelines
- Automating deployments and experiments
- Model performance assessment
- Model monitoring in production

Since edge AI is a new field, most MLOps systems are designed with the assumption that models will be “served” by web services, not deployed to edge devices. The unique nature of edge AI development involves some additional tasks, including:

- Capture of data from devices and sensors
- Digital signal processing and rule-based algorithms
- Estimation of on-device performance¹⁷
- Model compression and optimization
- Conversion and compilation for edge device support
- Tracking which model versions are currently in the field

A great way to think about MLOps is as a “stack”: a set of software tools that work together to enable development, deployment, and maintenance of an edge AI system. The company Valohai created the idea of an **MLOps stack template**: a diagram that shows how all of the components of the MLOps stack fit together. Their original stack template is based on a server-side context, but **Figure 5-4** shows the idea adapted to suit edge ML.

¹⁶ Listed in **State of MLOps** at ml-ops.org.

¹⁷ Including both model quality and computational performance.

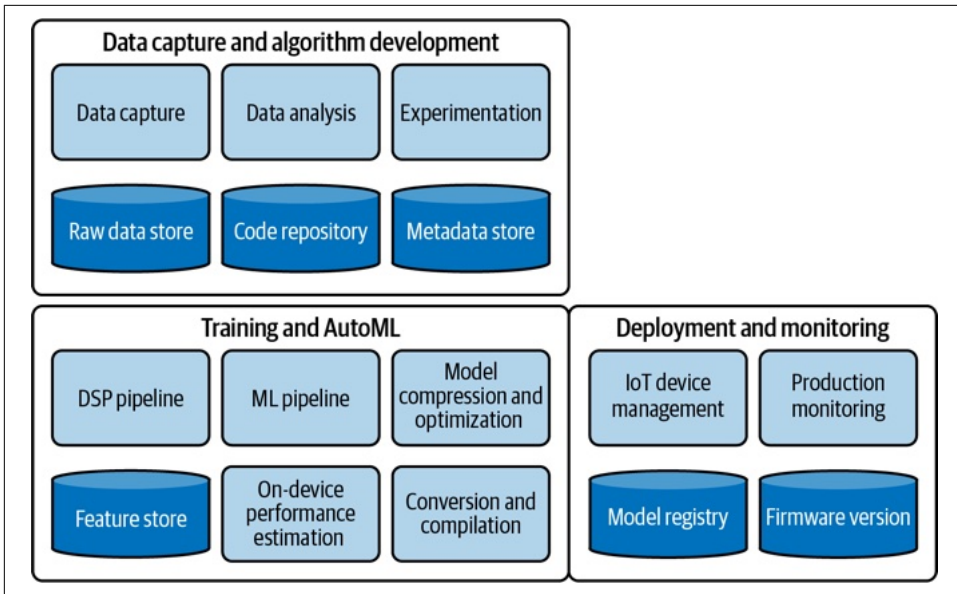


Figure 5-4. A stack template for ML at the edge; you’ll need a solution for each of the boxes, and probably some others depending on your particular use case

Over the course of development, you might choose to incrementally assemble your stack from various software components. On the other hand, you may benefit from working with a comprehensive MLOps platform that is designed specifically for edge AI, as we’ll encounter in “[End-to-End Platforms for Edge AI](#)” on page 162.

MLOps is a massive topic, too extensive to cover fully in a book about edge AI. If you’re looking to dig deeper, we recommend the following resources—with the caveat that most MLOps content is written with server-side models, not edge AI, in mind:

- The website ml-ops.org.
- [Introducing MLOps](#), a book by Mark Treveil et al. (O’Reilly, 2020).
- [Google Cloud’s introduction to MLOps](#), an excellent technical article.

Running Algorithms On-Device

Designing algorithms and training models requires one set of tools, while another is needed to run them efficiently on-device. These include both general-purpose C++ libraries and highly efficient implementations that are optimized for specific hardware architectures.

Math and DSP libraries

Various implementations of common mathematical operations are available, providing functionality for both DSP algorithms and deep learning ops—it would be time consuming to have to implement these fundamental algorithms from scratch. Some notable examples are:

- Fast Fourier transforms, used heavily in DSP, such as **KISS FFT** and **FFTW**.
- Matrix multiplication libraries such as **gemmlowp** and **ruy**.

Hardware devices often have features designed to improve the performance of common algorithms. These are available in hardware-specific libraries, such as the **CMSIS DSP Software Library** that provides optimized implementations of many popular DSP algorithms for Arm’s Cortex-M and Cortex-A hardware.

There are similarly optimized implementations available for deep learning kernels, such as the **CMSIS NN Software Library**. Equivalents exist for many modern processor architectures, including microcontrollers and SoCs. When choosing hardware, you should investigate the availability of optimized kernels, since they can make a huge (10–100x) difference in latency.

Machine learning inference

One way to run inference on an edge device is to write a custom program that implements a specific deep learning model in code that is hand-optimized for the target architecture. However, this would be time consuming and inflexible: you wouldn’t be able to reuse your code for new applications or with different hardware, and if you made any changes to your model you would have to change your entire program.

Developers have come up with various solutions to avoid this problem.¹⁸ The most common approaches are as follows:

Interpreters

An interpreter (or *runtime*) is a program that reads a file describing a model, including both its operations and its parameters, and then uses a set of prewritten operators to execute the model’s operations one after the other. Interpreters are very flexible: using an interpreter, an identical few lines of code can be used to run any model interchangeably. The trade-off is that the process of reading and interpreting a model introduces some operational overhead beyond what is required for the model’s operations. Interpreters consume additional RAM, ROM, and CPU cycles.

¹⁸ Pete Warden has an **excellent blog post** that outlines the technical challenges in this area.

The most widely used interpreters are both from the TensorFlow ecosystem. **TensorFlow Lite** was originally designed for cellphones but works on many popular SoCs, and **TensorFlow Lite for Microcontrollers** works well on microcontrollers and DSPs. Both of them are implemented in C++, but TensorFlow Lite provides Python and Java APIs for convenience. They both benefit from operator fusion and quantization provided by the **TensorFlow Lite converter**.

The kernels used by interpreters can be switched out depending on the device being targeted, so highly efficient optimized kernels can be used where available. These are readily available for several common devices and architectures.

Code generation compilers

With a code generation approach, a code-generating compiler takes a model file as input and transforms it into a program that implements it. For operator support, the program relies on a library of prewritten operators, calling them in the correct order and passing the appropriate parameters.

Code generation provides many of the same benefits as an interpreter-based approach but eliminates the majority of the overhead associated with the interpreter itself. Code generation may even make use of the wide array of prewritten operators available for interpreters: for example, Edge Impulse's **EON Compiler** is compatible with TensorFlow Lite for Microcontrollers kernels.

Bytecode compilers

It's possible for a compiler with knowledge of a target to directly generate the bytecode that implements a model, applying target-specific optimizations along the way. This results in a highly efficient implementation that makes use of whatever performance-enhancing features are available on the silicon. For example, Synaptics' **TENSAI Flow neural network compiler** is designed to compile models for deployment to Synaptics Katana Edge AI processors.

Virtual machines

The big downside of the bytecode compiler approach is that a compiler has to be written for each device that is going to be targeted, and writing a compiler is a difficult task. To get around this problem, some compilers target a so-called *virtual machine*: an abstraction layer that sits directly above the hardware and provides instructions that map to various low-level processor capabilities.

The abstraction layer slightly reduces the efficiency, but the benefits can outweigh the drawbacks—although the virtual machine still has to be ported to new processors. This approach is used by **Apache TVM**, which also uses an on-device runtime that can iteratively test different implementations to find the most efficient.

Hardware description language

A newly emerging trend is the use of special compilers to generate hardware description language (HDL), the code that describes processor architectures and is used to program FPGAs and ASICs. Using these techniques, it is possible to implement a model directly in hardware, which can be extremely efficient.

CFU Playground and **Tensil** are both open source tools that aim to make it easier to design custom accelerators using this approach.

Alternative methods

Some accelerator chips are programmed using systems that fall outside of the normal workflow of code and compilation. For example, some chips with hardware implementations of neural network kernels provide an interface via which a model's weights are written directly to a special memory buffer, separately from any application code.

Inference and Model Optimization

The optimization of kernels for high performance on specific devices is distinct from the optimization of *models* through compression and other techniques. Model optimizations tend to require their own kernel—and sometimes hardware—support.

For example, to run a quantized model, kernels compatible with the specific level of quantization must be available. A model quantized to 8-bit integer precision requires kernels designed to support it, and the same is true of other quantization levels. In fact, specific kernels are required depending on the data type used, whether that might be `int8`, `uint8`, `int16`, or so on.

The same is true of other optimization techniques. For example, pruning results in models that have a large amount of sparsity: they have lots of zeros. By itself, this doesn't make any difference to execution time—the model has to be run using special kernels or hardware that can make use of the sparsity to reduce computation time. These kernels and hardware have yet to enter wide availability, so pruning remains of limited utility in the field.

On-device learning

As we learned in “[On-Device Training](#)” on page 119, the data and computational requirements of deep learning training means that on-device training remains of limited utility. Most of the time, “on-device training” means a simple approach that involves calculating the distance between embedding vectors, for example if determining whether the embeddings of two fingerprints are a match.

It’s very rare for actual deep learning training to happen on an edge device. If you do have a device with the required amount of storage and compute—typically an SoC or mobile telephone—[TensorFlow Lite provides some functionality](#).

The problem remains that it is incredibly difficult to understand whether a model trained on-device is actually performing well. On-device deep learning is best avoided unless you have an extremely good reason to require it.¹⁹

Federated learning remains a topic of fascination for many people, but as we learned earlier in the book it is not a particularly good fit for the vast majority of problems. In addition, the tooling around federated learning is still primitive and experimental.²⁰ Many people feel drawn to follow the federated learning rabbit hole and end up wasting time: the chance that a project really needs it is very slim. However, if you really feel compelled to dig deeper, [TensorFlow Federated](#) is a good resource.

Embedded Software Engineering and Electronics

Edge AI is a subfield of embedded software engineering, which is closely tied to the practical disciplines of electrical engineering and electronics. Each of these areas involves multitudes of tools and techniques—there’s no way we’d have space in this book to cover them all.

Instead, we’ll step through the parts that specifically matter for developing AI at the edge.

19 Pete Warden’s blog post, “[Why Isn’t There More Training on the Edge?](#)” does a great job of illuminating this topic.

20 Although it will invariably improve over time.



Just Getting Started

If you're prototyping your own edge AI project but don't have much embedded experience, **Arduino** and **Arduino Pro** products are a great place to start. Arduino has created an embedded development environment that is easy for beginners to use but still powerful enough for building real applications—perfect if you're an ML engineer beginning to work with edge devices, or a newcomer to both fields. The Arduino team have understood the potential of the edge AI movement since the very beginning and have contributed a lot to its growth.

Embedded hardware tools

Developing embedded software is challenging due to the nature of embedded devices. Software is harder to debug when it's running on a separate device, especially one with limited ways to display its internal state. Embedded programs must take care of everything from basic hardware integration—it's common to have to write your own drivers for hardware such as sensors—to the complex handshakes of low-level communications protocols.

As such, embedded development requires some tools that would appear unusual to other software engineers. Some of these items include:

- Device programmers, which are pieces of hardware that allow a developer to upload new programs to an embedded device. They are often device specific.
- Debug probes, hardware devices that connect to embedded processors and allow analysis of a program at runtime. They are also device specific.
- USB to UART adapters, which send and receive arbitrary data between the developer's workstation and the embedded device. They are generic.
- Multimeters, which measure voltage, current, and resistance and can be used to understand the state of an embedded circuit as it is being controlled by a program.
- Oscilloscopes, which measure signals on the device or PCB, expressed as voltage over time.

These tools are necessary in order to reach into, manipulate, and understand the states of embedded devices. For example, to test a program is running correctly you might have it toggle a specific pin on the processor when it gets to a certain point. You would then use a multimeter to measure whether the pin has been toggled. Another common way to communicate with an embedded device is via a serial (UART) cable, which can send and receive data at a relatively low frequency—but still high enough to transfer sensor data in a reasonable timeframe.

Development boards

An embedded processor on its own is just a little piece of sand, wrapped in plastic. In order to actually do anything, it requires a small constellation of other electronic components to be wired up to it. As we saw in [“Boards and Devices” on page 79](#), development boards (or dev boards) provide a convenient ready-to-go platform that includes an embedded processor and various inputs and outputs, often including some sensors.

The goal of a dev board is to allow embedded engineers to evaluate a particular chip for suitability for a project, and allow software development to proceed without being blocked by the hardware development process. Once a working iteration of the product’s own hardware is ready, development can move there. The exception is with rapid prototyping platforms, such as [Arduino Pro](#), which are designed for use in small-batch production designs.

Dev boards are available for most families of embedded processors. When deciding on hardware, it’s a good idea to obtain a few different dev boards to experiment with. For example, you might try to run an early version of your deep learning model on a few different dev boards to understand their relative performance.

Some end-to-end platforms (see [“End-to-End Platforms for Edge AI” on page 162](#)) provide deep integration with dev boards, allowing you to capture data from their sensors or deploy and evaluate models without writing a single line of code. This can be extremely useful in development and testing.

Embedded software tools

For the purposes of edge AI, embedded software engineering generally means C++ development. This can be done in your text editor of choice, but it’s also common for embedded processor vendors to provide their own integrated development environments (IDEs) that integrate neatly with their hardware and make it easier to upload and debug code.

Vendors will often provide SDKs, drivers, and libraries that can be used on their hardware to help you access various processor features—but they are not always great quality, often provided more as a proof of concept than as production-quality code.

To reduce the amount of boilerplate code you need to write, you may choose to use a real-time operating system (RTOS). An RTOS provides the functionality of a simple operating system, but it arrives as a bunch of library code that you compile alongside your own program. You can then call the RTOS APIs to do things such as controlling peripherals or performing network communication.

Embedded development frequently involves complex toolchains: programs and scripts that are supplied by the hardware vendor and are used to take code from a text file, turn it into a program, and “flash” it onto a hardware device.

The workflow generally looks like this:

1. Make changes to your source code.
2. Run a compiler (supplied by the processor vendor) and linker to transform your code into a binary.
3. Run a script to flash your code onto the embedded device.
4. Use a serial connection to communicate with the device and test your code.

When your code is running on-device, you can often use a hardware tool called a *debug probe* to inspect it from your development machine. This allows you to debug as if you were running the code locally, setting break points, examining variables, and stepping through code.

Some parts of your code will be generic C++, and you'll be able to run it on your development machine with no problem, perhaps in the form of unit tests. However, you'll also end up with plenty of code that integrates with the specific hardware APIs of your processor. It's not possible to run that on your development machine—so you can either shrug your shoulders and test it on-device only, or you can attempt to use an emulator.

Emulators and simulators

An *emulator* is a piece of software that aims to reproduce a processor virtually, running on your development machine, so that you can execute your embedded code without having to flash it to the device. It will never be a perfect representation of the real hardware—for example, it won't necessarily run at the exact same speed as the program on real hardware—but it can be close enough to be a valuable tool.

If you need to determine how fast a program will run, for example in order to estimate the latency of an AI algorithm, a cycle-accurate simulator will allow you to determine the exact number of clock cycles that will run on the real hardware. You can divide this number by the clock rate to give you a precise estimate of latency. The emulator won't actually *run* at that speed, but it will give you the information you need to create an estimate.

Simulation is the use of software to simulate an entire device, including an emulated processor plus all of the other devices it may be attached to—including sensors and communications hardware. Some simulators can even represent multiprocessor boards, or entire networks of interconnected devices.

Emulators aren't available for all processors, but **Renode** is a powerful emulation and simulation environment for many common processor architectures, and **Arm Virtual Hardware** allows you to emulate Arm processors in the cloud.

Embedded Linux

Most of the specialized embedded tools we've mentioned so far are intended for working with microcontrollers and other bare-metal devices. SoCs and edge servers are another story: with enough computing power and memory to host a full-blown operating system, SoC development is much more similar to development for personal computers and web servers. This is one of their major benefits: developers don't need quite so many specialized skills.

A typical SoC will run a distribution of Linux, with all the helpful tools and libraries that that implies. Programming can be done in nearly any language, with the same trade-offs as on any other platform: low-level languages like C++ are fast and efficient, while high-level languages like Python are flexible and easy to use.

Google provides a **TensorFlow Lite runtime** that is prebuilt for some popular platforms, and you'll have the benefit of being able to use Python computing libraries directly: for example, you can use SciPy's digital signal processing functions within your application.

Embedded Linux devices can even make use of containerization for deployment: embedded applications can be packaged as Linux containers, making them easy to install and use.

With SoCs, it's relatively common to use an off-the-shelf board in a production installation. Many vendors exist who design and sell SoC-based platforms designed for specific applications. For example, you can buy devices in ruggedized housing designed for industrial deployments. To deploy, you just connect whatever sensors are required and install your application.

One challenge working with SoCs is that despite the familiar Linux environment, prebuilt packages are not always available. You may have to get used to building libraries from source in order to make your applications work, which can get a little involved at times.

It's important to think about security when working with devices that have fully fledged operating systems. The embedded Linux running on an SoC needs to be locked down as tightly as any other machine on your network to avoid it becoming a vector for attacks. Insecure IoT devices are notorious for being compromised by hackers and used to attack other systems.

Automated hardware testing

Modern software engineering best practices encourage the use of continuous integration tests: every code change is put through its paces by a suite of automated tests. Creating automated tests for embedded applications can be difficult, since code that interacts with hardware can't be tested on a development machine; it can only be tested on the target device itself.

However, it's easy for an embedded device to get into a state where it is unable to run tests. For example, if the program crashes it may not be possible to restart the device without physically interacting with it. Similarly, uploading new firmware may require physical intervention.

To get around this problem, developers build automated hardware testing systems that can interact with embedded devices to facilitate easier testing. These systems are a combination of software and hardware that can do things like flash new code, power cycle devices between tests, and even provide input to I/O ports or sensors.

Automated hardware testing systems are usually custom built. They are based around a host system (perhaps an embedded device itself) that is connected to whatever continuous integration tools the team is using, as well as being connected to the devices that are intended to run the code.

If integration with a sensor needs to be tested—for example, a microphone that is supposed to be detecting a keyword—the host system might even feature a speaker that can issue keywords on demand.

End-to-End Platforms for Edge AI

In an ideal world, any team with expertise in a certain domain would be able to capture its knowledge and deploy as edge AI. People with deep insight into diverse fields like healthcare, agriculture, manufacturing, and consumer technology should be able to take what they know and use it to build amazing AI-powered products.

Unfortunately, with so many moving parts and so much to learn, it's easy to feel overwhelmed by the edge AI development process. A huge amount of the workflow is focused not on domain knowledge but on the arcane engineering skills required to build a complex product across multiple fronts, including machine learning, digital signal processing, and low-level software engineering on embedded hardware.

In the early days, only a small number of technologists—who happened, by accident, to have all of the required skills—were able to work with edge AI technology. However, over the past few years a vibrant ecosystem of tools has sprung up that is designed to reduce the barriers to entry and make it possible for people without backgrounds in machine learning or embedded systems to build fantastic new products.

End-to-end edge AI platforms are designed to assist developers with the entire process of developing an application: collecting, managing, and exploring datasets; performing feature engineering and digital signal processing; training machine learning models; optimizing algorithms for embedded hardware; generating efficient low-level code; deploying to embedded systems; and evaluating systems' performance on real-world data. This flow is summarized in [Figure 5-5](#).

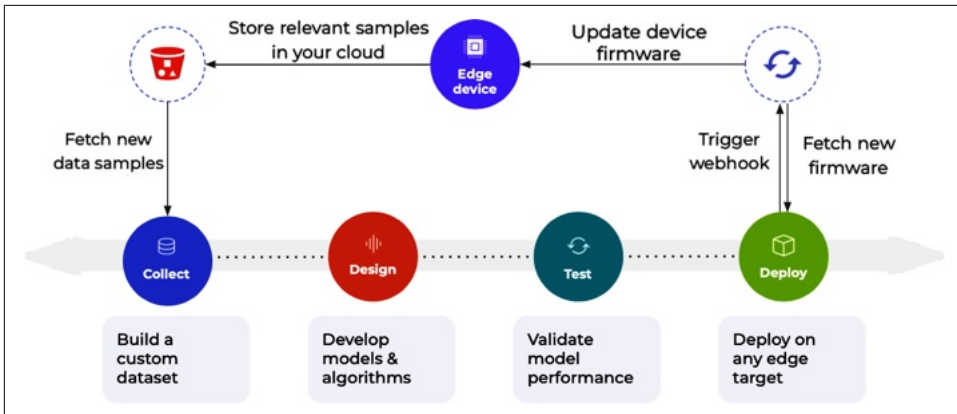


Figure 5-5. A major advantage of using an end-to-end platform is that it includes all of the components required for an iterative, data-driven feedback loop; that said, the most flexible platforms provide points for integration with external tools (Image courtesy of Edge Impulse Inc.)

End-to-end platforms are designed to apply the principles of MLOps to the specific process of creating algorithms that will run on embedded devices. As highly integrated tools, they are able to take most of the friction out of the development process: far less time is wasted in getting different parts of a toolchain to work together, and a holistic view of the entire process allows for helpful guidance that massively reduces exposure to risk.

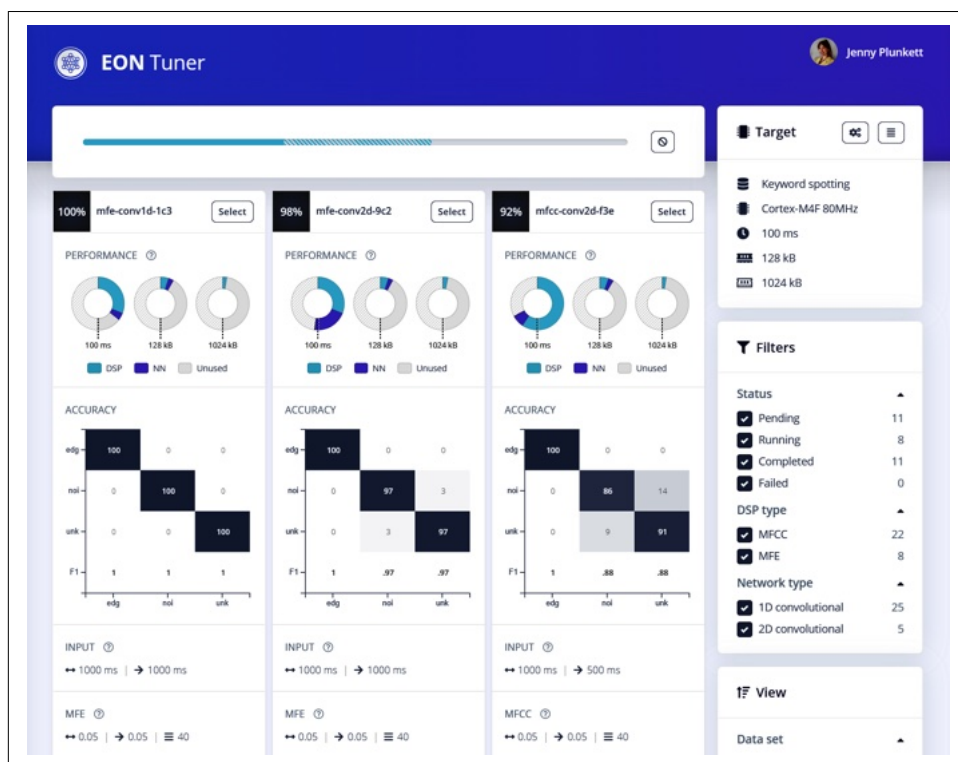
For example, an end-to-end platform can analyze a dataset in order to help a user select the type of ML model best suited to it, or it might provide estimates of on-device performance that can help the developer choose an algorithm or embedded processor. A platform may perform AutoML with the goal of finding the best possible combination of signal processing and model that will fit on a specific device, within a maximum specified latency, or within a limited power budget.²¹ A wide range of ready-to-deploy algorithms or architectures are typically available, preoptimized for various processors.

Platforms can also help teams collaborate. For instance, a cloud-based edge AI platform can act as a central repository for a team's datasets and workflow artifacts. APIs and configurable ML pipelines allow teams to automate routine tasks: so, for example, a new version of a model might be trained, tested, and deployed whenever new data is available. And visualizations and low-code user interfaces make it possible for

²¹ The paper by Kanav Anand et al., “Black Magic in Deep Learning: How Human Skill Impacts Network Training” (arXiv, 2020), shows that prior experience can have a strong impact on performance when tuning ML models by hand, which suggests the value of AutoML tools.

anyone on a team to contribute insight, not just those with existing data science or embedded engineering skills.

Cloud-based platforms also allow developers to benefit from distributed compute without having to administer their own systems. For example, data processing and model training might occur on powerful cloud servers that are managed by the platform, not by the user. This simplifies the process of running AutoML, where experiments may be run in parallel—as shown in [Figure 5-6](#).



*Figure 5-6. An AutoML sweep conducted using Edge Impulse’s **EON Tuner**; with end-to-end platforms, optimization of signal processing and machine learning algorithms can occur hand in hand, incorporating estimates of on-device latency and memory use*

The best end-to-end platforms have a focus on tightening all the feedback loops in the edge AI workflow. They make it possible to iterate rapidly, moving back and forth between development and testing with minimal overhead. This makes it far easier to build a successful product, since you’re able to immediately detect and course-correct on any issues.

Traditionally, getting an algorithm to run on-device for the first time—using real sensor data—has been a tricky process. Some end-to-end platforms provide prebuilt

firmwares for popular development boards, allowing you to capture sensor data and deploy and test models without writing any code. This enables you to close the loop between model development and real-world testing.

Another big benefit provided by end-to-end platforms is the ability to conveniently try out a variety of hardware to find the best fit. The same model can potentially be deployed in an optimized form to a multitude of microcontrollers, SoCs, and ML accelerators with a couple of clicks, allowing a development team to compare performance and determine the right choice for their application. Done by hand, this process could take weeks.

The AI ecosystem is built on top of open source tools, and good end-to-end platforms will allow you to continue using them however you want; they will integrate with industry standard technologies throughout the workflow and won't rely on vendor lock-in to keep you as a customer. You should be able to easily export your data, models, and training code, and it should be simple to create a mixed MLOps stack that incorporates parts of multiple solutions.

End-to-End or Roll Your Own?

You may be wondering which is a better choice: using an end-to-end platform or assembling your own custom set of tools from different sources. At the time of writing, it's pretty clear that the vast majority of projects will benefit from the productivity, structure, and cross-workflow integration provided by an end-to-end platform.

Even if you have strong existing skills in data science, signal processing, or embedded engineering, the process of setting up your own toolchain from scratch can be extremely tough. Depending on your target, you may find that you aren't even able to install the required tools side by side without resorting to containerization in order to isolate dependencies.

Beyond startup costs, individual tools on their own won't provide the type of immediate feedback and seamless iterative workflow required to build a successful project. You'll have to create your own automation between tools, which will result in countless scripts—and further dependencies—that need to be tracked, maintained, and scaled to serve your whole team.

In addition, end-to-end platforms are designed to provide the appropriate guidance to fill in your blind spots. Almost nobody has all of the required skills to build an edge AI application on their own. For example, someone with deep domain expertise is unlikely to also have a strong intuition about which deep learning model architectures are most efficient on a specific model of embedded processor.

The teams developing end-to-end platforms for edge AI have spent years building and improving them—so while it's certainly possible for a sufficiently large enterprise organization to build their own internal platform, it would be a multimillion dollar

investment in time and resources. It's very unlikely that the cost-benefit analysis would make sense, which is why some of the world's biggest, most sophisticated organizations—from government organizations like NASA to industrial giants like Bosch—are users of end-to-end platforms.

A major, valid concern is around flexibility and openness. If a company decides to use an end-to-end platform, what happens if they need to use techniques—for example, specific algorithms—that aren't available in the platform?

Fortunately, the best platforms already account for this and provide easy integration points for interoperability. New algorithms, data stores, deployment targets, and evaluation methods can be connected seamlessly, and APIs allow end-to-end platforms to be interwoven with other tools, including existing internal systems and alternative open source AI tooling.

Another concern is cost. End-to-end platforms are typically supported by an enterprise subscription fee that includes technical support and compute time, with many products also offering a free tier for individual projects. One platform, *Edge Impulse*, has a large, active community of free users who provide support to one another, including sharing example projects for inspiration and technical guidance.

If you don't have a budget, it's absolutely possible to use a product's free tier to build a successful project. If you do have a budget, platforms are typically quite affordable—especially when compared to the time cost of setting up and managing your own environment. The subscription cost typically buys you the type of heavy duty functionality that is required when dealing with big enterprise datasets and large teams.

Given the absurd complexity of the edge AI toolchain, it's very easy to recommend end-to-end edge AI platforms as the best starting point for the vast majority of projects. In the uncommon case that you require a feature that isn't supported, high-quality platforms make it simple to integrate with external tools—so you can use the platform as a foundation and extend it however you need.

At this point, it's worth addressing the fact that the authors of this book, Daniel and Jenny, are part of the team that has designed and developed *Edge Impulse*, an extremely popular end-to-end edge AI development platform. It's always important to take recommendations with a pinch of salt when the people doing the recommendation have a vested interest! Since we work on an end-to-end tool, how can we be expected to recommend anything else?

Hopefully, the history of this book provides some reassurance. One of the authors, Dan, was coauthor of *TinyML*—a book that helped introduce the field of embedded machine learning to a wider audience. *TinyML* introduces the process of building edge AI software using open source tools. At about 500 pages, it's not a short guide—

but it only covers the absolute basics, and it relies on its readers learning both Python and C++. Working directly with low-level tools is not a productive way to go.

Writing *TinyML* inspired both of its authors to try and make life easier for developers. Dan went on to join Edge Impulse as its founding engineer, inspired by a demo where the company’s CEO built and deployed a deep learning model for activity classification live in under ten minutes. The other coauthor of *TinyML*, Pete Warden, is working to simplify machine learning deployment by integrating sensors and ML as closely as possible.

Machine Learning Sensors

Building an effective edge AI product requires a lot of difficult work and specialized knowledge. Another concept that could help make the task easier is the idea of *machine learning sensors*. Proposed in a 2022 paper²² by a team led by Pete Warden, ML sensors are designed to be as simple to work with as ordinary sensors—but to include a dash of intelligence.

For example, a “person detector” ML sensor might be provided as a single chip that includes an image sensor, a processor, and a deep learning model that can identify human beings from images. As an interface, the ML sensor could expose a single digital pin that toggles high when a person is detected or low when there is nobody there.

Integrating with an ML sensor would be far easier than training and incorporating a machine learning model (along with all of the required dependencies) into an embedded application, making it trivial to add intelligence to devices. The trade-off is reduced flexibility—although if required, models could be customized via integration with end-to-end platforms.

At the time of writing, Pete’s company [Useful Sensors](#) is selling the Person Sensor, a small, low-power device that can detect and locate human faces. You can find more general information at [ML Sensors](#).

Summary

We’ve now encountered the people, skills, and tools that are prerequisites to successful edge AI projects. From the next chapter onward, we’ll be taking a journey through the iterative development workflow that real-world teams use to build applications.

22 Pete Warden et al., “[Machine Learning Sensors](#)”, arXiv, 2022.

Understanding and Framing Problems

The next five chapters provide a roadmap for working with edge AI. We'll establish best practices for:

- Viewing the problems you want to solve through the lens of edge AI
- Building datasets that allow you to train models and evaluate algorithms
- Designing applications that make use of edge AI technologies
- Developing effective applications through an iterative process
- Testing edge AI applications, deploying them, and monitoring them in the field

For this chapter in particular, we'll start by introducing a high-level, general workflow for edge AI projects. This should give you a sense of how everything will fit together. After that we'll learn how to evaluate projects to make sure they are a good fit for edge AI, then walk through the process of identifying which types of algorithms and hardware make sense for a given problem—and start to think about planning our implementation.

The Edge AI Workflow

Like any sophisticated engineering project, a typical edge AI project involves multiple tracks of work, some of which run in parallel. [Figure 6-1](#) shows them in context.

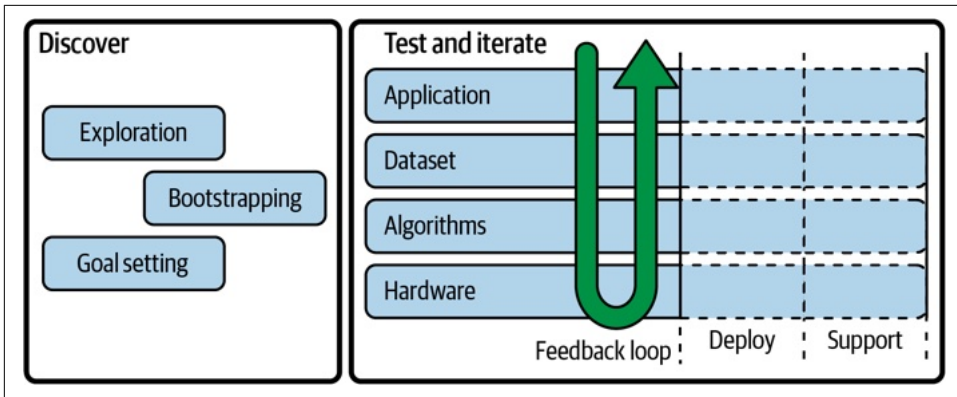


Figure 6-1. The edge AI workflow, grouped into the “discover” and “test and iterate” stages

The process can be split roughly into two chunks—labeled in the diagram as *discover* and *test and iterate*. The first chunk, *discover*, involves developing a deep understanding of the problem you are trying to solve, the resources you have at your disposal, and the space of possible solutions. This is where you do the up-front work of figuring out what you would like (and what is realistic) to achieve.

The second chunk, *test and iterate*, is a continual process of refinement that takes you from initial prototype through to a production-ready application. It spans time before and after development—in machine learning, your application is never truly *finished* but needs to be monitored, supported, and iterated on after being deployed in the field. This continual improvements happens across all parts of your project in parallel—application, dataset, algorithms, and hardware.

The most important part of this process is the *feedback loop* (see [Figure 6-2](#)) that enables continuous improvement. The more feedback you can create between different aspects of your project, the more successful your project will be. For example, the results of your model’s performance on different types of data can be fed back into the data gathering process, helping you build a diverse and representative dataset that covers the entire space of potential inputs.

We’ll be covering this whole workflow over the next few chapters. The *discover* stages are represented in Chapters 6, 7, and 8, while the *test and iterate* stages—including deployment and support—are covered in Chapters 9 and 10.

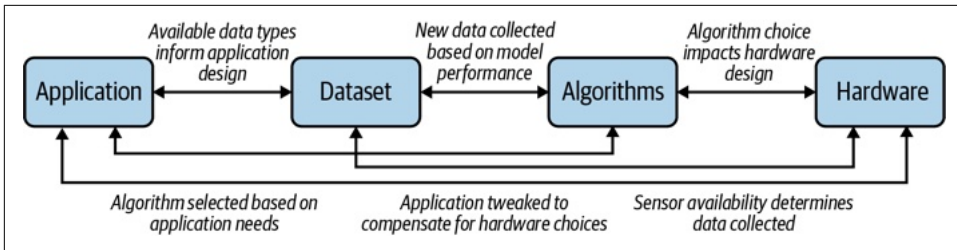


Figure 6-2. A few examples of possible feedback between application, dataset, algorithms, and hardware; the four parts will change and evolve as a project proceeds, and any change in one aspect will need to be reflected in another



Core to the success of any technology project (and arguably any project in general) is the task of managing risk. Edge AI projects are uniquely risky, thanks to their combination of hardware and software and their dependence on complex algorithms and data-driven development.

At each stage in the workflow, we'll learn techniques you can use to keep risk to a minimum and improve your chances of success.

Responsible AI in the Edge AI Workflow

As we've learned, AI applications are especially prone to propagating social harms. There are many types of issues that can lead to unexpectedly poor performance in the real world. This makes careful analysis of potential risks, and their likelihood of causing harm, a critical part of the edge AI development workflow.

It isn't enough to do a single ethical review at the start of a project, or a final one at the end. Since new information will come to light over the course of a project, and many decisions will be taken that have downstream effects, risk analysis needs to be happening at every stage along the way, giving you time to course-correct if required.

In this book we'll be reflecting on ethical design during every step of the process. You shouldn't think of this as an optional extra—it's a part of the core engineering and product management work that is necessary for a successful project. The nightmare scenario for teams working with edge AI is that issues are discovered only after a system has been deployed to production. Nobody wants to be responsible for a product recall or actual harm being caused.

By considering social factors in our risk analysis throughout the development process, we'll maximize our ability to identify challenges before they reach production—and increase the quality of our work.

Do I Need Edge AI?

Artificial intelligence and edge compute are both sophisticated technologies, each involving an entire landscape of considerations. Working with either of them involves making trade-offs between capability and complexity. For many projects, the burden of complexity may outweigh the benefits in capability that come from working with them.

With this in mind, for any potential application, it's very important to try to understand whether the risk is worth the reward. The answer depends heavily on context, including elements such as:

- The specific requirements of the application
- The skills of the team that will be building it
- The available budgets for engineering, data collection, and long-term support
- The amount of time available for delivery

In the following sections, we'll break down the questions that we need to ask in order to decide whether a project is a good fit for edge AI technologies. This is a great exercise to begin with since it will also shed light on many of the other necessary considerations for the discover phase of a project.



While it can be exciting to try to find new opportunities for edge AI, it's important to approach problems with an open mind and not assume that a technological solution is the right answer. Instead of trying to fit edge AI into a problem from the beginning, focus on understanding the problem and designing the right solution to address it. This solution may involve edge AI or it may not.

Describing a Problem

Describing a problem is the first step in figuring out whether edge AI is a good fit to solve it. You should try to summarize a problem in a few sentences and bullet points—keep it short and to the point. A good description should include:

- A high-level summary of the scenario, including any existing solution
- Problems currently faced
- Constraints that must be worked around

In **“Deep Dive: Spotting Rare Wildlife with Trail Cameras” on page 38**, we discussed a possible application for edge AI in wildlife monitoring. Following is an example of how we might capture that use case as a problem description.

Problem Description: Trail Cameras

Summary: Wildlife researchers sometimes need to estimate animal population and activity in remote areas. One way of doing this is to install trail cameras in remote locations to monitor specific animal species. The devices typically use a passive infrared (PIR) motion sensor to detect motion, triggering a camera to take a photograph. Photographs are saved to a memory card. The memory card is collected periodically to obtain the photos, which are then analyzed by researchers.

Problems:

- The PIR can be triggered by nontarget species or moving vegetation, filling up the memory card with useless photos and reducing battery life.
- No animal activity data is available until the memory card has been collected and analyzed.
- It's time consuming and expensive to send somebody to collect a memory card from a remote location.
- If the memory card is collected too infrequently, it will fill up and important data will be missed.
- If the memory card is collected too frequently, then money is being wasted on travel expenses.

Constraints:

- Trail cameras run on battery power and must be energy efficient.
- High-bandwidth data connections are expensive in the field.
- Research budgets are typically low.

The exact format of your problem description doesn't matter as much as the content. By capturing the exact problems and constraints, we can consider them while evaluating possible solutions.

Do I Need to Deploy to the Edge?

At this point in the book, we're very familiar with the model in the section [“To Understand the Benefits of Edge AI, Just BLERP” on page 14](#) for expressing the benefits of edge AI:

- Bandwidth
- Latency
- Economics

- Reliability
- Privacy

BLERP is the perfect tool to help us analyze our problem description and evaluate whether it might benefit from an edge architecture. A good way to do this is to create bullet points for each BLERP term.

To illustrate, let's explore *bandwidth*:

- Due to cost, trail cameras don't have access to much bandwidth. This makes doing work on-device important.
- If we could analyze photos on-device, we could send the resulting information (much smaller than raw images) up to the cloud.
- This could help avoid expensive trips into the field to collect memory cards.

By brainstorming the potential impact of each term, we start to understand whether BLERP benefits are important for this problem. Once you're done brainstorming and summarizing, you'll end up with something like the next BLERP analysis, taken from [“Deep Dive: Spotting Rare Wildlife with Trail Cameras” on page 38](#).

BLERP Analysis: Trail Cameras

Bandwidth

Camera traps are often deployed in remote areas with low connectivity—perhaps with expensive, low-bandwidth satellite as the only option. With edge AI, the number of photos taken can be reduced enough to make it possible to transmit them all.

Latency

Without edge AI, the latency involved with sending a researcher to collect photos from camera traps could be measured in months! With edge AI and a low-power radio connection, it's possible to analyze photos immediately and obtain useful information without having to wait.

Economics

Avoiding trips out into the field saves large amounts of money; so does avoiding unnecessary use of expensive satellite radio.

Reliability

If useless photos can be discarded, the memory card will take longer to fill up.

Privacy

An edge AI camera can discard photos of humans on the trail, preserving the privacy of other trail users (such as local people, or hikers).

In this case, there are clear and obvious benefits to deploying on the edge across multiple BLERP terms. In other cases, it may not be so evident—for example, there might not be benefits under every single term. That doesn't necessarily mean that edge deployment isn't a good fit. As long as there's enough compelling benefit in any category it is worth considering further.

Things that don't work well on the edge

In some situations, you may find that your problem does not fit BLERP very well at all. The following is an example description for a different problem.

Problem Description: Medical Imaging

A medical imaging device creates images that represent the interior of a patient's body. Specially trained doctors use these images to help diagnose certain medical conditions. The device is very large and is typically located in a major hospital. After scanning a patient, the device stores images on a hard disk attached to a computer network. Special software must be used to view the images.

Problems:

- Diagnosing medical conditions by looking at images is challenging and requires medical training.
- If trained doctors are not available, patients may have to wait for a diagnosis.
- Doctors can only view images on certain computers that have the imaging software installed.

Constraints:

- Images represent sensitive patient information that must be kept secure.
- Imaging devices are very large and cannot be moved around.
- Imaging devices are very expensive.

From the description, it's clear that there are some problems worth solving here: it's challenging for people to diagnose medical conditions based on imaging data, and patients may have to wait for a diagnosis depending on availability of equipment or experts. Perhaps there's some potential for AI to help doctors analyze images.

However, the question we need to answer is whether this is a good problem to try to solve using edge computing. To do this, let's try and brainstorm some potential benefits via BLERP (shown in the sidebar).

BLERP Analysis: Medical Imaging

Bandwidth

None. The imaging devices are located in major hospitals that typically have good internet connections, and they are already connected to a computer network. No benefit from reducing bandwidth requirements.

Latency

It would be helpful for patients to have faster access to the diagnosis.

Economics

Performing analysis using AI would reduce reliance on doctors' time, which is expensive.

Reliability

Doctors could use AI analysis to help improve their success at diagnosis.

Privacy

AI analysis could reduce the need to expose sensitive patient data.

Superficially, these sound like compelling reasons. However, if we dig a little deeper, it's clear that most of these benefits are attainable *without* edge computing. Because the imaging device is located in a hospital, there's no major benefit to performing analysis on a potentially resource-constrained "edge" device. Instead, we could use a standard computer, either attached to the hospital's network or in the cloud.¹ Privacy concerns could be addressed using techniques such as federated learning without using any tools specifically associated with edge AI.

In this case, a single fact—that a reliable network connection is already available—makes it unnecessary to use edge compute. But why not use it anyway? Does it make a difference whether we run compute on the edge or not?

Disadvantages of edge compute

While edge compute can have some massive benefits, especially in conjunction with AI, there are some very good reasons why most compute has moved into the cloud over the past decade. If the BLERP framework doesn't highlight some very good reasons to do work on the edge, you may be better off doing your information processing on a cloud server.

Here are some things that can make edge applications a challenge:

¹ While the case could be made that a computer on a hospital network is a form of edge device, many major hospitals have on-site data centers and are not subject to the typical constraints of edge computing.

Development complexity

Writing and maintaining embedded applications is difficult, especially with smaller targets. The simpler your embedded code, the better. Even if an embedded device is required in order to collect data, it may make sense to simplify engineering by hosting the more complex application logic in the cloud.

Staffing

Embedded development requires very specific skills, and while a cloud application can be built and maintained by many types of engineers, embedded engineering talent can be harder to find. If your organization doesn't have access to embedded engineering talent, it may make sense to de-risk a project by leaving compute in the cloud.

Limited compute

Even the most powerful edge devices are nowhere near as capable as a beefy cloud server with access to a GPU. Some applications require levels of compute that would be unreasonable to deliver in the field—for example, some language models are gigabytes in size and require a GPU to achieve low latency.

Deployment complexity

If you plan to update your application after it has been deployed, edge compute can create some problems. Updating edge firmware can be risky—devices can be “bricked” by a bug or a power outage at the wrong moment. Managing the application versions installed across a fleet of devices can also be a challenge. Working around these challenges is possible, but it requires engineering time. It may be simpler just to host your application logic in the cloud, where it can be updated with minimal fuss.

Hardware and support costs

Deploying and supporting a network of edge devices can be expensive. The expense can grow even higher if you require high-end devices with acceleration for machine learning workloads, or custom hardware designed for a specific purpose. Depending on the application, it may be cheaper to use less capable devices to collect data and send it to the cloud for processing.

Flexibility

If the workloads you wish to run outgrow your edge hardware, or your application changes substantially, you may need to buy new hardware to replace it. In contrast, cloud workloads can be scaled and modified at the click of a button.

Security

There is some security risk involved with allowing physical access to the implementations of your AI algorithms. In some cases, cloud compute may help reduce the risk. There'll be more on security later in the book.

As we saw in “[Multi-Device Architectures](#)” on page 82, it’s possible to split compute between edge devices and the cloud. This can be a helpful way to blend the benefits of each, especially when devices are deployed within controlled environments where reliable connectivity and power are available, such as homes or factories. For instance, a smart speaker can preserve privacy by doing wake word detection on the edge while still benefiting from powerful cloud servers to run large, highly sophisticated transcription and NLP models. In an industrial setting, an edge computer vision system could identify potential manufacturing defects with extremely low latency before invoking a cloud model to precisely categorize the defects and determine the appropriate response.

Do I Need Machine Learning?

As we learned in [Chapter 1](#), AI doesn’t always require machine learning. As a category, ML algorithms have various benefits and drawbacks that make them ideal for some applications but of limited utility for others.



It’s important to identify whether your use case is a good fit for ML early on in the development process. ML-based projects involve a substantially different workflow, which will impact your timeline and budget.

For a given edge AI problem, you’ll typically have to choose between a machine learning solution and a rule-based or heuristic solution. As we learned in “[Conditionals and heuristics](#)” on page 101, rule-based systems are designed by human beings using domain knowledge. They can make use of anything from basic arithmetic to incredibly complex physics equations. Here are some examples of applied heuristic algorithms in edge devices:

- A water kettle that shuts off when the temperature reaches the boiling point
- A diabetic insulin pump that dispenses precise doses of insulin based on blood glucose levels
- A driver assist feature that uses traditional computer vision (see “[Image feature detection](#)” on page 92) to identify lane markings and center a car between them
- The autopilot that flies a jumbo jet aircraft on international routes
- The guidance systems of a space rocket headed to Mars

Each of these examples, from simple to sophisticated, relies on domain knowledge. For example, the insulin pump’s algorithm is based on knowledge of the human blood sugar regulation system, and the space rocket’s guidance system is based on

knowledge of physics, aerodynamics (at least for part of the trip), and the handling characteristics of the vehicle.

In each case, the systems involved are governed by strict rules. These rules may be complex, and their discovery might have taken thousands of years of human history, but at the end of the day they can be described with acceptable accuracy by engineers using mathematical formulas.

Often, digital signal processing algorithms are used in conjunction with rule-based systems. A little processing can go a long way in making it possible to react to input using simple rules. For example, a driver assist feature might use DSP in the form of image feature detection in order to reduce a complex image into a set of simple vectors representing lane markings. This makes it much easier to determine whether to steer the car left or right.

The nice thing about rule-based systems is that their limits are known exactly, meaning that it can be proven when they work. A heuristic algorithm is based on a system that is well understood. It's possible to establish the mathematical correctness of an algorithm with regards to the underlying rules it is designed to model. This makes them reliable, trustworthy, and safe.



If there's a rule-based solution to your problem, you should almost certainly choose it. Many problems can be solved in an elegant manner using rules and heuristics, and they can prove much easier to develop, support, and interpret than the machine learning alternative. They also tend to be far less demanding in terms of computational power.

ML sounds exciting, but it's risky to use it unless you have a clear need. Heuristics are what landed man on the moon; there's a fair chance your problem is easier than that.

Unfortunately, not all problems can be solved with rule-based algorithms. Back in [“Conditionals and heuristics” on page 101](#), we encountered their two main weak points:

Problems with rules that are prohibitively difficult to discover.

For instance, it could require a huge amount of research and development to discover the system of algorithms that underlies your complex, noisy, high frequency sensor data. Even if it's possible to describe a system mathematically, it could be out of reach given your budget and time frame.

Problems with large numbers of variables.

For example, there may simply be too many inputs for a rule-based system to be feasible. This is a common problem with image data, which is incredibly highly dimensional and also very noisy. It's tough to write an equation that describes the appearance of a dog.

For nontrivial problems, good implementations of rule-based algorithms may depend on extensive research, domain knowledge, and relevant engineering skills. These are not always available for a given project. The weak spots for heuristics provide an opportunity for ML to shine.

Reasons to use ML

While rule-based systems often depend on a scientific understanding of the processes they interact with, ML algorithms can learn an approximation of the relationships between variables through exposure to the data itself.

This can certainly make life easier. Here are some situations where it could make sense to consider trying ML, in order of validity:

- Your situation and data are too complex or noisy to model by conventional means.
- Too much fundamental research would be required to find rule-based solutions.
- You do not have access to the domain expertise necessary to implement a rule-based system.²

If you find yourself in one of these situations, machine learning can be a huge help. The following sidebar provides a nice example.

Building an Artificial Nose

IoT engineer Benjamin Cabé wanted to **build an artificial nose**—a device that can identify objects and substances by their distinctive smell. His initial goal was to try and distinguish between different types of alcoholic spirits: vodka, rum, and Scotch (although it works for many foods and drinks, as you can see in **Figure 6-3**).

Benjamin had access to a cheap gas sensor designed to measure levels of several different types of gases. To build a rule-based algorithm for distinguishing between the spirits, he would have had to perform a chemical analysis of the various drinks, understand their composition, and work backwards from there to determine which

² As we'll see, domain expertise is still critical in training and evaluating ML models. However, ML-based approaches potentially enable experts with knowledge of a specific problem area to get good results without having to enlist as much outside help in other areas (such as signal processing).

gases to look out for. He'd also have to account for any gases that might be present in the surrounding environment, since they might lead to false positives.



Figure 6-3. Benjamin's artificial nose (Credit: blog.benjamin-cabe.com)

This type of research was beyond the scope of his project. Fortunately for Benjamin, he was familiar enough with machine learning that he knew it might be able to help.

Benjamin used his gas sensor to capture a small dataset of samples from several different types of drinks. He used this data to train a simple machine learning classification model to identify which gas readings were associated with which drink. The project was a success! Benjamin's system was able to discern between different types of drinks—even to the level of telling one brand of whiskey from another.

The ability of machine learning to extract rules from data allowed Benjamin to build a successful project without having to invest in chemical analysis of the drinks, which could have been time consuming and expensive. It also allowed him to avoid writing the sensitive, hand-tuned logic required to account for other gases present in the environment, since the dataset—collected in the real world—already factored this in.

It turns out that complex, noisy data is very common. In fact, most real-world data is a bit of a mess! One of the strengths of machine learning, especially deep learning models, is that given enough data they can learn to account for noise. During training, the parameters of the model are tuned in such a way that they filter out the noise from the data, leaving just the important information—which can be used to make a decision.

In addition, machine learning models are great at identifying the hidden patterns that exist within their training data. Relationships that would be invisible to the human eye, or too complex for our handcoded rules to represent, can become quite clear to machine learning models when they are provided with enough training data.

These advantages make machine learning a great pick if you are confronted with a bunch of noisy data describing unclear relationships. However, ML's use of data can also provide some risk.

The drawbacks of ML

From an engineering perspective, there are three major drawbacks to machine learning: data requirements, explainability, and bias.

It's well known that today's ML depends heavily on data.³ Large amounts of data are often required to train and test machine learning systems. Finding adequate data, and ensuring its quality, is the biggest challenge and expense associated with machine learning.

Data may seem plentiful—after all, don't we have “data warehouses” and “data lakes” filled with decades of IoT sensor data that has been carefully captured and logged? Unfortunately, raw data isn't enough. Most of today's machine learning techniques require data that has been *labeled*, meaning it has been tagged with information describing what it means. This tedious task often falls to humans, which makes it expensive and risky (since it's easy to get things wrong).

In addition, machine learning models can only make sense of situations they have seen before. This makes datasets highly context dependent. A model trained on a dataset collected using a specific type of sensor may not perform well if fed data captured by a different brand. A dataset of typical household items from one country may not be any help when trying to identify items from another.

Research is ongoing to mitigate these issues, and amazing progress is being made, but the fact remains that machine learning typically requires a lot of data. We'll learn more about this topic in the next chapter.

The second big drawback of machine learning, *explainability*, was mentioned already in “[Interpretability and Explainability](#)” on [page 105](#). While there are some highly explainable ML models, the more sophisticated a model gets the more challenging it can be to pinpoint exactly why it is making the predictions it does.

This problem is compounded by the fact that, thanks to their origins in statistics, many types of ML models don't give definitive answers. Ask a question of a

³ The effort to reduce the data requirements of ML algorithms is one of the most important and fascinating strands of ML research.

rule-based system and it will give you a nice, firm response, with clearly visible workings that you can double-check and review. Ask the same question of deep learning model and you'll get a fuzzy probability distribution that indicates the *potential* answer. Trace the answer back through the system and you'll meet an inscrutable mess of linear algebra that is beyond the comprehension of a human mind.

Their probabilistic nature means that ML models are great for dealing with fuzzy situations and nonobvious rules. Unfortunately, this means that their output has some of the same properties. This can be a challenge for many applications.

For example, the code that powers safety-related devices in fields like medical technology, automotive, and aerospace is often expected (through best practices and government regulation) to be *provably* correct. It's very difficult to meet this bar with a probabilistic model whose internal rules can only be gleaned through probing and experimentation.

The third big drawback of ML, *bias*, is a direct result of the first two challenges. We encountered this issue back in “[Black Boxes and Bias](#)” on page 46. When we create ML models, we rely on our datasets to both train them and to validate their performance. Our goal is to produce a model that works well in the real world. However, the real world is a big place, and it's quite challenging to capture all of its possible variation in a finite dataset.

If our dataset only includes a subset of all possibilities, our model may fail to perform correctly on the others. Even worse, because we don't have any examples of those other possibilities in our dataset, we'll have no idea that this is even a problem. Our model may appear to be working great when in fact it has some major issues.

To compound the issue, our model won't necessarily even *tell us* when it is having trouble.⁴ Instead, it will just make its best guess—which could be catastrophically wrong. Without data to test it, and without an easy way to analyze the internal rules of the model and understand where it might fall short, we'll have no way of knowing there is something wrong, beyond our application failing.

A Simple Example of ML Bias

Imagine you're building a system that uses audio to identify faults in an industrial machine. Working in your research and development lab, you collect a big dataset with thousands of audio samples representing correct and faulty operation. When you train and test the model in the lab, it works great. But when you deploy it to a customer's factory, it identifies far more faults than expected.

⁴ There are techniques that will help with this, such as those mentioned in “[Anomaly detection](#)” on page 99, though they may come with a burden of additional complexity.

On investigation, you discover that the model is detecting faults when *adjacent* machines are running. Because your dataset only contains data from your quiet lab, the model never learned to account for the ambient sounds of a factory floor. The model's biases reflect the lab conditions under which it was trained. It does not perform well in the real world.

Since we can never hope to sample the entire world in our dataset, bias is inevitable. However, we can manage the risk by rigorously understanding our application. In some situations, however, the consequences of bias may be so high that the application is not well suited to ML at all. The values held by many societies would not consider it appropriate for ML to be used to make certain life-or-death decisions, such as those resulting from the use of autonomous weapons systems. Similarly, there is much debate about the use of machine learning in automating judicial decisions, including sentencing.

Knowing when to use ML

There's a common saying about solving problems: "If all you have is a hammer, everything looks like a nail." Machine learning is far more exciting than the shiniest of hammers, and it's tempting to try to use it everywhere. Sadly, its complexity, limitations, and inherent risks make it a poor choice in many situations. Mat Kelcey, principal ML engineer at Edge Impulse, is fond of saying "the best ML is no ML at all."

There's no shame in using traditional algorithms to solve problems. As we saw in ["Artificial Intelligence" on page 6](#), the *intelligence* part of AI comes from "knowing the right thing to do at the right time." It doesn't matter to your users whether this knowledge is embedded in the form of an `if` statement or in the form of a deep learning model.

With that in mind, here's a checklist you can use to help decide whether ML might be appropriate for your application:

- There is no existing rule-based solution, and you don't have the resources to discover one.
- You have access to a high quality dataset or collecting one is within your budget.
- Your system can be designed to make use of fuzzy, probabilistic predictions.
- You do not need to explain the exact logic behind your system's decisions.
- Your system will not be exposed to inputs beyond those reflected in its training data.
- Your application can tolerate a degree of uncertainty.

Machine learning models are never perfect. The reason we use them is that compared to human intelligence, they are cheap and they scale well. One big trade-off is that they can fail in unintuitive ways. The decision of whether to use ML is a question of both “Is it good enough?” and “Can we handle the types of errors it is likely to make?”

Automation Versus Augmentation

There’s often an assumption that machine learning should be used to *automate* tasks: taking something that a person would have done and doing it automatically on the cheap. However, a slightly different approach to a problem can mitigate some of the risks associated with the strange ways that ML models can fail.

Rather than replacing a human role, it can be much more powerful to *augment* a person’s ability to do their job. Instead of putting a model in charge, the model’s output is provided as guidance for a person—who is able to combine it with their own insight, expertise, and common sense in order to decide how to act.

A combination of human and machine intelligence has been proven to work better than either technique alone in settings as complex as medicine.⁵ While there are some risks, including **automation bias**, where people over-rely on automated systems and ignore their own intuition, augmentation can be an effective way to derive value from an imperfect system.

Practical Exercise

Here are three scenarios that might potentially call for the use of edge AI. For each one, come up with a problem description and then decide whether it might make sense to apply edge compute, machine learning, or both. You may have to do some online research to fully understand each scenario:

Scenario 1: Fertilizer application

An agricultural producer wishes to save money and reduce environmental impact by applying less fertilizer to their crops. Rather than spraying a field uniformly with fertilizer, they would like to identify the areas within a field that would benefit the most from fertilizer (for example, due to variations in soil quality), and apply it there exclusively. They can understand whether fertilizer is needed by visually inspecting the growing plants.

Scenario 2: Hotel servicing

Hotels traditionally clean guest rooms during the daytime when guests are likely to be out. However, if a guest is still present in their room they may not want to

⁵ As documented in “**Human–Machine Partnership with Artificial Intelligence for Chest Radiograph Diagnosis**”, by Bhavik N. Patel et al. (National Library of Medicine, 2019).

be disturbed. A hotel chain would like to know whether any guest is present in a given room, so that their cleaning staff do not have to knock on the door and potentially disturb them.

Scenario 3: Tire life

Car tires wear out over time, and certain types of wear can indicate mechanical problems. This wear is usually spotted during periodic maintenance. A vehicle manufacturer would like their cars to identify some types of tire wear automatically, so that they can catch problems earlier.

There's no right or wrong answer for any of these prompts; they simply provide an opportunity to apply the analysis techniques we've learned so far.

Determining Feasibility

So, you have an idea for an edge AI project? Your first goal should be to determine its feasibility. There are many things to consider. Perhaps your project would be better served using server-side AI—or maybe your solution requires machine learning, but it's not feasible to collect a dataset. Alternatively, maybe it's the perfect fit!

The first step is to try and come up with an *ideal* solution to the problem in your problem description. If you try to forget about technological limitations and think at an extremely high level, what would you want the system to do?



Preconceived Notions

The purpose of our ideal solution is to give us something to aim for. If we constrain our search for ideas by what we think is feasible, we may miss promising solutions that are not immediately obvious. By keeping things *ideal*, we make sure we don't limit our own creativity.

The ideal solution also helps us avoid the temptation to use a certain technology because we are excited about it. It's a very common trap: we've just learned about some fascinating new technique and have been looking for an excuse to try it—so we overlook another method that would have given better results.

Once we have an ideal solution, we can consider feasibility from a few angles:

- Moral: Should it be done?
- Business: Is there value in doing it?
- Dataset: Do we have the raw materials?
- Technology: Can it be done?

For a project to be feasible overall, it needs to sit in the sweet spot of all four. Let's explore each of these angles one by one, using an example application to illustrate. In the real world, we would base our ideal solution on a detailed problem description. To keep things lightweight, we'll go with this simplified statement.

Edge AI for Warehouse Security

A warehouse filled with valuable products could be an attractive prospect to thieves. The first step in keeping the warehouse safe is to monitor it 24 hours per day.

Our ideal solution would be a system that is aware of every human being on the premises, understands contextually which of those people are *supposed* to be present, and informs a centralized authority of the location of any others.

There's no requirement that our solution is based on AI, or even uses technology. Depending on the context, the best system might be a team of human security guards. In the process of our feasibility investigation, we'll begin to determine the appropriate mix of human and technological abilities that will solve our problem in an elegant way.

Moral Feasibility

Ethical review is one of the most important parts of any feasibility analysis. A project that fails due to unaccounted technology risk may cost a lot of wasted money, but damage resulting from ethical issues is potentially unbounded—it can easily cost a company its reputation, trigger punitive regulatory measures, and cause direct harm to human beings.

“Building Applications Responsibly” on page 41 introduced some of the ethical issues that can affect AI products. As part of a feasibility analysis, it's critical to rigorously explore and document any potential ethical issues that could result from your application. Beyond the product itself, it's important to also understand any potential risks that come from the development process: data collection, deployment, and support.

This process needs to involve participation from the length and breadth of your team—driven by your ethical and domain experts. The material in **“Diversity” on page 126** outlines the importance of maximizing the number of perspectives you are able to draw from.

Some key questions to ask include:

- To whom can the solution cause harm? *Example: Some products (such as AI weapons) are designed to directly cause harm, while others may cause harm indirectly or due to negligent engagement with human stakeholders.*

- Can the required data be obtained without violating the rights of individuals or communities? *Example: Some datasets may not be feasible to collect without violating user privacy.*
- Is it possible to test the product without harming or violating the rights of stakeholders? *Example: An application that provides medical advice may cause harm during testing if the advice given is not reliable.*
- Have the potential risks been documented? How can they be mitigated? *Example: What is the potential harm if an application produces incorrect predictions?*
- Does the application work for all of its potential users? *Example: A safety-related speech detection application might be ineffective with speakers who have regional accents.*

It can be very difficult to understand the potential for indirect harm resulting from a project. For example, what if jobs are placed at risk as a result of an AI project, or if there are trade-offs between profit and environmental impact? Having a broad pool of expert advisors will help you navigate the nuance.

Warehouse Security Application Ethical Feasibility Review

A good way to begin an ethical review is to brainstorm a list of potential issues. Here's an example brainstorm for our warehouse security application. This is not intended to be an exhaustive list!

- Is the product itself ethical? The product might cause harm if it labels innocent people as suspicious. The product might cause harm if it results in the security team being reduced in size.
- Can the required data be obtained ethically? Available datasets may include photographs of individuals who did not consent to being included. If data is collected at the warehouse, employees may not feel comfortable with being included.
- Is it possible to test the product ethically? False alarms could lead to harm if an innocent person is labeled as suspicious. The testing process could distract the security team and lead to security issues.
- Does the application work for all of its potential users? The application may classify people as suspicious at different rates depending on their appearance, not their intent. The application may be difficult to use for security guards who are unfamiliar with similar technology.

Ethics is all about people and processes. One way to reduce the risk of unforeseen ethical issues is to make sure the ethical review is conducted by a diverse group of people from a representative set of demographics, including professionals who have direct expertise in the ethical evaluation of AI systems.

In addition to your own team, this group should include representation of stakeholders potentially affected by the product, as described in “Stakeholders” on page 128.

While the feasibility analysis begins with understanding moral feasibility, your long-term goal should be to enable a process of continuous ethical review during product conceptualization, development, deployment, and support. The work you do at this stage will pay dividends by helping throughout your product lifecycle.

Business Feasibility

There are two main ways that organizational issues can impact the feasibility of a project. First, for an AI application to be successful it needs to provide some clear benefit. In a business context, this could be to customers, executives, or the balance sheet. In a scientific context, it might mean allowing more work to be done with the same budget. At the outset of any project, it is vital to make sure that the proposed work will, if successful, add real value.

Second, AI application development is limited by the practical constraints faced by organizations. For example, there may not be enough budget to collect a dataset of sufficient size to train an effective ML model. Other common constraints include time, expertise, and long-term support from stakeholders.

Proving benefit

One particularly effective way of proving (and demonstrating) the benefits of an edge AI application is called a *Wizard of Oz prototype*. In the story of *The Wizard of Oz*, the titular wizard is first introduced as an impressive supernatural being. However, he is later revealed to be a normal man, hidden behind a curtain, who is controlling the illusion remotely.

In Wizard of Oz prototyping, a mock version of an AI product is produced. It is superficially functional, but in reality its functionality is controlled by hidden human actions. The mock product can be tested and experienced by stakeholders, allowing people to get a sense for how it might work in the real world, and to compare it against other options.

Wizard of Oz Testing for Warehouse Security

To test our warehouse security concept, a basic mobile application is developed. A security guard, equipped with the application, is tasked with his normal patrol. Periodically, a notification is sent to the guard by a human tester in order to simulate what would happen if an AI system detected an intruder. The guard reacts by investigating the issue. He records the amount of time it takes for him to respond.

Subsequent analysis finds that, given the response time, the application is not helpful: in the large warehouse complex, it takes so long for the guard to reach the site that

any thief would have long departed. The expense of developing the application would not be worth it, and the money would be better spent hiring additional guards.

Alternatively, it may have been found that the guard was able to attend to each situation rapidly, and that an AI application would help improve their ability to protect the site. The findings help persuade management that the investment is worthwhile.

In either case, the outcome of the Wizard of Oz test helps the organization save money.

This can be an extremely helpful exercise. Even without the technology portion of the project, the user experience can be explored, analyzed, and refined. If the experience is impressive, it can go a long way toward convincing stakeholders that a project is worthwhile. If things do not work well even in an artificial form, it's a good signal that you should go back to the drawing board.

For any evaluation to be successful, it's important to get stakeholder agreement on what "good" looks like. In our warehouse example, this might mean setting the minimum time it takes for a single guard to respond to an intrusion. During the test, we set up instrumentation that helps us monitor these metrics, deploy a baseline version of our application (for example, the fake Wizard of Oz application), and then evaluate it.

During this process, it's really important to test out the *current* solution alongside the proposed ones. This will provide a clear signal as to whether the solutions being evaluated are genuinely beneficial. This is an important thing to check throughout the development process, not just at the start.

Since we've established a specific threshold for good, it may even turn out that the current solution is good enough to satisfy the stakeholders, in which case the organization can save money by *not* adopting an AI-based solution. But if we prove that there's a benefit, the project can proceed with confidence.

Understanding constraints

Of course, beyond Wizard of Oz prototyping, there are many ways that organizations work to determine the risk, reward, and benefit of new projects. AI projects are no different and establishing that your idea is a good fit for the needs of the organization you work within will be critical to ensuring its success.

Part of this is identifying your organizational constraints and making sure they will not pose any problems. Here are some of the top constraints when developing AI applications:

Expertise

Although AI engineering is increasingly accessible, having experienced AI experts on hand will help de-risk your project. Make sure the skills you need are available to you before you begin.

Timeline

AI development is a data-driven, iterative process, and it's even more challenging to produce accurate time estimates for than traditional software engineering. Make sure you have room for contingencies, such as discovering late in the process that you need to collect more data.

Budget

The three most expensive parts of AI development are payroll, data collection, and testing in the field. If you are an individual or a small organization, compute time for training can become significant—although it will typically not exceed more than a few thousand dollars, since edge AI models tend to be small.

Long-term support

As we'll see in [Chapter 10](#), AI applications need long-term support to remain effective. As the world changes around them, ML models and hard-coded rules will need to be updated and refined. If your project will be deployed for more than a few months, it's important to know whether your organization can afford (and is willing) to support it.

You are the expert in the way your organization works, and it's up to you to understand whether it is capable of supporting the project you wish to develop.

Dataset Feasibility

Alongside technical feasibility, edge AI application development is constrained by data that is available. Machine learning is famously data hungry, but even hand-coded, rule-based approaches require substantial amounts of data to develop and test.⁶

Data collection is difficult, time consuming, and expensive, so it is challenging but vital to understand the data requirements of your project during the feasibility assessment stage. There are two steps to understanding data feasibility:

1. Estimating how much data is required to solve your problem
2. Understanding whether you will be able to obtain enough of it

⁶ Even if you are reproducing a well-known algorithm from a textbook, you'll still need to run data through it to make sure your application works end to end.

Both of these topics are covered in detail in [Chapter 7](#). As we'll see, understanding data requirements involves both research and engineering work. This means that you won't know your actual data requirements until you have invested a significant amount of time in a project.

At this stage, it's OK just to have a ballpark idea, perhaps based on some precedents that you have found via research (as we will discover in [Chapter 7](#)). If it doesn't look like you will have enough data, your project may not be feasible. It's critical that you rule this out at an early stage to avoid wasted development effort.

Dataset Feasibility for Warehouse Security

Our security application revolves around detecting people in a warehouse. To begin understanding the data requirements, it's helpful to identify some similar applications in scientific literature. For example, some web searching around the topic of person detection might unearth a reference to the *Visual Wake Words dataset* ([Chowdhery et al., 2019](#)), a dataset of 115,000 images of people in a wide range of contexts.

The literature shows it is possible to get >95% accurate performance on the dataset with a model that will run on a high-end MCU. This gives us at least some assurance that our use case might be feasible. In addition, the fact that it is publicly available means that we could potentially use it to help train our own model.

At this stage, this might be enough to convince us that our project is feasible from a dataset perspective. There's always some risk that things may not work out—for instance, we may discover that detecting people in a dark warehouse is more challenging than in the typical contexts that appear in the Visual Wake Words dataset. It's up to us to decide the level of risk we are willing to accept, and we can always reduce the risk through experimentation.

Data issues are one of the leading causes of ML project failure, so if you see negative signals during this part of your feasibility check, don't just cross your fingers and hope for the best.

Technological Feasibility

In [Chapters 3](#) and [4](#) we took a long walk through the technologies most important to edge AI. That material will be a great resource as you try to understand the feasibility of your ideas.

The first step is to map your idea onto edge AI concepts and methodologies. Here are some of the key ones, all of which are featured earlier in the book:

Sensors

How will you collect the data you need?

Data formats

What kinds of signals will your sensors output?

Feature engineering

What are the options available for processing raw signals?

Processors

How much compute can you afford, budgeting by cost and energy usage?

Connectivity

What type of communications are available to you?

Problem types

Do you need to perform classification, regression, or something else?

Rule-based or ML

Is it necessary to use machine learning, or can you get away with a rule-based or heuristic approach?

Choice of ML algorithm

Will classical ML suffice, or do you need a deep learning model?

Application architectures

Will you need a single edge device, or a more complex arrangement?

Finally, and most importantly, you'll need to consider the human element. Your eventual solution will be a system composed of people and technology, working together. Any technology decisions need to be viewed through a human lens.

At this point, it's still too early to try to answer all of these questions definitively. Instead, it makes sense to brainstorm a handful of possible solutions: start with four or five rough ideas, but feel free to capture more if they come easily to you. They don't all have to be fully thought out, but you should try to capture elements of the above.

An example in our warehouse security context is shown in the following sidebar.

Brainstorming Ideas for Warehouse Security

Even though we're trying to brainstorm an edge AI system, in most cases it's helpful to begin by establishing the simplest possible baseline. This gives us a known quantity to measure our other solutions against. You should also consider any incumbent solution that you are attempting to replace.

Solution 1: Security team

The warehouse is staffed by a team of trained trustworthy security guards that is large enough to maintain 24-hour visibility over its contents.

In some cases, a nontechnological baseline (or a non-AI solution) may turn out to be the best solution. You should always be open to this eventuality: our job here is to deliver value, not to find an excuse to use AI.

Similarly, since *edge* AI comes with significant challenges, it's also a good idea to reason about possible *cloud-based* solutions. Of course, in some cases there may not *be* a cloud-based solution—but it's always worth thinking about.

Solution 2: Cloud AI

The warehouse has numerous hardwired cameras, each streaming video to the cloud via networked internet access. A cloud server runs deep learning person detection on every video stream concurrently, messaging a security guard via an app if a person is detected in an unauthorized location.

We now have a cloud-based system to reason about. Let's see what happens if we push some of the compute back down to the edge.

Solution 3: Edge server

The warehouse has numerous hardwired cameras, each streaming video to an on-site edge server via a network. The edge server runs deep learning person detection on every video stream concurrently, messaging a security guard via an app if a person is detected in an unauthorized location.

This sounds interesting! It sounds achievable, and there are certainly some technology benefits—for example, we're no longer reliant on an internet connection. Now let's see if we can push even *more* compute to the edge.

Solution 4: On-device compute

The warehouse has numerous hardwired cameras, each of which is equipped with a high-end MCU. The MCU runs deep learning person detection on the camera's video stream, messaging a security guard via an app if a person is detected in an unauthorized location.

In addition to edge versus cloud, there are many other axes we can explore for ideas. For example, how about varying up the sensor type?

Solution 5: On-device compute with sensor fusion

The warehouse has numerous hardwired devices, each equipped with multiple sensors, including video, audio, and radar, along with a high-end MCU. The MCU uses sensor fusion to detect people, messaging a security guard via an app if a person is detected in an unauthorized location.

We now have five possible solutions to explore. Each one has its own benefits and drawbacks that can be analyzed, compared, and debated. It's not always obvious which solution makes the most sense; the correct answer will vary depending on everything from business requirements to the skillset of an organization. The key is to produce a portfolio of ideas so that you can begin to explore the options.

Framing problems

In order to fully explore the technology requirements of any AI solution, we need to be able to *frame* our problem in terms of the tools available to us. In “**Algorithm Types by Functionality**” on page 96, we met an assortment of useful techniques:

- Classification
- Regression
- Object detection and segmentation
- Anomaly detection
- Clustering
- Dimensionality reduction
- Transformation

To solve any problem, we first need to break it down into chunks that can be addressed using these techniques. A given problem may require multiple techniques to solve. For example, identifying intruders in unauthorized locations might involve both object detection (for spotting people) and anomaly detection (to identify when a person is behaving unusually, like sneaking around a warehouse aisle at night).

Each technique may require a different type of algorithm: for example, object detection might require a deep learning model and anomaly detection could be done using classical ML. By breaking down a problem into these techniques, we can better understand the computational burden of the work that must be done—which will aid with brainstorming solutions and help inform our hardware choices.

Any given problem can typically be broken down (or framed) in many different ways, each with its own set of technology requirements. For example, it might also be possible to spot intruders in a warehouse using dimensionality reduction: we could use an embedding model to describe any people in view, then compare their

embeddings to a database, allowing us to identify when a nonemployee is in the warehouse.

This would have different technology, data, business, and ethical considerations than an object detection-based system. As such, framing gives us another tool to explore the space of possible solutions and find something that fits our unique requirements.

Device capabilities and solution choice

For every edge AI project, there are innumerable hardware options. For instance, our warehouse security brainstorm resulted in solutions that could be implemented using MCUs, with an edge server, or in the cloud. Each individual solution has its own gigantic space of hardware choices—for example, for an MCU-based project we must select hardware from a list including dozens of silicon vendors, each with dozens of chips, all configurable in endless ways.

From a feasibility standpoint, we need to understand which of these hardware options are reasonable given the constraints of our problem—as described by our problem description. Constraints might include cost, in-house expertise, existing brownfield (“[Greenfield and Brownfield Projects](#)” on page 26) systems, or supply chain considerations. Capturing these constraints shrinks both the space of hardware options and the space of possible solutions.

After applying constraints, we might discover that there is no solution that fits. For example, in a brownfield project the only available hardware might not have enough memory to run an object detection model with suitable performance.

Table 3-1 provides a reference you can use to understand whether your application is in the ballpark of feasibility for the hardware options available to you. Remember, you can always split your application across multiple device types if you need additional flexibility. We’ll cover this in depth in “[Architectural Design](#)” on page 278.

Making a Final Decision

At this point, we’ve reviewed the feasibility of our project from the perspective of ethics, business, dataset, and technology. We should have enough information to make a call. If none of the solution ideas we have brainstormed seem like a good fit, our next steps should be as follows:

1. Update the problem description with the new constraints that we have identified during the review process.
2. Perform a new brainstorm of solutions, coming up with a new set of possible solutions that factor in our newly identified constraints.
3. Go through the same feasibility review process with our new solutions.

As with everything in AI, this can be an iterative process. You may have to repeat these steps several times in order to clarify your understanding of the constraints and arrive at a solution that works. It's worth having patience and being willing to revisit your assumptions. You will likely end up with a potential solution, even if it isn't what you envisioned at the start.

That said, in some cases there may simply not be a good edge AI solution for the problem you are trying to solve. If that happens, take note of the reasons. Perhaps they are ethical, and a sign that the project is too ethically risky to consider. Alternatively, they may be purely technological—which could mean the project may become feasible at some point down the line, as new hardware and techniques become available.

In any case, even if you have not been able to identify a promising solution, the process of exploring the solution space from a feasibility standpoint will have been incredibly instructive. You now likely know more about this problem space, from an AI perspective, than anybody out there.

If a solution does not pass the feasibility test, resist the temptation to continue anyway. If you've proven that it is too risky, trying to develop the project will result in wasted time and potential harm.



Even the knowledge that there is no reasonable edge AI solution to a problem is valuable information; the fact that you are aware of it is a form of competitive advantage. You may see other organizations wasting time pursuing it—but you will have the confidence to know that their efforts will fail and you will get better results focusing elsewhere.

If you have reached this point and your project seems feasible, congratulations—it's time to start making it a reality.

Planning an Edge AI Project

Edge AI development is a multistage process involving iterative development and potentially unbounded tasks (such as data collection, which as a process is never truly finished). With this in mind, it's important to come up with a plan before developing a solution.

In “[The Edge AI Workflow](#)” on [page 169](#) we saw the various workflow stages and how they are connected by a multitude of feedback loops. As an iterative process, you can spend as long as you like on each section. The two most important aspects of planning are:

- Defining acceptable performance
- Understanding time and resource constraints

Defining acceptable performance

The very first stage of your planning process should be to come up with a set of concrete standards for what acceptable performance will look like for your system. This must be done in conjunction with stakeholders and ethical analysis since an underperforming system may create risks in these areas.

Your task during development will be to step through the iterative process until you have met your goals with regards to acceptable performance. Once they have been satisfied, your stakeholders can confidently sign off on the project knowing that it is working well enough. Your goals should be set realistically—they need to be achievable—but they should also be set in such a way that the project will genuinely deliver value if they are met.

We'll learn about some key performance metrics in [“Useful Metrics” on page 322](#).

Understanding time and resource constraints

It's very important to understand both how much time you have and the resources you have available to help deliver a project—including funds, expertise, and hardware. Estimating development time is famously hard, and this is especially true with AI projects.

Due to the iterative nature of AI development, a traditional waterfall-style development model will not work well. Instead, you'll need to understand and manage risk as you move forward. A good approach is to aim to have something working end to end as rapidly as possible.

You can then iterate on the entire system as well as its individual components. Your very first version may be a Wizard of Oz prototype that uses off-the-shelf hardware and simple logic. You might iterate on this, training a simple ML model to replace the Wizard of Oz component and then creating a custom hardware design.

At every point during the process, your entire system should be tested end to end to determine whether it meets the performance standards you have defined. One benefit of this is that it removes the risk that you will spend all of your time on one stage of the project—for example, training a model—and not leave enough time for the remainder. Another benefit is that, sometimes, you may find that a simpler system than you originally envisioned is more than adequate to meet your performance goals.

We'll dig deeper into the topic of planning when we reach [Chapter 9](#).

Hardware Is Hard

Hardware projects come with unique challenges, and it's often important to begin the process of hardware design early in the process, concurrently with any software work. Once you have identified a suitable processor, obtain a development board so that you can begin working directly with hardware as soon as possible. This will help reduce the risk of unexpected friction when you try to deploy your logic to the hardware.

Due to the realities of the hardware supply chain, you may find that you need to pull the trigger on hardware orders before you feel entirely confident in your application's requirements. This is a major source of risk, but it may be an unavoidable one. It's best managed by getting your application deployed onto some development hardware as quickly as possible. Most silicon manufacturers provide plenty of development boards for this exact purpose.

If you are particularly worried, it may be helpful to select hardware with some excess capacity, giving you some headroom if you underestimate the requirements of your application code. Once you've gone to market and proven your algorithm works, you can potentially design a second version that is more cost effective. Some processor lines are even “pad compatible,” meaning you can start with a more capable processor but easily switch it out for a cheaper version without any changes to your circuit board.

Summary

We now understand the general workflow that applies to edge AI projects, and we've learned how to evaluate problems and generate promising solutions.

In the next chapter, we'll move forward with the first part of the edge AI workflow—collecting an effective dataset.

How to Build a Dataset

The dataset is the foundation of any edge AI project. With a great dataset, every task in the workflow becomes both easier and less risky—from selecting the right algorithm to understanding your hardware requirements and evaluating real-world performance.

Datasets are indisputably critical for machine learning projects, where data is used directly for training models. However, data is vital even if your edge AI application doesn't require machine learning. Datasets are necessary in order to select effective signal processing techniques, design heuristic algorithms, and test applications under realistic conditions.

Collecting a dataset is typically the most difficult, time-consuming, and expensive part of any edge AI project. It's also the most likely place you will make terrible, hard-to-detect mistakes that can doom your project to failure. This chapter is designed to introduce today's best practices for building an edge AI dataset. It's probably the most important section of this book.

What Does a Dataset Look Like?

Every dataset is made up of a bunch of individual items, known as *records*, each of which contains one or more pieces of information, known as *features*. Each feature may be a completely different data type: numbers, time series, images, and text are all common. This structure is shown in [Figure 7-1](#).

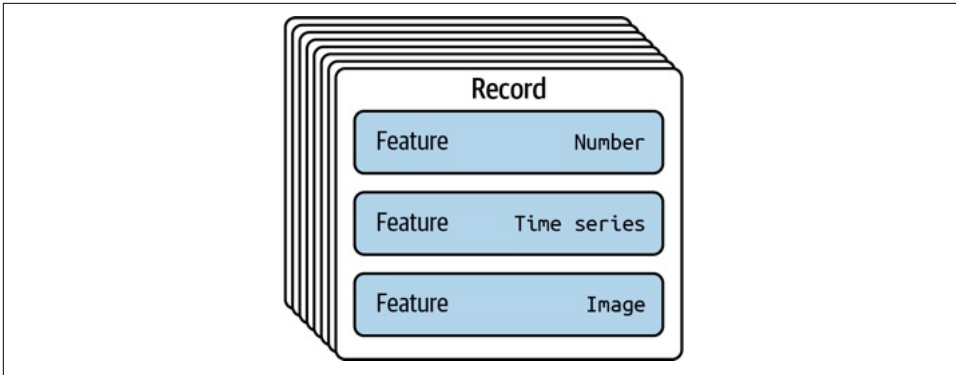


Figure 7-1. A dataset contains many records, each of which may contain many features; features can have different data types.

There are many different names for these components of datasets. Records are commonly referred to as *rows*, *samples*, *items*, *examples*, or *instances*. Features are also known as *columns* or *fields*.¹

Many datasets also contain *labels*, which are a special kind of feature that indicates the desired output of a model trained on that dataset—for example, the class returned by a classifier, or the bounding boxes returned by an object-detection model.

It is common for datasets to include something called *metadata*. This is special data that describes the data itself. For example, a record may include metadata that indicates the exact model of sensor its features were collected with, the precise date and time it was captured, or the sampling rate of the signal that makes up one of its features.



Datasets can be stored in many different ways: on a filesystem, in a database, in the cloud, or even in filing cabinets and cardboard boxes.

The structure of a dataset often evolves substantially during development. This may include changes in what its records and features represent. For example, imagine you are building a dataset of vibration data from industrial machines, since you wish to train a classifier to distinguish between different operational states.

¹ Bear in mind that samples at the dataset level are not the same thing as samples in an arbitrary digital signal. One dataset-level sample (aka record) might contain a feature that consists of multiple samples. As a multidisciplinary field, edge AI has a ton of these confusing terminology collisions!

You may begin by capturing 24 hours of data from 10 different machines. In this case, each record represents a specific period of time from a particular machine. You might then divide these records up, splitting each 24-hour record into sections that correspond to different operational states, and then adding the appropriate labels. Next, you might perform feature engineering on each record, creating additional features that can be fed into a machine learning model.

The Ideal Dataset

An ideal dataset has the following properties:

Relevant

Your dataset should contain information that is useful for the problem you are trying to solve. For example, if you're building a system that uses heart rate sensor data to estimate athletic performance, you'll need a dataset that includes both heart rate sensor data and some measure of performance. If you're planning on using a particular type of sensor, it's typically important that your dataset was collected using a similar device. If you're trying to solve a classification problem, it's important that your dataset contains discriminative information about the classes you care about.

Representative

To be representative, a dataset must include information about all of the different, varied types of conditions that might be encountered in the real world. For example, a dataset to be used in a health monitoring application would need to include data from a wide enough range of individuals to cover all of the different types of people who might be using the application. Unrepresentative datasets will result in bias, as described in [“Black Boxes and Bias” on page 46](#).

Balanced

Beyond being merely representative, ideal datasets contain a good *balance* of information from all of the relevant types of conditions. Many types of machine learning algorithms work best with balanced datasets, including deep learning models.

For example, in the US, 76% of commuters use a car to get to work, while only 10% of commuters use a bicycle.² If we were training a model to count vehicles driving across town, it would be important to use equal amounts of data for cars and bicycles—even though bikes would represent a minority of encounters. Otherwise, the model may perform better at identifying cars than bikes. This is another common way for bias to enter your system.

² According to [Statista Global Consumer Survey, 2022](#).

Reliable

An ideal dataset is consistently accurate. It contains as few errors as possible, and if there are errors, they exist uniformly across the data—as opposed to being concentrated in certain categories.³ If there’s noise in your data (which is common for sensor applications), it should be the same type and magnitude of noise that is present under real-world conditions. For example, we might want to train a classification model to identify different genres of music, using a dataset of music samples. In our dataset, it’s important that each sample is labeled with the correct genre, and that the samples include a similar amount of background noise as the expected real-world conditions.

Well formatted

The same data can be formatted in numerous different ways. For example, images can be represented in an infinite variety of different formats, resolutions, and color depths. In an ideal dataset, the data is formatted in the way that best suits the tasks you are using it for. At the very least, it’s helpful for a dataset to have consistent formatting throughout its samples.

Well documented

It’s critically important to understand where a dataset came from, how it was collected, and what all of its fields mean. Without this information, you will not be able to determine if the dataset meets your requirements. For example, imagine you wish to use a dataset of sensor data sourced from the internet. Without good documentation, you will have no way of knowing whether the data is relevant: it may come from sensors that are not equivalent to the ones you intend to use.

Appropriately sized

A machine learning model can learn the hidden rules in almost any system—as long as it is provided with sufficient data. For example, you may wish to train a model to identify different types of tennis shots using accelerometer data. If the dataset only includes a few samples of each shot, the model may struggle to learn a general representation of what characterizes each one. To generalize, more samples for each shot type may be necessary—the more the better.

However, larger datasets result in longer training times, and they are more difficult to work with from a technical perspective. In addition, different problems require different amounts of data to solve, so for every project there is a point of diminishing returns for collecting further data. Your goal should be to collect enough data to solve your problem.

³ See “The Uneven Distribution of Errors” on page 234.

If you are mostly using your dataset for testing (versus training a model), you can get away with a smaller one—but it's important that your dataset is big enough to be representative and balanced.

As you might expect, it's tricky to create a dataset with all of these ideal properties. As you build your dataset, you will likely have to do some work to get it into shape.



Datasets for Evaluation

While building and testing an edge AI system in the lab requires one type of dataset, evaluating its performance under *real-world conditions* may require another. **Chapter 10** will introduce several ways to evaluate edge AI systems and will explain how to collect the right types of data for the task.

Every AI project involves the distillation of domain expertise from a human mind into a computer system. The process of building a dataset is where the majority of this work happens. It must be conducted with care, intent, and careful consideration. The good news is that if you get it right, you'll massively increase your chances of success.

Datasets and Domain Expertise

Domain experts, also known as subject matter experts (SMEs), are the people with deep knowledge about the problem you are trying to tackle. No matter what the niche, there are people who have studied, experienced, and learned the subject inside out.

It's important to view domain expertise in your problem area as potentially distinct from the knowledge required to work with AI algorithms, signal processing, embedded engineering, or hardware design. While it's possible for a domain expert to also have skills in these areas, the fact that somebody is, say, a machine learning expert does not automatically make them qualified to design AI systems to solve any problem under the sun.

For example, imagine you are building an edge AI product for the healthcare market. In addition to hardware and software engineers, and people versed in building AI applications,⁴ your team will need to include domain experts who have a genuine understanding of the healthcare problem you are trying to solve. Otherwise, you'll risk building a product that does not work in the way you expect it to.

⁴ **Chapter 9** will talk more about the composition of the teams required to build edge AI products.

Datasets and domain experts are intimately connected. Every AI product reflects the dataset used to develop, train, and test it. When products use machine learning, the algorithms are dictated directly by the data. But even handcoded algorithms are only as good as the data that is used to test them.

This means that the outcomes of your entire project are dictated by the quality of your dataset. Moreover, the only people in your organization who are qualified to *understand* that quality are your domain experts. Their knowledge of the problem you are trying to solve must guide the construction and curation of your dataset. No matter how many talented data science experts you have on your team, their skills will be redundant without proper insight into the problem at hand.

In essence, your dataset acts as the main vector for domain expertise both within your product and your organization. Since it is constructed using knowledge from domain experts, it ends up being a representation of their knowledge in digital form—almost like an application programming interface (API) that provides access to their captured insights.

This encoded knowledge will be used by the rest of your team to help build your application. For example, the engineers working on your algorithms will use the dataset to tune or train them, and those responsible for testing your application will use it to ensure that it works well in all the situations you need it to.

All of this makes it critical that you have sufficient domain expertise on hand. In addition, since your domain experts may not necessarily be experts in building and assessing datasets, you will need them to work closely with the members of your team who have data science skills. It will take collaboration to build an effective dataset.

But what if you don't have access to domain expertise? The answer is frank and perhaps unwelcome. If your team lacks domain expertise in your problem area, it would be irresponsible for you to attempt to build a product. You will lack not only the knowledge to build an effective product but also the insight to understand whether you have built an *ineffective* one.

Data, Ethics, and Responsible AI

The quality of your dataset will shape your application's social consequences more than any other factor. No matter how carefully you have worked to investigate the ethical issues around your project, and to design an application that delivers benefit while being safe, the limitations of your dataset dictate your ability to understand and avoid unintentional harm.

From the perspective of responsible AI, your dataset provides two core things:

- The raw construction material for the system of algorithms that you are attempting to create
- Your most powerful tool for understanding the performance of your system

Your dataset is your only detailed representation of the real-world situation that your system is designed to interact with. Your entire application development feedback loop is mediated by it. As raw construction material, if your dataset is in any way lacking it will invariably and unavoidably lead to bad performance of your system. Even worse, the same failings will impact your ability to understand—or even notice—that the system is underperforming.

This is especially true for edge AI projects, since the nature of their edge deployment means that it is often challenging to capture information about how they are performing in the field. Your dataset often represents your only chance at evaluating your model's performance with any real precision.

With this in mind, it's beyond critical that you spend enough time on getting this part right.

Insufficient Data Causes a Tragic Death

In “[Building Applications Responsibly](#)” on page 41, we learned about a failure in Uber's self-driving car system that resulted in the death of a pedestrian. While the failure was systemic and involved the poor design of procedures and safety systems, the core issue was a lack of adequate training data.

The following quote, from a [Wired](#) article on the incident, provides some illustration:

The report says that the Uber vehicle, a modified Volvo XC90 SUV, had been in autonomous mode for 19 minutes and was driving at about 40 mph when it hit 49-year-old Elaine Herzberg as she was walking her bike across the street. The car's radar and lidar sensors detected Herzberg about six seconds before the crash—first identifying her as an unknown object, then as a vehicle, and then as a bicycle, each time adjusting its expectations for her path of travel...

Herzberg, walking a bike loaded with plastic bags and moving perpendicular to the car, outside the crosswalk and in a poorly lit spot, challenged Uber's system. “This points out that (a) classification is not always accurate, which all of us need to be aware of,” says Rajkumar, “and (b) Uber's testing likely did not have any, or at least not many, images of pedestrians with this profile.”

—Aarian Marshall and Alex Davies, *Wired*

Human beings are good at something called “zero-shot learning,” the ability to recognize and identify objects we have not seen before based on our prior understanding of the world. At present, it's very challenging to build AI systems that are capable of this.

If a human driver had seen Elaine Herzberg as she was crossing the road, they would have immediately understood that they were seeing a person wheeling a bicycle and could have applied the brakes immediately.

In some locations it is relatively rare to see a person wheeling a bike loaded with plastic bags across the street, so it was unlikely that Uber's self-driving dataset would include many instances of it. However, as we learned earlier, an ideal dataset is *balanced*: even if a situation is rare, the dataset will still contain sufficient instances of it to be able to train a model—or at least to be able to evaluate the model and show that it is ineffective in that situation.

Since Uber's self-driving algorithms weren't capable of zero-shot learning, they depended on their dataset in order to learn about situations such as Elaine Herzberg's crossing of the road. But since the dataset was imbalanced, it didn't contain sufficient examples of that type of situation for the model to be able to learn to recognize it.

This tragedy highlights one of the greatest challenges in dataset construction. The real world is varied to an almost absurd degree. There are a near infinite variety of human beings, bicycles, plastic bags, roads, and lighting conditions. It's impossible for a dataset to ever capture all of the possible combinations of these things.

Further, there are so many possible combinations of variations that even a domain expert may be unaware of some of them. For example, even if an expert on urban traffic was tasked with identifying objects that are critical for inclusion in a self-driving dataset, they may not have thought of including a bicycle loaded with plastic bags.

Minimizing Unknowns

As in Donald Rumsfeld's infamous quote, in dataset creation there are both "known unknowns" and "unknown unknowns." The only way to build an effective dataset is to minimize both of them. There are two main ways to do this.

The first, and most effective, is to limit the scope of the situation your model is going to interact with. A general-purpose, self-driving system could be considered a nightmare scenario for dataset construction. Self-driving cars must navigate across vast distances of messy reality, from city streets to country roads, encountering almost anything that can possibly be imagined. There's no possible way you can build a dataset that is representative of all that variety.

In contrast, consider a self-driving golf cart that is restricted to driving around a golf course. While it's still possible that it might encounter a bicycle as it roams around the fairways, it's quite unlikely—so it may be easier to build a dataset that is representative of the typical set of circumstances that exist in normal use. In the case

of a self-driving car, the principle of limited scope may guide you to limit vehicle operation to the geographic area that its algorithms were trained on.

The second way to avoid unknowns is to improve your domain expertise. The more expert knowledge available about a situation, the less “unknown unknowns” there may be. If Uber had employed a more effective panel of urban transportation experts to help build and evaluate their dataset, then they might potentially have averted a tragedy.

On a practical level, we can also derive a firm rule from this insight: we should never build edge AI applications for real-world usage in areas where we do not have access to domain expertise. Without domain expertise, the field of “unknown unknowns” is unbounded in size. It’s almost guaranteed that we will run into them.

Ensuring Domain Expertise

The amazing tools that now exist to assist with training machine learning models have massively lowered the barriers to entry. Unfortunately, this creates the temptation for developers to build applications in areas where they lack domain expertise.

During the COVID-19 pandemic, thousands of well-meaning researchers and engineers created projects designed to diagnose infection using medical imagery. A 2021 review published in *Nature Machine Intelligence*⁵ identified 2,212 such studies. Of these, only 62 passed a quality review, and not a single model was recommended for potential clinical use. The majority of issues found could likely have been resolved had clinical and machine learning domain expertise been applied.

The peer review system of academia provides a mechanism to analyze and critique attempts to solve problems with AI. However, in industry there is no such system. Models are deployed inside black box systems, with no accompanying documentation, and are allowed to interact with real-world systems in an unmediated and unmonitored way. This massively increases the chance that a catastrophic issue may make it into production.

Those of us who work in edge AI have a profound responsibility to build systems for ensuring adequate quality, both internally within organizations and through cross-organizational collaboration. A focus on dataset quality, and the corresponding deployment of domain knowledge, must be at the heart of any serious effort.

5 Michael Roberts et al., “Common Pitfalls and Recommendations for Using Machine Learning to Detect and Prognosticate for COVID-19 Using Chest Radiographs and CT Scans,” *Nat Mach Intell* 3(2021): 199–217, <https://doi.org/10.1038/s42256-021-00307-0>.

Data-Centric Machine Learning

Traditionally, machine learning practitioners have focused on selecting the best combination of feature engineering and learning algorithm to get good performance on a particular task. In this framework, datasets are considered fixed elements that are rarely manipulated beyond some basic cleanup. They provide an input, and a reference for correctness, but they are not considered something to be tuned and tweaked.

In recent years, it has been increasingly recognized that datasets should not be thought of as static objects. The makeup of a dataset has a strong impact on the performance of models that are trained on it, and practitioners have begun to modify datasets in order to achieve better performance on tasks.

This new way of thinking is referred to as “data-centric machine learning.” In a data-centric workflow, more emphasis is placed on improving the quality of datasets—as opposed to tweaking the parameters of algorithms.

Data-centric ML follows the age-old computing principle of “garbage in, garbage out”—the idea that it is unreasonable to expect a computer program to make good decisions if it is provided with poor-quality input.

Data-centric workflows and tools help developers understand the quality of their data and how to remedy issues within it. This could involve:

- Fixing or removing mislabeled samples
- Removing outliers
- Adding specific data to improve representation
- Resampling data to improve balance
- Adding and removing data to account for drift

It’s important to acknowledge that all of these tasks require domain knowledge. In some respects, the shift toward data-centric ML is a recognition of the importance of domain knowledge in getting satisfactory performance from machine learning systems.



Drift is the idea that the real world changes over time. Datasets and models must be continually updated to account for it. We’ll cover drift in detail later in this chapter.

The data-centric approach considers datasets to be living entities that require regular maintenance. This maintenance is worthwhile because it both reduces the amount of

algorithm work that needs to be done to train an effective model and reduces the amount of data that is required. A high-quality dataset with fewer samples is often superior to a low-quality dataset that has more.

Successful real-world projects often combine a data-centric approach with modern tools that automate the discovery of effective algorithmic parameters (such as AutoML systems, which we learned about in “Automated machine learning (AutoML)” on page 150). Presented with high-quality data, these tools can do an excellent job of exploring the design space and coming up with effective models.

This is the approach recommended by this book. It empowers domain experts to focus on the data that reflects their areas of expertise, while handing the grunt work of algorithm tuning to an automated system. These automated systems rely on high-quality data in order to evaluate models and select the best one for a task. By focusing on dataset quality, developers simultaneously improve both the raw inputs to the system and the mechanism for evaluating it.

Estimating Data Requirements

The most common question that people ask during the initial stages of an edge AI project is “How much data do I need?” Unfortunately, this isn’t a simple question to answer. Data requirements vary massively from project to project.

Typically, the data requirements of machine learning projects are much higher than those that rely only on signal processing, heuristics, and other handcoded algorithms. In these cases, you will primarily use data for testing—so while you will still need enough to guarantee that your dataset is representative, you won’t need the vast numbers of examples of each type of condition that is required by many machine learning algorithms.

The best way to know the data requirements for a problem is to look for precedents. Are there examples of this type of problem being solved that give you a sense of how much data is required?

The web is your best friend in this regard. A quick search will turn up scientific papers, benchmarks, open source projects, and technical blog posts that can provide a ton of insight. For example, the website [Papers with Code](#) has a “State-of-the-Art” section that lists benchmark datasets for various tasks and the performance that has been attained on them over time.

If we were developing a keyword-spotting application, we could take a look at the [results for the Google Speech Commands dataset](#), which at the time of writing has been solved with 98.37% accuracy. Digging into [the dataset itself](#) tells us that the task involves classifying among 10 keywords, and that the dataset has 1.5–4k utterances

for each keyword. If our task is sufficiently similar, these numbers give us a ballpark figure for how much data we might need.

Another good idea is to explore tools in your problem domain that are specifically designed to work with minimal data. Deep learning models can be especially data hungry: are there classical ML alternatives that could fit your use case? If your problem requires deep learning, are there any pretrained feature extractors available that might fit your use case, via transfer learning, or could you train one using an existing dataset?

For example, in the keyword-spotting domain, a paper from researchers at Harvard, “Few-Shot Keyword Spotting in Any Language” (Mazumder et al., 2021), provides evidence that a keyword-spotting model can be trained with only five examples of a keyword, along with a substantially larger dataset to verify its performance.

Table 7-1 provides a relative indication of how much data is required to train machine learning models for some common tasks.

Table 7-1. Data requirements for common tasks

Task	Relative data requirements	Notes
Time series classification	Low	DSP can do a lot of the hard work, making this task easier to train.
Time series regression	Medium	This is more challenging than classification due to finer-grained labels.
Nonvoice audio classification	Medium	Varied data is required to account for the diversity of background noise and environmental acoustics.
Voice audio classification	Low or High	This typically demanded many hours of data, but new few-shot techniques reduce this.
Image classification in visible spectrum	Low	Transfer learning using models trained on public datasets makes this a relatively simple task.
Object detection in visible spectrum	Medium	Transfer learning is available, but this is more challenging than classification.
Vision models for nonvisible spectrum	High	Transfer learning is not typically available, increasing data requirements.

It’s important to remember that even these relative requirements are highly approximate—they may vary greatly from project to project, which is why it’s difficult to give exact amounts. Data requirements will continue to evolve as new tooling and technology becomes available. The more common a task, the more likely there are signal processing or learning techniques that can help reduce data requirements.

The largest datasets in machine learning are the massive text datasets used for training language models from scratch. This is typically not a required task in edge AI, which limits the upper bounds of datasets that we’re required to deal with.

A Practical Workflow for Estimating Data Requirements

After our initial research, the next step is to dig out the tools and start doing some experimentation. Our core task here is to understand whether, given sufficient data, the feature engineering and machine learning pipeline we have selected will be able to achieve good enough results.

This task naturally ends up as part of the iterative approach to application development, which we'll be covering in more detail in [Chapter 9](#). For now, we'll go over the relevant tasks at a high level.



Defining what “good enough results” means for our project is an important step that will be explored in [“Scoping a Solution” on page 271](#).

Here is the basic process for estimating data requirements:

1. Capture and refine a small dataset. To be effective in estimating data requirements, this dataset should meet all the requirements of the ideal dataset described earlier in the chapter, aside from being appropriately sized. The rest of this chapter will help you understand the processes required to get it into good shape.
2. Based on your research into potential model types, select a candidate model. It's a good idea to begin with the simplest model that seems reasonable, since simpler models are typically easiest to train. Don't fall into the trap of wanting to try a hot new technology without having ruled out the simple and elegant alternatives.
3. Divide your dataset into multiple, same-sized chunks. Each chunk should have close to the same balance and distribution as the original dataset. To achieve this, you should use stratified random sampling.⁶ Begin with approximately eight chunks.
4. Train a simple model on one chunk of the dataset and record the resulting performance metrics. It may be helpful to use a hyperparameter optimization tool, as described in [“Automated machine learning \(AutoML\)” on page 150](#), to rule out the effects of hyperparameter selection.
5. Add another chunk to your training data, so it's now made up of two chunks worth of data. Train the same model again, from scratch (continuing to use hyperparameter optimization if you decided to use it) and record the metrics again.

⁶ This term is explained in [“How is data split?” on page 258](#).

6. Continue the process, adding a chunk of data, training the model, and collecting the performance metrics, until you are using the entire dataset.
7. Plot the performance metrics on a chart. It will look something like one of the charts in [Figure 7-2](#).

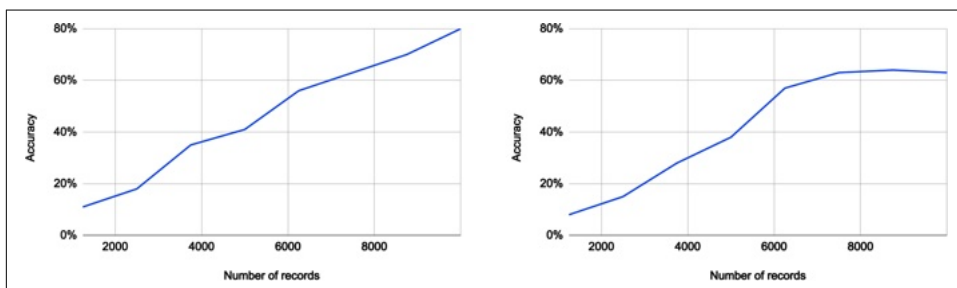


Figure 7-2. Each chart shows how a performance metric (in this case, accuracy) changes with the number of records. The chart on the left shows a situation where adding more data would likely result in better performance. The chart on the right shows a plateau: adding more data of the same type would be unlikely to result in much performance improvement.

In both charts, we can see that the model’s performance increases every time we add more data. By looking at the shape of the curve, we can understand the impact that new samples are having. In the lefthand chart, the curve indicates that performance would likely continue to increase if we were to add more data. The trend line provides a way to approximately estimate how much data would be required to achieve a given performance.

In the righthand chart we can see that the model has already reached a performance plateau. Adding more of the same type of data is unlikely to have any effect. In this case, it may be worth testing a different machine learning model or algorithm, or trying to improve on our feature engineering. You might also consider improving your dataset in ways beyond just increasing its size: perhaps it contains a lot of noise that could be reduced.

Of course, this technique depends entirely on the assumption that our dataset is “ideal.” In reality, there are likely to be issues with your dataset—and limitations of your feature engineering and machine learning algorithm—that prevent the real-world performance from matching the trend line as data is added. However, it’s still useful to obtain a ballpark figure—it can help you plan for the effort of collecting more data.

This technique will *not* tell you whether your dataset is representative, balanced, or reliable. These parts are entirely up to you.

Getting Your Hands on Data

A large part of the challenge of constructing a high-quality dataset is sourcing the data itself. These are some of the typical ways that data can be obtained:⁷

1. Collecting an entirely new dataset from scratch
2. Outsourcing the collection of data to another team or a third party
3. Using data from a public dataset
4. Repurposing existing data from a partner or collaborator
5. Repurposing existing data from an internal data store
6. Reusing data from a previous successful AI project

As you can see, there are a range of potential options. However, it's unlikely that all of them will be available for a given project. For example, if this is your first edge AI project you may not have any existing data to repurpose.

Each of these sources represents a different compromise between two important things: risk of quality issues and effort (which translates into cost). **Figure 7-3** shows how each of them compare.

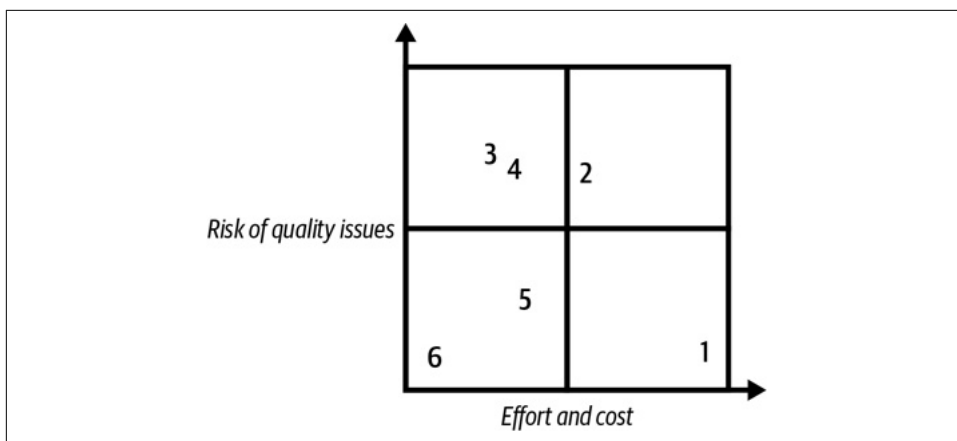


Figure 7-3. Data sources organized by quality risk and effort/cost.

The more control of the data collection process you have, the better you can guarantee quality. By far, the best option is being able to reuse data that you have used successfully in the past (6). If you're lucky enough to be able to take this option, you'll

⁷ Additionally, you may find yourself combining data from two or more of these sources.

already know the quality of the data, and you won't have to invest much effort to reuse it—as long as it remains relevant.

It's quite common for organizations to have existing stores of data that can be reused for AI projects (5). In these cases, it's potentially possible to understand the quality of the data since it was collected internally. However, it may require a bit of effort to get it into the form required for your AI project. For example, a manufacturer may already be collecting machine data using an existing IoT system. In this case, the data's provenance and the collection techniques are known, which helps reduce risk. However, the data may not be in a ready-to-use form and will likely require some cleanup. Existing data often lacks labels, which are expensive to add.

Often, data may be available from a partner organization or collaborator (4). In this case, since someone else has collected the data, there's no way to guarantee quality—and some cleanup may be required to make it usable.

The same is true for public datasets (3), which are typically used for academic research. Public datasets have the advantage of being scrutinized by many pairs of eyes, and may have useful benchmarks available, but they tend to either be cobbled together from low-quality data sources and contain a lot of errors or be very small. They may require significant cleanup to be usable, and they may contain biases that are not documented or obvious.⁸

It may be possible to outsource data collection to another team in your organization, or to a third party (2)—there are entire companies that exist to assist with data collection and labeling. While in theory you have significant control over the data collection process, this may still involve significant risk, since it's very difficult to guarantee that the third party will follow the correct procedures. This is typically a costly approach.

The approach that has the lowest risk is getting your hands dirty and collecting the data yourself (1). When the people who are designing the dataset and algorithms are the same ones who are leading the data collection effort, the risk of miscommunication or undetected error is minimized (assuming they have the required domain knowledge to do the job right). Unfortunately, this is also the most costly approach.

⁸ That said, even messy public datasets can be helpful in *evaluating* algorithms—they can be a good source of interesting corner cases.

Overcoming Data Limitations

It's often challenging to obtain enough data. If you run into data constraints, a good way to unblock your progress is to determine whether a slightly simpler approach to solving your problem might still work.

For example, imagine you're building a predictive maintenance system for a production line. Your initial goal might be to identify when a specific fault is likely to occur soon so that you can schedule preemptive repairs.

As you begin development, you might discover that no data is available that shows the specific fault you care about, and that you don't have the budget to collect any. Rather than give up on your project, you could instead modify your goals.

Instead of predicting when a specific fault is likely to occur, you might decide to create a more general system that identifies when *any* sort of change has happened. This system could be trained on nominal data, which may be easier to collect.

This general system might result in more false positives but depending on the situation it could potentially still help solve your maintenance issue and solve costs overall. By simplifying your goals, you're able to reduce data requirements and make the project feasible.

The Unique Challenges of Capturing Data at the Edge

The more common the use case, the more likely you are to find an easily accessible dataset that has been reviewed for quality. This makes life difficult for many edge applications, since there's a massive variety of niche use cases and exotic sensors. In addition, commercial entities don't tend to share their datasets since they represent potential competitive advantage.

If you need to collect your own data, there are some specific challenges to navigate:

Connectivity and bandwidth

Edge compute is often used in applications where bandwidth and connectivity is limited. This means that it can be difficult to collect data in the field. For example, if you are building an AI-powered camera for monitoring the movement of farm animals, you might wish to collect images of animals from the field. However, this may not be possible given the remote locations of many farms and the lack of connectivity.

To get around this issue, you could temporarily install networking hardware on-site (for example, a satellite connection might be used in remote regions)—or

rely on *sneakernet* capabilities.⁹ This is very expensive, but it may only need to be done temporarily, at the beginning of a project.

Brownfield hardware

As we learned in “[Greenfield and Brownfield Projects](#)” on page 26, it’s quite common to deploy edge AI applications on existing hardware. Unfortunately, the hardware was not always designed with data collection in mind. To succeed at data collection, brownfield hardware needs sufficient memory to store samples, sufficient networking capability to upload them, and sufficient energy budget to permit the process to occur frequently.

To work around this problem, it may make sense to temporarily install new hardware on-site that is better suited to the challenge of collecting data. Dedicated industrial [data loggers](#) exist for this purpose, and industrial-grade rapid IoT development platforms like [Arduino Pro](#) can be convenient to use.

Greenfield hardware

If an edge AI project involves the creation of new hardware, working hardware is likely not available until some time has passed. This can be a major challenge since it’s important to make progress with dataset and algorithm development in parallel with the hardware development process. It’s tricky to even know what hardware is required until at least some algorithm development has been done.

In this case, it’s important to try and get some representative data as quickly as possible. Similar to the brownfield case, it could make sense to use a rapid IoT development platform to start collecting data before your production hardware is ready.

Sensor differences

Sometimes, the sensor hardware currently available in the field may not be identical to the hardware you plan to use in a new device. In some cases, even the placement of a sensor may be different enough to cause problems.

If you suspect sensor differences might be a challenge, you should try as early as possible to evaluate the sensor data side by side and determine whether it is different enough to present a problem. If so, you can use the same approach recommended for working with inadequate brownfield hardware.

Labeling

One of the biggest challenges in working with edge AI data is the availability of labels. For example, imagine you are collecting accelerometer data from the ear tag of a farm animal with the goal of classifying how it is spending its time

⁹ The age-old practice of transmitting data by carrying a storage device from one place to another. See this [Wikipedia article](#).

between eating, walking, and sleeping. Even in a situation where it is trivial to collect the raw sensor data, it may be challenging to correlate this data with the actual activity of the animal. If you could already identify the animal's activity using the data, your project would not be necessary!

To work around this problem, you can try to collect additional data that may not be available during the normal operation of the device you are designing. For example, during initial data collection, you may choose to collect both accelerometer data and video from a camera that shows the animal's activity, with timestamps for both. You can then use the video to help you label the data.

Synthetic Data

Depending on your application, it may be possible to add *synthetic data* to your dataset. This is data that is created artificially rather than being captured. If it's realistic enough, it may help you meet your data requirements.

Here are some types of synthetic data:

- Simulation-based (e.g., time series from virtual sensors in a physics-based simulation of a machine)
- Procedural (e.g., algorithmically generated audio designed to simulate environmental noise)
- Generated images (e.g., realistic 3D renders, or images output directly from sophisticated deep learning models)

Synthetic data is typically helpful as a way to extend a dataset that contains real data. For example, artificially generated background noise could be mixed with real captured audio to help train a classifier to distinguish between background noise and human speech.

The concept of training a model on entirely simulated data, and then applying it to solve real-world tasks is known as *Sim2Real*.¹⁰ It is considered one of the most important and challenging tasks in robotics and is an area of ongoing research.

There are various software tools designed to help create synthetic data, or you can write your own with the help of a domain expert. At the time of writing, tools for generating artificial data are rapidly improving and available with commercial support.

¹⁰ In Sim2Real projects, synthetic data is used for training, and real-world data is used for testing.

Storing and Retrieving Data

As you begin to collect data, you'll need somewhere to store it. You'll also need a mechanism for getting data from devices into your data store, and from your data store to your training and testing infrastructure.

Storage requirements vary massively depending on how much data you expect your dataset to contain. The more data you have, the more sophisticated your solution needs to be. That said, edge AI datasets are typically relatively small and are unlikely to require technologies designed to operate at massive scale.

When choosing a solution, it's always preferable to go with the simplest you can get away with. If you're dealing with a quantity of data that can fit happily on a single workstation, there's no need to invest in fancy technology. The more direct access you have to your data for easy exploration and experimentation, the better—so from a convenience perspective, the ideal option is always your local filesystem.

Data Forms

Data tends to live in many different places throughout an organization's infrastructure. Some typical locations (which you may or may not have encountered) include:

- Production SQL databases
- Time series databases
- Log files
- Data lakes
- Data warehouses
- Cloud services
- IoT platforms

It's absolutely fine for data to live in different stores along its journey to become a part of your dataset. For example, you might find it convenient to store raw sensor data in one place, cleaned sensor data in another, and labels in a totally separate store.

When you're building a dataset for an AI project, you'll typically need to pull data from all of these disparate locations into a single place. You'll also need to reformat the data so it is compatible with the formats that are expected by the tools commonly used for developing signal processing and AI algorithms: Python-based software such as NumPy, pandas, scikit-learn, TensorFlow, and PyTorch, plus engineering software like MATLAB (see [Chapter 5](#)).

While there is no single standard, these tools typically expect data to be stored in simple, efficient, and filesystem-based formats. Even when training is done at massive

scale, in complex distributed infrastructure, the data itself is usually stored on disk in a relatively simple manner.

As such, you should expect to set up a pipeline for extracting data from your organizational data stores and transforming it into a simple format for training and evaluation. We'll learn about how to do this later.

Table 7-2 provides a quick reference to a variety of data storage solutions, with the advantages and disadvantages of each.

Table 7-2. Data storage solutions

Storage type	Advantages	Disadvantages
Local filesystem	Fast, simple, and easy to work with	No API, no backups, no distributed training; upper limit of a few terabytes
Network or cloud filesystem	Accessible by multiple machines; can store massive datasets	Slower than local filesystem; complex to set up mounts
Cloud object storage	Simple APIs for reading and writing data; massive scale	Data must be downloaded to use
Feature store	Data can be versioned and tracked; can store metadata; can query data	Data must be downloaded to use; more complex and costly than simple storage
End-to-end platform	Designed specifically for edge AI; data exploration tools built in; tight integration with data capture, training, and testing	More costly than simple storage

Data stored on a local filesystem is incredibly easy to use and can be accessed very fast. Even when using sophisticated cloud storage, data is typically copied to the local filesystem before training a model.

However, it's risky to store all of your valuable data on a single machine without a backup. It's also inconvenient if the data needs to be accessed by multiple people. Network shares, including cloud-based filesystems—like Amazon FSx, Azure Files, and Google Cloud Filestore—address this issue. However, they are relatively complex to access—they must be mounted as drives within an operating system.

Cloud object storage services—like Amazon S3, Azure Blob Storage, and Google Cloud Storage—provide HTTP APIs that make it much easier to get data in and out. These APIs can even be used by embedded devices to upload data from the edge, assuming the hardware is capable enough. However, they have slower access speeds than drive mounts, so data is typically downloaded to a local disk before it is used.

Feature stores are a relatively new trend in dataset storage. They are designed to offer simple APIs for data access and storage, along with additional features such as data versioning and the ability to query data. Feature store offerings from major providers include Amazon SageMaker Feature Store, Azure Databricks Feature Store,

and Google Cloud Vertex AI Feature Store. There are also open source equivalents you can host on your own infrastructure, such as Feast.

There are now several end-to-end platforms designed specifically for creating edge AI applications. Some of these include their own data storage solutions. These are typically equivalent to feature stores, but with the benefit of being designed specifically for edge AI projects. They may include tools for exploring and understanding sensor data or provide integration points with embedded software development tools. They are designed to integrate tightly with the other stages in the deep learning workflow. We learned more about these tools in [“Tools of the Trade” on page 136](#).

Data Versioning

In modern software engineering, it's expected that all source code is *versioned*—it exists within a system that keeps track of how it changes over time. It's important to know which version of code is deployed to production, or to a particular embedded device, so that problems can be traced back to the source.

In addition to code, machine learning systems are built using datasets. This means it makes a lot of sense to version your data, too. Data versioning tools allow you to record which data was used to train a given model. They can also help you understand where your data came from, which allows you to trace problems in production back to individual samples of data.

Data versioning is a powerful tool for data-centric ML, since it allows you to test different versions of your dataset and understand which performs better in the field. It is part of the practice of machine learning operations, described in [“Machine learning operations \(MLOps\)” on page 151](#).

Getting Data into Stores

If you're capturing sensor data for a project, how do you get it into your data store? The answer depends on your particular circumstances:

There is good connectivity on-site

If you have enough connectivity, bandwidth, and energy to send data directly from the edge, you can push data directly to APIs from your edge devices. This is easiest if you're using an end-to-end platform for edge AI that has APIs designed specifically for on-device use.

Another good option is to use an IoT platform. You can upload data to the platform using its purpose-built APIs, and then use another system to copy the data from the IoT platform into your dataset.

It's generally not a great idea to try to upload data directly to a cloud object store from an embedded device. Since the APIs were not designed for embedded use, they tend to use inefficient data structures, and their client libraries may not fit on small targets. This is less of an issue when working with embedded Linux devices, which have greater capabilities and access to a full OS.

There is poor or no connectivity on-site

If you lack decent connectivity, or you don't have the energy budget to send data from the very edge of the network, you may have to install some hardware to allow data to be stored at the edge and collected periodically.

This could mean modifying your existing hardware to add data storage. It might also mean adding another independent system, situated nearby, that is able to receive and store data from the device that is generating it. This separate system could be equipped with better connectivity, or it could be physically collected on a periodic basis.

Collecting Metadata

As we learned earlier, an ideal dataset is well documented. When designing your system for collecting data, you should be sure to capture as much information as possible about the context in which the data is being collected.

This additional information, known as metadata, can be included in your dataset alongside the sensor data itself. It may include things such as:

- The date and time that data was captured
- The specific device that collected the data
- The exact model of sensors used
- The location of the device on the data collection site
- Any people involved with the collection of the data

Metadata can be relevant to an entire dataset, to any subset of its records, or to individual records themselves. The paper “[Datasheets for Datasets](#)” (Geburu et al., 2018) defines a standard for collecting documentation that describes a dataset in aggregate, along with subsets of its records. While this is extremely valuable and should be considered a best practice, there are major benefits to collecting metadata on a more structured, granular, and machine-readable basis.

In many cases, you will be collecting samples of data that relate to individual entities. For example, you might be monitoring vibrations from a specific machine, capturing samples of keywords spoken by particular human beings, or logging biosignal data from individual farm animals.

In these cases, it's crucially important to capture as much relevant metadata as possible about each individual entity. In the case of a machine, you might capture:

- The exact make and model
- The machine's production run
- The place the machine is installed
- The work the machine is being used for

In the case of a person who is speaking keywords, you might try and capture any conceivable property that might affect their voice. For example:

- Physical characteristics, such as age, gender, or medical conditions
- Cultural characteristics, such as accent, race, or nationality
- Personal characteristics, such as profession or income level

You should attach this metadata to the individual samples that it relates to. This will allow you to split your dataset into subgroups according to metadata. You can use this ability to understand two things in great depth:

- During algorithm development, you will understand the makeup of your dataset and where you are missing representation and balance.
- During evaluation of your system, you will understand the weak areas of your model in terms of subgroups of your dataset.

For example, imagine you are training a model to detect faults in a machine. By analyzing your metadata, you may discover that most of your data samples have come from machines of a specific production run. In this case, you may wish to collect data from other production runs to improve representation of your dataset.

In a different situation, you may be using a keyword dataset to evaluate a keyword-spotting model. By cross referencing the model's performance on different data samples with the samples' metadata, you may discover that the model performs better on samples taken from older speakers versus younger ones. In this case, you may be able to collect more training data from younger speakers in order to improve the performance.

In this way, metadata helps reduce risk. Without sample-level metadata, you are blind to the composition of your dataset and the way your model performs on different groups within it. When you're armed with detailed information about the provenance of your data, you're able to build better products.

Ensuring Data Quality

Earlier in this chapter we listed the properties that an ideal dataset should have:

- Relevant
- Representative
- Balanced
- Reliable
- Well formatted
- Well documented
- Appropriately sized

As we learned in “[Data-Centric Machine Learning](#)” on page 210, a high-quality dataset reduces both the amount of data that is required and the impact of algorithm choice on creating an effective system. It’s a lot easier for machine learning systems to get useful results when they are trained and evaluated with good data.

But what is the best way to understand the quality of your dataset? The truth is that it comes down to domain expertise. If you have deep insight into the problem domain you are tackling, you’ll be able to draw on that insight to help evaluate your data.

Ensuring Representative Datasets

The most important property of a dataset is that it is representative. The reason for this is that the goal of an AI algorithm is to model a real-world situation in order to make decisions. The only mechanism it has for learning about the real world is the dataset used to train or design it. This means that if a dataset is not representative, the resulting algorithms will fail to represent the real world.

For example, imagine you are building an AI system to help recognize different types of plant disease, using photographs of afflicted plants. If your dataset does not include photographs of the correct plants, or the appropriate symptoms, there is no way the AI system you design can be effective, no matter how sophisticated its algorithms.

Even worse, since the dataset is also used to evaluate the system’s performance, we’ll have no idea that there is even a problem until we deploy the model to the field.¹¹

This is where domain expertise comes in. If you are an expert in plant diseases, you can use your knowledge to help understand whether the dataset is representative of real-world conditions. For example, perhaps your dataset is missing photos of some species of plants that are affected by the disease you wish to identify.

¹¹ In this case our field may be literal, not metaphorical!

Metadata is incredibly helpful for this process. If your dataset contains metadata that indicates the plant species in each photograph, a domain expert can simply review the list of species and immediately notice whether one is missing.

Another useful way to use metadata is to plot the distributions of specific metadata attributes throughout the data. For example, you might choose to plot the numbers of samples that belong to each species. If this distribution does not look sensible with regards to real-world conditions, you may need to collect more data. For example, you may have significantly more records from one species than another, as in [Figure 7-4](#).

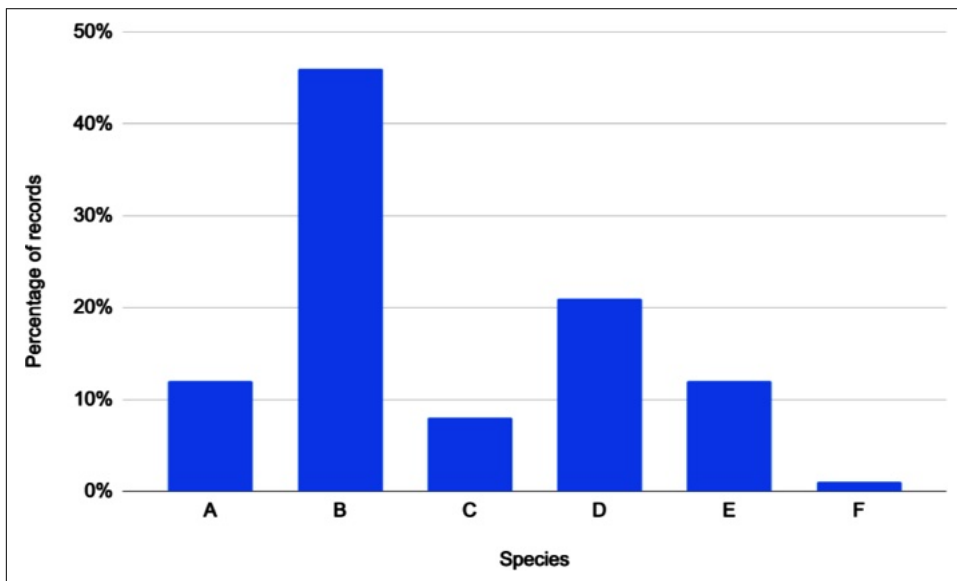


Figure 7-4. This dataset features many more records for some species than others. This may cause problems with fairness; for example, it's likely your algorithm will perform much better with species B than with species F.

In addition to the dataset at large, it's important that representation is maintained across labels. For example, you should ensure there is equally good representation of every affected species within each class of plant disease you are attempting to identify.

If your dataset represents individual entities, you should also check to ensure that your dataset is balanced with regards to those entities. For instance, perhaps all of the photos of one species were taken from a single plant, while the photos of another species were taken from multiple plants.

A domain expert should be able to help identify the axes of data that are important to explore in this way. But what if adequate metadata is not available? This is very

common when using data that was not deliberately collected for a specific project. In this case, you'll have to embark on a systemic review of the dataset.

Representation and Time

One of the most important aspects of data quality is the idea that your dataset captures all of the variation present in the real-world context it is designed to represent. This is why we care so much about subgroup representation, like the species of plants in our plant disease dataset.

Plant species is an obvious subgroup in this example, but there is another major property that affects almost all datasets. That property is *time*. Our world is in a constant state of change, which means that data collected about a system at one moment in time does not necessarily represent the state of the system in the future.

For example, imagine we collect our photographs of plants during the springtime. Plant appearance naturally varies throughout the course of the year, as they grow and change with the seasons. If our dataset only includes photos of plants in the springtime, a model trained on it may not perform well during the autumn when the appearance of the plants has changed. This property of data is known as *seasonality*.

To combat this risk, we need to apply domain expertise. A domain expert will understand if there is likely to be seasonal variation in the data and can guide the data collection process accordingly—for example, to make sure that we collect images of plants throughout the year.

The idea of seasonality affects all datasets, not just those containing plants, and a *season* in this context can be any period of time. For example, a fitness wearable may need to account for natural changes in the human body that occur over the course of a day. If its dataset was collected only during the morning, it may be unreliable at night.

It's also worth checking for seasonality in your data even if your domain expert is not particularly concerned about it. There may be other variables at play, such as the effect of varying ambient temperatures on sensor noise. There are many algorithmic techniques you can use to identify seasonality in data, and you can also test your model on different time-based subgroups of your dataset to help identify any issues.

Reviewing Data by Sampling

The challenge of reviewing data quality, especially when there is limited accompanying metadata, is that it's often infeasible to look over every sample of data individually. Datasets can be gigantic, and the time of domain experts is precious (and expensive).

Fortunately, *sampling* gives us a way to review data without having to inspect every last item. For a given dataset, a sufficiently sized random sample of records will have

approximately the same representation and balance as the larger dataset. This smaller sample can be inspected thoroughly by a domain expert in order to understand the quality of the dataset as a whole.

The tricky part is determining how large a sample needs to be. It needs to be large enough to have a reasonable probability of including the characteristics we care about but small enough to be reviewed in a reasonable time.

For example, imagine our domain expert is trying to understand whether the dataset contains enough instances of a certain species of plant. To do this, they can count the instances of that plant species in a sample of the data and calculate the ratio between instances of that species and any others. But how big a sample size is required in order for us to assume that the ratio between plant species is equivalent between the sample and the entire dataset?

There's actually a formula we can use to estimate the sample size. It looks like this:

$$\text{Sample size} = \frac{(Z \text{ score})^2 * \text{standard deviation} * (1 - \text{standard deviation})}{(\text{margin of error})^2}$$

In this formula, the *margin of error* represents the amount of difference we're willing to tolerate between the ratio in our sample and in our full dataset. It's common to set this to 5%, meaning we'll be OK with the ratio in our sample being either 2.5% higher or 2.5% lower than the ratio in the entire dataset.

The Z score expresses our confidence level, or how confident we need to be that the number we get will *actually* fall within the bounds of our margin of error. A reasonable confidence level is 95%, which would give us a Z score of 1.96,¹² assuming a dataset of typical size (anything more than tens of thousands of samples).

Finally, the standard deviation represents how much we expect the data to vary. Since there's no real way to know this ahead of time, we can just play it safe and set it to 0.5, which maximizes the sample size.

If we plug all of these together, we'll get the following:

$$\text{Sample size} = \frac{(1.96)^2 * 0.5 * (1 - 0.5)}{(0.05)^2} = \frac{0.9604}{0.0025} = 384.16$$

Since there's no such thing as a fraction of a sample, we can round the sample size up to 385. This tells us we'll need to randomly sample 385 items in order to have 95%

¹² The Z score can be looked up in a table such as the one [hosted by Wikipedia](#).

confidence that the ratio of one species to another lies within 5% of the value we see in our random sample.

It turns out that this number doesn't vary too much with the size of the dataset, at least for datasets of sizes that are relevant for machine learning. It's most sensitive to changes in the margin of error: if you want a margin of error of only 1%, you'll need to review a sample with 9,604 items. Qualtrics provides a [handy online calculator](#) that makes it easy to experiment.

All this goes to say that it should generally suffice to randomly select a few hundred samples from your dataset.¹³ This should be a manageable number to review and will also give you some reasonable insight into whether your dataset has acceptable quality.

Of course, this assumes that the subgroups you are looking for are large enough to fit within the error bounds. For example, if a plant species represents less than 5% of the data, then it's unlikely we will find it in a sample of 385 items. However, if you are hunting for underrepresented subgroups, then this will still be a helpful result: it will guide you to add more data, eventually allowing the group to be detectable with random sampling.

Label Noise

Beyond representativeness, another major source of dataset quality issues comes from something called *label noise*. Labels give us the values that we are trying to use AI to predict. For example, if we're training a plant disease classifier, a photo of an unhealthy plant might be labeled with the exact disease that is afflicting it. Labels don't have to be classes, though—for example, if we are solving a regression problem we would expect the data to be labeled with the number that we are trying to predict.

Unfortunately, the labels attached to data are not always correct. Since most data is labeled by human beings, it's common for errors to creep in. These errors can be quite significant. A research team from MIT found that an average of 3.4% of samples are incorrectly labeled across a set of commonly used public datasets¹⁴—they even [built a website to showcase the errors](#).

Label noise isn't a total catastrophe. Machine learning models are pretty good at learning to cope with noise. But it does have a significant impact,¹⁵ and to squeeze the

¹³ To make sure the sample is truly random, it's a good idea to use sampling tools such as those provided by [NumPy](#).

¹⁴ Curtis G. Northcutt et al., "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," arXiv, 2021, <https://oreil.ly/Zrcu1>.

¹⁵ For a good exploration of the impact of label noise, check out Gökem Algan and Ilkay Ulusoy, "Label Noise Types and Their Effects on Deep Learning," arXiv, 2020, <https://oreil.ly/1LZKl>.

most performance out of your models it can be worth trying to clean up noisy labels. The constraints of edge AI already place a premium on model performance. Cleaning up noisy labels may deliver a good return on investment versus spending more time on algorithm design or model optimization.

The simplest way to identify label noise is by reviewing random samples of data, but with large datasets this can be like looking for needles in a haystack. Instead of sampling randomly, it's better to focus the search more intelligently.

A good method for doing this is by hunting for outliers within a class. If a sample is misclassified, it likely appears significantly different from the other members of the class it is mislabeled as. For simple data, this may be easy using standard data science tools. For high-dimensional data, like images or audio, it can be more challenging.

The end-to-end edge AI platform that Edge Impulse uses has an interesting solution to this problem. Edge Impulse's feature explorer uses an unsupervised dimensionality reduction algorithm to project complex data into a simplified 2D space, where proximity correlates with similarity. This approach **makes it easy to spot outliers**, as in **Figure 7-5**.

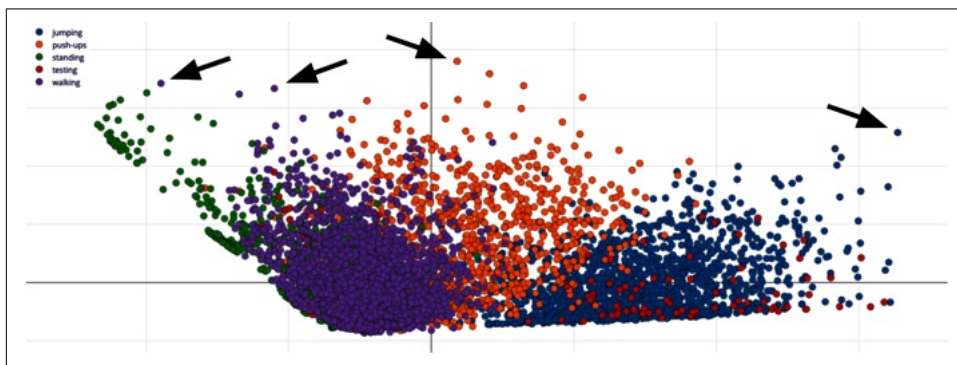


Figure 7-5. Each dot represents a sample of data, and the distance between dots represents their similarity. Outliers, such as those highlighted with arrows, are unusual samples. When samples appear close to those belonging to other classes, it's worth investigating to see if they have been mislabeled.

Another simple way to hunt for noisy class labels is to assume that a model trained on the data will be *less confident* at classifying noisy samples. If training samples are ranked in order of a trained model's confidence at assigning them a class, it's likely that mislabeled samples will appear toward the bottom of the list.

In datasets for problems other than classification, label noise looks a bit different. For example, label noise in a regression dataset consists of error in the target values, while label noise in an object detection or segmentation dataset means the bounding